

TRATADO COMPLETO DE ALGORITMO Y DATOS

Nahuel Villfañe

```
function busquedaBinaria (A, x)
  l = 0; r = length(A) - 1
  while l <= r
    m = (l + r) // 2
    if A[m] == x
      return m
    else if A[m] < x
      l = m + 1
    else
      r = m - 1
  return -1
```

```
DFS(v)
  visitado[v] = true
  for each u in ady(v)
    if not visitado[u]
      DFS(u)
```



TRATADO DE ALGORITMOS Y DATOS I

Desde Cero con el Lenguaje C

Nahuel Villafañe

Versión 1.0

2026

Índice general

Introducción	11
Unidad I: Los Cimientos de la Programación en C	13
Concepto de Algoritmo	13
Concepto de Lenguaje Estructurado	13
Comparación de C con otros lenguajes	14
Concepto de Compiladores e Intérpretes	14
El Lenguaje de Programación C	15
Un vistazo a un programa básico	15
Variables, Constantes y Operadores	15
Concepto de Variable	15
Declaración, Inicialización y Asignación	15
Reglas para Identificadores	16
Palabras Reservadas	16
Tipos de Variables según su Alcance (Ámbito)	16
Variables Locales	16
Parámetros Formales	16
Variables Globales	16
Constantes	17
Constantes Literales	17
Constantes Simbólicas	17
Operadores en C	18
Aritméticos	18
Asignación	18
Incremento y Decremento	18
Relacionales	18
Lógicos	18
Operadores de Bits	18
Conversiones y Modificaciones de Tipo	19
Conversión Implícita	19
Conversión Explícita (Casting)	19
Modificadores de Tipo	19
Especificadores de Tipo de Almacenamiento	19
Arreglos de Caracteres (Strings)	20
Tipos Enumerativos (enum)	20
La declaración typedef	20

El operador <code>sizeof</code>	21
Resumen de la Unidad I	21
Unidad II: El Control del Flujo - Tomando Decisiones y Repitiendo Tareas	23
Las Construcciones Fundamentales del Lenguaje	23
Sentencias de Bloque y Expresión	23
Sentencias de Expresión	23
Sentencias de Bloque	24
Selección: Tomar Decisiones	25
La sentencia <code>if</code>	25
La sentencia <code>if-else</code>	26
La sentencia <code>else if</code>	27
La sentencia <code>switch</code>	27
Iteración: Repetir Tareas (Bucles)	29
El bucle <code>while</code>	29
El bucle <code>do-while</code>	30
El bucle <code>for</code>	31
Sentencias de Salto	32
<code>break</code>	32
<code>continue</code>	33
<code>goto</code> (El Gran Evitado)	34
<code>return</code>	34
Sentencias de Etiquetado	35
Resumen de la Unidad II	35
Unidad III: Arrays y Punteros - El Corazón de C	37
Arrays (Arreglos)	37
Declaración e Inicialización de Arrays	38
Acceso a los elementos	38
Arrays Multidimensionales	39
Declaración e Inicialización	39
Introducción a los Punteros	40
Declaración de Punteros	41
Operadores de Punteros	41
Asignación de Punteros	42
Aritmética de Punteros	42
Arrays y Punteros: Una Relación Íntima	43
Punteros a Punteros	44
Arrays de Punteros	44
Punteros a Funciones	45
Argumentos en la Línea de Comandos	46
Funciones para Manejo de Memoria Dinámica	46
<code>malloc()</code>	47
<code>realloc()</code>	47
Resumen de la Unidad III	49

Unidad IV: Principios de Diseño de Algoritmos y Programas - Escribiendo

Código de Calidad	51
Principios Fundamentales de Diseño	52
Denominaciones Significativas (Naming)	52
Documentación y Formato (Estilo de Código)	53
Refinamiento (Top-Down Design)	54
Codificación y Pruebas	55
Codificación	55
Pruebas (Testing)	55
Modularización: Divide y Vencerás	56
Criterios y Principios de Modularización	57
Reusabilidad	58
Abstracción de Datos	58
Aspectos Acerca de la Calidad del Software	60
Resumen de la Unidad IV	61

Unidad V: Funciones - Los Bloques de Construcción de Programas

Concepto de Función	63
Sintaxis Básica de una Función	63
La Instrucción return	65
Valores de Retorno	66
Reglas de Alcance de Funciones	67
Argumentos de Funciones	67
Llamada por Valor (Pass by Value)	67
Llamada por Referencia (Simulada con Punteros)	68
Comparación: Por Valor vs Por Referencia	69
Prototipos de Funciones	69
Retorno de Punteros	70
Modelos de Memoria en C	72
Segmentos de Memoria	73
Ejemplo de ubicación de variables	73
Recursividad	74
Componentes de una Función Recursiva	74
Ejemplo Clásico: Factorial	74
Ejemplo: Fibonacci	75
Recursividad vs Iteración	76
Resumen de la Unidad V	76

Unidad VI: Compilación, Enlace y el Preprocesador - Construyendo Pro-

yectos Profesionales	77
El Proceso Completo: De Código Fuente a Ejecutable	77
Fase 1: Preprocesamiento	77
Fase 2: Compilación	78
Fase 3: Ensamblado	79
Fase 4: Enlace (Link)	79
El Preprocesador de C en Profundidad	80

Las Directivas del Preprocesador	80
Compilación Condicional	82
Directivas Principales	82
Usos Comunes de la Compilación Condicional	83
Archivos Cabecera (.h) - La Interfaz Pública	84
Qué debe ir en un archivo .h	84
Qué NO debe ir en un archivo .h	84
Ejemplo Completo de un Módulo Bien Diseñado	85
Concepto de Proyecto	86
Concepto de Makefile	87
Estructura básica de una regla en Makefile	87
Ejemplo de Makefile para nuestro proyecto	87
Concepto de Biblioteca (Librería)	88
Bibliotecas Estáticas (.a en Linux/Mac, .lib en Windows)	88
Bibliotecas Dinámicas (Compartidas) (.so en Linux, .dylib en Mac, .dll en Windows)	89
Bibliotecas Estándar de C	89
Biblioteca Estándar de Entrada-Salida (<stdio.h>)	90
Biblioteca para Manejo de Strings (<string.h>)	91
Otras Bibliotecas Estándar Importantes	93
Presentación Comparativa de Varios Compiladores	95
Comparativa de Mensajes de Error (Un ejemplo vale más que mil palabras)	95
Compilando con Diferentes Compiladores	96
Resumen de la Unidad VI	96
Unidad VIII: Entrada y Salida - Comunicación con el Mundo Exterior	99
El Concepto de Stream (Flujo)	99
Entrada y Salida Estándar	99
Concepto de Archivo	100
Tipos de Archivos (desde la perspectiva de C)	100
Operaciones Básicas con Archivos	101
Apertura y Cierre: fopen() y fclose()	101
Escritura en Archivos	102
Lectura de Archivos	102
La Importancia de fclose()	104
Funciones de Streams Avanzadas	104
Verificación de Fin de Archivo (EOF)	104
Posicionamiento en el Archivo: ftell(), fseek(), rewind()	105
Gestión de Errores	106
Funciones Tipo Unix de Manejo de I/O (Bajo Nivel)	106
¿Por qué usar funciones de bajo nivel?	107
Las funciones clave (en sistemas Unix/POSIX)	107
Tabla Comparativa: Funciones Estándar vs. Unix	109
Ejemplo: Copiar un archivo con funciones de bajo nivel	109
Resumen de la Unidad VIII	110

,

Abstract

Hola! Que tal? Nos encontramos de nuevo. Soy Nahuel Villafañe, un estudiante UBA Exactas, como veras y te topaste con mi nuevo apunte que hice. Espero te sirva, lo hice con mucho amor, paciencia, notas, investigacion, consultas y con muchas ganas de ayudar. Como digo siempre, mi idea es ayudar haciendo estos apuntes para las distintas carreras, porque se lo dificil que es la facultad. Ademas, lo escribi con mis propias palabras y con ayuda de la IA para verificar que este bien. Te deseo exitos en la cursada, si te sirve, no te rindas, estudiar es aprender. Uno aprende hasta perfeccionarse porque de eso se trata , se trata de aprender y ser persistente. Lo digo por experiencia propia. En fin, exitos!



Figura 1: *El secreto de la existencia humana no sólo está en vivir, sino también en saber para qué se vive* Manuel Sadosky

Introducción: El Arte de Instruir a una Máquina

Imagina que quieres enseñarle a un amigo a hacer un café. No le dices simplemente "haz un café", porque esa orden es demasiado abstracta. En cambio, le das una secuencia de pasos muy detallada:

1. Ve a la cocina.
2. Toma la cafetera.
3. Vierte agua hasta la marca de 2 tazas.
4. Coloca un filtro.
5. Añade dos cucharadas de café.
6. Presiona el botón de encendido.
7. Espera a que termine.
8. Sirve el café.

Eso es, en esencia, un **algoritmo**: una serie de pasos ordenados y finitos para resolver un problema.

Nuestro "amigo" en este caso es la **computadora**. Es una máquina increíblemente rápida y obediente, pero solo entiende un lenguaje muy básico de unos y ceros (lenguaje máquina). La **programación** es el puente entre nuestra capacidad para resolver problemas y la capacidad de la máquina para ejecutar órdenes.

Unidad I: Los Cimientos de la Programación en C

Concepto de Algoritmo

Un algoritmo no es solo una receta. Debe cumplir con características muy estrictas:

- **Preciso:** Cada paso debe estar definido sin ambigüedad.
- **Determinista:** Dada una misma entrada, debe producir siempre la misma salida.
- **Finito:** Debe terminar después de un número finito de pasos.

Ejemplo: Algoritmo para calcular el área de un rectángulo

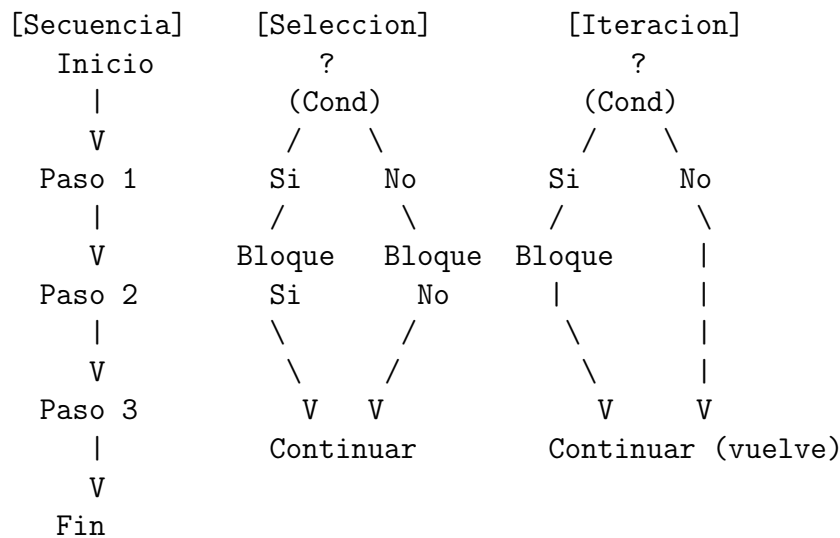
1. **INICIO**
2. Leer el valor de la base (B).
3. Leer el valor de la altura (H).
4. Calcular el área: $A = B \times H$.
5. Mostrar el resultado (A).
6. **FIN**

Concepto de Lenguaje Estructurado

Un lenguaje estructurado nos permite escribir programas siguiendo una lógica clara, basada en tres estructuras fundamentales:

Estructura	Descripción
Secuencia	Instrucciones se ejecutan una detrás de otra, en orden.
Selección	Permite elegir entre caminos alternativos basándose en una condición (ej. <code>if-else</code>).
Iteración	Permite repetir un bloque de instrucciones mientras se cumpla una condición (ej. <code>while</code> , <code>for</code>).

Representación Gráfica



Comparación de C con otros lenguajes

C es el "latín" de la programación. Muchos lenguajes modernos derivan de él.

- **C vs. Python:** Python es de alto nivel e interpretado (automático). C es de nivel medio y compilado (control total). C es más rápido y te da control del hardware, pero es más complejo.
- **C vs. Java:** Java está orientado a objetos y corre en una máquina virtual. C es más directo y procedural.
- **C vs. C++:** C++ es C con orientación a objetos y otras características añadidas. C es un subconjunto de C++.

Ventajas de C: Eficiencia, velocidad, control de memoria, portabilidad. **Desventajas de C:** Gestión manual de la memoria, curva de aprendizaje pronunciada.

Concepto de Compiladores e Intérpretes

La computadora solo entiende código máquina. Necesitamos un traductor.

- **Compilador:** Traduce todo el programa de una vez a un ejecutable. (Ej: C, C++, Go).
- **Intérprete:** Traduce y ejecuta el programa línea por línea cada vez. (Ej: Python, JavaScript).

[Compilador]

programa.c ---> [COMPILADOR] ---> programa.exe ---> [EJECUCION] ---> Resultado

[Intérprete]

programa.py ---> [INTERPRETE (ej. python)] ---> Resultado

El Lenguaje de Programación C

Un vistazo a un programa básico

```
1 /* Este es mi primer programa. */
2 #include <stdio.h> // Incluye funciones de entrada/salida
3
4 int main() {        // Funcion principal
5     printf("Hola, mundo!\n"); // Imprime en pantalla
6     return 0;        // Indica que termino correctamente
7 }
```

Listing 1: Nuestro primer programa en C

Variables, Constantes y Operadores

Concepto de Variable

Una **variable** es un espacio en memoria donde guardamos un valor que puede cambiar.

Nombre: edad

```
+-----+
|  25   | <-- Valor
+-----+
```

Direccion: 0x7FFF

- **Nombre:** La etiqueta para referirnos a ella.
- **Tipo:** Define qué datos puede guardar y cuánto espacio ocupa.
- **Valor:** El dato actual.

Declaración, Inicialización y Asignación

```
1 #include <stdio.h>
2 int main() {
3     int edad;           // Declaracion (valor "basura" ↔
4                          // inicial)
5     edad = 25;          // Asignacion
6
7     float altura = 1.75; // Declaracion e inicializacion
8     char inicial = 'N';
9
10    edad = 26;           // Re-asignacion
11    return 0;
12 }
```

Listing 2: Trabajando con variables

Reglas para Identificadores

- Pueden contener letras, dígitos y guión bajo (_).
- Deben comenzar con letra o guión bajo.
- C es sensible a mayúsculas/minúsculas ('Edad' $\neq$ 'edad').
- No se pueden usar **palabras reservadas** ('int', 'float', 'return', etc.).

Palabras Reservadas

Palabras con significado especial para el compilador. auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while.

Tipos de Variables según su Alcance (Ámbito)

Variables Locales

Declaradas dentro de una función o bloque “. Solo existen allí.

```
1 int main() {  
2     int x = 10; // Local a main  
3     if (x > 5) {  
4         int y = 20; // Local al if  
5     }  
6     // y ya no existe aqui  
7 }
```

Parámetros Formales

Variables en la definición de una función. Actúan como locales.

```
1 int sumar(int a, int b) { // a y b son parametros formales  
2     return a + b;  
3 }
```

Variables Globales

Declaradas fuera de todas las funciones. Accesibles desde cualquier función. Su uso excesivo es mala práctica.

```
1 int contador = 0; // Global  
2  
3 void inc() {
```

```

4     contador++;
5 }

```

[Ambito de Variables]

```

+-----+
| PROGRAMA                                     |
| int global = 100;                           |
|                                             |
| +-----+                                   |
| | main()                                     | |
| | int local_main = 10;                     |
| | +-----+                               |
| | | if {                                     |
| | | int local_if = 20;                     |
| | +-----+                               |
| +-----+                                   |
|                                             |
| +-----+                                   |
| | otra_funcion( int param )                 |
| | int local_otra = 30;                     |
| +-----+                                   |
+-----+

```

Constantes

Valores que no cambian.

Constantes Literales

- **Enteras:** 10, -5
- **Reales:** 3.14, -0.5
- **Carácter:** 'A', '7' (comillas simples)
- **Cadena:** "Hola" (comillas dobles)

Constantes Simbólicas

```

1 #define PI 3.1416 // Directiva del preprocesador
2
3 const int DIAS = 7; // Palabra clave 'const'

```

Operadores en C

Aritméticos

+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo (resto)

Nota:

Dividir enteros da entero: $5/2 = 2$. Para decimal, usa $5/2.0$.

Asignación

= asigna. Compuestos: +=, -=, *=, /=, %=.

```
1 a += 5; // Equivale a a = a + 5;
```

Incremento y Decremento

```
1 int x=5, y=5;
2 int a = x++; // a=5, x=6 (post-incremento)
3 int b = ++y; // y=6, b=6 (pre-incremento)
```

Relacionales

Comparan, resultado 1 (verdadero) o 0 (falso). ==, !=, >, <, >=, <=

Lógicos

Combinan expresiones booleanas. && (AND), || (OR), ! (NOT).

Operadores de Bits

Actúan sobre bits. Avanzado. &, |, ^, ~, «, »

Ej: $6 \& 3 = 2$

6 = 00000110

3 = 00000011

& = 00000010 = 2

Conversiones y Modificaciones de Tipo

Conversión Implícita

El compilador convierte automáticamente tipos "menores" a "mayores".

```
1 float r = 5 + 2.5; // 5 se convierte a 5.0 implícitamente
```

Conversión Explícita (Casting)

El programador fuerza la conversión.

```
1 float pi = 3.14;
2 int p = (int) pi; // p = 3 (se trunca)
3 float div = (float) 10 / 3; // 3.333...
```

Modificadores de Tipo

- `short`, `long`: Modifican el rango de `int`.
- `signed`: Con signo (por defecto).
- `unsigned`: Solo positivos (duplica el rango positivo).

```
1 unsigned int cont = 0;
2 long long int numero_grande;
```

Especificadores de Tipo de Almacenamiento

- `auto`: Por defecto para locales (raro usarlo).
- `register`: Sugiere almacenar en registro de CPU.
- `static`: Variable local que conserva su valor entre llamadas; o variable global con visibilidad limitada al archivo.

```
1 void contar() {
2     static int c = 0; // Se inicializa una sola vez
3     c++;
4 }
```

Arreglos de Caracteres (Strings)

En C, un string es un arreglo de char que termina en `\0` (carácter nulo).

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char nombre1[] = "Nahuel"; // El compilador agrega '\0'
6     char nombre2[20] = "Nahuel";
7
8     printf("%s\n", nombre1);
9     printf("Longitud: %zu\n", strlen(nombre1)); // strlen ←
10     del <string.h>
11     return 0;
12 }
```

nombre1 en memoria:

[0]	[1]	[2]	[3]	[4]	[5]	[6]
N	a	h	u	e	l	\0

Tipos Enumerativos (enum)

Crea un conjunto de constantes con nombre.

```
1 enum dias { LUN, MAR, MIE, JUE, VIE, SAB, DOM };
2
3 int main() {
4     enum dias hoy = MIE; // hoy = 2 (por defecto empieza ←
5     en 0)
6     return 0;
7 }
```

La declaración typedef

Crea un alias para un tipo existente.

```
1 typedef int Entero;
2 typedef unsigned char Byte;
3
4 int main() {
5     Entero edad = 30; // Lo mismo que 'int edad'
6     return 0;
7 }
```

El operador sizeof

Devuelve el tamaño en bytes de un tipo o variable.

```
1 #include <stdio.h>
2 int main() {
3     int a;
4     double b;
5     int arr[10];
6
7     printf("Tamaño de int: %zu bytes\n", sizeof a);
8     printf("Tamaño de arr: %zu bytes\n", sizeof arr);
9     printf("Elementos en arr: %zu\n", sizeof arr / sizeof ↵
10         arr[0]);
11     return 0;
12 }
```

Resumen de la Unidad I

- Un **algoritmo** es una secuencia de pasos para resolver un problema.
- C es un lenguaje **estructurado** y **compilado**.
- Las **variables** son espacios en memoria con un nombre y un tipo.
- Las **constantes** son valores fijos.
- Los **operadores** permiten manipular datos.
- El **ámbito** de una variable define dónde existe.
- Los **strings** son arreglos de char terminados en `\0`.
- `enum`, `typedef` y `sizeof` son herramientas fundamentales.

Unidad II: El Control del Flujo - Tomando Decisiones y Repitiendo Tareas

En la Unidad I, aprendimos a declarar variables y a realizar operaciones con ellas. Nuestros programas eran una simple **secuencia** de instrucciones que se ejecutaban de arriba a abajo, una detrás de otra. Para crear programas que resuelvan problemas del mundo real, necesitamos poder controlar el **flujo de ejecución**: la capacidad de tomar decisiones (elegir qué camino seguir) y de repetir bloques de código (ejecutar algo varias veces). Esta unidad está dedicada por completo a dominar estas estructuras de control.

Las Construcciones Fundamentales del Lenguaje

Como vimos en la unidad anterior, la programación estructurada se basa en tres pilares. Ahora los exploraremos a fondo en C.

1. **Secuencia**: La ejecución lineal y ordenada de instrucciones. Es el modo por defecto.
2. **Selección (o Decisión)**: Permite ejecutar un bloque de código u otro en función de una condición. Las herramientas en C son: `if`, `else if`, `else` y `switch`.
3. **Iteración (o Ciclo o Bucle)**: Permite ejecutar un bloque de código repetidamente mientras se cumpla una condición. Las herramientas en C son: `while`, `do-while` y `for`.

Además de estas, C nos proporciona sentencias para alterar el flujo de manera más fina: las **sentencias de salto** (`break`, `continue`, `goto`, `return`).

Sentencias de Bloque y Expresión

Antes de sumergirnos en las estructuras de control, debemos entender dos conceptos básicos pero cruciales.

Sentencias de Expresión

En C, la mayoría de las líneas de código son **sentencias de expresión**. Una sentencia de expresión es cualquier expresión válida en C seguida de un punto y coma (;). La

expresión se evalúa, pero el resultado de esa evaluación (si lo hay) se descarta. Su propósito es el **efecto secundario**, como cambiar el valor de una variable o llamar a una función.

```

1 #include <stdio.h>
2 int main() {
3     int x = 5;           // Sentencia de expresion: ←
4                           // asignacion
5     x = x + 2;           // Sentencia de expresion: ←
6                           // asignacion
7     x++;                 // Sentencia de expresion: ←
8                           // incremento
9     printf("Hola");      // Sentencia de expresion: ←
10                           // llamada a funcion
11     3 + 4;               // Sentencia de expresion VALIDA ←
12                           // pero INUTIL (no tiene efecto secundario)
13     ;                    // Sentencia vacia (tambien valida)
14     return 0;            // Sentencia de salto (return)
15 }
```

Listing 3: Sentencias de expresión

Como ves, casi todo lo que termina en `;` es una sentencia de expresión. La línea `3 + 4;` es sintácticamente correcta, pero el resultado de la suma se calcula y se pierde, por lo que no hace nada útil.

Sentencias de Bloque

Un **bloque** (o **sentencia compuesta**) es un grupo de declaraciones y sentencias encerradas entre llaves `{ }`. El bloque se considera una sola unidad sintáctica. Es decir, donde el lenguaje espera una única sentencia (por ejemplo, después de un `if`), podemos poner un bloque con muchas sentencias. Además, las variables declaradas dentro de un bloque tienen ámbito local a ese bloque.

```

1 #include <stdio.h>
2 int main() {
3     int a = 10;
4     printf("Valor de a fuera del bloque: %d\n", a);
5
6     { // Inicio del bloque
7         int b = 20; // b solo existe dentro de este bloque
8         a = a + b;  // Podemos modificar a, que es externa
9         printf("Dentro del bloque: a=%d, b=%d\n", a, b);
10    } // Fin del bloque, b se destruye
11
12    // printf("b = %d", b); // ERROR! b no existe aqui
13    printf("Valor de a despues del bloque: %d\n", a);
14    return 0;
15 }
```

Listing 4: Sentencia de bloque

Selección: Tomar Decisiones

La capacidad de tomar decisiones es lo que dota de inteligencia.^a nuestros programas. En C, la decisión se basa en el valor de una **expresión condicional**: si la expresión es verdadera (cualquier valor distinto de cero), se ejecuta un bloque; si es falsa (cero), se ejecuta otro (opcionalmente).

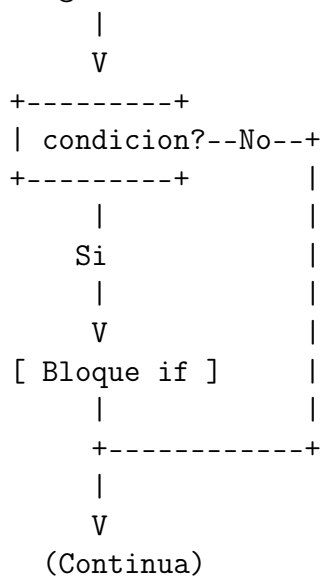
La sentencia if

Es la estructura de decisión más simple.

```
1 if (condicion) {
2     // Bloque de código que se ejecuta SI la condicion es ←
    verdadera (distinta de cero)
3 }
```

Listing 5: Sintaxis de if

[Diagrama de if]



```
1 #include <stdio.h>
2 int main() {
3     int temperatura = 30;
4
5     if (temperatura > 25) {
6         printf("Hace calor. Prende el aire ←
           acondicionado.\n");
7     }
8
9     printf("Esta línea siempre se ejecuta.\n");
10    return 0;
11 }
```

Listing 6: Ejemplo de if

La sentencia if-else

Amplía el if para ejecutar un bloque alternativo cuando la condición es falsa.

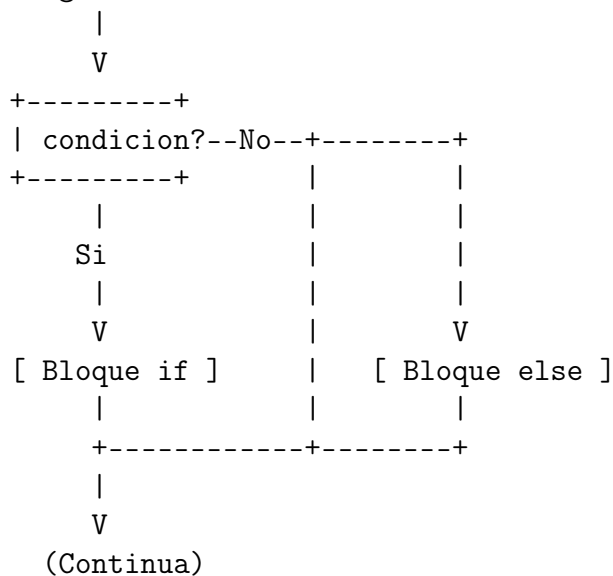
```

1 if (condicion) {
2     // Bloque que se ejecuta si la condicion es verdadera
3 } else {
4     // Bloque que se ejecuta si la condicion es falsa
5 }

```

Listing 7: Sintaxis de if-else

[Diagrama de if-else]



```

1 #include <stdio.h>
2 int main() {
3     int edad = 16;
4
5     if (edad >= 18) {
6         printf("Eres mayor de edad. Puedes votar.\n");
7     } else {
8         printf("Eres menor de edad. Aun no puedes ↵
9             votar.\n");
10    }
11    return 0;

```

Listing 8: Ejemplo de if-else

La sentencia else if

Cuando tenemos más de dos caminos posibles, podemos encadenar varias condiciones con `else if`. Es, en esencia, un `if` dentro del `else` de otro `if`, pero con una sintaxis más limpia.

```
1 if (condicion1) {  
2     // Bloque si condicion1 es verdadera  
3 } else if (condicion2) {  
4     // Bloque si condicion1 es falsa Y condicion2 es ←  
5     verdadera  
6 } else if (condicion3) {  
7     // Bloque si condicion1 y condicion2 son falsas Y ←  
8     condicion3 es verdadera  
9 } else {  
10    // Bloque si todas las condiciones anteriores son falsas  
11 }
```

Listing 9: Sintaxis de else if

```
1 #include <stdio.h>  
2 int main() {  
3     int nota = 85;  
4  
5     if (nota >= 90) {  
6         printf("Calificacion: A (Excelente)\n");  
7     } else if (nota >= 80) {  
8         printf("Calificacion: B (Bien)\n");  
9     } else if (nota >= 70) {  
10        printf("Calificacion: C (Suficiente)\n");  
11    } else {  
12        printf("Calificacion: D (Reprobado)\n");  
13    }  
14    return 0;  
15 }
```

Listing 10: Ejemplo de else if

La sentencia switch

La sentencia `switch` es una forma especializada de selección múltiple. Se usa cuando queremos comparar el valor de una **única expresión** contra una lista de **constantes enteras** (o de tipo carácter, que en el fondo son enteros). Es más eficiente y legible que una larga cadena de `else if` en estos casos.

```
1 switch (expresion) {  
2     case constante1:  
3         // Bloque de codigo si expresion == constante1  
4         break; // Opcional, pero crucial
```

```

5     case constante2:
6         // Bloque de código si expresion == constante2
7         break;
8     // ... mas casos ...
9     default:
10        // Bloque de código si expresion no coincide con ←
11        // ninguna constante
12        // (opcional)
13 }

```

Listing 11: Sintaxis de switch

Puntos clave del switch:

- La expresion debe ser de un tipo entero (int, char, enum) o poder convertirse a entero.
- Cada case va seguido de una constante (un valor literal, no una variable).
- Al final de cada bloque de case se suele poner break;. El break hace que el programa salga del switch. Si no se pone, ocurre algo llamado **fall through** (caída): la ejecución continúa con el siguiente case, ignorando su condición, hasta encontrar un break o el final del switch. A veces, el *fall through* es intencional y útil.

```

1 #include <stdio.h>
2 int main() {
3     char operador;
4     float num1, num2, resultado;
5
6     printf("Ingresa un operador (+, -, *, /): ");
7     scanf("%c", &operador);
8     printf("Ingresa dos numeros: ");
9     scanf("%f %f", &num1, &num2);
10
11     switch (operador) {
12         case '+':
13             resultado = num1 + num2;
14             printf("Resultado: %.2f\n", resultado);
15             break;
16         case '-':
17             resultado = num1 - num2;
18             printf("Resultado: %.2f\n", resultado);
19             break;
20         case '*':
21             resultado = num1 * num2;
22             printf("Resultado: %.2f\n", resultado);
23             break;
24         case '/':
25             if (num2 != 0) {
26                 resultado = num1 / num2;

```

```

27         printf("Resultado: %.2f\n", resultado);
28     } else {
29         printf("Error: No se puede dividir por ↵
        cero.\n");
30     }
31     break;
32     default:
33         printf("Error: Operador no valido.\n");
34 }
35 return 0;
36 }

```

Listing 12: Ejemplo de switch

Iteración: Repetir Tareas (Bucles)

Los bucles nos permiten ejecutar un bloque de código múltiples veces. Son la base para procesar listas de datos, realizar cálculos repetitivos y mucho más.

El bucle while

El bucle **while** ejecuta un bloque de código **mientras** una condición sea verdadera. La condición se evalúa **antes** de cada iteración (incluso antes de la primera). Si la condición es falsa desde el principio, el bloque no se ejecuta nunca.

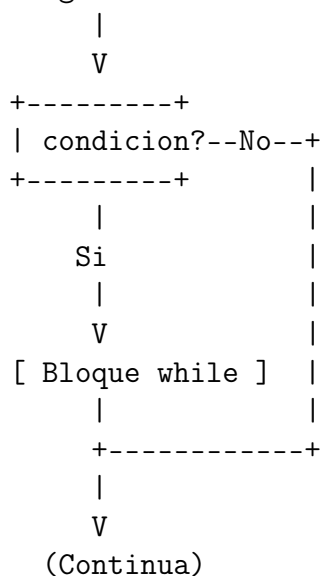
```

1 while (condicion) {
2     // Bloque de código a repetir
3 }

```

Listing 13: Sintaxis de while

[Diagrama de while]



```

1 #include <stdio.h>
2 int main() {
3     int contador = 1;
4     int limite = 5;
5
6     while (contador <= limite) {
7         printf("Iteracion numero: %d\n", contador);
8         contador++; // ES CRUCIAL modificar la variable ←
                      de la condicion para no crear un bucle infinito
9     }
10    printf("Bucle terminado.\n");
11    return 0;
12 }

```

Listing 14: Ejemplo de while

El bucle do-while

El bucle do-while es muy similar al `while`, con una diferencia fundamental: la condición se evalúa **después** de ejecutar el bloque de código. Esto garantiza que el bloque se ejecute **al menos una vez**.

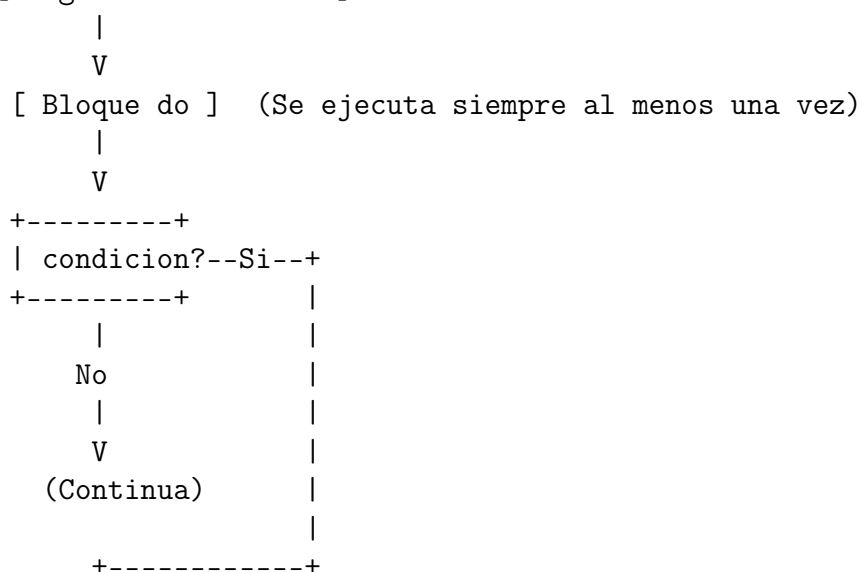
```

1 do {
2     // Bloque de codigo a repetir
3 } while (condicion); // <--- Nota el punto y coma al final

```

Listing 15: Sintaxis de do-while

[Diagrama de do-while]



```

1 #include <stdio.h>
2 int main() {
3     int numero;
4
5     do {
6         printf("Adivina el numero (del 1 al 10): ");
7         scanf("%d", &numero);
8     } while (numero != 7); // Se repite MIENTRAS el ←
        numero NO sea 7
9
10    printf("Correcto! El numero era 7.\n");
11    return 0;
12 }

```

Listing 16: Ejemplo de do-while

En este ejemplo, el programa siempre pide un número al menos una vez, y sigue pidiendo hasta que el usuario acierta.

El bucle for

El bucle for es el más compacto y se usa típicamente cuando sabemos de antemano cuántas iteraciones queremos realizar (por ejemplo, para recorrer un arreglo). Agrupa en una sola línea tres partes: la **inicialización**, la **condición** y la **actualización**.

```

1 for (inicializacion; condicion; actualizacion) {
2     // Bloque de codigo a repetir
3 }

```

Listing 17: Sintaxis de for

El flujo de ejecución de un for es:

1. Se ejecuta la **inicializacion** (solo una vez, al principio).
2. Se evalúa la **condicion**. Si es falsa, se sale del bucle.
3. Si es verdadera, se ejecuta el **bloque de codigo**.
4. Se ejecuta la **actualizacion**.
5. Se vuelve al paso 2.

[Diagrama de for]

```

|
V
[ Inicializacion ]
|
V
+-----+

```



```

| condicion?--No--+
+-----+
|
| Si
|
| V
[ Bloque for ]
|
| V
[ Actualizacion ]
|
+-----+
|
| V
(Continua)

```

```

1 #include <stdio.h>
2 int main() {
3     int i;
4
5     for (i = 1; i <= 5; i++) {
6         printf("Iteracion numero: %d\n", i);
7     }
8
9     printf("Bucle terminado. i vale: %d\n", i); // i vale 6
10    return 0;
11 }

```

Listing 18: Ejemplo de for

Este ejemplo produce exactamente la misma salida que el ejemplo de `while` con contador, pero es mucho más conciso.

Sentencias de Salto

Estas sentencias nos permiten transferir el control del programa a otro punto, "saltando" por encima de la ejecución lineal.

break

Ya lo vimos en `switch`. Dentro de un bucle (`while`, `do-while`, `for`), `break` provoca la **salida inmediata** del bucle más interno que lo contiene. La ejecución continúa en la primera línea después del bucle.

```

1 #include <stdio.h>
2 int main() {
3     int i;

```

```
4     for (i = 1; i <= 10; i++) {  
5         if (i == 5) {  
6             printf("Encontre el 5! Saliendo del bucle.\n");  
7             break; // El bucle termina aqui  
8         }  
9         printf("Valor de i: %d\n", i);  
10    }  
11    printf("Bucle terminado en i=%d\n", i); // i=5  
12    return 0;  
13 }
```

Listing 19: Ejemplo de `break`

Este programa imprimirá los valores del 1 al 4, luego el mensaje de "Encontre el 5", y luego "Bucle terminado en i=5". Las iteraciones del 6 al 10 nunca ocurren.

`continue`

La sentencia `continue` solo puede usarse dentro de bucles. Hace que se **omit** el resto del cuerpo del bucle para la iteración actual y que el programa pase directamente a la siguiente iteración (evaluando la condición y, en el caso del `for`, ejecutando la actualización).

```
1 #include <stdio.h>  
2 int main() {  
3     int i;  
4     for (i = 1; i <= 5; i++) {  
5         if (i == 3) {  
6             printf("Saltando la iteracion %d...\n", i);  
7             continue; // Salta al incremento i++ y luego ←  
8                 a la condicion  
9         }  
10        printf("Valor de i: %d\n", i);  
11    }  
12    return 0;  
13 }
```

Listing 20: Ejemplo de `continue`

Este programa imprimirá:

```
Valor de i: 1  
Valor de i: 2  
Saltando la iteracion 3...  
Valor de i: 4  
Valor de i: 5
```

Observa que cuando `i` vale 3, la línea de `printf` inferior se omite y el bucle continúa con la siguiente iteración.

goto (El Gran Evitado)

La sentencia `goto` permite saltar incondicionalmente a una **etiqueta** en cualquier lugar de la misma función. Es una reliquia de la programación desestructurada y su uso está **muy desaconsejado** porque crea código "espagueti", difícil de leer y mantener. Casi siempre hay una forma mejor y más clara de hacer lo mismo con las estructuras de control vistas. Lo mencionamos solo por completitud.

```

1 #include <stdio.h>
2 int main() {
3     int x = 0;
4
5     inicio: // Esta es una etiqueta
6     printf("x = %d\n", x);
7     x++;
8     if (x < 3) {
9         goto inicio; // Salta de nuevo a la etiqueta ↩
10        'inicio'
11    }
12    printf("Fin.\n");
13    return 0;
14 }

```

Listing 21: Ejemplo de `goto` (No lo hagas en casa)

return

La sentencia `return` termina la ejecución de la función actual y devuelve el control a la función que la llamó (en el caso de `main`, se la devuelve al sistema operativo). Puede devolver un valor (que debe coincidir con el tipo de retorno de la función) o puede usarse sin valor en funciones declaradas como `void`.

```

1 #include <stdio.h>
2
3 int sumar(int a, int b) {
4     int resultado = a + b;
5     return resultado; // Devuelve el valor y termina la ↩
6     funcion 'sumar'
7 }
8
9 void saludar() {
10    printf("Hola, mundo!\n");
11    return; // Opcional en funciones void. Termina la ↩
12    funcion.
13    printf("Esta linea nunca se ejecutara.\n");
14 }
15
16 int main() {
17     int s = sumar(5, 3);

```

```
16     printf("La suma es: %d\n", s);
17     saludar();
18     return 0; // Termina main y el programa
19 }
```

Listing 22: Ejemplo de `return`

Sentencias de Etiquetado

En C, podemos poner una **etiqueta** a una sentencia. Una etiqueta es un identificador seguido de dos puntos (:). Su único uso práctico en C moderno es como destino de un `goto` (que ya vimos que es mala práctica) o dentro de la sentencia `switch` (con las etiquetas `case` y `default`), que ya hemos visto. Por lo tanto, no profundizaremos más en ellas, ya que su aplicación principal ya la hemos cubierto con `switch`.

Resumen de la Unidad II

Hemos cubierto las herramientas que nos permiten escribir programas con una lógica compleja y no meramente lineal.

- **Secuencia:** La base de todo, instrucción tras instrucción.
- **Selección:** Usamos `if`, `else if` y `else` para decisiones basadas en cualquier expresión booleana. Usamos `switch` para comparar una expresión con múltiples constantes enteras, prestando atención al `break` para evitar *fall through* no deseado.
- **Iteración:**
 - `while`: Repite mientras una condición sea verdadera (evalúa al inicio).
 - `do-while`: Repite mientras una condición sea verdadera, pero garantiza al menos una ejecución (evalúa al final).
 - `for`: Ideal para repeticiones con un contador, agrupa inicialización, condición y actualización.
- **Sentencias de Salto:**
 - `break`: Sale inmediatamente de un bucle o `switch`.
 - `continue`: Salta al final del cuerpo del bucle para la iteración actual y pasa a la siguiente.
 - `return`: Termina la función actual y devuelve un valor (si corresponde).
 - `goto`: Salta a una etiqueta. Su uso está desaconsejado.
- **Sentencias de Bloque y Expresión:** Un bloque (`{}`) agrupa sentencias. Una sentencia de expresión es cualquier expresión seguida de `;`.

Con estas herramientas, ya podemos escribir programas que realmente resuelvan problemas: podemos procesar listas de números, validar entradas de usuario, y crear una lógica de decisión compleja. En la siguiente unidad, abordaremos uno de los temas más poderosos y distintivos de C: los **arrays y punteros**, que nos darán un control aún más fino sobre la memoria y los datos.

Unidad III: Arrays y Punteros - El Corazón de C

Bienvenido a la unidad más crucial y, a menudo, la más desafiante del lenguaje C. Los **arrays** (arreglos) y los **punteros** son las herramientas que le dan a C su increíble poder y flexibilidad para manejar la memoria de manera directa. Dominar estos conceptos es lo que separa a un programador de C casual de un verdadero experto.

Imaginemos la memoria RAM como una larga calle con muchas casas. Cada casa tiene una dirección única (su **dirección de memoria**) y dentro de cada casa podemos guardar un dato (el **valor** de una variable). Hasta ahora, trabajábamos con variables individuales, que son como casas aisladas. Los arrays nos permiten tener una **hilera de casas contiguas**, todas del mismo tamaño, y los punteros nos permiten crear **carteles** que contienen la dirección de una casa (o de la primera de una hilera). Con un puntero, podemos "señalar.^a una variable y luego movernos por la memoria.

Arrays (Arreglos)

Un **array** es una colección de elementos del mismo tipo, almacenados en posiciones de memoria **contiguas**. Piensa en él como una fila de casilleros numerados.

Representación de un Array en Memoria

```
int edades[5]; // Un array de 5 enteros
```

Dirección de memoria (ejemplo):

```
+-----+-----+-----+-----+-----+
| edades[0] | edades[1] | edades[2] | edades[3] | edades[4] |
+-----+-----+-----+-----+-----+
0x100      0x104      0x108      0x10C      0x110
```

(Asumiendo que int ocupa 4 bytes en este sistema)

Características clave:

- Todos los elementos son del mismo tipo (definido en la declaración).
- Ocupan bloques de memoria contiguos.
- Cada elemento se accede mediante un **índice**. El primer elemento tiene índice **0**.
- El nombre del array (**edades**, en el ejemplo) es un puntero **constante** a la dirección del primer elemento (**&edades[0]**).

Declaración e Inicialización de Arrays

Declaración

La sintaxis es: `tipo nombre[tamaño];`

```
1 int numeros[10];           // Array de 10 enteros (índices 0 ↵  
    a 9)  
2 float promedios[5];       // Array de 5 floats  
3 char nombre[50];          // Array de 50 caracteres (para ↵  
    un string)
```

Listing 23: Declaración de arrays

Inicialización

Podemos inicializar un array al momento de declararlo de varias formas.

```
1 #include <stdio.h>  
2  
3 int main() {  
4     // Inicializacion completa  
5     int a[5] = {10, 20, 30, 40, 50};  
6     // a[0]=10, a[1]=20, a[2]=30, a[3]=40, a[4]=50  
7  
8     // Inicializacion parcial: los elementos no ↵  
        inicializados se ponen a 0  
9     int b[5] = {1, 2}; // b = {1, 2, 0, 0, 0}  
10  
11    // Inicializacion sin especificar tama o (el ↵  
        compilador lo cuenta)  
12    int c[] = {5, 10, 15}; // c tendra tama o 3  
13  
14    // Inicializacion con designadores (C99 en adelante)  
15    int d[10] = { [2]=100, [5]=200 };  
16    // d = {0, 0, 100, 0, 0, 200, 0, 0, 0, 0}  
17  
18    // Inicializacion a cero  
19    int e[10] = {0}; // Todos los elementos a 0  
20  
21    return 0;  
22 }
```

Listing 24: Inicialización de arrays

Acceso a los elementos

Se usa el nombre del array y el índice entre corchetes `[]`.

```

1 #include <stdio.h>
2 int main() {
3     int notas[4] = {85, 90, 78, 95};
4
5     printf("La primera nota es: %d\n", notas[0]); // ↩
6     notas[0] = 85
7     printf("La tercera nota es: %d\n", notas[2]); // ↩
8     notas[2] = 78
9
10    // Modificar un elemento
11    notas[1] = 92;
12    printf("La segunda nota ahora es: %d\n", notas[1]);
13
14    // Recorrer un array con un bucle
15    int suma = 0;
16    for (int i = 0; i < 4; i++) {
17        suma += notas[i];
18    }
19    printf("La suma de las notas es: %d\n", suma);
20    return 0;
21 }

```

Listing 25: Acceso a elementos de un array

Arrays Multidimensionales

Podemos tener arrays de más de una dimensión. Un array bidimensional es como una tabla con filas y columnas, o una matriz. En memoria, se almacenan por filas (fila 0 completa, luego fila 1, etc.).

Representación de un Array Bidimensional

```
int matriz[3][4]; // 3 filas, 4 columnas
```

Visualmente:

	Col 0	Col 1	Col 2	Col 3
Fila 0:	[0,0]	[0,1]	[0,2]	[0,3]
Fila 1:	[1,0]	[1,1]	[1,2]	[1,3]
Fila 2:	[2,0]	[2,1]	[2,2]	[2,3]

En memoria (contiguo):

```
[0,0] [0,1] [0,2] [0,3] [1,0] [1,1] [1,2] [1,3] [2,0] [2,1] [2,2] [2,3]
```

Declaración e Inicialización


```

1 #include <stdio.h>
2 int main() {
3     // Declaracion e inicializacion de una matriz 2x3
4     int matriz[2][3] = {
5         {1, 2, 3},    // Fila 0
6         {4, 5, 6}     // Fila 1
7     };
8
9     // Otra forma de inicializar (menos legible)
10    int matriz2[2][3] = {1, 2, 3, 4, 5, 6};
11
12    // Inicializacion parcial (los no inicializados a 0)
13    int matriz3[2][3] = { {1}, {4, 5} };
14    // Resultado: {{1,0,0}, {4,5,0}}
15
16    // Acceso
17    printf("Elemento en fila 1, columna 2: %d\n", ←
18           matriz[1][2]); // 6
19
20    // Recorrido con bucles anidados
21    for (int i = 0; i < 2; i++) {           // i = fila
22        for (int j = 0; j < 3; j++) {       // j = columna
23            printf("%d ", matriz[i][j]);
24        }
25        printf("\n"); // Nueva linea al final de cada fila
26    }
27    return 0;
28 }

```

Listing 26: Arrays multidimensionales

Introducción a los Punteros

Un **puntero** es una variable que **almacena una dirección de memoria**. Es decir, en lugar de guardar un valor como 25, guarda la dirección donde está guardado el 25.

Comparación: Variable Normal vs. Puntero

Variable normal 'edad':

```

+-----+
|  25   |
+-----+
0x7FFF  (direccion)

```

Puntero 'ptr':

```

+-----+
| 0x7FFF | (almacena una direccion)
+-----+
0x7FF0   (su propia direccion)
|
| apunta a
V
+-----+

```

```

|   25   |
+-----+
0x7FFF   (direccion de edad)

```

Declaración de Punteros

Se declara usando el símbolo `*` antes del nombre de la variable. El tipo del puntero indica a qué tipo de dato apunta.

```

1 int *ptr_int;      // Puntero a un entero
2 float *ptr_float; // Puntero a un float
3 char *ptr_char;    // Puntero a un caracter (muy usado ↵
    para strings)
4 void *ptr_void;    // Puntero generico (puede apuntar a ↵
    cualquier tipo)

```

Listing 27: Declaración de punteros

Operadores de Punteros

Hay dos operadores fundamentales para trabajar con punteros:

- **& (operador de dirección):** Devuelve la dirección de memoria de una variable.
- *** (operador de indirección o desreferencia):** Dado un puntero, accede al valor almacenado en la dirección a la que apunta.

```

1 #include <stdio.h>
2 int main() {
3     int edad = 25;      // Variable normal
4     int *ptr;           // Declaramos un puntero a entero
5
6     ptr = &edad;        // ptr ahora contiene la ↵
        direccion de edad
7
8     printf("Valor de edad: %d\n", edad);      // 25
9     printf("Direccion de edad: %p\n", &edad); // ↵
        0x7fff...
10    printf("Valor de ptr (direccion que guarda): %p\n", ↵
        ptr); // misma direccion
11    printf("Valor al que apunta ptr: %d\n", *ptr); // 25 ↵
        (desreferencia)
12
13    // Modificar edad a traves del puntero
14    *ptr = 30;           // Cambia el valor en la ↵
        direccion apuntada por ptr
15
16    printf("Nuevo valor de edad: %d\n", edad); // 30

```

```

17     return 0;
18 }

```

Listing 28: Uso de & y *

Asignación de Punteros

Podemos asignar a un puntero la dirección de otra variable, o el valor de otro puntero del mismo tipo.

```

1 int x = 10, y = 20;
2 int *p1, *p2;
3
4 p1 = &x;    // p1 apunta a x
5 p2 = p1;    // p2 ahora apunta a donde apunta p1, es ↔
               decir, a x
6 p1 = &y;    // p1 ahora apunta a y, p2 sigue apuntando a x
7
8 // CUIDADO: No confundir * en declaracion con * en ↔
               asignacion
9 *p2 = 30;   // Cambia x a 30 (desreferencia p2)

```

Listing 29: Asignación de punteros

Aritmética de Punteros

Esta es una de las características más potentes de C. Podemos realizar operaciones aritméticas con punteros: suma, resta, incremento, decremento. **No es aritmética normal de enteros.** Cuando sumamos 1 a un puntero, este avanza al siguiente elemento del tipo al que apunta, no al siguiente byte.

```

1 #include <stdio.h>
2 int main() {
3     int arr[] = {10, 20, 30, 40, 50};
4     int *p = arr; // p apunta al primer elemento (arr[0])
5
6     printf("p apunta a: %d\n", *p); // 10
7
8     p++; // Ahora p apunta al siguiente entero (arr[1])
9     printf("p++ apunta a: %d\n", *p); // 20
10
11    p = p + 2; // Ahora p apunta 2 posiciones mas alla ↔
               (arr[3])
12    printf("p+2 apunta a: %d\n", *p); // 40
13
14    // Podemos recorrer el array con un puntero
15    printf("\nRecorriendo el array con punteros:\n");

```

```

16     for (p = arr; p < arr + 5; p++) {
17         printf("%d ", *p);
18     }
19     printf("\n");
20     return 0;
21 }

```

Listing 30: Aritmética de punteros

Visualización de la aritmética de punteros

```
int arr[5] = {10, 20, 30, 40, 50};
```

Memoria (cada bloque son 4 bytes asumiendo int de 32 bits):

```

arr:  [10] [20] [30] [40] [50]
dir:  100  104  108  112  116

```

```

int *p = arr;    // p = 100
p++;            // p = 104 (NO 101)
p = p + 2;      // p = 112 (104 + 2*4)

```

La regla general es: `ptr + n` avanza `n * sizeof(tipo_al_que_apunta)` bytes.

Arrays y Punteros: Una Relación Íntima

En C, el nombre de un array es un **puntero constante al primer elemento**. Es decir, no podemos cambiar a dónde apunta (no podemos hacer `arr++`), pero podemos usar aritmética de punteros para acceder a sus elementos.

```

1  #include <stdio.h>
2  int main() {
3      int arr[5] = {10, 20, 30, 40, 50};
4
5      // Son equivalentes:
6      printf("%d\n", arr[2]);      // Notacion de indice
7      printf("%d\n", *(arr + 2)); // Notacion de punteros
8
9      // Para recorrer el array:
10     int *p;
11     for (p = arr; p < arr + 5; p++) {
12         printf("%d ", *p);
13     }
14     printf("\n");
15
16     // arr++ es ILEGAL (arr es constante)
17     // arr = p; // ILEGAL
18     return 0;
19 }

```

Listing 31: Relación array-puntero

Punteros a Punteros

Un puntero puede apuntar a otro puntero. Es decir, una variable que almacena la dirección de otra variable que también es un puntero. Se usa mucho en matrices de strings y para modificar punteros desde funciones.

Visualización de puntero a puntero

```
int x = 10;
int *p = &x;    // p apunta a x
int **pp = &p;  // pp apunta a p
```

```
pp ---> p ---> x
(0x200) (0x100) (0x50)
```

```
1 #include <stdio.h>
2 int main() {
3     int x = 10;
4     int *p = &x;
5     int **pp = &p;
6
7     printf("x = %d\n", x);
8     printf("*p = %d\n", *p);
9     printf("**pp = %d\n", **pp); // Accedemos al valor de x
10
11     **pp = 20; // Modificamos x a traves de pp
12     printf("x ahora es: %d\n", x);
13     return 0;
14 }
```

Listing 32: Puntero a puntero

Arrays de Punteros

Podemos tener arrays cuyos elementos son punteros. Es muy común para manejar listas de strings.

```
1 #include <stdio.h>
2 int main() {
3     // Array de 3 punteros a char (strings)
4     char *nombres[] = {"Ana", "Carlos", "Maria"};
5 }
```

```
6 // nombres[0] es un puntero al string "Ana"
7 // nombres[1] es un puntero al string "Carlos", etc.
8
9 for (int i = 0; i < 3; i++) {
10     printf("Nombre %d: %s\n", i, nombres[i]);
11 }
12
13 // Tambien podemos tener arrays de punteros a enteros
14 int a = 1, b = 2, c = 3;
15 int *arr_ptr[] = {&a, &b, &c};
16
17 printf("Valor apuntado por arr_ptr[1]: %d\n", ←
18     *arr_ptr[1]); // 2
19 return 0;
20 }
```

Listing 33: Array de punteros

Punteros a Funciones

Sí, ¡las funciones también tienen dirección! Un puntero a función almacena la dirección de una función, permitiendo pasarla como argumento a otra función o almacenarla en estructuras. Es avanzado pero muy útil.

```
1 #include <stdio.h>
2
3 // Dos funciones con la misma firma (tipo de retorno y ←
4 // parametros)
5 int sumar(int a, int b) { return a + b; }
6 int restar(int a, int b) { return a - b; }
7
8 int main() {
9     // Declaracion de un puntero a funcion que recibe dos ←
10     // ints y devuelve un int
11     int (*operacion)(int, int);
12
13     operacion = &sumar; // o simplemente = sumar (el & es ←
14     // opcional)
15     printf("Suma: %d\n", operacion(10, 5));
16
17     operacion = restar;
18     printf("Resta: %d\n", operacion(10, 5));
19
20     // Ejemplo mas util: pasar una funcion como parametro
21     // (Lo veremos en detalle mas adelante)
22     return 0;
23 }
```

Listing 34: Punteros a funciones

Argumentos en la Línea de Comandos

¿Has notado que programas como `ls -l` reciben argumentos? En C, la función `main` puede recibir parámetros para procesar estos argumentos.

```
1 #include <stdio.h>
2
3 // argc: numero de argumentos (argument count)
4 // argv: array de strings con los argumentos (argument ↵
   vector)
5 int main(int argc, char *argv[]) {
6     printf("Numero de argumentos: %d\n", argc);
7     printf("Nombre del programa: %s\n", argv[0]); // ↵
       argv[0] es el nombre del ejecutable
8
9     for (int i = 1; i < argc; i++) {
10         printf("Argumento %d: %s\n", i, argv[i]);
11     }
12     return 0;
13 }
```

Listing 35: Argumentos de línea de comandos

Si compilamos este programa como `programa` y lo ejecutamos como:

```
./programa hola mundo 123
```

La salida será:

```
Numero de argumentos: 4
Nombre del programa: ./programa
Argumento 1: hola
Argumento 2: mundo
Argumento 3: 123
```

Funciones para Manejo de Memoria Dinámica

Hasta ahora, toda la memoria que usábamos era **estática** o **automática** (en la pila). Sabíamos de antemano el tamaño de los arrays. Pero, ¿y si no sabemos cuántos datos va a ingresar el usuario hasta que el programa se está ejecutando? Necesitamos **memoria dinámica**: memoria que podemos solicitar y liberar en tiempo de ejecución desde el **montón** (heap).

Las funciones para esto están en `<stdlib.h>`.

malloc()

`malloc` (memory allocation) reserva un bloque de memoria del tamaño especificado (en bytes) y devuelve un puntero `void*` al inicio de ese bloque. Si no hay suficiente memoria, devuelve `NULL`.

```
1 #include <stdio.h>
2 #include <stdlib.h> // Para malloc, free, etc.
3
4 int main() {
5     int n;
6     printf("Cuantos numeros quieres guardar? ");
7     scanf("%d", &n);
8
9     // Reservamos memoria para n enteros
10    int *arr = (int*) malloc(n * sizeof(int));
11
12    // SIEMPRE verificar que malloc no haya fallado
13    if (arr == NULL) {
14        printf("Error: No se pudo reservar memoria.\n");
15        return 1; // Salida con error
16    }
17
18    // Usamos el array como si fuera normal
19    for (int i = 0; i < n; i++) {
20        arr[i] = i * 10;
21    }
22
23    for (int i = 0; i < n; i++) {
24        printf("arr[%d] = %d\n", i, arr[i]);
25    }
26
27    // IMPORTANTE: Liberar la memoria cuando ya no la ↔
28    // necesitamos
29    free(arr);
30
31    return 0;
32 }
```

Listing 36: Uso de `malloc`

realloc()

`realloc` (reallocation) cambia el tamaño de un bloque de memoria previamente asignado con `malloc`, `calloc` o `realloc`. Puede agrandarlo o reducirlo. Si es posible, intenta mantener los datos existentes.

```
1 #include <stdio.h>
2 #include <stdlib.h>
```



```

3
4 int main() {
5     // Empezamos con espacio para 5 enteros
6     int *arr = (int*) malloc(5 * sizeof(int));
7     if (arr == NULL) { /* manejar error */ }
8
9     for (int i = 0; i < 5; i++) {
10         arr[i] = i;
11     }
12
13     printf("Array original:\n");
14     for (int i = 0; i < 5; i++) {
15         printf("%d ", arr[i]);
16     }
17     printf("\n");
18
19     // Ahora necesitamos espacio para 10 enteros
20     int *temp = (int*) realloc(arr, 10 * sizeof(int));
21
22     if (temp == NULL) {
23         // realloc fallo! arr sigue siendo valido, pero ↵
24         // no tenemos mas espacio
25         printf("Error al redimensionar.\n");
26         free(arr); // Liberamos la memoria original
27         return 1;
28     } else {
29         arr = temp; // Asignamos el nuevo bloque (puede ↵
30         // ser la misma direccion u otra)
31
32         // Inicializamos los nuevos elementos
33         for (int i = 5; i < 10; i++) {
34             arr[i] = i * 10;
35         }
36
37         printf("Array redimensionado:\n");
38         for (int i = 0; i < 10; i++) {
39             printf("%d ", arr[i]);
40         }
41         printf("\n");
42
43         free(arr); // Liberamos la memoria
44         return 0;
45     }
46 }

```

Listing 37: Uso de realloc

Otras funciones relacionadas:

- `calloc()`: Similar a `malloc`, pero inicializa todos los bytes a 0. Recibe el número

de elementos y el tamaño de cada uno.

- **free()**: Libera la memoria asignada dinámicamente. Es crucial para evitar **fugas de memoria** (memory leaks).

Resumen de la Unidad III

Hemos cubierto el tema más fundamental de C:

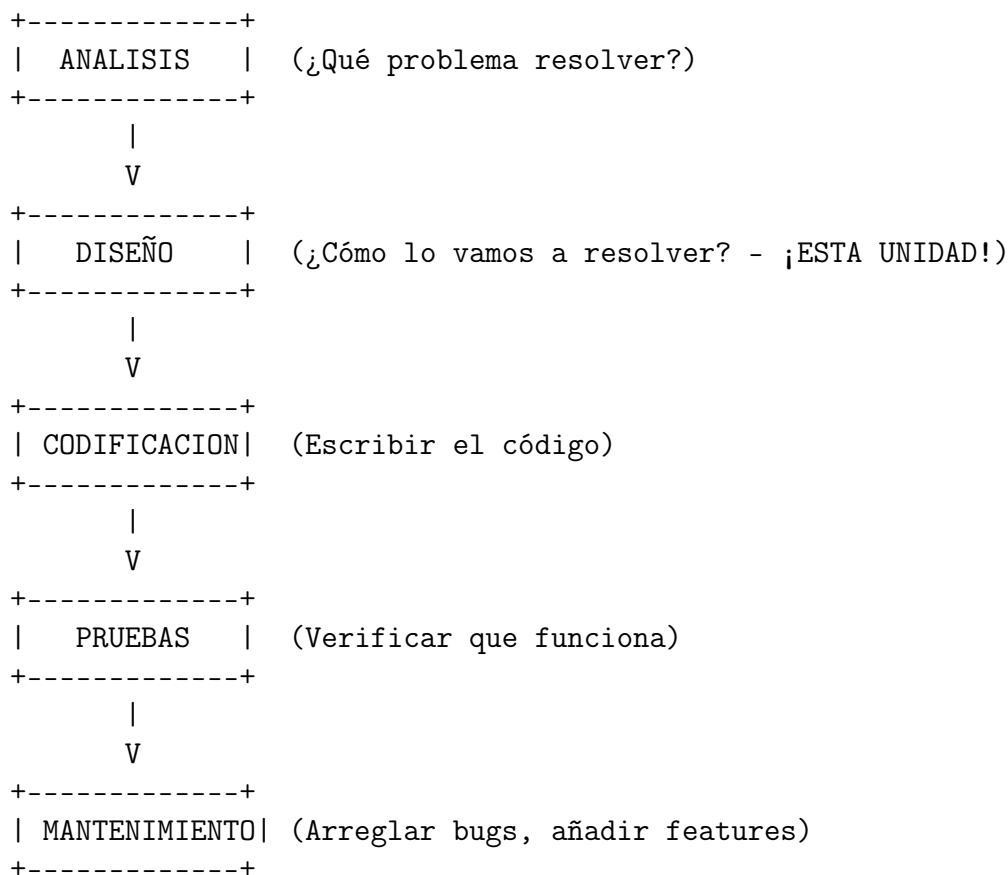
- **Arrays**: Colecciones de elementos del mismo tipo en memoria contigua. Se acceden por índice.
- **Arrays multidimensionales**: Arrays de arrays, como matrices.
- **Punteros**: Variables que guardan direcciones de memoria. Operadores **&** (dirección) y ***** (desreferencia).
- **Aritmética de punteros**: Sumar/restar a un puntero avanza según el tamaño del tipo al que apunta.
- **Relación arrays-punteros**: El nombre de un array es un puntero constante al primer elemento. `arr[i]` es equivalente a `*(arr + i)`.
- **Punteros a punteros**: Útiles para matrices de strings y modificar punteros en funciones.
- **Arrays de punteros**: Muy usado para listas de strings.
- **Punteros a funciones**: Permiten pasar funciones como argumentos.
- **Argumentos de línea de comandos**: `argc` y `argv` en `main`.
- **Memoria dinámica**: `malloc`, `realloc`, `free` para gestionar memoria en tiempo de ejecución.

Esta unidad es la base para todo lo que sigue. Tómate tu tiempo para asimilarla, experimenta con los ejemplos y, sobre todo, **¡ practica mucho!** La comprensión de punteros es la clave del dominio de C.

Unidad IV: Principios de Diseño de Algoritmos y Programas - Escribiendo Código de Calidad

Hasta ahora, nos hemos enfocado en aprender la sintaxis de C y en resolver problemas específicos. Sin embargo, en el mundo real, los programas no son solo ejercicios aislados; son sistemas complejos que evolucionan con el tiempo, son mantenidos por diferentes personas y deben ser confiables. Esta unidad trata sobre las **buenas prácticas** y los **principios fundamentales** que transforman un código que "funciona" en un código **de calidad**, que es fácil de entender, modificar y mantener.

El Ciclo de Vida del Software (Visión Simplificada)



Principios Fundamentales de Diseño

Diseñar un programa no es sentarse y empezar a teclear código. Es un proceso de reflexión y planificación. Los siguientes principios nos guían en este proceso.

Denominaciones Significativas (Naming)

El nombre que le damos a las variables, funciones, estructuras, etc., es la primera forma de documentación. Un buen nombre explica por sí mismo el propósito del elemento.

```

1 #include <stdio.h>
2
3 // MAL: Nombres cr p ticos y sin significado
4 int f(int a, int b) {
5     int c = a * b;
6     int d = c / 2;
7     return d;
8 }
9
10 // BIEN: Nombres autoexplicativos
11 int calcular_media(int suma_total, int ←
    numero_de_elementos) {
12     int media = suma_total / numero_de_elementos;
13     return media;
14 }
15
16 int main() {
17     int n1 = 10; // MAL: Qu es n1?
18     int edad_del_usuario = 25; // BIEN: Queda claro que es
19
20     // MAL: nombres de una letra en contexto amplio
21     int a = 5;
22     int b = 3;
23     int r = f(a, b);
24
25     // BIEN: Contexto claro
26     int notas[] = {85, 90, 78};
27     int num_notas = 3;
28     int suma_notas = 85 + 90 + 78;
29     int promedio = calcular_media(suma_notas, num_notas);
30
31     return 0;
32 }
```

Listing 38: Malas y buenas denominaciones

Consejos:

- Usa nombres en inglés o español, pero sé consistente.

- Para variables, usa sustantivos: `contador`, `nombre_completo`.
- Para funciones, usa verbos: `calcular_promedio()`, `imprimir_factura()`.
- Evita abreviaturas crípticas (`tmp` está bien para un temporal de corta vida, pero `sz` no está claro).

Documentación y Formato (Estilo de Código)

El código se lee muchas más veces de las que se escribe. Un código bien formateado y comentado es más fácil de leer y mantener.

Comentarios

Los comentarios deben explicar el **porqué**, no el **cómo**. El **cómo** debería ser evidente por el código mismo si está bien escrito.

```
1 #include <stdio.h>
2
3 // MAL: Comentario redundante
4 int i = 0; // Inicializo i a 0
5
6 // BIEN: Explica una decision o logica compleja
7 int calcular_interes(double saldo, int dias) {
8     // Usamos la regla de 365/360 segun la normativa ↵
9     bancaria XYZ
10    const int DIAS_BASE = 360;
11    double interes = saldo * 0.05 * dias / DIAS_BASE;
12    return interes;
13 }
14
15 /*
16  * Funcion: procesar_archivo
17  * Proposito: Lee un archivo de configuraci n y carga ↵
18  * las opciones.
19  * Parametros:
20  *     nombre_archivo: Ruta al archivo a procesar.
21  * Retorna: 0 si OK, -1 si error.
22  */
23 int procesar_archivo(char *nombre_archivo) {
24     // ... codigo ...
25 }
```

Listing 39: Buenos y malos comentarios

Formato (Indentación y Espacios)

La indentación consistente (usando espacios o tabs, pero no mezclar) es crucial para mostrar la estructura del programa.

```

1 // DIFICIL DE LEER
2 #include <stdio.h>
3 int main(){
4     int x=5;
5     if(x>0){
6         printf("Positivo");
7         for(int i=0;i<3;i++){
8             printf("%d",i);
9         }
10    }
11    return 0;}
12
13 // FACIL DE LEER
14 #include <stdio.h>
15 int main() {
16     int x = 5;
17     if (x > 0) {
18         printf("Positivo");
19         for (int i = 0; i < 3; i++) {
20             printf("%d", i);
21         }
22     }
23     return 0;
24 }

```

Listing 40: Mal y buen formato

Refinamiento (Top-Down Design)

El refinamiento sucesivo, también conocido como diseño de arriba hacia abajo (*top-down design*), es una técnica donde empezamos con una descripción de alto nivel del problema y la vamos desglosando en pasos cada vez más detallados hasta llegar a instrucciones que podamos codificar directamente.

Ejemplo: Calcular el promedio de notas de un estudiante

Nivel 0 (Problema):

Calcular el promedio de notas.

Nivel 1 (Subproblemas):

1. Obtener las notas del usuario.
2. Calcular la suma de las notas.
3. Calcular el promedio (suma / cantidad).
4. Mostrar el resultado.

Nivel 2 (Detalle de "Obtener notas"):

- 1.1. Preguntar cuantas notas va a ingresar.

- 1.2. Para cada nota:
 - 1.2.1. Pedir la nota.
 - 1.2.2. Almacenarla.

Nivel 3 (Detalle de "Calcular suma"):

- 2.1. Inicializar suma = 0.
- 2.2. Para cada nota almacenada:
 - 2.2.1. Sumarla al acumulador.

... y así sucesivamente.

Este proceso nos lleva directamente a una estructura de programa modular y clara.

Codificación y Pruebas

Una vez diseñado, pasamos a codificar. Pero la codificación y las pruebas van de la mano.

Codificación

Es la traducción de nuestro diseño a un lenguaje de programación (C). Seguir las convenciones de formato y denominación es parte de esta fase.

Pruebas (Testing)

El objetivo es encontrar errores (*bugs*). Nunca asumas que el programa funciona porque compiló porque "probé un caso".

- **Pruebas Unitarias:** Verificar que cada función o módulo por separado funciona correctamente.
- **Pruebas de Integración:** Verificar que los módulos funcionan bien juntos.
- **Pruebas de Regresión:** Al hacer un cambio, verificar que no se rompió nada que ya funcionaba.

Estrategias de prueba

- **Camino Feliz (Happy Path):** Probar con entradas típicas y esperadas.
- **Casos Límite (Edge Cases):** Probar con valores extremos (0, números muy grandes, listas vacías, etc.).
- **Casos de Error:** Probar con entradas inválidas (división por cero, punteros NULL, etc.).


```

1 #include <stdio.h>
2 #include <assert.h> // Para pruebas simples con assert
3
4 // Funcion que queremos probar
5 int dividir(int a, int b, float *resultado) {
6     if (b == 0) {
7         return -1; // Indicar error
8     }
9     *resultado = (float)a / b;
10    return 0; // Exito
11 }
12
13 // Pruebas
14 void test_dividir() {
15     float res;
16
17     // Happy path
18     assert(dividir(10, 2, &res) == 0 && res == 5.0f);
19
20     // Edge case: numeros negativos
21     assert(dividir(-10, 2, &res) == 0 && res == -5.0f);
22
23     // Edge case: division que da decimal
24     assert(dividir(5, 2, &res) == 0 && res == 2.5f);
25
26     // Caso de error: division por cero
27     assert(dividir(10, 0, &res) == -1);
28
29     printf("Todas las pruebas pasaron!\n");
30 }
31
32 int main() {
33     test_dividir();
34     return 0;
35 }

```

Listing 41: Ejemplo de función a probar y casos de prueba

Modularización: Divide y Vencerás

La **modularización** es el proceso de dividir un programa grande en partes más pequeñas y manejables llamadas **módulos** (en C, generalmente funciones y archivos separados). Cada módulo tiene una responsabilidad bien definida.

Criterios y Principios de Modularización

- **Alta Cohesión:** Un módulo debe realizar una única tarea bien definida. Todas sus partes deben estar estrechamente relacionadas. Una función que calcula promedios y imprime facturas tiene baja cohesión.
- **Bajo Acoplamiento:** Los módulos deben depender lo menos posible unos de otros. Deben comunicarse a través de interfaces pequeñas y bien definidas (parámetros y valores de retorno). Cambiar un módulo no debería forzar cambios masivos en otros.

Visualización de Cohesión y Acoplamiento

BAJA COHESION (Mala):

```
+-----+
| Modulo "Utilidades" |
| - leer_archivo()    |
| - calcular_media()  |
| - imprimir()        |
| - enviar_email()    |
+-----+
```

ALTA COHESION (Buena):

```
+-----+
| Modulo "Calculos"   |
| - calcular_media()  |
| - calcular_suma()    |
+-----+
```

ALTO ACOPLAMIENTO (Mala):

```
+-----+ +-----+
| Mod A |----->| Mod B |
| (usa  |         | (usa  |
| datos |<-----| datos |
| internos |      | internos |
| de B) |         | de A) |
+-----+ +-----+
```

```
^
|
+-----+
(Dependencia circular)
```

BAJO ACOPLAMIENTO (Buena):

```
+-----+ +-----+
| Mod A |----->| Mod B |
| (llama |         | (recibe|
| f()    |----->| args, |
| con    |         | devuelve
| args) |         | valor)|
+-----+ +-----+
```

```
|
|
+-----+
(Comunicacion limpia)
```

```
1 // archivo: operaciones.h (Interfaz)
2 #ifndef OPERACIONES_H
3 #define OPERACIONES_H
4
5 float calcular_media(int suma, int cantidad);
6 int calcular_suma(int *datos, int cantidad);
7
8 #endif
9
10 // archivo: operaciones.c (Implementacion)
11 #include "operaciones.h"
12
13 float calcular_media(int suma, int cantidad) {
14     if (cantidad == 0) return 0.0f;
```

```

15     return (float)suma / cantidad;
16 }
17
18 int calcular_suma(int *datos, int cantidad) {
19     int suma = 0;
20     for (int i = 0; i < cantidad; i++) {
21         suma += datos[i];
22     }
23     return suma;
24 }
25
26 // archivo: main.c (Uso de los modulos)
27 #include <stdio.h>
28 #include "operaciones.h"
29
30 int main() {
31     int notas[] = {85, 90, 78, 95};
32     int num_notas = 4;
33     int suma = calcular_suma(notas, num_notas);
34     float media = calcular_media(suma, num_notas);
35     printf("La media es: %.2f\n", media);
36     return 0;
37 }

```

Listing 42: Ejemplo de modularización

Reusabilidad

La **reusabilidad** es la capacidad de usar un módulo o componente en diferentes programas o en diferentes partes del mismo programa sin modificarlo. Los beneficios son enormes: ahorro de tiempo, menos errores (porque el código ya ha sido probado) y mayor consistencia.

¿Cómo logramos reusabilidad?

- Escribiendo funciones genéricas que no dependan de datos globales.
- Creando bibliotecas (librerías) de funciones útiles (como hicimos con `operaciones.h`).
- Documentando claramente cómo usar el módulo (su interfaz).

Abstracción de Datos

La **abstracción de datos** es el concepto de ocultar los detalles de implementación de una estructura de datos y exponer solo las operaciones que se pueden realizar sobre ella. El usuario del tipo de dato no necesita saber cómo está implementado internamente, solo qué operaciones puede hacer.

En C, esto se logra típicamente con:

- Tipos de datos definidos por el usuario (struct).
- Archivos de cabecera (.h) que declaran las operaciones (la interfaz pública).
- Archivos de implementación (.c) que contienen el código y los detalles ocultos.
- Uso de static para hacer privadas las funciones internas.

```
1 // punto.h - INTERFAZ PUBLICA
2 #ifndef PUNTO_H
3 #define PUNTO_H
4
5 // Declaracion adelantada (opaca) del tipo Punto
6 // El usuario solo sabe que existe, no su estructura interna
7 typedef struct Punto Punto;
8
9 // Operaciones publicas
10 Punto* punto_crear(float x, float y);
11 void punto_destruir(Punto *p);
12 void punto_mover(Punto *p, float dx, float dy);
13 float punto_distancia_origen(Punto *p);
14 void punto_imprimir(Punto *p);
15
16 #endif
17
18 // punto.c - IMPLEMENTACION PRIVADA
19 #include "punto.h"
20 #include <stdio.h>
21 #include <stdlib.h>
22 #include <math.h>
23
24 // Aqui SI definimos la estructura (oculta al usuario)
25 struct Punto {
26     float x;
27     float y;
28 };
29
30 Punto* punto_crear(float x, float y) {
31     Punto *nuevo = (Punto*) malloc(sizeof(Punto));
32     if (nuevo != NULL) {
33         nuevo->x = x;
34         nuevo->y = y;
35     }
36     return nuevo;
37 }
38
39 void punto_destruir(Punto *p) {
40     free(p);
41 }
```

```

42
43 void punto_mover(Punto *p, float dx, float dy) {
44     if (p != NULL) {
45         p->x += dx;
46         p->y += dy;
47     }
48 }
49
50 float punto_distancia_origen(Punto *p) {
51     return sqrt(p->x * p->x + p->y * p->y);
52 }
53
54 void punto_imprimir(Punto *p) {
55     printf("Punto(%.2f, %.2f)", p->x, p->y);
56 }
57
58 // main.c - USO
59 #include <stdio.h>
60 #include "punto.h"
61
62 int main() {
63     Punto *mi_punto = punto_crear(3.0, 4.0);
64     punto_imprimir(mi_punto);
65     printf(" -> Distancia al origen: %.2f\n", ←
        punto_distancia_origen(mi_punto));
66
67     punto_mover(mi_punto, -1.0, 2.0);
68     printf("Despues de mover: ");
69     punto_imprimir(mi_punto);
70     printf("\n");
71
72     punto_destruir(mi_punto);
73     return 0;
74 }

```

Listing 43: Ejemplo de abstracción de datos: Un Punto en 2D

En este ejemplo, el usuario de `Punto` no sabe (y no le importa) si las coordenadas son `float` o `double`, o si la distancia se calcula con `sqrt` o de otra forma. Solo conoce las funciones que puede usar. Esto es abstracción de datos.

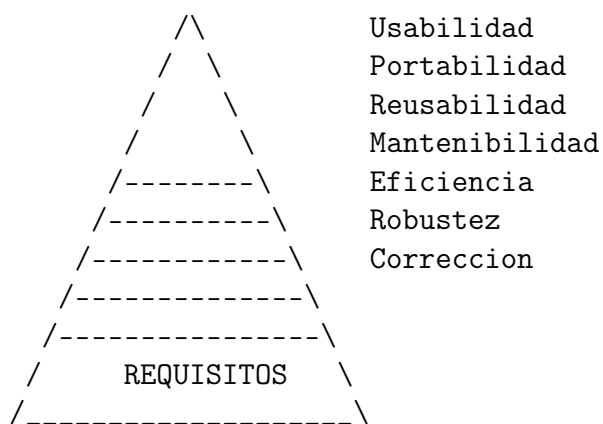
Aspectos Acerca de la Calidad del Software

¿Qué hace que un software sea de calidad? No es solo que funcione. Hay varios atributos clave:

- **Corrección:** El software hace lo que debe hacer (satisface los requisitos).
- **Robustez:** El software se comporta de manera predecible ante situaciones inesperadas (errores de entrada, fallos de hardware, etc.). No *“explota”*.

- **Eficiencia:** El software usa los recursos (tiempo de CPU, memoria) de manera óptima.
- **Mantenibilidad:** Es fácil de modificar para corregir errores o añadir nuevas funcionalidades. Esto es una consecuencia directa de la modularización, la documentación y las buenas prácticas de código.
- **Portabilidad:** El software puede ejecutarse en diferentes plataformas (sistemas operativos, hardware) con pocos o ningún cambio.
- **Usabilidad:** Es fácil de usar para el usuario final.
- **Reusabilidad:** Ya lo vimos, sus componentes pueden usarse en otros proyectos.

La Pirámide de la Calidad del Software



La base es la corrección (hacer lo que se pide). Sobre ella se construye la robustez (hacerlo de forma fiable), y así sucesivamente.

Resumen de la Unidad IV

Esta unidad nos alejó de la sintaxis pura de C y nos introdujo en el arte y la ingeniería de la programación.

- **Denominaciones:** Usa nombres claros y descriptivos. El código se lee más de lo que se escribe.
- **Documentación y formato:** Comenta el *porqué*, no el *cómo*. Usa un estilo de código consistente.
- **Refinamiento:** Diseña de arriba hacia abajo, descomponiendo problemas grandes en subproblemas más pequeños.
- **Codificación y pruebas:** Prueba tu código sistemáticamente (casos felices, límites y de error).
- **Modularización:** Divide el programa en módulos con alta cohesión (una sola responsabilidad) y bajo acoplamiento (interfaces simples).

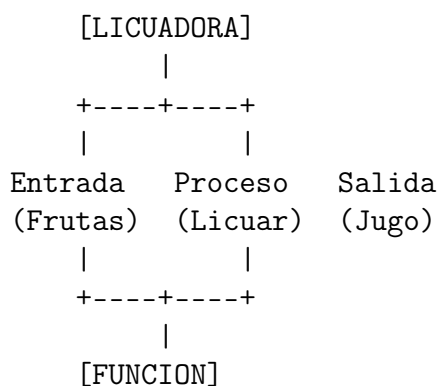
- **Reusabilidad:** Escribe código pensando en que podría ser usado de nuevo.
- **Abstracción de datos:** Oculta los detalles de implementación y expone solo una interfaz limpia. Es la base de la programación orientada a objetos (que veremos más adelante).
- **Calidad del software:** No basta con que funcione. Buscamos corrección, robustez, eficiencia, mantenibilidad, etc.

Estos principios son universales y te servirán sin importar el lenguaje o el tipo de proyecto en el que trabajes. Aplicarlos desde el principio es la marca de un buen programador.

Unidad V: Funciones - Los Bloques de Construcción de Programas

Las funciones son el corazón de la programación estructurada en C. Desde el primer día hemos usado `main()` y `printf()`, pero ahora es momento de entender a fondo cómo crearlas, usarlas y dominarlas. Una función es, esencialmente, un bloque de código reutilizable que realiza una tarea específica.

Analogía: Una función es como un electrodoméstico



Así como una licuadora recibe frutas (entrada), las procesa y produce jugo (salida), una función recibe argumentos, realiza operaciones y devuelve un resultado.

Concepto de Función

Una **función** es un bloque de código con un nombre, que puede recibir parámetros y puede devolver un valor. Nos permite:

- **Modularizar:** Dividir problemas complejos en partes más pequeñas.
- **Reutilizar:** Escribir código una vez y usarlo muchas veces.
- **Abstraer:** Ocultar detalles complejos y usar la función como una "caja negra".

Sintaxis Básica de una Función


```

1 tipo_de_retorno nombre_de_la_funcion(tipo_param1 ←
    nombre_param1, tipo_param2 nombre_param2) {
2     // Cuerpo de la funcion
3     // ... instrucciones ...
4     return valor_del_tipo_de_retorno;
5 }

```

Listing 44: Estructura de una función en C

- **tipo_de_retorno:** El tipo de dato que la función devuelve (int, float, char, etc.). Si no devuelve nada, usamos void.
- **nombre_de_la_funcion:** Identificador con las mismas reglas que las variables.
- **parámetros:** Lista de variables que recibe la función (puede estar vacía: void).
- **cuerpo:** Bloque de código entre llaves {} que implementa la lógica.
- **return:** Instrucción que devuelve el resultado y termina la función.

```

1 #include <stdio.h>
2
3 // Funcion que suma dos enteros y devuelve el resultado
4 int sumar(int a, int b) {
5     int resultado = a + b;
6     return resultado;
7 }
8
9 // Funcion que no recibe parametros y no devuelve nada
10 void saludar(void) {
11     printf("Hola, mundo!\n");
12     // No necesita return, pero se puede usar "return;" ←
13     // para salir antes
14 }
15
16 int main() {
17     int x = 5, y = 3;
18     int suma = sumar(x, y); // Llamada a la funcion
19     printf("La suma es: %d\n", suma);
20     saludar();
21     return 0;
22 }

```

Listing 45: Ejemplo de función simple

La Instrucción return

La instrucción `return` tiene dos propósitos fundamentales:

1. **Terminar la ejecución de la función:** Cuando se encuentra un `return`, la función finaliza inmediatamente y el control vuelve al punto donde fue llamada.
2. **Devolver un valor:** Opcionalmente, puede devolver un valor al llamante.

```
1 #include <stdio.h>
2 #include <stdbool.h> // Para usar bool, true, false
3
4 // 1. Funcion que devuelve un valor
5 int cuadrado(int x) {
6     return x * x;
7 }
8
9 // 2. Funcion que puede terminar antes
10 int buscar_primer_par(int numeros[], int size) {
11     for (int i = 0; i < size; i++) {
12         if (numeros[i] % 2 == 0) {
13             return i; // Termina aqui, devuelve el indice
14         }
15     }
16     return -1; // Si no encuentra, devuelve -1
17 }
18
19 // 3. Funcion void que usa return para salir antes
20 void procesar(int valor) {
21     if (valor < 0) {
22         printf("Valor negativo, no se procesa.\n");
23         return; // Salimos de la funcion sin hacer nada mas
24     }
25     printf("Procesando valor: %d\n", valor);
26     // Aqui no hay return explicito, pero al final de la ←
27     // funcion se retorna implicitamente
28 }
29
30 int main() {
31     printf("Cuadrado de 5: %d\n", cuadrado(5));
32
33     int arr[] = {1, 3, 5, 6, 7};
34     int pos = buscar_primer_par(arr, 5);
35     if (pos != -1) {
36         printf("Primer par encontrado en indice: %d\n", ←
37             pos);
38     } else {
39         printf("No se encontro ningun par\n");
40     }
41 }
```

```

39
40     procesar(10);
41     procesar(-5);
42
43     return 0;
44 }

```

Listing 46: Diferentes usos de `return`

Valores de Retorno

El valor de retorno puede ser de cualquier tipo (int, float, char, puntero, struct, etc.). Es importante que el tipo de la expresión después de `return` sea compatible con el tipo de retorno declarado.

```

1  #include <stdio.h>
2
3  // Retorna un entero
4  int obtener_edad() {
5      return 25;
6  }
7
8  // Retorna un float
9  float obtener_promedio(int a, int b) {
10     return (a + b) / 2.0f;
11 }
12
13 // Retorna un caracter
14 char obtener_inicial(char *nombre) {
15     return nombre[0];
16 }
17
18 // Retorna un puntero a char (string)
19 char *obtener_saludo() {
20     return "Hola, mundo!";
21 }
22
23 int main() {
24     printf("Edad: %d\n", obtener_edad());
25     printf("Promedio: %.2f\n", obtener_promedio(10, 15));
26     printf("Inicial: %c\n", obtener_inicial("Carlos"));
27     printf("Saludo: %s\n", obtener_saludo());
28     return 0;
29 }

```

Listing 47: Funciones con diferentes tipos de retorno

Reglas de Alcance de Funciones

El **ámbito (scope)** de una función se refiere a dónde puede ser llamada.

- Por defecto, todas las funciones tienen ámbito **global**: pueden ser llamadas desde cualquier parte del programa (siempre que hayan sido declaradas o definidas antes del punto de llamada).
- Podemos restringir el ámbito de una función al archivo actual usando la palabra clave **static**.

```
1 #include <stdio.h>
2
3 // Funcion global: puede ser llamada desde cualquier archivo
4 void funcion_global() {
5     printf("Puedo ser llamada desde cualquier lado.\n");
6 }
7
8 // Funcion estatica: solo visible en este archivo
9 static void funcion_privada() {
10     printf("Solo visible en este archivo.\n");
11 }
12
13 void llamar_privada() {
14     funcion_privada(); // OK: dentro del mismo archivo
15 }
16
17 int main() {
18     funcion_global(); // OK
19     llamar_privada(); // OK, llama a funcion_privada ↔
20     // funcion_privada(); // Tambien OK porque estamos en ↔
21     // el mismo archivo
22     return 0;
23 }
```

Listing 48: Ámbito de funciones

Argumentos de Funciones

Los argumentos (o parámetros) son los valores que le pasamos a una función para que trabaje con ellos.

Llamada por Valor (Pass by Value)

En C, **todos los argumentos se pasan por valor por defecto**. Esto significa que la función recibe una **copia** del valor del argumento. Modificar el parámetro dentro de la función no afecta a la variable original.

```

1 #include <stdio.h>
2
3 void duplicar(int x) {
4     x = x * 2; // Modifica la copia
5     printf("Dentro de duplicar: x = %d\n", x);
6 }
7
8 int main() {
9     int a = 5;
10    printf("Antes de llamar: a = %d\n", a);
11    duplicar(a);
12    printf("Despues de llamar: a = %d\n", a); // a sigue ←
13        siendo 5!
14    return 0;
15 }

```

Listing 49: Paso por valor

Visualización del paso por valor

main():	duplicar():
a = 5	x = 5 (copia)
+-----> duplicar(a)	-----> x = x*2 (x = 10)
	(x se destruye al salir)
a sigue siendo 5 <-----+	

Llamada por Referencia (Simulada con Punteros)

C no tiene paso por referencia como otros lenguajes (C++, Java). Sin embargo, podemos simularlo pasando **punteros**. Así, la función recibe una copia de la **dirección** de la variable, y usando el operador de desreferencia ***** podemos modificar el valor original.

```

1 #include <stdio.h>
2
3 void duplicar_referencia(int *ptr) {
4     *ptr = *ptr * 2; // Modifica el valor en la direccion ←
5     printf("Dentro de duplicar_referencia: *ptr = %d\n", ←
6         *ptr);
7 }
8
9 int main() {
10    int a = 5;
11    printf("Antes de llamar: a = %d\n", a);
12    duplicar_referencia(&a); // Pasamos la direccion de a

```

```

12     printf("Despues de llamar: a = %d\n", a); // a ahora ←
13         es 10!
14     return 0;
15 }

```

Listing 50: Simulación de paso por referencia con punteros

Visualización del paso por referencia (con punteros)

```

main():                                duplicar_referencia():
a = 5 en direccion 0x100                ptr = 0x100 (copia de la direccion)
|                                       |
+-----> duplicar_referencia(&a)      |
        (pasa 0x100) -----+         |
                                       |
                                       V
                                       *ptr = *ptr * 2
                                       (accede a la direccion 0x100
                                       y modifica a a 10)

main(): a ahora es 10 <-----+

```

Comparación: Por Valor vs Por Referencia

Paso por Valor	Paso por Referencia (con punteros)
Se pasa una copia del dato.	Se pasa la dirección del dato.
La función no puede modificar la variable original.	La función puede modificar la variable original.
Más seguro (no hay efectos secundarios no deseados).	Más flexible (permite modificar múltiples variables).
Útil cuando solo necesitas el valor para calcular.	Necesario para modificar el argumento o para evitar copiar grandes estructuras.

Prototipos de Funciones

Un **prototipo** (o declaración) de función le dice al compilador: "Existe una función con este nombre, que recibe estos parámetros y devuelve este tipo". Esto permite llamar a la función antes de que el compilador haya visto su definición completa.

```

1 #include <stdio.h>
2
3 // Prototipos (declaraciones)
4 int sumar(int a, int b);
5 float dividir(float a, float b);

```

```

6 void saludar(char *nombre);
7
8 int main() {
9     // Podemos usar las funciones aunque su definicion ←
10     este despues
11     printf("Suma: %d\n", sumar(5, 3));
12     printf("Division: %.2f\n", dividir(10.0, 3.0));
13     saludar("Ana");
14     return 0;
15 }
16 // Definiciones de las funciones
17 int sumar(int a, int b) {
18     return a + b;
19 }
20
21 float dividir(float a, float b) {
22     if (b == 0) return 0;
23     return a / b;
24 }
25
26 void saludar(char *nombre) {
27     printf("Hola, %s!\n", nombre);
28 }

```

Listing 51: Prototipos de funciones

Beneficios de los prototipos:

- Permiten organizar el código con `main()` al principio y las funciones después.
- El compilador puede verificar que las llamadas a funciones coincidan con los tipos esperados.
- Son esenciales cuando trabajamos con múltiples archivos (los prototipos van en archivos `.h`).

Retorno de Punteros

Las funciones pueden devolver punteros. Esto es muy útil para devolver strings, estructuras asignadas dinámicamente, o para indicar la posición de un elemento en un array.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // 1. Retornar un puntero a un string literal
6 char *obtener_mensaje() {
7     return "Este es un mensaje";
8 }

```

```

9
10 // 2. Retornar un puntero a un elemento de un array
11 int *buscar_elemento(int *arr, int size, int valor) {
12     for (int i = 0; i < size; i++) {
13         if (arr[i] == valor) {
14             return &arr[i]; // Retorna la direccion del ←
15                             elemento encontrado
16         }
17     }
18     return NULL; // No encontrado
19 }
20
21 // 3. Retornar un puntero a memoria dinamica
22 char *duplicar_string(const char *original) {
23     if (original == NULL) return NULL;
24     int len = strlen(original);
25     char *copia = (char*) malloc(len + 1); // +1 para el ←
26     '\0'
27     if (copia != NULL) {
28         strcpy(copia, original);
29     }
30     return copia; // El llamante debe liberar esta ←
31     memoria con free()
32 }
33
34 int main() {
35     // Ejemplo 1
36     printf("%s\n", obtener_mensaje());
37
38     // Ejemplo 2
39     int numeros[] = {10, 20, 30, 40};
40     int *ptr = buscar_elemento(numeros, 4, 30);
41     if (ptr != NULL) {
42         printf("Elemento encontrado: %d en direccion ←
43         %p\n", *ptr, ptr);
44     }
45
46     // Ejemplo 3
47     char *copia = duplicar_string("Hola, mundo!");
48     if (copia != NULL) {
49         printf("Copia: %s\n", copia);
50         free(copia); // LIBERAR MEMORIA! Muy importante
51     }
52     return 0;
53 }

```

Listing 52: Funciones que retornan punteros

¡PRECAUCIÓN! Nunca retornes un puntero a una variable local automática, porque esa variable se destruye al salir de la función y el puntero quedará colgando (apuntando

a memoria inválida).

```

1 int *funcion_mal() {
2     int x = 10;
3     return &x; // GRAVE ERROR! x es local y deja de ↵
                existir al salir
4 } // Al salir de la funcion, la direccion de x es invalida

```

Listing 53: ERROR: Retornar puntero a variable local

Modelos de Memoria en C

Para entender mejor cómo funcionan las variables y funciones, necesitamos conocer los diferentes segmentos de memoria donde se almacenan los datos.

Organización de la Memoria en C

+-----+	Direcciones altas (ej. 0xFFFF)
STACK (Pila)	
(Variables locales,	
parametros, direcciones	
de retorno)	
(crece hacia	
V abajo)	
+-----+	
V	
HEAP (Montón)	
(Memoria dinamica:	
malloc, calloc, realloc)	
^	
(crece hacia	
arriba)	
+-----+	
DATOS (Data)	
(Variables globales	
y static inicializadas)	
+-----+	
BSS (Block	
Started by Symbol)	
(Variables globales	
y static no iniciadas,	
se inicializan a 0)	
+-----+	

	TEXT (Código)	
	(El programa en si,	
	las instrucciones)	
+-----+ Direcciones bajas (ej. 0x0000)		

Segmentos de Memoria

- **TEXT (Código):** Contiene las instrucciones del programa. Es de solo lectura.
- **DATOS (Data):** Variables globales y `static` que tienen un valor inicial distinto de cero.
- **BSS:** Variables globales y `static` que no se inicializan explícitamente (el sistema las pone a 0).
- **HEAP (Montón):** Memoria asignada dinámicamente con `malloc()`, `calloc()`, `realloc()`. Crece hacia arriba.
- **STACK (Pila):** Variables locales automáticas, parámetros de funciones, direcciones de retorno. Se crea un "marco de pila" (stack frame) por cada llamada a función. Crece hacia abajo.

Ejemplo de ubicación de variables

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int global_inicializada = 100;           // DATOS
5 int global_no_inicializada;             // BSS (se ←
    inicializa a 0)
6
7 static int estatica_inicializada = 200; // DATOS
8 static int estatica_no_inicializada;    // BSS
9
10 int main() {
11     int local_auto = 10;                 // STACK
12     static int local_estatica = 300;     // DATOS (aunque ←
        sea local, por 'static')
13     int *ptr_dinamico;
14
15     ptr_dinamico = (int*) malloc(sizeof(int)); // HEAP
16     *ptr_dinamico = 50;
17
18     printf("global_inicializada (DATOS): %d\n", ←
        global_inicializada);
19     printf("local_auto (STACK): %d\n", local_auto);
20     printf("local_estatica (DATOS): %d\n", local_estatica);
21     printf("ptr_dinamico (STACK) apunta a HEAP: %d\n", ←
        *ptr_dinamico);

```

```

22
23     free(ptr_dinamico);
24     return 0;
25 }

```

Listing 54: ¿Dónde se almacena cada variable?

Recursividad

Una función **recursiva** es aquella que se llama a sí misma. Es una técnica poderosa para resolver problemas que pueden descomponerse en subproblemas más pequeños de la misma naturaleza.

Componentes de una Función Recursiva

1. **Caso base:** Una condición que detiene la recursión. Sin caso base, la función se llamaría infinitamente (hasta desbordar la pila).
2. **Llamada recursiva:** La función se llama a sí misma, pero con un argumento modificado que se acerca al caso base.

Ejemplo Clásico: Factorial

El factorial de un número n ($n!$) se define como:

- $0! = 1$ (caso base)
- $n! = n \times (n - 1)!$ para $n > 0$ (paso recursivo)

```

1  #include <stdio.h>
2
3  int factorial(int n) {
4      // Caso base
5      if (n == 0) {
6          return 1;
7      }
8      // Llamada recursiva
9      return n * factorial(n - 1);
10 }
11
12 int main() {
13     int num = 5;
14     printf("Factorial de %d = %d\n", num, ↵
        factorial(num)); // 120
15     return 0;
16 }

```

Listing 55: Factorial recursivo

Visualización de la pila en la recursión de factorial(5)

Llamadas sucesivas (ida):

```
main() llama a factorial(5)
  factorial(5) llama a factorial(4)
    factorial(4) llama a factorial(3)
      factorial(3) llama a factorial(2)
        factorial(2) llama a factorial(1)
          factorial(1) llama a factorial(0)
```

Resolviendo (vuelta):

```
    factorial(0) devuelve 1
  factorial(1) devuelve 1 * 1 = 1
factorial(2) devuelve 2 * 1 = 2
factorial(3) devuelve 3 * 2 = 6
factorial(4) devuelve 4 * 6 = 24
factorial(5) devuelve 5 * 24 = 120
main() recibe 120
```

Ejemplo: Fibonacci

La secuencia de Fibonacci: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$.

```
1 #include <stdio.h>
2
3 int fibonacci(int n) {
4     if (n <= 1) {
5         return n; // Casos base: F(0)=0, F(1)=1
6     }
7     return fibonacci(n - 1) + fibonacci(n - 2);
8 }
9
10 int main() {
11     for (int i = 0; i <= 10; i++) {
12         printf("F(%d) = %d\n", i, fibonacci(i));
13     }
14     return 0;
15 }
```

Listing 56: Fibonacci recursivo

Nota: Esta implementación es muy ineficiente porque recalcula los mismos valores muchas veces. Es un buen ejemplo de recursión simple, pero en la práctica se usaría iteración o memoización.

Recursividad vs Iteración

Recursividad	Iteración
Código más elegante y cercano a la definición matemática.	Código generalmente más eficiente (sin overhead de llamadas a función).
Útil para estructuras de datos recursivas (árboles, listas).	Usa bucles (while , for).
Puede causar desbordamiento de pila (stack overflow) si la profundidad es muy grande.	No tiene riesgo de desbordamiento de pila por repetición.
Cada llamada consume memoria en la pila.	No consume memoria extra por iteración.

Resumen de la Unidad V

Hemos profundizado en el concepto más importante de la programación estructurada: las funciones.

- **Funciones:** Bloques reutilizables de código con un nombre, parámetros y valor de retorno.
- **return:** Termina la función y opcionalmente devuelve un valor.
- **Ámbito:** Las funciones pueden ser globales o estáticas (privadas al archivo).
- **Paso de argumentos:**
 - **Por valor:** Se pasa una copia (el original no se modifica).
 - **Por referencia (simulado):** Se pasa un puntero (se puede modificar el original).
- **Prototipos:** Declaraciones que permiten usar funciones antes de su definición.
- **Retorno de punteros:** Útil para strings y memoria dinámica, pero cuidado con retornar punteros a locales.
- **Modelos de memoria:** TEXT (código), DATOS/BSS (globales/static), HEAP (dinámica), STACK (locales y llamadas a función).
- **Recursividad:** Función que se llama a sí misma. Requiere un caso base y una llamada que se acerque a él. Elegante pero puede ser ineficiente.

Las funciones son la herramienta fundamental para construir programas modulares, legibles y mantenibles. En la siguiente unidad, exploraremos cómo organizar múltiples funciones en proyectos más grandes usando compilación separada y el preprocesador.

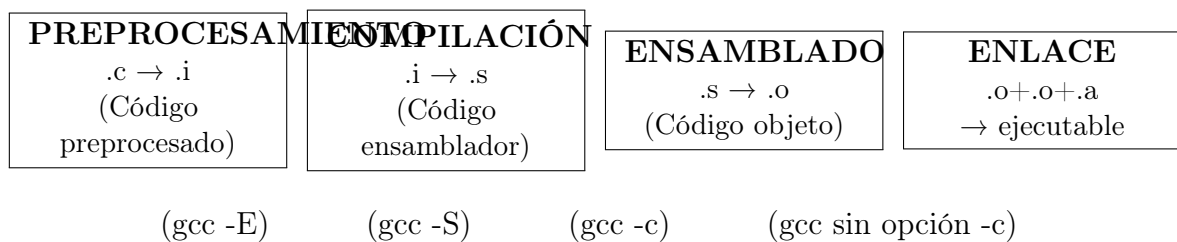
Unidad VI: Compilación, Enlace y el Preprocesador - Construyendo Proyectos Profesionales

Hasta ahora hemos trabajado con programas de un solo archivo, ejecutando comandos simples como `gcc programa.c -o programa`. Sin embargo, los proyectos de software del mundo real constan de decenas, cientos o miles de archivos, y su construcción es un proceso complejo que debe ser automatizado. Esta unidad te llevará al siguiente nivel: entenderás cómo se transforma un conjunto de archivos fuente en un programa ejecutable, cómo organizar tu código en módulos, y cómo usar las herramientas que C proporciona para gestionar todo el proceso.

El Proceso Completo: De Código Fuente a Ejecutable

Cuando escribimos un programa en C, el código fuente no es directamente ejecutable por la máquina. Debe pasar por un proceso de transformación que consta de cuatro fases principales. Es crucial entender cada una para diagnosticar errores y optimizar la compilación.

Las 4 Fases de la Transformación del Código



Fase 1: Preprocesamiento

El preprocesador (comúnmente llamado `cpp`) es la primera herramienta que actúa sobre nuestro código. No compila nada; simplemente realiza manipulaciones textuales basadas en directivas que comienzan con el símbolo `#`.

```
1 #include <stdio.h>
2 #define PI 3.14159
3 #define CUADRADO(x) ((x)*(x))
```

```

4
5 int main() {
6     double area = PI * CUADRADO(5);
7     printf("Area: %f\n", area);
8     return 0;
9 }

```

Listing 57: Código antes del preprocesamiento (entrada.c)

```

1 // ... contenido literal de stdio.h (cientos de lineas) ...
2
3 int main() {
4     double area = 3.14159 * ((5)*(5));
5     printf("Area: %f\n", area);
6     return 0;
7 }

```

Listing 58: Código después del preprocesamiento (salida.i) - simplificado

Acciones del preprocesador:

- Elimina todos los comentarios.
- Expande las macros (`#define`).
- Inserta el contenido de los archivos incluidos (`#include`).
- Procesa las directivas de compilación condicional (`#ifdef`, `#ifndef`, etc.).

Para ver el resultado del preprocesamiento, podemos usar:

```
gcc -E programa.c -o programa.i
```

Fase 2: Compilación

El compilador toma el archivo preprocesado (`.i`) y lo traduce a código **ensamblador** (`.s`). El código ensamblador es un lenguaje de bajo nivel específico de la arquitectura del procesador, pero aún legible por humanos.

```
gcc -S programa.i -o programa.s
```

Un fragmento de código ensamblador (para x86-64) podría verse así:

```

main:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 16
    movsd   xmm0, QWORD PTR .LC0[rip]
    movsd   QWORD PTR [rbp-8], xmm0
    mov     eax, 0

```

```

    leave
    ret
.LC0:
    .long    1374389535
    .long    1074339512 # Esto es 3.14159 en representacion de punto flotante

```

Fase 3: Ensamblado

El ensamblador convierte el código ensamblador (.s) en **código máquina** (binario) y lo guarda en un **archivo objeto** (.o en Linux/Mac, .obj en Windows). Este archivo contiene instrucciones ejecutables, pero aún no es un programa completo porque las direcciones de funciones externas (como printf) no están resueltas.

```

gcc -c programa.s -o programa.o
# O directamente desde el fuente C:
gcc -c programa.c -o programa.o

```

Podemos inspeccionar un archivo objeto con herramientas como nm (Linux/Mac) para ver sus símbolos:

```

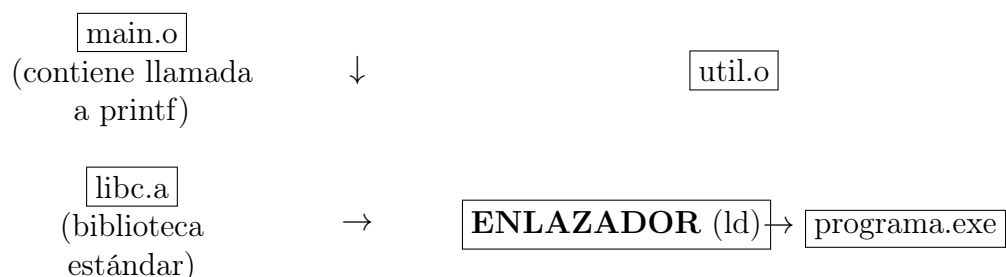
$ nm programa.o
0000000000000000 T main
                 U printf # La 'U' significa "Undefined" (indefinido)

```

Fase 4: Enlace (Link)

El **enlazador** (linker) es la pieza final del rompecabezas. Su trabajo es combinar uno o más archivos objeto y las bibliotecas necesarias para producir un único archivo **ejecutable**. Resuelve las referencias externas: cuando encuentra U printf en nuestro programa.o, busca la definición de printf en las bibliotecas (por defecto, la biblioteca estándar de C, libc.so o libc.a) y conecta la llamada con el código de la función.

El Proceso de Enlace



```

gcc programa.o util.o -o programa
# O directamente desde los fuentes (gcc llama automaticamente al enlazador):
gcc programa.c util.c -o programa

```


El Preprocesador de C en Profundidad

El preprocesador es una herramienta poderosa que actúa antes de la compilación. Todas sus directivas comienzan con `#` y terminan con un salto de línea (no llevan punto y coma).

Las Directivas del Preprocesador

`#include` - Inclusión de Archivos

Inserta el contenido de otro archivo en el archivo actual. Hay dos formas:

```
1 #include <stdio.h>           // Busca en los directorios del ↵
    sistema (ej. /usr/include)
2 #include "mi_libreria.h" // Busca primero en el ↵
    directorio actual, luego en los del sistema
```

Listing 59: Formas de `#include`

`#define` - Definición de Macros

Crea macros, que son como "abreviaturas" que el preprocesador expande textualmente.

```
1 #define PI 3.14159
2 #define EDAD_MAXIMA 150
3 #define NOMBRE_PROGRAMA "Calculadora v1.0"
4 #define ERROR -1
5
6 int main() {
7     double radio = 10.0;
8     double area = PI * radio * radio; // Se expande a: ↵
9         3.14159 * radio * radio
10    if (area > 1000) {
11        return ERROR; // Se expande a: ↵
12        return -1;
13    }
14    return 0;
15 }
```

Listing 60: Macros simples

Macros con Parámetros (Macros tipo función)

Podemos crear macros que reciben parámetros. El preprocesador las expande haciendo una sustitución textual.

```
1 #define CUADRADO(x) ((x) * (x))
2 #define MAX(a, b) ((a) > (b) ? (a) : (b))
3 #define MIN(a, b) ((a) < (b) ? (a) : (b))
```

```

4 #define ES_PAR(n) ((n) % 2 == 0)
5
6 int main() {
7     int a = 5, b = 10;
8     int c = CUADRADO(a);           // c = ((5) * (5)) = 25
9     int max = MAX(a, b);           // max = ((5) > (10) ? ←
                                   // (5) : (10)) = 10
10    int d = CUADRADO(a + b);        // d = ((5+10) * (5+10)) ←
                                   // = 225
11    int e = CUADRADO(++a);           // PELIGRO ! Se expande ←
                                   // a ((++a) * (++a))
12                                   // Comportamiento ←
                                   // indefinido (a se ←
                                   // incrementa dos veces)
13    return 0;
14 }

```

Listing 61: Macros con parámetros

¡PRECAUCIÓN CON LAS MACROS! Como la expansión es textual, pueden ocurrir efectos secundarios inesperados. Por eso, es buena práctica:

- Encerrar cada parámetro entre paréntesis en la definición: `((x)*(x))`.
- Encerrar toda la macro entre paréntesis.
- Evitar pasar expresiones con efectos secundarios (como `++a`) a macros.

#undef - Eliminar una Macro

Permite eliminar una definición previa.

```

1 #define TEMP 100
2 // ... código que usa TEMP ...
3 #undef TEMP
4 // A partir de aquí, TEMP ya no está definida
5 #define TEMP 200 // Se puede volver a definir

```

Operadores Especiales del Preprocesador

- `#` (operador de stringización): Convierte el parámetro de una macro en un string literal.
- `##` (operador de concatenación): Concatena dos tokens.

```

1 #define STRINGIFY(x) #x
2 #define CONCAT(a, b) a##b
3
4 int main() {

```

```

5     printf(Stringify(Hola mundo)); // Se expande a: ↵
        printf("Hola mundo");
6
7     int mi_variable_123 = 10;
8     int CONCAT(mi_variable_, 123) = 20; // Se expande a: ↵
        int mi_variable_123 = 20;
9     // Cuidado ! Esto puede ser confuso y difícil de ↵
        depurar.
10    return 0;
11 }

```

Listing 62: Operadores # y ##

Compilación Condicional

Esta es una de las características más útiles del preprocesador. Permite incluir o excluir partes del código basándose en condiciones que se evalúan en tiempo de preprocesamiento.

Directivas Principales

```

1 #include <stdio.h>
2
3 #define VERSION 2
4 #define DEBUG    // Definida, pero sin valor
5
6 int main() {
7     #ifdef DEBUG    // Si DEBUG esta definido (con o sin ↵
        valor)
8         printf("Modo DEBUG activado.\n");
9     #endif
10
11     #if VERSION == 1
12         printf("Version 1 del programa\n");
13     #elif VERSION == 2
14         printf("Version 2 del programa\n");
15     #else
16         printf("Version desconocida\n");
17     #endif
18
19     #ifndef RELEASE    // Si RELEASE NO esta definido
20         printf("Esta es una compilacion de desarrollo.\n");
21     #endif
22
23     return 0;
24 }

```

Listing 63: Compilación condicional básica

Usos Comunes de la Compilación Condicional

1. Código específico de plataforma:

```

1 #ifdef _WIN32
2     #include <windows.h>
3     #define LIMPIAR_PANTALLA system("cls")
4 #elif __linux__
5     #include <unistd.h>
6     #define LIMPIAR_PANTALLA system("clear")
7 #elif __APPLE__
8     #define LIMPIAR_PANTALLA system("clear")
9 #endif

```

2. Depuración (debug):

```

1 #define DEBUG 1
2
3 if (error) {
4     #if DEBUG
5         printf("Error en linea %d del archivo %s\n", ←
6             __LINE__, __FILE__);
7     #endif
8     // Manejo del error...
9 }

```

3. Guardas de inclusión (*include guards*) - ¡ESENCIALES!

Cuando varios archivos incluyen el mismo archivo de cabecera, podemos terminar con definiciones duplicadas. Las guardas de inclusión lo evitan.

```

1 #ifndef MI_LIBRERIA_H // Si NO esta definido ←
2     MI_LIBRERIA_H
3 #define MI_LIBRERIA_H // Lo definimos
4
5 // Todo el contenido del archivo va aqui
6 int sumar(int a, int b);
7 int restar(int a, int b);
8 #endif // Fin de la guarda

```

Listing 64: Archivo *mi_biblioteca.h* con guardas de inclusión

La primera vez que se incluye este archivo, `MI_LIBRERIA_H` no está definido, por lo que el preprocesador incluye el contenido y define la macro. Las siguientes veces, la macro ya está definida, por lo que el contenido se omite.

4. Macros predefinidas útiles: El estándar define varias macros que siempre están disponibles:

```

1 #include <stdio.h>
2 int main() {
3     printf("Archivo: %s\n", __FILE__);        // Nombre ↵
4     printf("Linea: %d\n", __LINE__);          // Linea ↵
5     printf("Fecha: %s\n", __DATE__);          // Fecha ↵
6     printf("Hora: %s\n", __TIME__);           // Hora de ↵
7     printf("ANSI C: %d\n", __STDC__);         // 1 si el ↵
8     return 0;
9 }

```

Archivos Cabecera (.h) - La Interfaz Pública

Los archivos de cabecera son la clave para la modularización en C. Actúan como el **contrato** entre el implementador de un módulo y sus usuarios. Contienen las **declaraciones** (qué hay disponible), mientras que los archivos fuente (.c) contienen las **definiciones** (cómo funciona).

Qué debe ir en un archivo .h

- **Prototipos de funciones públicas:** Las funciones que otros archivos pueden llamar.
- **Declaraciones de tipos:** typedef, struct, enum, union que deben ser conocidos por los usuarios.
- **Macros públicas:** Constantes y macros tipo función que forman parte de la interfaz.
- **Declaraciones de variables globales:** Usando extern (la definición real va en un .c).
- **Guardas de inclusión:** Siempre, para evitar múltiples inclusiones.

Qué NO debe ir en un archivo .h

- **Definiciones de funciones:** A menos que sean static inline (un caso avanzado). Si pones la definición completa de una función en un .h y ese .h se incluye en dos archivos .c diferentes, tendrás un error de "definición múltiple".^{en} la fase de enlace.
- **Definiciones de variables:** Sin extern o static, causarán el mismo problema de definición múltiple.
- **Código ejecutable:** El .h es para declaraciones, no para lógica.

Ejemplo Completo de un Módulo Bien Diseñado

```
1 #ifndef CALCULOS_H
2 #define CALCULOS_H
3
4 // Constantes
5 #define PI 3.1415926535
6 #define GRAVEDAD 9.81
7
8 // Tipos
9 typedef struct {
10     float x;
11     float y;
12 } Punto;
13
14 // Prototipos de funciones publicas
15 float calcular_area_circulo(float radio);
16 float calcular_distancia(Punto p1, Punto p2);
17 int es_par(int numero);
18
19 #endif
```

Listing 65: calculos.h - El archivo de cabecera

```
1 #include "calculos.h"
2 #include <math.h> // Para sqrt
3
4 // Definiciones de las funciones declaradas en el .h
5 float calcular_area_circulo(float radio) {
6     return PI * radio * radio;
7 }
8
9 float calcular_distancia(Punto p1, Punto p2) {
10     float dx = p2.x - p1.x;
11     float dy = p2.y - p1.y;
12     return sqrt(dx*dx + dy*dy);
13 }
14
15 int es_par(int numero) {
16     return numero % 2 == 0;
17 }
18
19 // Funcion privada (no declarada en el .h, solo visible ←
    en este archivo)
20 static void funcion_privada() {
21     // Solo puede ser llamada desde dentro de calculos.c
22 }
```

Listing 66: calculos.c - La implementación

```

1 #include <stdio.h>
2 #include "calculos.h" // Incluimos nuestra interfaz
3
4 int main() {
5     float area = calcular_area_circulo(5.0);
6     printf("Area del circulo: %.2f\n", area);
7
8     Punto a = {0.0, 0.0};
9     Punto b = {3.0, 4.0};
10    float dist = calcular_distancia(a, b);
11    printf("Distancia: %.2f\n", dist);
12
13    if (es_par(10)) {
14        printf("10 es par.\n");
15    }
16
17    // funcion_privada(); // ERROR! No esta declarada en ←
18    // calculos.h
19    return 0;
20 }

```

Listing 67: main.c - Usando el módulo

Para compilar este proyecto:

```

gcc -c calculos.c -o calculos.o
gcc -c main.c -o main.o
gcc main.o calculos.o -o programa -lm

```

Concepto de Proyecto

Un **proyecto** en programación es un conjunto organizado de archivos que, al ser compilados y enlazados juntos, producen uno o más programas o bibliotecas. Un proyecto típico en C incluye:

```

mi_proyecto/
src/           # Código fuente (.c)
    main.c
    calculos.c
    io.c
include/       # Archivos de cabecera (.h) publicos
    calculos.h
    io.h
lib/           # Bibliotecas externas (opcional)
obj/           # Archivos objeto (.o) (generados)
bin/           # Ejecutables (generados)
tests/        # Pruebas unitarias

```

```
docs/          # Documentacion
Makefile       # Automatizacion de la compilacion
```

Concepto de Makefile

Cuando un proyecto tiene muchos archivos, compilar manualmente cada uno es tedioso y propenso a errores. Además, si solo cambias un archivo, quieres recompilar solo ese archivo y enlazar de nuevo, no todo el proyecto.

Make es una herramienta de automatización de compilación que existe desde los años 70. Lee un archivo llamado **Makefile** que contiene reglas sobre cómo construir el proyecto.

Estructura básica de una regla en Makefile

objetivo: dependencias
comandos

- **objetivo:** El archivo que queremos generar (ej. programa, main.o).
- **dependencias:** Archivos de los que depende el objetivo. Si alguna dependencia es más nueva que el objetivo, Make ejecuta los comandos.
- **comandos:** Instrucciones (precedidas por un tabulador, ¡no espacios!) para generar el objetivo.

Ejemplo de Makefile para nuestro proyecto

```
1 # Variables (para facilitar cambios)
2 CC = gcc
3 CFLAGS = -Wall -Wextra -std=c99 -I./include
4 LDFLAGS = -lm
5 TARGET = bin/programa
6 SRCDIR = src
7 INCDIR = include
8 OBJDIR = obj
9 SOURCES = $(wildcard $(SRCDIR)/*.c)
10 OBJECTS = $(patsubst $(SRCDIR)/%.c, $(OBJDIR)/%.o, ↵
    $(SOURCES))
11
12 # Regla por defecto (la primera)
13 $(TARGET): $(OBJECTS) | $(BINDIR)
14     $(CC) $^ -o $@ $(LDFLAGS)
15
16 # Como generar archivos .o a partir de .c
17 $(OBJDIR)/%.o: $(SRCDIR)/%.c | $(OBJDIR)
18     $(CC) $(CFLAGS) -c $< -o $@
19
20 # Crear directorios si no existen
```



```

21 $(OBJDIR):
22     mkdir -p $(OBJDIR)
23
24 $(BINDIR):
25     mkdir -p $(BINDIR)
26
27 # Limpiar archivos generados
28 clean:
29     rm -rf $(OBJDIR) $(TARGET)
30
31 # Regla para ejecutar el programa
32 run: $(TARGET)
33     ./$$(TARGET)
34
35 # Reglas phony (no representan archivos reales)
36 .PHONY: clean run

```

Listing 68: Makefile completo

Cómo usarlo:

```

$ make          # Compila y enlaza todo
$ make clean    # Limpia archivos objeto y ejecutable
$ make run      # Compila (si es necesario) y ejecuta el programa

```

¿Por qué es tan útil? Si modificas solo `src/calculos.c`, Make ve que `calculos.o` es más antiguo que `calculos.c`, así que recompila solo `calculos.o`. Luego ve que `programa` depende de `calculos.o` (y de `main.o`), y como `calculos.o` se actualizó, re-enlaza `programa`. ¡Ningún otro archivo se recompila innecesariamente!

Concepto de Biblioteca (Librería)

Una **biblioteca** es una colección de código precompilado que puede ser reutilizado por múltiples programas. En C, hay dos tipos principales:

Bibliotecas Estáticas (.a en Linux/Mac, .lib en Windows)

- El código de la biblioteca se copia directamente dentro del ejecutable en tiempo de enlace.
- El ejecutable final es más grande, pero es independiente (no necesita archivos adicionales para ejecutarse).
- Si la biblioteca se actualiza, todos los programas que la usan deben ser recompilados.

Crear y usar una biblioteca estática

```

# 1. Compilar los archivos fuente a objetos
gcc -c calculos.c -o calculos.o

```

```
gcc -c io.c -o io.o
```

```
# 2. Crear la biblioteca estatica (ar = archiver)
ar rcs libmimath.a calculos.o io.o
```

```
# 3. Usar la biblioteca al compilar un programa
gcc main.c -L. -lmimath -o programa
```

Bibliotecas Dinámicas (Compartidas) (.so en Linux, .dylib en Mac, .dll en Windows)

- El código de la biblioteca NO se copia al ejecutable. En su lugar, el ejecutable contiene referencias a la biblioteca.
- La biblioteca se carga en memoria en tiempo de ejecución (cuando el programa se inicia, o incluso después).
- Los ejecutables son más pequeños.
- Múltiples programas pueden compartir la misma biblioteca en memoria.
- Si la biblioteca se actualiza, los programas se benefician sin necesidad de recompilar (siempre que la interfaz no cambie).

Crear y usar una biblioteca dinámica

```
# 1. Compilar con -fPIC (Position Independent Code)
gcc -c -fPIC calculos.c -o calculos.o
gcc -c -fPIC io.c -o io.o
```

```
# 2. Crear la biblioteca compartida
gcc -shared -o libmimath.so calculos.o io.o
```

```
# 3. Usar la biblioteca
gcc main.c -L. -lmimath -o programa
```

```
# 4. Para ejecutar, la biblioteca debe estar en un lugar donde el sistema la encuentre
export LD_LIBRARY_PATH=./:$LD_LIBRARY_PATH # Linux
./programa
```

Bibliotecas Estándar de C

El estándar de C define un conjunto de bibliotecas que deben estar disponibles en cualquier implementación del lenguaje.

Biblioteca Estándar de Entrada-Salida (<stdio.h>)

Es la biblioteca que más hemos usado. Proporciona funciones para interactuar con el usuario (consola) y con archivos.

Funciones para consola

```

1 #include <stdio.h>
2
3 int main() {
4     // Salida
5     printf("Hola, %s! Tienes %d anos.\n", "Carlos", 25);
6     puts("Una linea simple"); // A ade \n automaticamente
7     putchar('A');           // Imprime un caracter
8
9     // Entrada
10    char nombre[50];
11    int edad;
12    printf("Ingresa tu nombre: ");
13    scanf("%s", nombre);      // Cuidado ! No verifica ←
14                                longitud
15    printf("Ingresa tu edad: ");
16    scanf("%d", &edad);
17
18    // Entrada de linea completa (mas segura)
19    getchar(); // Para consumir el \n que quedo del scanf ←
20                                anterior
21    printf("Ingresa una frase: ");
22    fgets(nombre, sizeof(nombre), stdin); // Lee ←
23    // incluyendo espacios
24
25    return 0;
26 }
```

Listing 69: Funciones de consola en stdio.h

Funciones para archivos

```

1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo;
5     char linea[100];
6
7     // --- Escribir en un archivo ---
8     archivo = fopen("datos.txt", "w"); // "w" = write ←
9     // (sobrescribe)
10    if (archivo == NULL) {
```

```

10     printf("Error al abrir el archivo para ↵
        escritura\n");
11     return 1;
12 }
13
14 fprintf(archivo, "Linea 1: Hola mundo\n");
15 fputs("Linea 2: Otra linea\n", archivo);
16 fputc('X', archivo);
17
18 fclose(archivo); // Siempre cerrar el archivo!
19
20 // --- Leer de un archivo ---
21 archivo = fopen("datos.txt", "r"); // "r" = read
22 if (archivo == NULL) {
23     printf("Error al abrir el archivo para lectura\n");
24     return 1;
25 }
26
27 printf("Contenido del archivo:\n");
28 while (fgets(linea, sizeof(linea), archivo) != NULL) {
29     printf("%s", linea);
30 }
31
32 fclose(archivo);
33
34 // --- Otras formas de apertura ---
35 // "a" = append (agregar al final)
36 // "r+" = lectura y escritura (el archivo debe existir)
37 // "w+" = lectura y escritura (crea o sobrescribe)
38 // "a+" = lectura y escritura (agrega al final)
39
40 // Modo binario (aadir 'b'): "rb", "wb", "ab"
41
42 return 0;
43 }

```

Listing 70: Manejo de archivos en C

Biblioteca para Manejo de Strings (<string.h>)

Proporciona funciones para manipular cadenas de caracteres (arrays de char terminados en \0).

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char destino[50];
6     char origen[] = "Hola, mundo!";

```

```

7     char otra[] = "Hola, C!";
8
9     // 1. Copiar strings
10    strcpy(destino, origen); // Copia origen a destino
11    printf("strcpy: %s\n", destino);
12
13    // Copia segura con limite
14    strncpy(destino, otra, 5); // Copia solo 5 caracteres
15    destino[5] = '\0'; // IMPORTANTE ! strncpy NO agrega ←
16    printf("strncpy: %s\n", destino);
17
18    // 2. Longitud
19    printf("Longitud de '%s': %zu\n", origen, ←
20          strlen(origen));
21
22    // 3. Concatenar
23    char saludo[100] = "Hola, ";
24    strcat(saludo, "Carlos");
25    printf("strcat: %s\n", saludo);
26
27    // Concatenacion segura
28    strncat(saludo, " Perez", 5); // Agrega " Perez" pero ←
29    printf("strncat: %s\n", saludo);
30
31    // 4. Comparar
32    if (strcmp(origen, otra) == 0) {
33        printf("Las cadenas son iguales\n");
34    } else {
35        printf("Las cadenas son diferentes\n");
36    }
37
38    // 5. Buscar
39    char *pos = strchr(saludo, 'C'); // Busca 'C' en saludo
40    if (pos != NULL) {
41        printf("Encontrada 'C' en posicion: %ld\n", pos - ←
42              saludo);
43    }
44
45    pos = strstr(saludo, "arlos"); // Busca subcadena
46    if (pos != NULL) {
47        printf("Subcadena encontrada: %s\n", pos);
48    }
49
50    // 6. Tokenizacion (dividir string)
51    char str[] = "uno,dos,tres,cuatro";
52    char *token = strtok(str, ",");
53    while (token != NULL) {

```

```

52     printf("Token: %s\n", token);
53     token = strtok(NULL, ",");
54 }
55
56 return 0;
57 }

```

Listing 71: Funciones clave de string.h

¡PRECAUCIÓN! Muchas funciones de `string.h` (como `strcpy`, `strcat`) no verifican los límites de los buffers y pueden causar desbordamientos de búfer (una grave vulnerabilidad de seguridad). Siempre que sea posible, usa las versiones con `n`: `strncpy`, `strncat`, o funciones aún más seguras como `strlcpy` (no estándar, pero disponible en algunos sistemas).

Otras Bibliotecas Estándar Importantes

■ <stdlib.h> (Standard Library):

```

1  #include <stdlib.h>
2  int a = atoi("123");           // String a entero
3  double d = atof("3.14");       // String a double
4  long l = strtol("0xFF", NULL, 16); // String a long ←
    en base 16
5
6  int *p = malloc(10 * sizeof(int)); // Asignar memoria
7  if (p != NULL) {
8      // usar p
9      free(p);                     // Liberar memoria
10 }
11
12 int r = rand();                 // Numero aleatorio (0 a ←
    RAND_MAX)
13 srand(time(NULL));             // Semilla para rand
14
15 exit(0);                       // Terminar programa ←
    inmediatamente
16
17 // Ordenar y buscar (avanzado)
18 qsort(array, size, sizeof(int), comparar); // ←
    Ordenamiento rapido

```

■ <math.h> (Matemáticas):

```

1  #include <math.h>
2  double x = sqrt(25.0);          // Raiz cuadrada ←
    (resultado 5.0)
3  double y = pow(2.0, 3.0);       // Potencia (2^3 = 8.0)
4  double z = sin(3.14159 / 2);    // Seno

```

```

5 double a = fabs(-5.5);           // Valor absoluto (5.5)
6 double b = ceil(3.2);           // Redondear hacia ↵
    arriba (4.0)
7 double c = floor(3.8);          // Redondear hacia ↵
    abajo (3.0)
8 double d = round(3.5);          // Redondear al entero ↵
    mas cercano (4.0)

```

Nota: En Linux, al compilar con `<math.h>`, hay que añadir `-lm` al final del comando de compilación: `gcc programa.c -o programa -lm`.

■ **<ctype.h>** (Clasificación de caracteres):

```

1 #include <ctype.h>
2 if (isalpha('A')) { }           // Es letra?
3 if (isdigit('5')) { }           // Es dígito?
4 if (isspace(' ')) { }           // Es espacio en blanco?
5 if (isupper('A')) { }           // Es mayúscula?
6 char may = toupper('a');         // Convertir a ↵
    mayuscula ('A')
7 char min = tolower('Z');         // Convertir a ↵
    minuscula ('z')

```

■ **<time.h>** (Tiempo y fecha):

```

1 #include <stdio.h>
2 #include <time.h>
3
4 int main() {
5     time_t t = time(NULL);       // Tiempo actual ↵
        (segundos desde 1970)
6     printf("Segundos desde 1970: %ld\n", t);
7
8     // Convertir a formato local
9     struct tm *info = localtime(&t);
10    printf("Fecha: %d/%d/%d\n", info->tm_mday, ↵
        info->tm_mon + 1, info->tm_year + 1900);
11
12    // Medir tiempo de ejecución
13    clock_t inicio = clock();
14    // ... código a medir ...
15    for (long i = 0; i < 100000000; i++);
16    clock_t fin = clock();
17
18    double tiempo = (double)(fin - inicio) / ↵
        CLOCKS_PER_SEC;
19    printf("Tiempo: %.2f segundos\n", tiempo);
20
21    return 0;

```

22 }

Presentación Comparativa de Varios Compiladores

Existen múltiples compiladores de C, cada uno con sus fortalezas y plataformas objetivo.

Compilador	Plataformas Principales	Características Destacadas	Caso de Uso Típico
GCC	Linux, Windows (MinGW, Cygwin), macOS, BSD, embebidos.	Estándar en Unix/Linux. Muy maduro, múltiples optimizaciones, soporta muchos dialectos de C. Amplísima comunidad.	Desarrollo general en Linux, sistemas embebidos, código abierto.
Clang	Linux, Windows, macOS (compilador por defecto en Xcode), BSD.	Mensajes de error excepcionalmente claros. Compilación rápida. Buen soporte para análisis estático.	Desarrollo en macOS/iOS, proyectos donde se valoren buenos diagnósticos.
MSVC	Windows.	Integración perfecta con Visual Studio. Excelente soporte para API de Windows. Depurador muy potente.	Desarrollo de aplicaciones nativas para Windows, videojuegos.
TCC	Windows, Linux.	Extremadamente rápido y pequeño. Compila casi al instante. Perfecto para aprender o para scripts.	Aprendizaje, ejecución de código C como script, entornos con recursos limitados.
Intel C++	Linux, Windows.	Altamente optimizado para procesadores Intel. Código más rápido en hardware Intel.	Computación de alto rendimiento (HPC), aplicaciones críticas en hardware Intel.

Comparativa de Mensajes de Error (Un ejemplo vale más que mil palabras)

```

1 int main() {
2     int x = "hola"; // Error: asignar string a int
3     printf("Hola\n") // Error: falta punto y coma
4     return 0;
5 }
```

Listing 72: Código con errores

Mensaje de GCC:

```
error: initialization of 'int' from 'char *' makes integer from pointer without a cast
error: expected ';' before 'return'
```

Mensaje de Clang:

```
error: incompatible pointer to integer conversion initializing 'int' with an expression of type 'char *'
error: expected ';' after expression
```

La diferencia es sutil, pero Clang tiende a ser más descriptivo y a sugerir correcciones.

Compilando con Diferentes Compiladores

GCC

```
gcc -Wall -Wextra -std=c99 programa.c -o programa
```

Clang (los parametros son compatibles)

```
clang -Wall -Wextra -std=c99 programa.c -o programa
```

MSVC (en una terminal de desarrollador de Visual Studio)

```
cl /W4 programa.c
```

TCC

```
tcc programa.c -o programa.exe
```

Resumen de la Unidad VI

Hemos dado un salto cualitativo, pasando de programas sencillos a proyectos complejos y profesionales.

- **Proceso de compilación:** Comprende 4 fases (preprocesamiento, compilación, ensamblado, enlace). Cada fase transforma el código hasta obtener un ejecutable.
- **Preprocesador:** Procesa directivas # (include, define, condicionales). Es una herramienta textual que actúa antes de la compilación real.
- **Compilación condicional:** Permite incluir/excluir código basado en condiciones de preprocesamiento (#ifdef, #ifndef, #if, #elif, #else). Esencial para código multiplataforma y depuración.
- **Archivos cabecera (.h):** Son las interfaces de los módulos. Contienen declaraciones, prototipos y macros. Usan guardas de inclusión (#ifndef ... #define ... #endif) para evitar inclusiones múltiples.
- **Proyecto:** Estructura organizada de archivos fuente, cabeceras, bibliotecas y herramientas de construcción.
- **Makefile:** Automatiza la compilación, recompilando solo lo necesario y evitando trabajo manual repetitivo.

- **Bibliotecas:** Código precompilado reutilizable. Estáticas (`.a`, `.lib`) se copian al ejecutable; dinámicas (`.so`, `.dll`) se cargan en tiempo de ejecución.
- **Bibliotecas estándar:** Conjunto esencial de funcionalidades: entrada/salida (`stdio.h`), strings (`string.h`), utilidades generales (`stdlib.h`), matemáticas (`math.h`), clasificación de caracteres (`ctype.h`), tiempo (`time.h`), etc.
- **Compiladores:** Diferentes opciones (GCC, Clang, MSVC, TCC) con ventajas según la plataforma y el caso de uso. Conocer sus diferencias ayuda a elegir la herramienta adecuada.

Con esta unidad, ya estás preparado para trabajar en proyectos de cualquier tamaño, organizar tu código de manera profesional y entender todo el proceso que hay detrás de un simple comando de compilación.

Unidad VIII: Entrada y Salida - Comunicación con el Mundo Exterior

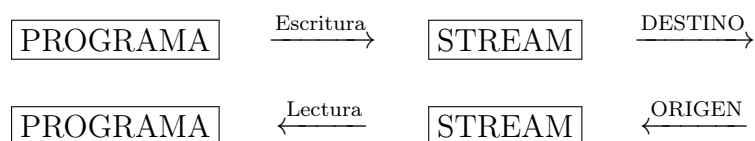
Hasta ahora, nuestros programas han sido entidades relativamente aisladas. Han recibido datos a través de argumentos de línea de comandos o de la función `scanf()`, y han mostrado resultados por la pantalla con `printf()`. Sin embargo, en el mundo real, los programas necesitan comunicarse con el exterior de formas mucho más diversas: leer archivos de configuración, procesar grandes volúmenes de datos almacenados en disco, generar reportes, comunicarse con otros programas, etc.

Esta unidad está dedicada por completo a las operaciones de **entrada y salida (I/O)**, el mecanismo fundamental que permite a nuestros programas interactuar con el mundo exterior.

El Concepto de Stream (Flujo)

Antes de hablar de archivos o de consola, debemos entender el concepto unificador de **stream** (flujo) en C. Un stream es una abstracción que representa un flujo de datos, independientemente de su origen o destino. Piensa en él como una tubería por la que circulan bytes.

Abstracción de Stream



La gran ventaja de esta abstracción es que las funciones de I/O en C trabajan con streams, no directamente con dispositivos. El mismo código que escribe en un archivo puede escribir en la consola o en una conexión de red simplemente cambiando el stream al que se dirige.

Entrada y Salida Estándar

Cuando un programa en C comienza, el sistema operativo le proporciona tres streams predefinidos, accesibles a través de las constantes definidas en `<stdio.h>`:

Stream	Constante	Pr
Entrada estándar	<code>stdin</code>	Normalmente conectada al teclado
Salida estándar	<code>stdout</code>	Normalmente conectada a la pantalla
Salida de error estándar	<code>stderr</code>	Normalmente conectada a la pantalla, pero separada de <code>stdout</code>

```

1 #include <stdio.h>
2
3 int main() {
4     // Escribir en salida estandar
5     fprintf(stdout, "Este es un mensaje normal.\n");
6
7     // Escribir en salida de error (inmediato, sin buffer)
8     fprintf(stderr, "Este es un mensaje de ERROR.\n");
9
10    // Leer de entrada estandar
11    char nombre[50];
12    fprintf(stdout, "Como te llamas? ");
13    fscanf(stdin, "%s", nombre); // Equivalente a scanf("%s", nombre);
14
15    fprintf(stdout, "Hola, %s!\n", nombre);
16
17    return 0;
18 }

```

Listing 73: Uso de streams estándar

Redirección: Una característica poderosa de los sistemas operativos tipo Unix (y también en Windows) es que podemos redirigir estos streams. Por ejemplo:

```
$ ./programa < entrada.txt > salida.txt 2> error.txt
```

Esto hará que `stdin` lea de `entrada.txt`, `stdout` escriba en `salida.txt` y `stderr` escriba en `error.txt`, sin cambiar ni una línea de código.

Concepto de Archivo

Un **archivo** es una colección de datos almacenada en un dispositivo persistente (disco duro, SSD, memoria USB, etc.). En C, un archivo no es más que un stream que está conectado a un archivo en el sistema de archivos.

Tipos de Archivos (desde la perspectiva de C)

- **Archivos de texto:** Contienen datos legibles por humanos, organizados en líneas. Cuando se leen o escriben, pueden ocurrir traducciones de caracteres (por ejemplo, `\n` en C puede convertirse en `\r\n` en Windows al escribir).

- **Archivos binarios:** Contienen datos en el mismo formato en que se almacenan en memoria. No hay traducción de caracteres. Son más eficientes para almacenar datos numéricos o estructuras complejas.

Operaciones Básicas con Archivos

Trabajar con archivos en C sigue un patrón de tres pasos: **abrir**, **operar** (leer/escribir), **cerrar**.

Apertura y Cierre: `fopen()` y `fclose()`

```

1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo; // Puntero a FILE (la estructura que ↵
        representa un stream)
5
6     // Abrir archivo para escritura
7     archivo = fopen("datos.txt", "w"); // "w" = write ↵
        (crea o sobrescribe)
8
9     if (archivo == NULL) {
10         fprintf(stderr, "Error: No se pudo abrir el ↵
            archivo para escritura.\n");
11         return 1;
12     }
13
14     // ... operaciones de escritura ...
15
16     // Cerrar el archivo ( IMPORTANTE !)
17     fclose(archivo);
18
19     return 0;
20 }
```

Listing 74: Abrir y cerrar un archivo

Modos de apertura (el segundo argumento de `fopen()`)

Modo	Significado	Observaciones
<code>r</code>	Lectura (read)	El archivo debe existir.
<code>w</code>	Escritura (write)	Crea el archivo si no existe; si existe, lo sobrescribe.
<code>a</code>	Añadir (append)	Crea el archivo si no existe; escribe al final.
<code>r+</code>	Lectura y escritura	El archivo debe existir.
<code>w+</code>	Lectura y escritura	Crea o sobrescribe.
<code>a+</code>	Lectura y añadir	Crea si no existe; lectura desde cualquier posición, escritura siempre al final.
<code>rb</code> , <code>wb</code> , etc.	Modo binario	Añadir <code>'b'</code> para modo binario (ej. <code>rb</code> para leer binario).

Escritura en Archivos

Las funciones de escritura son análogas a las de consola, pero reciben el stream como primer argumento.

```

1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo = fopen("salida.txt", "w");
5
6     if (archivo == NULL) {
7         perror("Error al abrir archivo"); // perror ←
8         muestra el error del sistema
9         return 1;
10    }
11
12    // fprintf: escritura formateada (como printf)
13    fprintf(archivo, "Entero: %d, Float: %.2f, String: ←
14        %s\n", 42, 3.14159, "hola");
15
16    // fputs: escribe una cadena (sin formato, no a ade ←
17        \n automáticamente)
18    fputs("Esta es una línea de texto.\n", archivo);
19    fputs("Esta es otra línea.\n", archivo);
20
21    // fputc: escribe un caracter
22    fputc('A', archivo);
23    fputc('\n', archivo);
24
25    // fwrite: escritura binaria (para datos en bruto)
26    int numeros[] = {10, 20, 30, 40, 50};
27    size_t elementos_escritos = fwrite(numeros, ←
28        sizeof(int), 5, archivo);
29    if (elementos_escritos != 5) {
30        fprintf(stderr, "Error al escribir datos ←
31            binarios\n");
32    }
33
34    fclose(archivo);
35    return 0;
36 }
```

Listing 75: Escritura en archivos

Lectura de Archivos

Análogamente, tenemos funciones para leer.

```

1 #include <stdio.h>
```

```
2
3 int main() {
4     FILE *archivo = fopen("salida.txt", "r");
5
6     if (archivo == NULL) {
7         perror("Error al abrir archivo");
8         return 1;
9     }
10
11     // fscanf: lectura formateada (como scanf)
12     int entero;
13     float flotante;
14     char cadena[50];
15     fscanf(archivo, "Entero: %d, Float: %f, String: ↵
16         %s\n", &entero, &flotante, cadena);
17     printf("Leido: %d, %f, %s\n", entero, flotante, cadena);
18
19     // fgets: lee una linea (MAS SEGURA que fscanf para ↵
20     strings)
21     char linea[100];
22     fgets(linea, sizeof(linea), archivo); // Lee hasta ↵
23     99 chars o hasta \n
24     printf("Linea: %s", linea); // fgets incluye el \n ↵
25     si hay espacio
26
27     fgets(linea, sizeof(linea), archivo);
28     printf("Linea: %s", linea);
29
30     // fgetc: lee un caracter
31     int c = fgetc(archivo); // Retorna int, no char, ↵
32     para poder retornar EOF
33     if (c != EOF) {
34         printf("Caracter: %c\n", c);
35     }
36
37     // fread: lectura binaria
38     int numeros_leidos[5];
39     size_t elementos_leidos = fread(numeros_leidos, ↵
40     sizeof(int), 5, archivo);
41     if (elementos_leidos == 5) {
42         printf("Numeros leidos: ");
43         for (int i = 0; i < 5; i++) {
44             printf("%d ", numeros_leidos[i]);
45         }
46         printf("\n");
47     }
48
49     fclose(archivo);
50     return 0;
51 }
```



```
45 }
```

Listing 76: Lectura de archivos

La Importancia de fclose()

Siempre debemos cerrar los archivos que abrimos por varias razones:

1. Los datos escritos con funciones como `fprintf()` o `fwrite()` pueden estar en un **búfer** en memoria, no en el disco físicamente. `fclose()` vacía (flusha) el búfer y garantiza que todos los datos se escriban.
2. Libera recursos del sistema (como descriptores de archivo) que son limitados.
3. Si el programa termina abruptamente sin cerrar, los datos pueden corromperse o perderse.

Funciones de Streams Avanzadas

Además de las funciones básicas, hay otras muy útiles para manipular streams.

Verificación de Fin de Archivo (EOF)

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo = fopen("datos.txt", "r");
5     if (archivo == NULL) {
6         perror("Error");
7         return 1;
8     }
9
10    char linea[100];
11    while (fgets(linea, sizeof(linea), archivo) != NULL) {
12        printf("Leido: %s", linea);
13    }
14
15    if (feof(archivo)) {
16        printf("Fin de archivo alcanzado.\n");
17    } else if (ferror(archivo)) {
18        printf("Error de lectura.\n");
19    }
20
21    fclose(archivo);
22    return 0;
23 }
```

Listing 77: Detectar fin de archivo

Posicionamiento en el Archivo: ftell(), fseek(), rewind()

Podemos movernos a cualquier posición dentro de un archivo (útil para archivos binarios o de acceso aleatorio).

```
1 #include <stdio.h>
2
3 int main() {
4     FILE *archivo = fopen("datos.txt", "r");
5     if (archivo == NULL) {
6         perror("Error");
7         return 1;
8     }
9
10    // Obtener la posición actual (desde el inicio)
11    long pos = ftell(archivo);
12    printf("Posición actual: %ld\n", pos); // 0
13
14    // Leer una línea
15    char linea[100];
16    fgets(linea, sizeof(linea), archivo);
17    printf("Leído: %s", linea);
18
19    pos = ftell(archivo);
20    printf("Posición ahora: %ld\n", pos);
21
22    // Volver al principio
23    rewind(archivo); // Equivalente a fseek(archivo, 0, ←
24    SEEK_SET);
25    printf("Después de rewind, posición: %ld\n", ←
26    ftell(archivo));
27
28    // fseek: moverse a una posición
29    fseek(archivo, 10, SEEK_SET); // Desde el principio ←
30    (SEEK_SET), byte 10
31    printf("Posición después de fseek(...,10,SEEK_SET): ←
32    %ld\n", ftell(archivo));
33
34    fseek(archivo, 5, SEEK_CUR); // 5 bytes desde la ←
35    posición actual
36    printf("Posición después de SEEK_CUR +5: %ld\n", ←
37    ftell(archivo));
38
39    fseek(archivo, -3, SEEK_END); // 3 bytes antes del ←
40    final
41    printf("Posición desde el final: %ld\n", ←
42    ftell(archivo));
43
44    fclose(archivo);
45    return 0;
46 }
```

```
38 }
```

Listing 78: Posicionamiento en archivos

Gestión de Errores

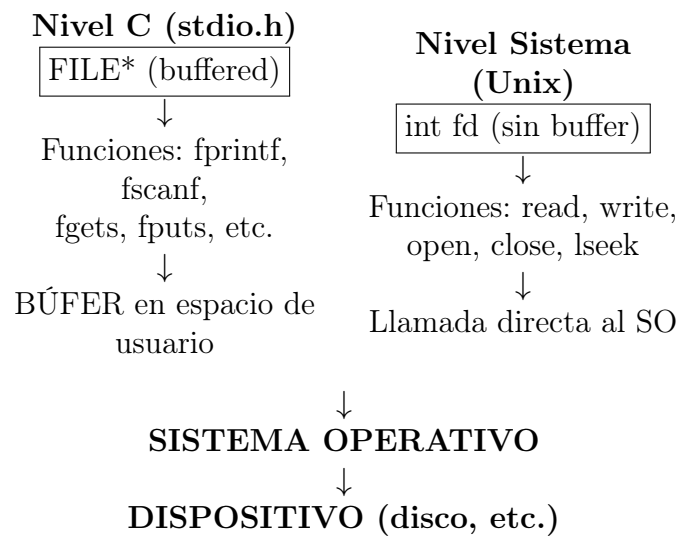
```
1 #include <stdio.h>
2 #include <errno.h>
3 #include <string.h>
4
5 int main() {
6     FILE *archivo = fopen("archivo_que_no_existe.txt", "r");
7
8     if (archivo == NULL) {
9         // Forma 1: perror() - muestra el mensaje de ↵
10         // error del sistema
11         perror("Error al abrir archivo");
12
13         // Forma 2: errno y strerror()
14         fprintf(stderr, "Error %d: %s\n", errno, ↵
15             strerror(errno));
16
17         return 1;
18     }
19
20     // ... operaciones ...
21
22     if (ferror(archivo)) {
23         fprintf(stderr, "Error durante operaciones de ↵
24         archivo.\n");
25         clearerr(archivo); // Limpiar el indicador de error
26     }
27
28     fclose(archivo);
29     return 0;
30 }
```

Listing 79: Manejo de errores en archivos

Funciones Tipo Unix de Manejo de I/O (Bajo Nivel)

Hasta ahora hemos usado las funciones de la biblioteca estándar de C (`stdio.h`), que son **portables** (funcionan en cualquier sistema que tenga un compilador de C). Sin embargo, en sistemas Unix/Linux (y también en Windows con ciertas bibliotecas), existen funciones de **bajo nivel** que son llamadas directas al sistema operativo. Estas funciones trabajan con **descriptores de archivo** (file descriptors), que son números enteros que identifican un archivo abierto, en lugar de con punteros `FILE*`.

Comparación: Streams vs. Descriptores



¿Por qué usar funciones de bajo nivel?

- **Control más fino:** Puedes hacer cosas que las funciones estándar no permiten (ej. operaciones de control de dispositivo específicas con `ioctl`).
- **Eficiencia:** Evitas el buffering doble (aunque el sistema también hace buffering).
- **Comunicación con el sistema:** Necesarias para trabajar con tuberías (pipes), sockets, y otros mecanismos de IPC (Comunicación entre Procesos).

Las funciones clave (en sistemas Unix/POSIX)

Estas funciones están declaradas en `<unistd.h>`, `<fcntl.h>`, etc.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>           // read, write, close
4 #include <fcntl.h>           // open, modos O_RDONLY, etc.
5 #include <string.h>
6 #include <errno.h>
7
8 int main() {
9     int fd; // File descriptor (un entero)
10    ssize_t bytes_leidos, bytes_escritos;
11    char buffer[100];
12
13    // 1. ABRIR ARCHIVO (similar a fopen)
14    // O_RDONLY: solo lectura
15    // O_WRONLY: solo escritura
16    // O_RDWR: lectura y escritura
17    // O_CREAT: crear si no existe
18    // O_TRUNC: truncar si existe
  
```

```

19 // O_APPEND: a adir al final
20 // Los modos (tercer argumento) son los permisos si se crea el archivo
21 fd = open("archivo.txt", O_RDWR | O_CREAT | O_TRUNC, 0644);
22
23 if (fd == -1) {
24     perror("Error al abrir archivo");
25     return 1;
26 }
27
28 // 2. ESCRIBIR (write)
29 char *mensaje = "Hola, mundo desde bajo nivel!\n";
30 bytes_escritos = write(fd, mensaje, strlen(mensaje));
31
32 if (bytes_escritos == -1) {
33     perror("Error al escribir");
34     close(fd);
35     return 1;
36 }
37 printf("Escritos %ld bytes\n", bytes_escritos);
38
39 // 3. POSICIONAMIENTO (lseek)
40 // SEEK_SET: desde el inicio
41 // SEEK_CUR: desde la posicion actual
42 // SEEK_END: desde el final
43 off_t pos = lseek(fd, 0, SEEK_SET); // Volver al principio
44 if (pos == -1) {
45     perror("Error en lseek");
46     close(fd);
47     return 1;
48 }
49
50 // 4. LEER (read)
51 bytes_leidos = read(fd, buffer, sizeof(buffer) - 1);
52 if (bytes_leidos == -1) {
53     perror("Error al leer");
54     close(fd);
55     return 1;
56 }
57
58 buffer[bytes_leidos] = '\0'; // Asegurar terminacion nula
59 printf("Leidos %ld bytes: %s", bytes_leidos, buffer);
60
61 // 5. CERRAR (close)
62 close(fd);
63

```

```

64 // --- Ejemplo de lectura de entrada estandar (stdin ←
    tiene fd = 0) ---
65 printf("\nEscribe algo (max 50 chars): ");
66 fflush(stdout); // Asegurar que el mensaje se muestra
67
68 bytes_leidos = read(0, buffer, 50); // 0 es ←
    STDIN_FILENO
69 if (bytes_leidos > 0) {
70     buffer[bytes_leidos] = '\0';
71     printf("Leido del teclado: %s", buffer);
72 }
73
74 return 0;
75 }

```

Listing 80: Funciones de I/O de bajo nivel

Tabla Comparativa: Funciones Estándar vs. Unix

Operación	Estándar (stdio.h)	Unix (unistd.h, fcntl.h)
Abrir archivo	fopen()	open()
Cerrar archivo	fclose()	close()
Leer	fread(), fscanf(), fgets()	read()
Escribir	fwrite(), fprintf(), fputs()	write()
Posicionarse	fseek(), ftell(), rewind()	lseek()
Manejo de errores	ferror(), perror()	errno, perror()
Stream estándar	stdin, stdout, stderr	0 (STDIN_FILENO), 1 (STDOUT_FILENO), 2 (STDERR_FILENO)
Buffering	Sí (controlable con setbuf())	No (depende del sistema operativo)
Portabilidad	Alta (C estándar)	Baja (solo sistemas Unix/POSIX)

Ejemplo: Copiar un archivo con funciones de bajo nivel

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <fcntl.h>
5
6 #define BUFFER_SIZE 4096
7
8 int main(int argc, char *argv[]) {
9     if (argc != 3) {
10         fprintf(stderr, "Uso: %s origen destino\n", ←
            argv[0]);
11         return 1;
12     }
13
14     int fd_origen = open(argv[1], O_RDONLY);

```

```

15     if (fd_origen == -1) {
16         perror("Error abriendo origen");
17         return 1;
18     }
19
20     int fd_destino = open(argv[2], O_WRONLY | O_CREAT | ↵
21         O_TRUNC, 0644);
22     if (fd_destino == -1) {
23         perror("Error creando destino");
24         close(fd_origen);
25         return 1;
26     }
27
28     char buffer[BUFFER_SIZE];
29     ssize_t bytes_leidos, bytes_escritos;
30
31     while ((bytes_leidos = read(fd_origen, buffer, ↵
32         BUFFER_SIZE)) > 0) {
33         bytes_escritos = write(fd_destino, buffer, ↵
34             bytes_leidos);
35         if (bytes_escritos != bytes_leidos) {
36             perror("Error escribiendo");
37             close(fd_origen);
38             close(fd_destino);
39             return 1;
40         }
41     }
42
43     if (bytes_leidos == -1) {
44         perror("Error leyendo");
45     }
46
47     close(fd_origen);
48     close(fd_destino);
49
50     printf("Archivo copiado exitosamente.\n");
51     return 0;
52 }

```

Listing 81: Programa cp simplificado

Resumen de la Unidad VIII

Hemos explorado en profundidad cómo los programas en C se comunican con el exterior.

- **Streams:** Abstracción unificadora para cualquier flujo de datos. La biblioteca estándar trabaja con FILE*.

- **Entrada/Salida estándar:** `stdin`, `stdout`, `stderr` son streams preabiertos para consola.
- **Archivos:** Conjunto de datos persistentes. Se acceden siguiendo el patrón abrir-operar-cerrar.
- **Modos de apertura:** `r`, `w`, `a`, y sus variantes con `+` y `b` para binario.
- **Funciones de stream:** `fopen`, `fclose`, `fprintf`, `fscanf`, `fgets`, `fputs`, `fread`, `fwrite`, `fseek`, `ftell`, `rewind`, `feof`, `ferror`.
- **Funciones de bajo nivel (Unix):** `open`, `close`, `read`, `write`, `lseek`. Trabajan con descriptores de archivo (enteros). Ofrecen más control pero menos portabilidad.
- **Gestión de errores:** `perror` y `strerror(errno)` para obtener mensajes descriptivos de los errores del sistema.

Con estas herramientas, nuestros programas ya pueden interactuar plenamente con el sistema de archivos, procesar datos persistentes y comunicarse con otros programas a través de tuberías y redirecciones. En la siguiente unidad, aplicaremos estos conocimientos para explorar algoritmos clásicos de ordenamiento y búsqueda.

Unidad IX: Algoritmos de Ordenamiento y Búsqueda - Organizando y Encontrando Datos

En las unidades anteriores, hemos aprendido a almacenar datos en arrays y a manipularlos. Sin embargo, en el mundo real, los datos rara vez vienen en el orden que necesitamos. Imagina una guía telefónica sin ordenar, o una lista de estudiantes de la que queremos encontrar a alguien por su nombre. Sería ineficiente hasta el punto de ser inútil.

Esta unidad está dedicada a dos familias de algoritmos fundamentales en computación:

- **Ordenamiento:** Organizar los elementos de una lista según un criterio (numérico, alfabético, etc.).
- **Búsqueda:** Encontrar un elemento específico dentro de una colección de datos.

Entender estos algoritmos no solo es crucial para resolver problemas prácticos, sino que también nos enseña a pensar en términos de **eficiencia** y **complejidad computacional**.

Conceptos Preliminares

Antes de sumergirnos en los algoritmos, debemos establecer un marco común.

¿Por qué ordenar?

- **Búsqueda eficiente:** En una lista ordenada, podemos encontrar elementos mucho más rápido (búsqueda binaria vs. búsqueda secuencial).
- **Presentación de datos:** Los humanos entendemos mejor la información ordenada.
- **Eliminación de duplicados:** Es más fácil detectar duplicados en una lista ordenada.
- **Optimización de otros algoritmos:** Muchos algoritmos (como ciertos de grafos) asumen datos ordenados.

Estabilidad en ordenamiento

Un algoritmo de ordenamiento es **estable** si mantiene el orden relativo de elementos con claves iguales. Por ejemplo, si ordenamos una lista de personas por edad, y dos personas tienen la misma edad, un algoritmo estable mantendrá el orden en que aparecían originalmente.

Notación de Complejidad (Breve introducción)

Para comparar algoritmos, usamos la **notación Big O** que describe cómo crece el tiempo de ejecución (o el uso de memoria) a medida que crece el tamaño de la entrada (n).

- **$O(1)$** : Tiempo constante (no depende de n).
- **$O(\log n)$** : Tiempo logarítmico (muy eficiente).
- **$O(n)$** : Tiempo lineal (crece proporcionalmente a n).
- **$O(n \log n)$** : Tiempo linealítmico (eficiente para ordenar).
- **$O(n^2)$** : Tiempo cuadrático (ineficiente para grandes n).

Búsqueda Secuencial (Lineal)

Es el algoritmo de búsqueda más simple. Recorremos el elemento por elemento desde el principio hasta que encontramos el que buscamos o llegamos al final.

Visualización de Búsqueda Secuencial

Array: [5, 2, 8, 1, 9, 3]

Buscar: 8

Paso 1: [5] -> ¿Es 8? No.

Paso 2: [5, 2] -> ¿Es 8? No.

Paso 3: [5, 2, 8] -> ¿Es 8? ¡SÍ! Encontrado en índice 2.

Implementación en C

```

1 #include <stdio.h>
2
3 // Busca 'valor' en 'arr' de tamaño 'n'
4 // Retorna el índice si lo encuentra, -1 si no
5 int busqueda_secuencial(int arr[], int n, int valor) {
6     for (int i = 0; i < n; i++) {
7         if (arr[i] == valor) {
8             return i; // Encontrado
9         }
10    }

```

```
10     }
11     return -1;    // No encontrado
12 }
13
14 int main() {
15     int numeros[] = {5, 2, 8, 1, 9, 3};
16     int n = sizeof(numeros) / sizeof(numeros[0]);
17     int buscar = 8;
18
19     int indice = busqueda_secuencial(numeros, n, buscar);
20
21     if (indice != -1) {
22         printf("Valor %d encontrado en indice %d\n", ←
23             buscar, indice);
24     } else {
25         printf("Valor %d no encontrado\n", buscar);
26     }
27
28     return 0;
29 }
```

Listing 1: Búsqueda secuencial

Análisis

- **Complejidad temporal:**
 - Mejor caso: $O(1)$ (el elemento está al principio).
 - Peor caso: $O(n)$ (el elemento está al final o no está).
 - Caso promedio: $O(n)$.
- **Ventajas:** Simple, funciona en arrays no ordenados.
- **Desventajas:** Ineficiente para arrays grandes.

Búsqueda Binaria

La búsqueda binaria es mucho más rápida, pero requiere que el array esté **ordenado**. Funciona dividiendo repetidamente el intervalo de búsqueda a la mitad.

Visualización de Búsqueda Binaria

Array ordenado: [1, 3, 5, 7, 9, 11, 13, 15, 17]

Buscar: 11

Paso 1: [1, 3, 5, 7, 9, 11, 13, 15, 17]
 ^ bajo ^ medio ^ alto


```
26         return -1; // Caso base: no encontrado
27     }
28
29     int medio = bajo + (alto - bajo) / 2;
30
31     if (arr[medio] == valor) {
32         return medio;
33     }
34     else if (arr[medio] < valor) {
35         return busqueda_binaria_recursiva(arr, medio + 1, ↵
36             alto, valor);
37     }
38     else {
39         return busqueda_binaria_recursiva(arr, bajo, ↵
40             medio - 1, valor);
41     }
42 }
43
44 int main() {
45     int numeros[] = {1, 3, 5, 7, 9, 11, 13, 15, 17};
46     int n = sizeof(numeros) / sizeof(numeros[0]);
47     int buscar = 11;
48
49     int indice1 = busqueda_binaria_iterativa(numeros, n, ↵
50         buscar);
51     int indice2 = busqueda_binaria_recursiva(numeros, 0, ↵
52         n-1, buscar);
53
54     printf("Iterativa: Valor %d en indice %d\n", buscar, ↵
55         indice1);
56     printf("Recursiva: Valor %d en indice %d\n", buscar, ↵
57         indice2);
58
59     return 0;
60 }
```

Listing 2: Búsqueda binaria iterativa

Análisis

- **Complejidad temporal:** $O(\log n)$ en todos los casos (mejor, peor y promedio).
- **Ventajas:** Extremadamente rápida incluso para arrays muy grandes.
- **Desventajas:** Requiere que el array esté ordenado.

Ordenamiento por Inserción (Insertion Sort)

Es uno de los algoritmos más intuitivos. Funciona como ordenamos naipes en la mano: tomamos un elemento y lo insertamos en la posición correcta entre los ya ordenados.

Visualización de Inserción

Array inicial: [5, 2, 4, 6, 1, 3]

Paso 1 (i=1, valor=2): Comparar con 5. $2 < 5$, desplazar 5 a la derecha.

[5, 5, 4, 6, 1, 3] -> Insertar 2 en posición 0.

[2, 5, 4, 6, 1, 3]

Paso 2 (i=2, valor=4): Comparar con 5. $4 < 5$, desplazar 5.

[2, 5, 5, 6, 1, 3] -> Comparar con 2. $4 > 2$, insertar en posición 1.

[2, 4, 5, 6, 1, 3]

Paso 3 (i=3, valor=6): Comparar con 5. $6 > 5$, no hay desplazamiento.

[2, 4, 5, 6, 1, 3]

Paso 4 (i=4, valor=1): Comparar con 6,5,4,2. Desplazar todos y insertar al inicio.

[1, 2, 4, 5, 6, 3]

Paso 5 (i=5, valor=3): Comparar con 6,5,4. Insertar entre 2 y 4.

[1, 2, 3, 4, 5, 6] -> Ordenado!

Implementación en C

```

1 #include <stdio.h>
2
3 void insertion_sort(int arr[], int n) {
4     for (int i = 1; i < n; i++) {
5         int valor_actual = arr[i];
6         int j = i - 1;
7
8         // Desplazar elementos mayores que valor_actual a ←
           la derecha
9         while (j >= 0 && arr[j] > valor_actual) {
10             arr[j + 1] = arr[j];
11             j--;
12         }
13
14         // Insertar valor_actual en su lugar
15         arr[j + 1] = valor_actual;
16
17         // Mostrar paso a paso (opcional)
18         printf("Paso %d: ", i);

```

```
19     for (int k = 0; k < n; k++) {
20         printf("%d ", arr[k]);
21     }
22     printf("\n");
23 }
24 }
25
26 int main() {
27     int numeros[] = {5, 2, 4, 6, 1, 3};
28     int n = sizeof(numeros) / sizeof(numeros[0]);
29
30     printf("Array original: ");
31     for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
32     printf("\n\n");
33
34     insertion_sort(numeros, n);
35
36     printf("\nArray ordenado: ");
37     for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
38     printf("\n");
39
40     return 0;
41 }
```

Listing 3: Ordenamiento por inserción

Análisis

- **Complejidad temporal:**
 - Mejor caso (array ya ordenado): $O(n)$ (solo compara, no desplaza).
 - Peor caso (array en orden inverso): $O(n^2)$.
 - Caso promedio: $O(n^2)$.
- **Complejidad espacial:** $O(1)$ (in-place, no usa memoria adicional).
- **Estabilidad:** Sí, es estable.
- **Ventajas:** Simple, eficiente para arrays pequeños o casi ordenados, ordena en línea (puede ordenar mientras recibe datos).
- **Desventajas:** Ineficiente para arrays grandes.

Ordenamiento por Selección (Selection Sort)

Este algoritmo funciona seleccionando repetidamente el elemento más pequeño (o más grande) del array desordenado y colocándolo al principio.

Visualización de Selección

Array inicial: [64, 25, 12, 22, 11]

Paso 1: Encontrar el mínimo entre [0..4] (11). Intercambiar con posición 0.
[11, 25, 12, 22, 64]

Paso 2: Encontrar el mínimo entre [1..4] (12). Intercambiar con posición 1.
[11, 12, 25, 22, 64]

Paso 3: Encontrar el mínimo entre [2..4] (22). Intercambiar con posición 2.
[11, 12, 22, 25, 64]

Paso 4: Encontrar el mínimo entre [3..4] (25). Ya está en su lugar.
[11, 12, 22, 25, 64]

Paso 5: Solo queda un elemento, fin.

Implementación en C

```

1 #include <stdio.h>
2
3 void selection_sort(int arr[], int n) {
4     for (int i = 0; i < n - 1; i++) {
5         // Encontrar el índice del mínimo elemento en ↔
6         // arr[i..n-1]
7         int indice_minimo = i;
8         for (int j = i + 1; j < n; j++) {
9             if (arr[j] < arr[indice_minimo]) {
10                 indice_minimo = j;
11             }
12         }
13
14         // Intercambiar el mínimo encontrado con el ↔
15         // elemento en posición i
16         if (indice_minimo != i) {
17             int temp = arr[i];
18             arr[i] = arr[indice_minimo];
19             arr[indice_minimo] = temp;
20         }
21
22         // Mostrar paso a paso
23         printf("Paso %d: ", i+1);
24         for (int k = 0; k < n; k++) {
25             printf("%d ", arr[k]);
26         }
27         printf("\n");
28     }
29 }
```



```
27 }
28
29 int main() {
30     int numeros[] = {64, 25, 12, 22, 11};
31     int n = sizeof(numeros) / sizeof(numeros[0]);
32
33     printf("Array original: ");
34     for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
35     printf("\n\n");
36
37     selection_sort(numeros, n);
38
39     printf("\nArray ordenado: ");
40     for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
41     printf("\n");
42
43     return 0;
44 }
```

Listing 4: Ordenamiento por selección

Análisis

- **Complejidad temporal:** $O(n^2)$ en todos los casos (siempre hace el mismo número de comparaciones).
- **Complejidad espacial:** $O(1)$ (in-place).
- **Estabilidad:** No es estable (el intercambio puede alterar el orden de elementos iguales).
- **Ventajas:** Simple, hace el mínimo número de intercambios ($n-1$), útil cuando escribir en memoria es costoso.
- **Desventajas:** Siempre es $O(n^2)$, incluso si el array está ordenado.

Ordenamiento Shell (Shell Sort)

El Shell Sort es una optimización del ordenamiento por inserción. Fue creado por Donald Shell en 1959. La idea es permitir el intercambio de elementos que están lejanos, no solo adyacentes.

Funciona dividiendo el array original en subarrays que consisten en elementos separados por un "gap" (hueco). Estos subarrays se ordenan con inserción. Luego se reduce el gap y se repite, hasta que el gap es 1, que es un ordenamiento por inserción normal, pero para entonces el array ya está casi ordenado".

Visualización de Shell Sort (con gap decreciente)

Array inicial: [9, 8, 7, 6, 5, 4, 3, 2, 1]

Gap = 4 (floor(9/2)): Subarrays con índices separados por 4:

Subarray1 (índices 0,4,8): [9, 5, 1] -> ordenado: [1, 5, 9]

Subarray2 (índices 1,5): [8, 4] -> ordenado: [4, 8]

Subarray3 (índices 2,6): [7, 3] -> ordenado: [3, 7]

Subarray4 (índices 3,7): [6, 2] -> ordenado: [2, 6]

Resultado tras gap=4: [1, 4, 3, 2, 5, 8, 7, 6, 9]

Gap = 2: Subarrays con índices separados por 2:

Subarray1 (índices 0,2,4,6,8): [1, 3, 5, 7, 9] -> ya ordenado

Subarray2 (índices 1,3,5,7): [4, 2, 8, 6] -> ordenado: [2, 4, 6, 8]

Resultado: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Gap = 1: Inserción normal sobre array casi ordenado (muy rapido)

Implementación en C

```

1 #include <stdio.h>
2
3 void shell_sort(int arr[], int n) {
4     // Comenzar con un gap grande, luego reducirlo
5     for (int gap = n / 2; gap > 0; gap /= 2) {
6
7         // Hacer una ordenacion por insercion para este gap
8         for (int i = gap; i < n; i++) {
9             int temp = arr[i];
10            int j;
11
12            // Desplazar elementos separados por gap
13            for (j = i; j >= gap && arr[j - gap] > temp; ←
14                j -= gap) {
15                arr[j] = arr[j - gap];
16            }
17
18            arr[j] = temp;
19        }
20
21        // Mostrar progreso (opcional)
22        printf("Gap = %d: ", gap);
23        for (int k = 0; k < n; k++) {
24            printf("%d ", arr[k]);
25        }
26        printf("\n");
27    }
28 }

```

```
29 int main() {
30     int numeros[] = {9, 8, 7, 6, 5, 4, 3, 2, 1};
31     int n = sizeof(numeros) / sizeof(numeros[0]);
32
33     printf("Array original: ");
34     for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
35     printf("\n\n");
36
37     shell_sort(numeros, n);
38
39     printf("\nArray ordenado: ");
40     for (int i = 0; i < n; i++) printf("%d ", numeros[i]);
41     printf("\n");
42
43     return 0;
44 }
```

Listing 5: Ordenamiento Shell

Análisis

- **Complejidad temporal:** Depende de la secuencia de gaps. Para la secuencia original (dividir por 2), es aproximadamente $O(n^2)$ en el peor caso, pero en la práctica es mucho mejor. Con secuencias optimizadas, puede llegar a $O(n \log^2 n)$ o incluso $O(n \log n)$ en algunos casos.
- **Complejidad espacial:** $O(1)$ (in-place).
- **Estabilidad:** No es estable.
- **Ventajas:** Mucho más rápido que inserción y selección para arrays medianos/grandes. No usa memoria adicional.
- **Desventajas:** Su análisis teórico es complejo. El rendimiento depende críticamente de la secuencia de gaps elegida.

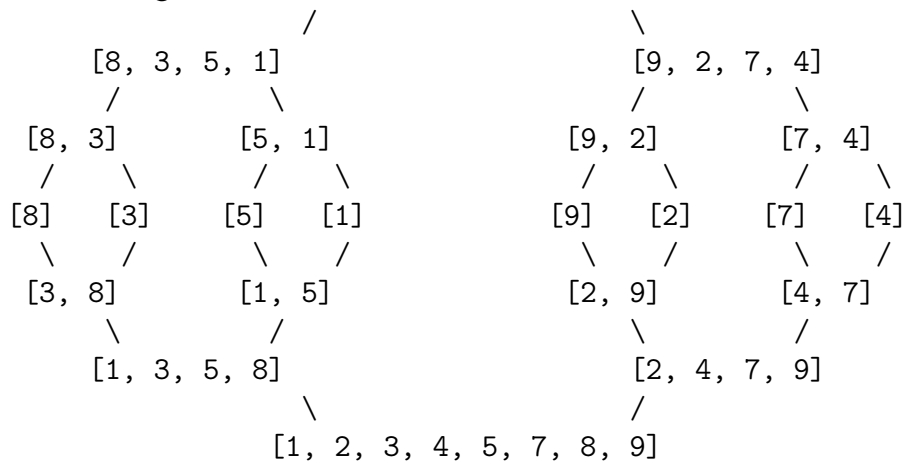
Dividir y Conquistar (Divide and Conquer)

La estrategia de "Dividir y Conquistar" es uno de los paradigmas más poderosos en el diseño de algoritmos. Consiste en tres pasos:

1. **Dividir:** El problema se divide en subproblemas más pequeños, generalmente de la misma naturaleza.
2. **Conquistar:** Los subproblemas se resuelven recursivamente.
3. **Combinar:** Las soluciones de los subproblemas se combinan para obtener la solución del problema original.

Visualización de Dividir y Conquistar

Problema original: [8, 3, 5, 1, 9, 2, 7, 4]



Algoritmos Basados en Dividir y Conquistar

Si bien el programa de estudios no especifica MergeSort o QuickSort en esta unidad, es imposible hablar de "Dividir y Conquistar" sin mencionarlos. Estos algoritmos son la aplicación más clara de esta estrategia.

Merge Sort (Ordenamiento por Mezcla)

Merge Sort es un algoritmo de ordenamiento estable que sigue el paradigma de dividir y conquistar.

1. **Dividir:** El array se divide en dos mitades (aproximadamente iguales).
2. **Conquistar:** Cada mitad se ordena recursivamente aplicando Merge Sort.
3. **Combinar:** Las dos mitades ordenadas se fusionan (merge) en un único array ordenado.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Funcion para fusionar dos subarrays ordenados
5 void merge(int arr[], int izquierda, int medio, int ←
    derecha) {
6     int i, j, k;
7     int n1 = medio - izquierda + 1;
8     int n2 = derecha - medio;
9
10    // Crear arrays temporales
11    int *L = (int*)malloc(n1 * sizeof(int));
12    int *R = (int*)malloc(n2 * sizeof(int));
  
```

```
13
14 // Copiar datos a los arrays temporales
15 for (i = 0; i < n1; i++)
16     L[i] = arr[izquierda + i];
17 for (j = 0; j < n2; j++)
18     R[j] = arr[medio + 1 + j];
19
20 // Fusionar los arrays temporales de vuelta a ↵
21   arr[izquierda..derecha]
22 i = 0; // Indice inicial del primer subarray
23 j = 0; // Indice inicial del segundo subarray
24 k = izquierda; // Indice inicial del array fusionado
25
26 while (i < n1 && j < n2) {
27     if (L[i] <= R[j]) {
28         arr[k] = L[i];
29         i++;
30     } else {
31         arr[k] = R[j];
32         j++;
33     }
34     k++;
35 }
36
37 // Copiar los elementos restantes de L[], si los hay
38 while (i < n1) {
39     arr[k] = L[i];
40     i++;
41     k++;
42 }
43
44 // Copiar los elementos restantes de R[], si los hay
45 while (j < n2) {
46     arr[k] = R[j];
47     j++;
48     k++;
49 }
50
51 free(L);
52 free(R);
53 }
54
55 // Funcion principal de Merge Sort
56 void merge_sort(int arr[], int izquierda, int derecha) {
57     if (izquierda < derecha) {
58         // Calcula el punto medio
59         int medio = izquierda + (derecha - izquierda) / 2;
60         // Ordenar las dos mitades recursivamente
```

```

61     merge_sort(arr, izquierda, medio);
62     merge_sort(arr, medio + 1, derecha);
63
64     // Fusionar las mitades ordenadas
65     merge(arr, izquierda, medio, derecha);
66 }
67 }
68
69 void print_array(int arr[], int n) {
70     for (int i = 0; i < n; i++)
71         printf("%d ", arr[i]);
72     printf("\n");
73 }
74
75 int main() {
76     int arr[] = {38, 27, 43, 3, 9, 82, 10};
77     int n = sizeof(arr) / sizeof(arr[0]);
78
79     printf("Array original: ");
80     print_array(arr, n);
81
82     merge_sort(arr, 0, n - 1);
83
84     printf("Array ordenado: ");
85     print_array(arr, n);
86
87     return 0;
88 }

```

Listing 6: Merge Sort en C

Análisis de Merge Sort

- **Complejidad temporal:** $O(n \log n)$ en todos los casos (mejor, peor y promedio).
- **Complejidad espacial:** $O(n)$ (requiere memoria adicional para los arrays temporales).
- **Estabilidad:** Sí, es estable.
- **Ventajas:** Rendimiento garantizado $O(n \log n)$, estable.
- **Desventajas:** Requiere memoria extra $O(n)$.

Quick Sort (Ordenamiento Rápido)

Quick Sort es otro algoritmo de dividir y conquistar, pero con un enfoque diferente. Es el algoritmo de ordenamiento más utilizado en la práctica.

1. **Dividir:** Se selecciona un elemento como pivote. El array se particiona de modo que todos los elementos menores que el pivote estén a su izquierda, y todos los mayores a su derecha.
2. **Conquistar:** Las subarrays izquierdo y derecho se ordenan recursivamente aplicando Quick Sort.
3. **Combinar:** No hay paso de combinación, ya que después de la partición, los subarrays están en el lugar correcto relativo al pivote.

```
1 #include <stdio.h>
2
3 // Funcion para intercambiar dos elementos
4 void swap(int *a, int *b) {
5     int temp = *a;
6     *a = *b;
7     *b = temp;
8 }
9
10 // Funcion de particion: coloca el pivote en su posicion ←
    correcta
11 // y retorna su indice
12 int partition(int arr[], int bajo, int alto) {
13     // Elegir el ultimo elemento como pivote
14     int pivote = arr[alto];
15
16     // Indice del elemento mas pequeno (indica la ←
        posicion correcta del pivote)
17     int i = (bajo - 1);
18
19     for (int j = bajo; j <= alto - 1; j++) {
20         // Si el elemento actual es menor o igual que el ←
            pivote
21         if (arr[j] <= pivote) {
22             i++; // Incrementar el indice del elemento ←
                mas pequeno
23             swap(&arr[i], &arr[j]);
24         }
25     }
26     swap(&arr[i + 1], &arr[alto]);
27     return (i + 1);
28 }
29
30 // Funcion principal de Quick Sort
31 void quick_sort(int arr[], int bajo, int alto) {
32     if (bajo < alto) {
33         // pi es el indice de particion, arr[pi] ya esta ←
            en su lugar
34         int pi = partition(arr, bajo, alto);
```

```

35
36     // Ordenar recursivamente los elementos antes y ←
37     // después de la partición
38     quick_sort(arr, bajo, pi - 1);
39     quick_sort(arr, pi + 1, alto);
40 }
41
42 void print_array(int arr[], int n) {
43     for (int i = 0; i < n; i++)
44         printf("%d ", arr[i]);
45     printf("\n");
46 }
47
48 int main() {
49     int arr[] = {10, 7, 8, 9, 1, 5};
50     int n = sizeof(arr) / sizeof(arr[0]);
51
52     printf("Array original: ");
53     print_array(arr, n);
54
55     quick_sort(arr, 0, n - 1);
56
57     printf("Array ordenado: ");
58     print_array(arr, n);
59
60     return 0;
61 }

```

Listing 7: Quick Sort en C

Análisis de Quick Sort

- **Complejidad temporal:**
 - Mejor caso: $O(n \log n)$ (cuando el pivote siempre divide el array en mitades iguales).
 - Peor caso: $O(n^2)$ (cuando el pivote es siempre el mínimo o máximo, ej. array ya ordenado y se elige el último como pivote).
 - Caso promedio: $O(n \log n)$.
- **Complejidad espacial:** $O(\log n)$ (por la pila de recursión, es in-place).
- **Estabilidad:** No es estable (el intercambio puede alterar el orden de elementos iguales).
- **Ventajas:** Muy rápido en la práctica, in-place (no requiere memoria extra significativa).
- **Desventajas:** Peor caso $O(n^2)$, no es estable.

Comparativa de Algoritmos de Ordenamiento

Algoritmo	Mejor Caso	Peor Caso	Promedio	Espacio
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell	$O(n \log n)$	$O(n^2)$	Depende	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$

Aplicación Práctica: Búsqueda Binaria en Array Ordenado

Ahora que sabemos ordenar, podemos aplicar búsqueda binaria de manera eficiente.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Funciones de ordenamiento (seleccion, por simplicidad)
5 void selection_sort(int arr[], int n) {
6     for (int i = 0; i < n-1; i++) {
7         int min_idx = i;
8         for (int j = i+1; j < n; j++)
9             if (arr[j] < arr[min_idx])
10                 min_idx = j;
11         if (min_idx != i) {
12             int temp = arr[i];
13             arr[i] = arr[min_idx];
14             arr[min_idx] = temp;
15         }
16     }
17 }
18
19 // Búsqueda binaria
20 int busqueda_binaria(int arr[], int n, int valor) {
21     int bajo = 0, alto = n-1;
22     while (bajo <= alto) {
23         int medio = bajo + (alto - bajo) / 2;
24         if (arr[medio] == valor)
25             return medio;
26         else if (arr[medio] < valor)
27             bajo = medio + 1;
28         else
29             alto = medio - 1;
30     }
31     return -1;
32 }
33
34 int main() {

```

```

35     int datos[] = {42, 17, 8, 99, 23, 56, 11, 34, 77, 5};
36     int n = sizeof(datos) / sizeof(datos[0]);
37     int buscar = 23;
38
39     printf("Datos originales: ");
40     for (int i = 0; i < n; i++) printf("%d ", datos[i]);
41     printf("\n");
42
43     // Ordenar
44     selection_sort(datos, n);
45     printf("Datos ordenados: ");
46     for (int i = 0; i < n; i++) printf("%d ", datos[i]);
47     printf("\n");
48
49     // Buscar
50     int indice = busqueda_binaria(datos, n, buscar);
51     if (indice != -1) {
52         printf("Valor %d encontrado en indice %d\n", ←
53             buscar, indice);
54     } else {
55         printf("Valor %d no encontrado\n", buscar);
56     }
57
58     return 0;
59 }

```

Listing 8: Sistema completo: ordenar y buscar

Resumen de la Unidad IX

Hemos explorado los algoritmos fundamentales para organizar y encontrar datos.

■ Búsqueda:

- **Secuencial:** Simple, $O(n)$, funciona en arrays no ordenados.
- **Binaria:** Muy eficiente $O(\log n)$, pero requiere array ordenado.

■ Ordenamiento:

- **Inserción:** $O(n^2)$, bueno para arrays pequeños o casi ordenados, estable.
- **Selección:** $O(n^2)$ siempre, minimiza intercambios, inestable.
- **Shell:** Mejora de inserción, mucho más rápido, $O(n \log n)$ en la práctica.
- **Merge Sort:** $O(n \log n)$ garantizado, estable, requiere memoria extra.
- **Quick Sort:** $O(n \log n)$ promedio, in-place, más rápido en la práctica, pero $O(n^2)$ en el peor caso.

- **Dividir y conquistar:** Paradigma poderoso para diseñar algoritmos eficientes (Merge Sort, Quick Sort, y muchos otros).

Estos algoritmos no solo son herramientas prácticas, sino que también nos enseñan a pensar en términos de eficiencia y a elegir la estrategia adecuada según el contexto. En la siguiente unidad, exploraremos cómo estos conceptos evolucionan en el paradigma de la programación orientada a objetos.

Unidad X: Introducción a la Programación Orientada a Objetos - Un Nuevo Paradigma

Hasta ahora, hemos trabajado exclusivamente con C, un lenguaje **procedural** (o estructurado). En este paradigma, nos enfocamos en las **funciones** que realizan tareas y en los **datos** que esas funciones procesan. Los datos y las funciones están separados.

Sin embargo, a medida que los programas crecen en complejidad, este enfoque puede volverse difícil de manejar. Surge entonces un nuevo paradigma: la **Programación Orientada a Objetos (POO)**. En POO, combinamos datos y funciones en una sola entidad llamada **objeto**. Los objetos son como cápsulas que contienen tanto información (atributos) como comportamientos (métodos).

En esta unidad, exploraremos los conceptos fundamentales de la POO. Aunque C no es un lenguaje orientado a objetos, sentaremos las bases conceptuales y luego haremos una introducción a **C++**, que es una extensión de C que sí soporta POO.

Paradigma Procedural vs. Orientado a Objetos

PROCEDURAL:

+-----+		
Datos (struct)	<---- Separados ----	Funciones que operan
- nombre		sobre los datos
- edad		calcular_imc(Persona p)
- altura		imprimir_persona(Persona p)
+-----+		cambiar_nombre(Persona p, char *nuevo)

ORIENTADO A OBJETOS:

+-----+		
	OBJETO	
	+-----+	
	ATRIBUTOS (datos)	
	- nombre	
	- edad	
	- altura	
	+-----+	
	MÉTODOS (funciones)	
	+ calcular_imc()	

```

| | + imprimir() | |
| | + cambiar_nombre() | |
| +-----+ |
+-----+

```

Conceptos Fundamentales de la POO

La POO se basa en cuatro pilares fundamentales:

Abstracción de Datos

La **abstracción** consiste en simplificar la realidad modelando solo las características esenciales de un objeto, ignorando los detalles irrelevantes. Es como cuando usamos un celular: no necesitamos saber cómo funciona internamente el procesador para hacer una llamada; solo necesitamos conocer la interfaz (la pantalla, los botones).

En programación, la abstracción nos permite:

- Ocultar la complejidad interna.
- Exponer solo las operaciones relevantes (la interfaz pública).
- Cambiar la implementación interna sin afectar a quienes usan el objeto.

Ya vimos un ejemplo de abstracción en C con los Tipos de Datos Abstractos (TDA) en la Unidad IV, donde ocultamos la estructura interna de un **Punto** y solo expusimos funciones para manipularlo. La POO lleva este concepto al siguiente nivel, integrando datos y operaciones en una sola entidad.

Clases y Objetos

- **Clase:** Es una "plantilla."o "molde"que define las características (atributos) y comportamientos (métodos) que tendrán los objetos de ese tipo. Es un tipo de dato definido por el programador. No ocupa memoria; es solo una definición.
- **Objeto:** Es una **instancia** concreta de una clase. Tiene valores específicos para sus atributos y ocupa memoria. Podemos tener muchos objetos de la misma clase.

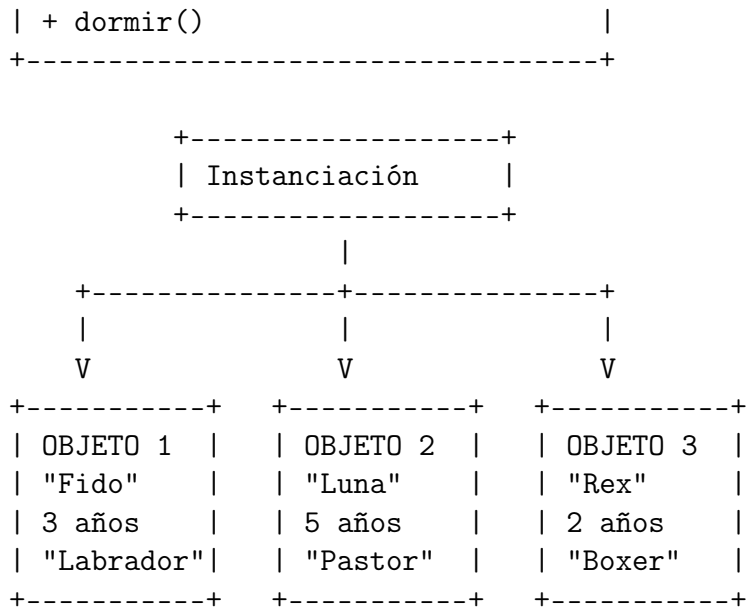
Clase vs. Objetos

CLASE: "Perro"

```

+-----+
| Atributos: |
| - nombre |
| - edad |
| - raza |
| Métodos: |
| + ladrar() |
| + comer() |

```



Variables de Instancia (Atributos)

Son las variables que pertenecen a cada objeto individualmente. Cada objeto tiene su propia copia de estas variables. Definen el **estado** del objeto.

Métodos (Funciones Miembro)

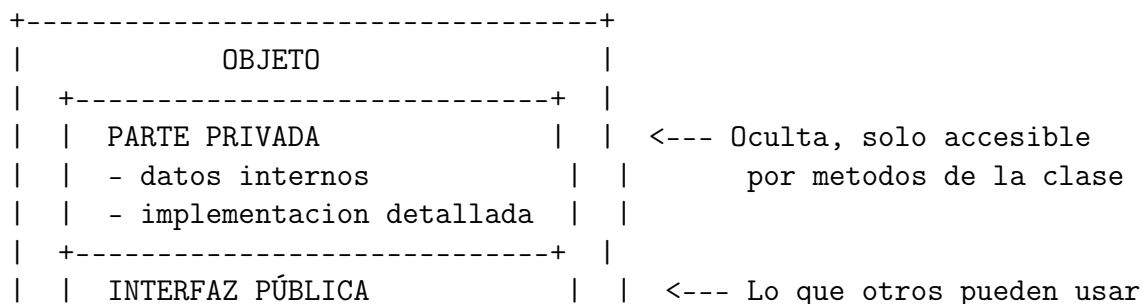
Son las funciones que pertenecen a la clase y definen el **comportamiento** de los objetos. Pueden acceder y modificar las variables de instancia.

Encapsulamiento

Es el mecanismo que **oculta** los detalles internos de un objeto y protege los datos de accesos no autorizados. En POO, esto se logra mediante modificadores de acceso:

- **public:** Accesible desde cualquier parte del programa.
- **private:** Accesible solo desde dentro de la misma clase.
- **protected:** Accesible desde la clase y sus clases derivadas (herencia).

Visualización del Encapsulamiento



```

|  |  + metodo_publico1()          |  |
|  |  + metodo_publico2()          |  |
|  +-----+                      |  |
+-----+

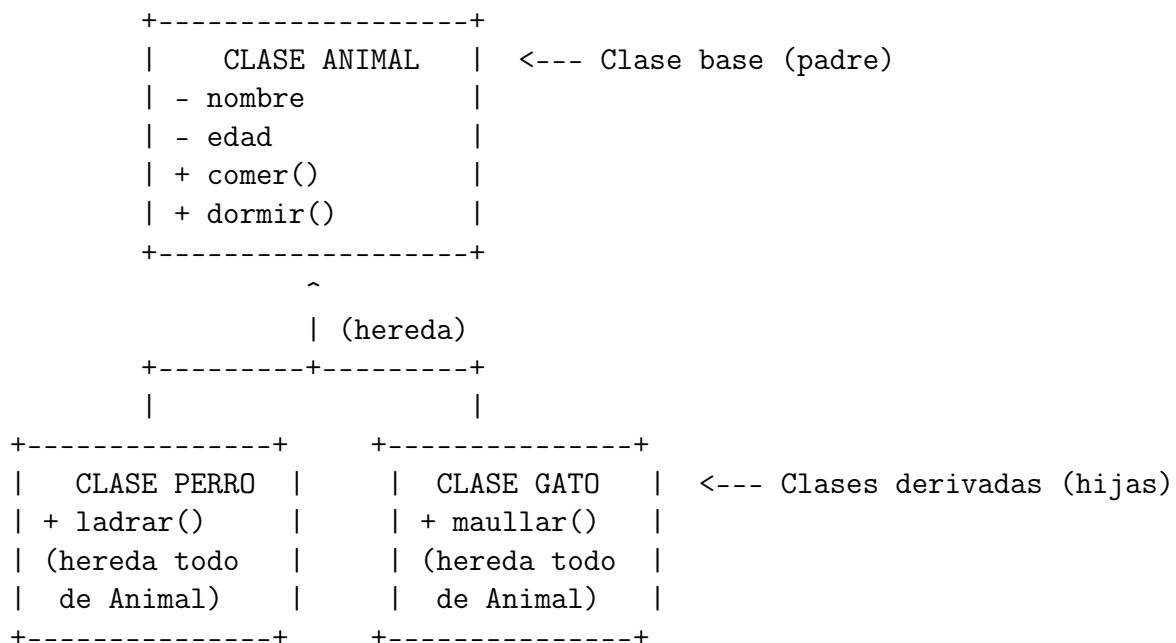
```

Herencia

La **herencia** permite crear nuevas clases basadas en clases existentes. La clase nueva (derivada o hija) **hereda** los atributos y métodos de la clase base (padre), y puede añadir nuevos o modificar los existentes.

Esto promueve la **reutilización de código** y permite modelar relaciones de tipo **es-un** (is-a). Por ejemplo, un "Perro" **es** un "Animal", un "Empleado" **es** una "Persona".

Diagrama de Herencia



Polimorfismo

El **polimorfismo** (del griego "muchas formas") es la capacidad de un objeto para comportarse de diferentes maneras según el contexto. En POO, se manifiesta principalmente de dos formas:

1. **Polimorfismo en tiempo de compilación (sobrecarga):** Varias funciones con el mismo nombre pero diferentes parámetros. El compilador decide cuál llamar basándose en los argumentos.
2. **Polimorfismo en tiempo de ejecución (sobreescritura):** Una clase derivada puede proporcionar su propia implementación de un método definido en la clase base. Cuando se llama a ese método a través de un puntero o referencia a la clase base, se ejecuta la versión de la clase derivada (gracias a las funciones virtuales).

Identidad de Objetos

Cada objeto tiene una **identidad** única, independientemente de su estado. Dos objetos pueden tener exactamente los mismos valores en sus atributos, pero son objetos diferentes (ocupan diferentes posiciones de memoria). En C++, la dirección de memoria suele usarse como identidad.

Introducción a C++

C++ es una extensión de C que añade, entre otras cosas, soporte para programación orientada a objetos. Fue desarrollado por Bjarne Stroustrup a principios de los 80. La mayoría de los programas en C son también programas válidos en C++ (aunque no al revés).

De C a C++: Primeras Diferencias

```
1 // En C:
2 #include <stdio.h>
3 int main() {
4     printf("Hola, mundo!\n");
5     return 0;
6 }
7
8 // En C++:
9 #include <iostream> // Iostream en lugar de stdio.h
10 using namespace std; // Para no tener que escribir std:: ←
    antes de cout
11
12 int main() {
13     cout << "Hola, mundo!" << endl; // cout en lugar de ←
        printf
14     return 0;
15 }
```

Listing 9: Comparación C vs C++ básico

Clases en C++

La palabra clave **class** se usa para definir una clase. Por defecto, los miembros de una clase son **private**, a diferencia de **struct** en C++ (que es público por defecto, aunque en C++ también se puede usar **struct** para clases con miembros públicos).

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Definición de la clase Persona
```

```

6 class Persona {
7 private: // Atributos privados (encapsulamiento)
8     string nombre;
9     int edad;
10    float altura;
11
12 public: // Metodos publicos (interfaz)
13     // Constructor
14     Persona(string nom, int ed, float alt) {
15         nombre = nom;
16         edad = ed;
17         altura = alt;
18     }
19
20     // Metodos
21     void saludar() {
22         cout << "Hola, soy " << nombre << " y tengo " << ←
23         edad << " años." << endl;
24     }
25
26     void cumplir_anios() {
27         edad++;
28         cout << "Feliz cumpleaños! Ahora tengo " << edad ←
29         << " años." << endl;
30     }
31
32     float calcular_imc(float peso) {
33         return peso / (altura * altura);
34     }
35
36     // Getter (metodo para acceder a un atributo privado)
37     string get_nombre() {
38         return nombre;
39     }
40
41     // Setter (metodo para modificar un atributo privado)
42     void set_nombre(string nuevo_nombre) {
43         if (nuevo_nombre.length() > 0) {
44             nombre = nuevo_nombre;
45         }
46     }
47 };
48
49 int main() {
50     // Crear objetos (instancias de la clase)
51     Persona persona1("Ana", 25, 1.65);
52     Persona persona2("Carlos", 30, 1.75);
53
54     // Usar metodos

```

```
53     persona1.saludar();
54     persona2.saludar();
55
56     persona1.cumplir_anios();
57
58     float imc = persona2.calcular_imc(70.5);
59     cout << "IMC de Carlos: " << imc << endl;
60
61     // Usar getters/setters
62     cout << "Nombre original: " << persona1.get_nombre() << endl;
63     persona1.set_nombre("Ana Maria");
64     cout << "Nombre nuevo: " << persona1.get_nombre() << endl;
65
66     return 0;
67 }
```

Listing 10: Definición de una clase en C++

Constructores y Destructores

Constructores

Un **constructor** es un método especial que se llama automáticamente cuando se crea un objeto. Su propósito es inicializar el objeto (dar valores iniciales a los atributos, reservar recursos, etc.).

Características:

- Tiene el mismo nombre que la clase.
- No tiene tipo de retorno (ni siquiera void).
- Puede tener parámetros (constructor parametrizado).
- Puede haber varios constructores (sobrecarga de constructores).
- Si no definimos ningún constructor, el compilador genera uno por defecto (sin parámetros) que hace nada.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Rectangulo {
6 private:
7     double ancho;
8     double alto;
```

```

9
10 public:
11     // Constructor por defecto (sin parametros)
12     Rectangulo() {
13         ancho = 1.0;
14         alto = 1.0;
15         cout << "Constructor por defecto llamado." << endl;
16     }
17
18     // Constructor parametrizado
19     Rectangulo(double a, double al) {
20         ancho = a;
21         alto = al;
22         cout << "Constructor parametrizado llamado." << ↵
23             endl;
24     }
25
26     // Constructor de copia (crea un objeto a partir de ↵
27     otro)
28     Rectangulo(const Rectangulo &otro) {
29         ancho = otro.ancho;
30         alto = otro.alto;
31         cout << "Constructor de copia llamado." << endl;
32     }
33
34     double area() {
35         return ancho * alto;
36     }
37
38     double perimetro() {
39         return 2 * (ancho + alto);
40     }
41
42     void mostrar() {
43         cout << "Rectangulo: " << ancho << " x " << alto ↵
44             << endl;
45     }
46 };
47
48 int main() {
49     Rectangulo r1;                // Constructor por ↵
50     defecto
51     r1.mostrar();
52
53     Rectangulo r2(5.0, 3.0);      // Constructor ↵
54     parametrizado
55     r2.mostrar();
56
57     Rectangulo r3 = r2;           // Constructor de copia

```

```

53     r3.mostrar();
54
55     cout << "Area de r2: " << r2.area() << endl;
56
57     return 0;
58 }

```

Listing 11: Diferentes tipos de constructores

Destruyores

Un **destructor** es un método especial que se llama automáticamente cuando un objeto se destruye (sale de su ámbito o se libera con `delete`). Su propósito es liberar recursos (memoria dinámica, archivos abiertos, conexiones, etc.).

Características:

- Tiene el mismo nombre que la clase, precedido por `~`.
- No tiene parámetros.
- No tiene tipo de retorno.
- Solo puede haber un destructor por clase.
- Si no lo definimos, el compilador genera uno por defecto que no hace nada.

```

1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  class Cadena {
6  private:
7      char *buffer;
8      int longitud;
9
10 public:
11     // Constructor
12     Cadena(const char *texto) {
13         longitud = strlen(texto);
14         buffer = new char[longitud + 1]; // Reservar ←
15         strcpy(buffer, texto);
16         cout << "Constructor: Se creo Cadena \"" << ←
17             buffer << "\"" << endl;
18     }
19
20     // Destructor
21     ~Cadena() {

```

```

21     cout << "Destructor: Liberando Cadena \"" << ↵
        buffer << "\"" << endl;
22     delete[] buffer; // Liberar memoria dinamica
23 }
24
25 void mostrar() {
26     cout << "Contenido: " << buffer << endl;
27 }
28
29 int get_longitud() {
30     return longitud;
31 }
32 };
33
34 int main() {
35     cout << "Inicio de main" << endl;
36
37     {
38         Cadena saludo("Hola, mundo!");
39         saludo.mostrar();
40         cout << "Longitud: " << saludo.get_longitud() << ↵
            endl;
41     } // Aqui se llama al destructor de 'saludo'
42
43     cout << "Fin de main" << endl;
44
45     return 0;
46 }

```

Listing 12: Clase con constructor y destructor

Sobrecarga de Funciones

La **sobrecarga de funciones** (function overloading) permite tener múltiples funciones con el **mismo nombre** pero diferentes parámetros (diferente número, tipo u orden). El compilador decide cuál llamar basándose en los argumentos proporcionados.

Esto es una forma de **polimorfismo en tiempo de compilación**.

```

1 #include <iostream>
2 using namespace std;
3
4 // Funcion para sumar dos enteros
5 int sumar(int a, int b) {
6     cout << "Sumando enteros: ";
7     return a + b;
8 }
9
10 // Sobrecarga: sumar tres enteros

```

```
11 int sumar(int a, int b, int c) {
12     cout << "Sumando tres enteros: ";
13     return a + b + c;
14 }
15
16 // Sobrecarga: sumar dos doubles
17 double sumar(double a, double b) {
18     cout << "Sumando doubles: ";
19     return a + b;
20 }
21
22 // Sobrecarga: sumar un entero y un double
23 double sumar(int a, double b) {
24     cout << "Sumando entero y double: ";
25     return a + b;
26 }
27
28 // Sobrecarga: concatenar strings (no es suma numerica)
29 string sumar(string a, string b) {
30     cout << "Concatenando strings: ";
31     return a + b;
32 }
33
34 int main() {
35     cout << sumar(5, 3) << endl;
36     cout << sumar(5, 3, 2) << endl;
37     cout << sumar(2.5, 3.7) << endl;
38     cout << sumar(10, 3.14) << endl;
39     cout << sumar(string("Hola "), string("Mundo")) << endl;
40
41     return 0;
42 }
```

Listing 13: Sobrecarga de funciones

Sobrecarga de Operadores

La **sobrecarga de operadores** es una característica muy potente de C++ que permite darle un significado especial a los operadores (+, -, *, /, ==, !=, <, >, etc.) cuando se usan con objetos de nuestras clases.

Esto hace que el código sea más intuitivo y legible. Por ejemplo, podemos sumar dos objetos `Complejo` con el operador +, o comparar dos `Rectangulo` con ==.

Sintaxis básica

Para sobrecargar un operador, usamos la palabra clave `operator` seguida del símbolo del operador.

```

1  #include <iostream>
2  using namespace std;
3
4  class Complejo {
5  private:
6      double real;
7      double imaginario;
8
9  public:
10     Complejo(double r = 0.0, double i = 0.0) : real(r),
        imaginario(i) {}
11
12     // Sobrecarga del operador + (como metodo de la clase)
13     Complejo operator+(const Complejo &otro) const {
14         return Complejo(real + otro.real, imaginario +
        otro.imaginario);
15     }
16
17     // Sobrecarga del operador - (como metodo)
18     Complejo operator-(const Complejo &otro) const {
19         return Complejo(real - otro.real, imaginario -
        otro.imaginario);
20     }
21
22     // Sobrecarga del operador * (multiplicacion de
        complejos)
23     Complejo operator*(const Complejo &otro) const {
24         double r = real * otro.real - imaginario *
        otro.imaginario;
25         double i = real * otro.imaginario + imaginario *
        otro.real;
26         return Complejo(r, i);
27     }
28
29     // Sobrecarga del operador ==
30     bool operator==(const Complejo &otro) const {
31         return (real == otro.real && imaginario ==
        otro.imaginario);
32     }
33
34     // Sobrecarga del operador !=
35     bool operator!=(const Complejo &otro) const {
36         return !(*this == otro);
37     }
38
39     // Metodo para mostrar el numero complejo
40     void mostrar() const {
41         cout << real << (imaginario >= 0 ? " + " : " - ")

```



```

42         << abs(imaginario) << "i";
43     }
44 };
45
46 // Sobrecarga del operador << (como funcion externa, para ←
    poder usarlo con cout)
47 ostream& operator<<(ostream &os, const Complejo &c) {
48     os << c.real << (c.imaginario >= 0 ? " + " : " - ")
49     << abs(c.imaginario) << "i";
50     return os;
51 }
52
53 int main() {
54     Complejo c1(3.0, 4.0);
55     Complejo c2(1.0, 2.0);
56
57     Complejo suma = c1 + c2;
58     Complejo resta = c1 - c2;
59     Complejo producto = c1 * c2;
60
61     cout << "c1 = " << c1 << endl;
62     cout << "c2 = " << c2 << endl;
63     cout << "c1 + c2 = " << suma << endl;
64     cout << "c1 - c2 = " << resta << endl;
65     cout << "c1 * c2 = " << producto << endl;
66
67     if (c1 == c2) {
68         cout << "c1 y c2 son iguales" << endl;
69     } else {
70         cout << "c1 y c2 son diferentes" << endl;
71     }
72
73     return 0;
74 }

```

Listing 14: Sobrecarga de operadores en una clase Complejo

Ejemplo más completo: Clase Vector3D

```

1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 class Vector3D {
6 private:
7     double x, y, z;
8
9 public:

```

```

10     Vector3D(double xv = 0.0, double yv = 0.0, double zv ←
11         = 0.0)
12         : x(xv), y(yv), z(zv) {}
13
14     // Operadores aritmeticos
15     Vector3D operator+(const Vector3D &v) const {
16         return Vector3D(x + v.x, y + v.y, z + v.z);
17     }
18
19     Vector3D operator-(const Vector3D &v) const {
20         return Vector3D(x - v.x, y - v.y, z - v.z);
21     }
22
23     Vector3D operator*(double escalar) const {
24         return Vector3D(x * escalar, y * escalar, z * ←
25             escalar);
26     }
27
28     // Amigo para permitir multiplicacion escalar * ←
29     // vector (commutativa)
30     friend Vector3D operator*(double escalar, const ←
31         Vector3D &v);
32
33     // Producto escalar (dot product)
34     double operator*(const Vector3D &v) const {
35         return x * v.x + y * v.y + z * v.z;
36     }
37
38     // Producto vectorial (cross product)
39     Vector3D operator^(const Vector3D &v) const {
40         return Vector3D(y * v.z - z * v.y,
41             z * v.x - x * v.z,
42             x * v.y - y * v.x);
43     }
44
45     // Operadores de comparacion
46     bool operator==(const Vector3D &v) const {
47         return (x == v.x && y == v.y && z == v.z);
48     }
49
50     bool operator!=(const Vector3D &v) const {
51         return !(*this == v);
52     }
53
54     // Operador de asignacion
55     Vector3D& operator=(const Vector3D &v) {
56         if (this != &v) { // Evitar auto-asignacion
57             x = v.x;
58             y = v.y;

```

```

55         z = v.z;
56     }
57     return *this;
58 }
59
60 // Operador de indice (acceso como array)
61 double& operator[](int indice) {
62     if (indice == 0) return x;
63     if (indice == 1) return y;
64     return z;
65 }
66
67 const double& operator[](int indice) const {
68     if (indice == 0) return x;
69     if (indice == 1) return y;
70     return z;
71 }
72
73 // Metodos utilitarios
74 double magnitud() const {
75     return sqrt(x*x + y*y + z*z);
76 }
77
78 Vector3D normalizar() const {
79     double mag = magnitud();
80     if (mag > 0) {
81         return Vector3D(x/mag, y/mag, z/mag);
82     }
83     return *this;
84 }
85
86 void mostrar() const {
87     cout << "(" << x << ", " << y << ", " << z << ")";
88 }
89 };
90
91 // Implementacion de la funcion amiga
92 Vector3D operator*(double escalar, const Vector3D &v) {
93     return Vector3D(v.x * escalar, v.y * escalar, v.z * ←
94         escalar);
95 }
96
97 // Sobrecarga del operador <<
98 ostream& operator<<(ostream &os, const Vector3D &v) {
99     os << "(" << v.x << ", " << v.y << ", " << v.z << ")";
100    return os;
101 }
102
103 int main() {

```

```

103     Vector3D v1(1.0, 2.0, 3.0);
104     Vector3D v2(4.0, 5.0, 6.0);
105
106     cout << "v1 = " << v1 << endl;
107     cout << "v2 = " << v2 << endl;
108
109     cout << "v1 + v2 = " << (v1 + v2) << endl;
110     cout << "v1 - v2 = " << (v1 - v2) << endl;
111     cout << "v1 * 2.5 = " << (v1 * 2.5) << endl;
112     cout << "3.0 * v2 = " << (3.0 * v2) << endl;
113
114     cout << "Producto escalar v1 * v2 = " << (v1 * v2) << ↵
        endl;
115     cout << "Producto vectorial v1 ^ v2 = " << (v1 ^ v2) ↵
        << endl;
116
117     cout << "Magnitud de v1: " << v1.magnitud() << endl;
118     cout << "v1 normalizado: " << v1.normalizar() << endl;
119
120     // Usando operador []
121     cout << "v1[0] = " << v1[0] << ", v1[1] = " << v1[1] ↵
        << ", v1[2] = " << v1[2] << endl;
122
123     return 0;
124 }

```

Listing 15: Clase Vector3D con operadores sobrecargados

Resumen de la Unidad X

Hemos dado un salto conceptual hacia la Programación Orientada a Objetos:

- **Abstracción:** Modelamos la realidad enfocándonos en lo esencial, ocultando la complejidad.
- **Clases y Objetos:** Las clases son moldes; los objetos son instancias concretas con estado y comportamiento.
- **Encapsulamiento:** Protegemos los datos (privados) y exponemos solo una interfaz pública.
- **Variables de instancia:** Atributos que definen el estado de cada objeto.
- **Constructores:** Métodos especiales que inician objetos al crearlos.
- **Destructores:** Métodos especiales que liberan recursos al destruir objetos.
- **Sobrecarga de funciones:** Múltiples funciones con el mismo nombre pero diferentes parámetros.

- **Sobrecarga de operadores:** Damos significado a los operadores (+, -, *, etc.) para nuestros objetos.
- **Herencia (concepto):** Crear nuevas clases basadas en clases existentes (reutilización).
- **Polimorfismo (concepto):** La capacidad de un objeto para tomar múltiples formas.

Esta unidad es solo una introducción a la POO. C++ ofrece muchas más características avanzadas como herencia múltiple, funciones virtuales, templates, manejo de excepciones, etc., que se exploran en cursos más avanzados.

Unidad XI: Manejo de Recursos del Sistema - Interactuando con el Sistema Operativo

Hasta ahora, nuestros programas han operado en un entorno relativamente aislado. Han procesado datos, realizado cálculos, leído y escrito archivos, pero siempre dentro de los límites que el sistema operativo nos ha permitido de forma transparente. Sin embargo, en aplicaciones más avanzadas, necesitamos interactuar directamente con el sistema operativo para gestionar recursos como procesos, memoria, señales y el tiempo.

Esta unidad explora cómo un programa en C puede comunicarse con el sistema operativo (especialmente en entornos Unix/Linux) para tener un control más fino sobre la ejecución y los recursos del sistema.

¿Qué son los Recursos del Sistema?

Un **recurso del sistema** es cualquier componente del sistema informático que un programa puede necesitar para ejecutarse. Incluye:

- **Procesador (CPU):** Tiempo de ejecución.
- **Memoria:** RAM, memoria virtual.
- **Dispositivos de E/S:** Archivos, terminales, impresoras.
- **Procesos:** Capacidad de crear y gestionar otros programas.
- **Temporizadores:** Medir y controlar el tiempo.
- **Señales:** Mecanismos de notificación asíncrona.

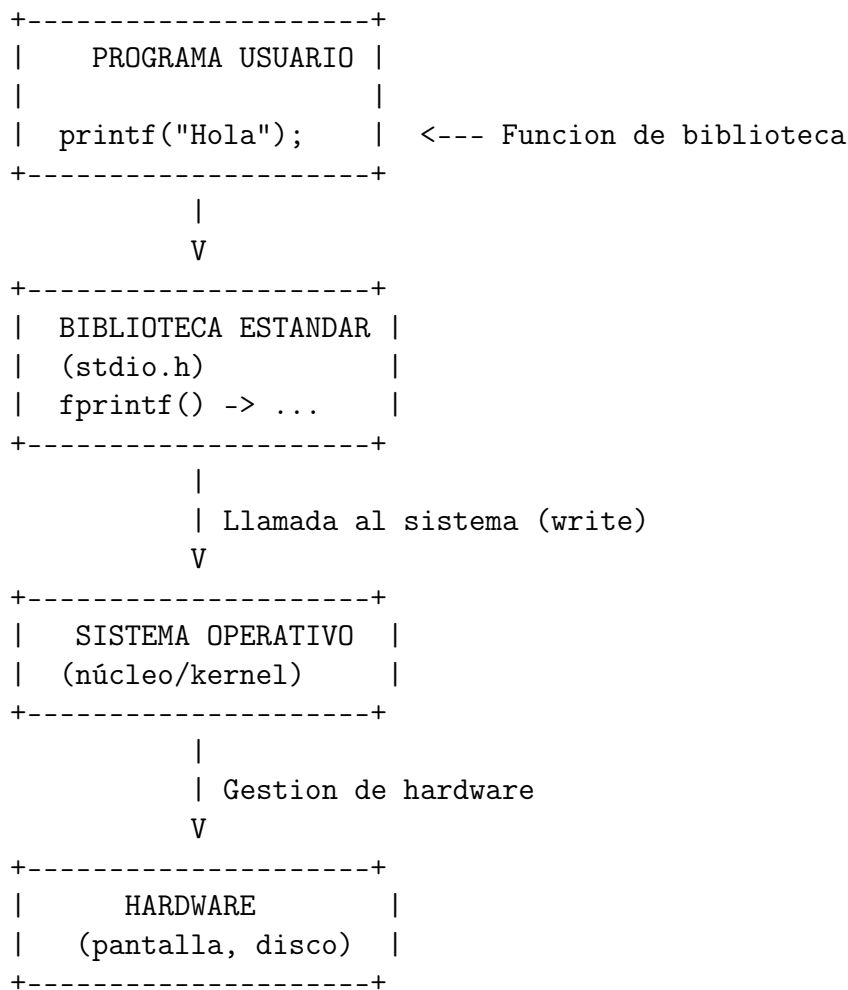
El **sistema operativo** es el administrador de todos estos recursos. Cuando nuestro programa necesita un recurso, realiza una **llamada al sistema** (system call) para solicitarlo al SO.

Llamadas al Sistema vs. Funciones de Biblioteca

Es crucial entender la diferencia:

- **Funciones de biblioteca:** Son funciones como `printf()`, `scanf()`, `malloc()`, etc. Están en la biblioteca estándar de C y son **portables** (funcionan en cualquier sistema con un compilador de C). Por debajo, muchas de ellas utilizan llamadas al sistema.
- **Llamadas al sistema:** Son interfaces directas al núcleo del sistema operativo. Son específicas del SO (no portables) y permiten un control más fino. Ejemplos: `read()`, `write()`, `open()`, `fork()`, `exec()`.

Relación entre Programa, Biblioteca y Sistema Operativo



Gestión de Procesos

Un **proceso** es un programa en ejecución. Cada proceso tiene su propio espacio de direcciones de memoria, sus propios descriptores de archivo, y su propio estado.

Obteniendo Información del Proceso: getpid(), getppid()


```

1 #include <stdio.h>
2 #include <unistd.h>    // Para getpid(), getppid()
3
4 int main() {
5     printf("ID de este proceso (PID): %d\n", getpid());
6     printf("ID del proceso padre (PPID): %d\n", getppid());
7
8     // Podemos ejecutar 'ps' en otra terminal para verificar
9     printf("Ejecuta 'ps' en otra terminal para ver este ←
10         proceso.\n");
11     sleep(10); // Espera 10 segundos para que podamos verlo
12
13     return 0;
14 }

```

Listing 16: Identificadores de proceso

Creación de Procesos: fork()

La llamada al sistema `fork()` es fundamental en Unix. Crea un nuevo proceso (el **hijo**) que es una copia casi exacta del proceso que la llama (el **padre**).

Visualización de fork()

Antes de `fork()`:

```

+-----+
| PROCESO |
| PADRE   |
| pid=1000 |
+-----+

```

```

|
| fork()
V

```

PROCESO	PROCESO
PADRE	HIJO
pid=1000	pid=1001
fork() devuelve	fork() devuelve
el PID del hijo	0
(1001)	

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h> // Para wait()

```

```

4
5 int main() {
6     pid_t pid = fork(); // Crea un nuevo proceso
7
8     if (pid < 0) {
9         // Error al crear el proceso
10        perror("fork fallo");
11        return 1;
12    }
13    else if (pid == 0) {
14        //Codigo que ejecuta el proceso HIJO
15        printf("SOY EL HIJO: Mi PID = %d, PID de mi padre ←
16              = %d\n",
17              getpid(), getppid());
18
19        // El hijo puede hacer algo diferente
20        for (int i = 0; i < 5; i++) {
21            printf("Hijo: mensaje %d\n", i);
22            sleep(1);
23        }
24
25        printf("Hijo: terminando.\n");
26    }
27    else {
28        //Codigo que ejecuta el proceso PADRE
29        printf("SOY EL PADRE: Mi PID = %d, PID de mi hijo ←
30              = %d\n",
31              getpid(), pid);
32
33        printf("Padre: esperando a que el hijo ←
34              termine...\n");
35        wait(NULL); // Espera a que cualquier hijo termine
36
37        printf("Padre: hijo termino, yo tambien ←
38              termino.\n");
39    }
40
41    return 0;
42 }

```

Listing 17: Ejemplo de fork()

¿Qué imprime este programa? Depende del orden de ejecución, pero típicamente verás una mezcla de mensajes del padre y del hijo, demostrando que son dos procesos independientes ejecutándose concurrentemente.

Ejecución de Nuevos Programas: exec()

La familia de funciones `exec()` (`execlp`, `execvp`, etc.) reemplaza la imagen del proceso actual con un nuevo programa. Es decir, el proceso actual deja de ejecutar su código y comienza a ejecutar otro programa completamente diferente.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int main() {
6     pid_t pid = fork();
7
8     if (pid == 0) {
9         // Proceso hijo: ejecuta el comando "ls -l"
10        printf("Hijo: voy a ejecutar 'ls -l'\n");
11
12        // execlp busca el programa en el PATH y lo ejecuta
13        execlp("ls", "ls", "-l", NULL);
14
15        // Si execlp falla, se ejecuta esta linea
16        perror("execlp fallo");
17        return 1;
18    }
19    else if (pid > 0) {
20        // Proceso padre
21        printf("Padre: esperando a que el hijo ejecute ↵
22               ls...\n");
23        wait(NULL);
24        printf("Padre: hijo termino.\n");
25    }
26    else {
27        perror("fork fallo");
28        return 1;
29    }
30
31    return 0;
32 }
```

Listing 18: Uso de `execlp()`

Combinando `fork()` y `exec()`

Esta combinación es la base para lanzar nuevos programas desde otro programa (como hace el shell cuando ejecutas un comando).

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
```

```

4
5 int main() {
6     char *comando = "ls";
7     char *argumentos[] = {"ls", "-l", "-a", NULL};
8
9     pid_t pid = fork();
10
11     if (pid == 0) {
12         // Proceso hijo
13         printf("Ejecutando: %s\n", comando);
14         execvp(comando, argumentos);
15
16         // Si llegamos aqui, execvp fallo
17         perror("execvp fallo");
18         return 1;
19     }
20     else if (pid > 0) {
21         // Proceso padre
22         int status;
23         wait(&status); // Espera al hijo y obtiene su ←
24                         estado
25
26         if (WIFEXITED(status)) {
27             printf("Hijo termino con codigo: %d\n", ←
28                     WEXITSTATUS(status));
29         }
30     }
31     else {
32         perror("fork fallo");
33         return 1;
34     }
35
36     return 0;
37 }

```

Listing 19: Crear un nuevo proceso que ejecute otro programa

Esperando a los Hijos: wait() y waitpid()

Cuando un proceso hijo termina, se convierte en un **proceso zombi** hasta que su padre recoja su estado de salida con `wait()` o `waitpid()`.

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int main() {
6     pid_t pid = fork();
7

```

```

8     if (pid == 0) {
9         // Hijo
10        printf("Hijo: trabajando...\n");
11        sleep(2);
12        printf("Hijo: terminando con codigo 42\n");
13        return 42; //Codigo de salida
14    }
15    else {
16        // Padre
17        int status;
18        pid_t terminado = waitpid(pid, &status, 0); // ←
19        // Espera a ese hijo especifico
20
21        if (terminado == pid) {
22            if (WIFEXITED(status)) {
23                printf("Padre: hijo termino con codigo ←
24                        %d\n", WEXITSTATUS(status));
25            }
26            else if (WIFSIGNALED(status)) {
27                printf("Padre: hijo termino por se al ←
28                        %d\n", WTERMSIG(status));
29            }
30        }
31    }
32
33    return 0;
34 }

```

Listing 20: Uso de waitpid()

Señales (Signals)

Las **señales** son notificaciones asíncronas que se envían a un proceso para indicarle que ha ocurrido un evento. Pueden ser enviadas por el sistema operativo, por otro proceso, o por el propio proceso (a sí mismo).

Ejemplos de señales comunes:

- SIGINT: Interrupción desde el teclado (Ctrl+C).
- SIGKILL: Forzar la terminación del proceso (no se puede ignorar ni capturar).
- SIGTERM: Solicitud de terminación (se puede capturar).
- SIGSEGV: Violación de segmento (acceso a memoria inválida).
- SIGALRM: Alarma temporizador.
- SIGCHLD: Enviada al padre cuando un hijo termina o se detiene.

Manejadores de Señales: signal() y sigaction()

Podemos capturar ciertas señales y ejecutar una función cuando ocurran.

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 // Manejador de la se al
6 void manejador_sigint(int sig) {
7     printf("\n Recibi SIGINT (Ctrl+C)! Pero no quiero ↵
8         terminar.\n");
9     printf("Puedes intentarlo de nuevo, o usar 'kill' con ↵
10         SIGTERM.\n");
11 }
12
13 int main() {
14     // Instalar el manejador para SIGINT
15     if (signal(SIGINT, manejador_sigint) == SIG_ERR) {
16         perror("Error instalando manejador");
17         return 1;
18     }
19
20     printf("Presiona Ctrl+C para enviar SIGINT.\n");
21     printf("Esperando... (PID = %d)\n", getpid());
22
23     // Bucle infinito
24     while (1) {
25         printf("Trabajando...\n");
26         sleep(2);
27     }
28
29     return 0;
30 }

```

Listing 21: Capturando SIGINT (Ctrl+C)

```

1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4 #include <string.h>
5
6 void manejador(int sig) {
7     printf("Recibi se al %d\n", sig);
8 }
9
10 int main() {
11     struct sigaction sa;
12
13     // Configurar la estructura sigaction

```

```

14  memset(&sa, 0, sizeof(sa));
15  sa.sa_handler = manejador; // Funcion manejadora
16  sigemptyset(&sa.sa_mask); // No bloquear otras se
    se ales durante el manejador
17  sa.sa_flags = 0; // Sin flags especiales
18
19  // Instalar el manejador para SIGUSR1
20  if (sigaction(SIGUSR1, &sa, NULL) == -1) {
21      perror("sigaction");
22      return 1;
23  }
24
25  printf("Envia SIGUSR1 al proceso %d con: kill -USR1 ↵
    %d\n", getpid(), getpid());
26
27  while (1) {
28      pause(); // Espera hasta que llegue una se al
29  }
30
31  return 0;
32 }

```

Listing 22: Usando sigaction() (más control)

Enviando Señales: kill()

La función `kill()` (a pesar de su nombre) envía una señal a un proceso o grupo de procesos.

```

1  #include <stdio.h>
2  #include <signal.h>
3  #include <unistd.h>
4
5  int main() {
6      pid_t pid_hijo;
7
8      pid_hijo = fork();
9
10     if (pid_hijo == 0) {
11         // Proceso hijo: ignora SIGTERM y espera
12         signal(SIGTERM, SIG_IGN); // Ignora SIGTERM
13         printf("Hijo: ignorando SIGTERM. Mi PID: %d\n", ↵
            getpid());
14
15         while (1) {
16             printf("Hijo: trabajando...\n");
17             sleep(2);
18         }
19     }

```

```

20     else {
21         // Proceso padre
22         sleep(3);
23         printf("Padre: enviando SIGTERM al hijo...\n");
24         kill(pid_hijo, SIGTERM); // Envía SIGTERM al hijo
25
26         sleep(2);
27         printf("Padre: enviando SIGKILL al hijo... (este ←
28             no se puede ignorar)\n");
29         kill(pid_hijo, SIGKILL);
30
31         wait(NULL);
32         printf("Padre: hijo termino.\n");
33     }
34
35     return 0;
36 }

```

Listing 23: Enviar una señal

Señal de Alarma: alarm()

La función `alarm()` programa que el sistema envíe una señal `SIGALRM` al proceso después de un número especificado de segundos.

```

1  #include <stdio.h>
2  #include <signal.h>
3  #include <unistd.h>
4
5  void manejador_alarma(int sig) {
6      printf(" ALARMA ! Han pasado 3 segundos.\n");
7      alarm(3); // Reprogramar otra alarma
8  }
9
10 int main() {
11     signal(SIGALRM, manejador_alarma);
12
13     alarm(3); // Primera alarma en 3 segundos
14
15     printf("Alarma programada. Esperando...\n");
16
17     while (1) {
18         pause(); // Esperar seales
19     }
20
21     return 0;
22 }

```

Listing 24: Usando alarm()

Gestión de Memoria Avanzada

Ya vimos malloc(), free(), etc. Pero hay llamadas al sistema para un control más fino.

Memoria Anónima con mmap()

mmap() (memory map) permite mapear archivos o dispositivos en memoria, o crear regiones de memoria anónima compartida entre procesos.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/mman.h>
4 #include <sys/wait.h>
5 #include <unistd.h>
6 #include <string.h>
7
8 int main() {
9     int *memoria_compartida;
10
11     // Crear una region de memoria compartida anonima
12     memoria_compartida = mmap(NULL, sizeof(int),
13                               PROT_READ | PROT_WRITE,
14                               MAP_SHARED | MAP_ANONYMOUS,
15                               -1, 0);
16
17     if (memoria_compartida == MAP_FAILED) {
18         perror("mmap fallo");
19         return 1;
20     }
21
22     *memoria_compartida = 0; // Inicializar
23
24     pid_t pid = fork();
25
26     if (pid == 0) {
27         // Proceso hijo
28         for (int i = 0; i < 5; i++) {
29             (*memoria_compartida)++;
30             printf("Hijo: incremento contador = %d\n", ←
31                  *memoria_compartida);
32             sleep(1);
33         }
34     }
35     else if (pid > 0) {
36         // Proceso padre
37         for (int i = 0; i < 5; i++) {
38             printf("Padre: contador = %d\n", ←
39                  *memoria_compartida);
```

```

38         sleep(1);
39     }
40
41     wait(NULL);
42
43     // Liberar la memoria mapeada
44     munmap(memoria_compartida, sizeof(int));
45 }
46 else {
47     perror("fork fallo");
48     return 1;
49 }
50
51 return 0;
52 }

```

Listing 25: Memoria compartida con mmap()

Tuberías (Pipes)

Las **tuberías** permiten la comunicación unidireccional entre procesos. Un proceso escribe en un extremo y otro lee del otro extremo.

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <string.h>
5
6 int main() {
7     int pipefd[2]; // pipefd[0] para lectura, pipefd[1] ←
8                     para escritura
9     pid_t pid;
10    char buffer[100];
11
12    // Crear la tubería
13    if (pipe(pipefd) == -1) {
14        perror("pipe fallo");
15        return 1;
16    }
17
18    pid = fork();
19
20    if (pid == 0) {
21        // Proceso hijo: cierra el extremo de escritura, ←
22        lee del pipe
23        close(pipefd[1]); // Cerrar escritura
24
25        read(pipefd[0], buffer, sizeof(buffer));
26    }
27
28    // Proceso padre: cierra el extremo de lectura, ←
29    write al pipe
30    close(pipefd[0]);
31
32    wait(&pid);
33
34    printf("Proceso padre: %s\n", buffer);
35
36    return 0;
37 }

```

```

24     printf("Hijo recibio: %s\n", buffer);
25
26     close(pipefd[0]);
27 }
28 else if (pid > 0) {
29     // Proceso padre: cierra el extremo de lectura, ←
30     // escribe en el pipe
31     close(pipefd[0]); // Cerrar lectura
32
33     char *mensaje = "Hola desde el padre!";
34     write(pipefd[1], mensaje, strlen(mensaje) + 1);
35
36     close(pipefd[1]);
37
38     wait(NULL);
39 }
40 else {
41     perror("fork fallo");
42     return 1;
43 }
44
45 return 0;
46 }

```

Listing 26: Comunicación padre-hijo con pipe()

Tuberías con Nombre (FIFOs)

Las tuberías sin nombre solo funcionan entre procesos relacionados. Las FIFOs (tuberías con nombre) permiten comunicación entre procesos no relacionados.

```

1 // Creador de la FIFO (primer programa)
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7 #include <string.h>
8
9 int main() {
10     const char *fifo_path = "/tmp/mi_fifo";
11
12     // Crear la FIFO (si no existe)
13     mkfifo(fifo_path, 0666);
14
15     // Abrir la FIFO para escritura
16     int fd = open(fifo_path, O_WRONLY);
17     if (fd == -1) {
18         perror("open");

```

```

19     return 1;
20 }
21
22 char *mensaje = "Hola desde el programa escritor!";
23 write(fd, mensaje, strlen(mensaje) + 1);
24
25 close(fd);
26
27 // Opcional: eliminar la FIFO
28 unlink(fifo_path);
29
30 return 0;
31 }

```

Listing 27: Crear y usar una FIFO

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6
7 int main() {
8     const char *fifo_path = "/tmp/mi_fifo";
9     char buffer[100];
10
11     // Abrir la FIFO para lectura (espera hasta que ←
12     // alguien abra para escritura)
13     int fd = open(fifo_path, O_RDONLY);
14     if (fd == -1) {
15         perror("open");
16         return 1;
17     }
18
19     read(fd, buffer, sizeof(buffer));
20     printf("Recibido: %s\n", buffer);
21
22     close(fd);
23
24     return 0;
25 }

```

Listing 28: Lector de la FIFO (segundo programa)

Gestión de Tiempo y Temporizadores

Ya vimos `time()` y `clock()`. Ahora exploraremos temporizadores de mayor resolución.

Temporizadores de Alta Resolución: gettimeofday()

```
1 #include <stdio.h>
2 #include <sys/time.h>
3
4 int main() {
5     struct timeval inicio, fin;
6     long segundos, microsegundos;
7
8     gettimeofday(&inicio, NULL);
9
10    // Simular trabajo
11    for (long i = 0; i < 100000000; i++) {
12        // Nada, solo consumir tiempo
13    }
14
15    gettimeofday(&fin, NULL);
16
17    segundos = fin.tv_sec - inicio.tv_sec;
18    microsegundos = fin.tv_usec - inicio.tv_usec;
19
20    if (microsegundos < 0) {
21        microsegundos += 1000000;
22        segundos--;
23    }
24
25    printf("Tiempo transcurrido: %ld segundos, %ld ←
26           microsegundos\n",
27           segundos, microsegundos);
28
29    return 0;
30 }
```

Listing 29: Medir tiempo con gettimeofday()

Temporizadores con setitimer()

setitimer() permite temporizadores más flexibles que alarm(), con resolución de microsegundos y diferentes modos.

```
1 #include <stdio.h>
2 #include <signal.h>
3 #include <sys/time.h>
4 #include <unistd.h>
5
6 void manejador_timer(int sig) {
7     static int contador = 0;
8     printf("Timer expirado! (%d)\n", ++contador);
9 }
```

```

9  }
10
11 int main() {
12     struct itimerval timer;
13
14     signal(SIGALRM, manejador_timer);
15
16     // Configurar el timer
17     timer.it_value.tv_sec = 2;          // Primer expiracion ←
18     // en 2 segundos
19     timer.it_value.tv_usec = 0;
20     timer.it_interval.tv_sec = 1;      // Luego, cada 1 segundo
21     timer.it_interval.tv_usec = 0;
22
23     setitimer(ITIMER_REAL, &timer, NULL);
24
25     printf("Timer iniciado. Esperando...\n");
26
27     while (1) {
28         pause();
29     }
30
31     return 0;
32 }

```

Listing 30: Temporizador con setitimer()

Límites del Sistema

Podemos consultar los límites que el sistema impone a los procesos.

```

1  #include <stdio.h>
2  #include <sys/resource.h>
3  #include <unistd.h>
4
5  int main() {
6     struct rlimit lim;
7
8     // Limite de archivos abiertos
9     getrlimit(RLIMIT_NOFILE, &lim);
10    printf("Limite de archivos abiertos: soft = %ld, hard ←
11           = %ld\n",
12           (long)lim.rlim_cur, (long)lim.rlim_max);
13
14    // Limite de memoria (para el proceso)
15    getrlimit(RLIMIT_AS, &lim);
16    printf("Limite de memoria (bytes): soft = %ld, hard = ←
17           %ld\n",

```

```

16         (long)lim.rlim_cur, (long)lim.rlim_max);
17
18     // Tama o maximo de archivo que puede crear
19     getrlimit(RLIMIT_FSIZE, &lim);
20     printf("Limite de tama o de archivo: soft = %ld, ↵
21         ↵
22         ↵
23         ↵
24         ↵
25         ↵
26         ↵
27         ↵
28         ↵
29         ↵
30         ↵
31         ↵
32         ↵
33         ↵
34         ↵
35         ↵
36         ↵
37         ↵
38         ↵
39         ↵
40         ↵
41         ↵
42         ↵
43         ↵
44         ↵
45         ↵
46         ↵
47         ↵
48         ↵
49         ↵
50         ↵
51         ↵
52         ↵
53         ↵
54         ↵
55         ↵
56         ↵
57         ↵
58         ↵
59         ↵
60         ↵
61         ↵
62         ↵
63         ↵
64         ↵
65         ↵
66         ↵
67         ↵
68         ↵
69         ↵
70         ↵
71         ↵
72         ↵
73         ↵
74         ↵
75         ↵
76         ↵
77         ↵
78         ↵
79         ↵
80         ↵
81         ↵
82         ↵
83         ↵
84         ↵
85         ↵
86         ↵
87         ↵
88         ↵
89         ↵
90         ↵
91         ↵
92         ↵
93         ↵
94         ↵
95         ↵
96         ↵
97         ↵
98         ↵
99         ↵
100        ↵
101        ↵
102        ↵
103        ↵
104        ↵
105        ↵
106        ↵
107        ↵
108        ↵
109        ↵
110        ↵
111        ↵
112        ↵
113        ↵
114        ↵
115        ↵
116        ↵
117        ↵
118        ↵
119        ↵
120        ↵
121        ↵
122        ↵
123        ↵
124        ↵
125        ↵
126        ↵
127        ↵
128        ↵
129        ↵
130        ↵
131        ↵
132        ↵
133        ↵
134        ↵
135        ↵
136        ↵
137        ↵
138        ↵
139        ↵
140        ↵
141        ↵
142        ↵
143        ↵
144        ↵
145        ↵
146        ↵
147        ↵
148        ↵
149        ↵
150        ↵
151        ↵
152        ↵
153        ↵
154        ↵
155        ↵
156        ↵
157        ↵
158        ↵
159        ↵
160        ↵
161        ↵
162        ↵
163        ↵
164        ↵
165        ↵
166        ↵
167        ↵
168        ↵
169        ↵
170        ↵
171        ↵
172        ↵
173        ↵
174        ↵
175        ↵
176        ↵
177        ↵
178        ↵
179        ↵
180        ↵
181        ↵
182        ↵
183        ↵
184        ↵
185        ↵
186        ↵
187        ↵
188        ↵
189        ↵
190        ↵
191        ↵
192        ↵
193        ↵
194        ↵
195        ↵
196        ↵
197        ↵
198        ↵
199        ↵
200        ↵
201        ↵
202        ↵
203        ↵
204        ↵
205        ↵
206        ↵
207        ↵
208        ↵
209        ↵
210        ↵
211        ↵
212        ↵
213        ↵
214        ↵
215        ↵
216        ↵
217        ↵
218        ↵
219        ↵
220        ↵
221        ↵
222        ↵
223        ↵
224        ↵
225        ↵
226        ↵
227        ↵
228        ↵
229        ↵
230        ↵
231        ↵
232        ↵
233        ↵
234        ↵
235        ↵
236        ↵
237        ↵
238        ↵
239        ↵
240        ↵
241        ↵
242        ↵
243        ↵
244        ↵
245        ↵
246        ↵
247        ↵
248        ↵
249        ↵
250        ↵
251        ↵
252        ↵
253        ↵
254        ↵
255        ↵
256        ↵
257        ↵
258        ↵
259        ↵
260        ↵
261        ↵
262        ↵
263        ↵
264        ↵
265        ↵
266        ↵
267        ↵
268        ↵
269        ↵
270        ↵
271        ↵
272        ↵
273        ↵
274        ↵
275        ↵
276        ↵
277        ↵
278        ↵
279        ↵
280        ↵
281        ↵
282        ↵
283        ↵
284        ↵
285        ↵
286        ↵
287        ↵
288        ↵
289        ↵
290        ↵
291        ↵
292        ↵
293        ↵
294        ↵
295        ↵
296        ↵
297        ↵
298        ↵
299        ↵
300        ↵
301        ↵
302        ↵
303        ↵
304        ↵
305        ↵
306        ↵
307        ↵
308        ↵
309        ↵
310        ↵
311        ↵
312        ↵
313        ↵
314        ↵
315        ↵
316        ↵
317        ↵
318        ↵
319        ↵
320        ↵
321        ↵
322        ↵
323        ↵
324        ↵
325        ↵
326        ↵
327        ↵
328        ↵
329        ↵
330        ↵
331        ↵
332        ↵
333        ↵
334        ↵
335        ↵
336        ↵
337        ↵
338        ↵
339        ↵
340        ↵
341        ↵
342        ↵
343        ↵
344        ↵
345        ↵
346        ↵
347        ↵
348        ↵
349        ↵
350        ↵
351        ↵
352        ↵
353        ↵
354        ↵
355        ↵
356        ↵
357        ↵
358        ↵
359        ↵
360        ↵
361        ↵
362        ↵
363        ↵
364        ↵
365        ↵
366        ↵
367        ↵
368        ↵
369        ↵
370        ↵
371        ↵
372        ↵
373        ↵
374        ↵
375        ↵
376        ↵
377        ↵
378        ↵
379        ↵
380        ↵
381        ↵
382        ↵
383        ↵
384        ↵
385        ↵
386        ↵
387        ↵
388        ↵
389        ↵
390        ↵
391        ↵
392        ↵
393        ↵
394        ↵
395        ↵
396        ↵
397        ↵
398        ↵
399        ↵
400        ↵
401        ↵
402        ↵
403        ↵
404        ↵
405        ↵
406        ↵
407        ↵
408        ↵
409        ↵
410        ↵
411        ↵
412        ↵
413        ↵
414        ↵
415        ↵
416        ↵
417        ↵
418        ↵
419        ↵
420        ↵
421        ↵
422        ↵
423        ↵
424        ↵
425        ↵
426        ↵
427        ↵
428        ↵
429        ↵
430        ↵
431        ↵
432        ↵
433        ↵
434        ↵
435        ↵
436        ↵
437        ↵
438        ↵
439        ↵
440        ↵
441        ↵
442        ↵
443        ↵
444        ↵
445        ↵
446        ↵
447        ↵
448        ↵
449        ↵
450        ↵
451        ↵
452        ↵
453        ↵
454        ↵
455        ↵
456        ↵
457        ↵
458        ↵
459        ↵
460        ↵
461        ↵
462        ↵
463        ↵
464        ↵
465        ↵
466        ↵
467        ↵
468        ↵
469        ↵
470        ↵
471        ↵
472        ↵
473        ↵
474        ↵
475        ↵
476        ↵
477        ↵
478        ↵
479        ↵
480        ↵
481        ↵
482        ↵
483        ↵
484        ↵
485        ↵
486        ↵
487        ↵
488        ↵
489        ↵
490        ↵
491        ↵
492        ↵
493        ↵
494        ↵
495        ↵
496        ↵
497        ↵
498        ↵
499        ↵
500        ↵
501        ↵
502        ↵
503        ↵
504        ↵
505        ↵
506        ↵
507        ↵
508        ↵
509        ↵
510        ↵
511        ↵
512        ↵
513        ↵
514        ↵
515        ↵
516        ↵
517        ↵
518        ↵
519        ↵
520        ↵
521        ↵
522        ↵
523        ↵
524        ↵
525        ↵
526        ↵
527        ↵
528        ↵
529        ↵
530        ↵
531        ↵
532        ↵
533        ↵
534        ↵
535        ↵
536        ↵
537        ↵
538        ↵
539        ↵
540        ↵
541        ↵
542        ↵
543        ↵
544        ↵
545        ↵
546        ↵
547        ↵
548        ↵
549        ↵
550        ↵
551        ↵
552        ↵
553        ↵
554        ↵
555        ↵
556        ↵
557        ↵
558        ↵
559        ↵
560        ↵
561        ↵
562        ↵
563        ↵
564        ↵
565        ↵
566        ↵
567        ↵
568        ↵
569        ↵
570        ↵
571        ↵
572        ↵
573        ↵
574        ↵
575        ↵
576        ↵
577        ↵
578        ↵
579        ↵
580        ↵
581        ↵
582        ↵
583        ↵
584        ↵
585        ↵
586        ↵
587        ↵
588        ↵
589        ↵
590        ↵
591        ↵
592        ↵
593        ↵
594        ↵
595        ↵
596        ↵
597        ↵
598        ↵
599        ↵
600        ↵
601        ↵
602        ↵
603        ↵
604        ↵
605        ↵
606        ↵
607        ↵
608        ↵
609        ↵
610        ↵
611        ↵
612        ↵
613        ↵
614        ↵
615        ↵
616        ↵
617        ↵
618        ↵
619        ↵
620        ↵
621        ↵
622        ↵
623        ↵
624        ↵
625        ↵
626        ↵
627        ↵
628        ↵
629        ↵
630        ↵
631        ↵
632        ↵
633        ↵
634        ↵
635        ↵
636        ↵
637        ↵
638        ↵
639        ↵
640        ↵
641        ↵
642        ↵
643        ↵
644        ↵
645        ↵
646        ↵
647        ↵
648        ↵
649        ↵
650        ↵
651        ↵
652        ↵
653        ↵
654        ↵
655        ↵
656        ↵
657        ↵
658        ↵
659        ↵
660        ↵
661        ↵
662        ↵
663        ↵
664        ↵
665        ↵
666        ↵
667        ↵
668        ↵
669        ↵
670        ↵
671        ↵
672        ↵
673        ↵
674        ↵
675        ↵
676        ↵
677        ↵
678        ↵
679        ↵
680        ↵
681        ↵
682        ↵
683        ↵
684        ↵
685        ↵
686        ↵
687        ↵
688        ↵
689        ↵
690        ↵
691        ↵
692        ↵
693        ↵
694        ↵
695        ↵
696        ↵
697        ↵
698        ↵
699        ↵
700        ↵
701        ↵
702        ↵
703        ↵
704        ↵
705        ↵
706        ↵
707        ↵
708        ↵
709        ↵
710        ↵
711        ↵
712        ↵
713        ↵
714        ↵
715        ↵
716        ↵
717        ↵
718        ↵
719        ↵
720        ↵
721        ↵
722        ↵
723        ↵
724        ↵
725        ↵
726        ↵
727        ↵
728        ↵
729        ↵
730        ↵
731        ↵
732        ↵
733        ↵
734        ↵
735        ↵
736        ↵
737        ↵
738        ↵
739        ↵
740        ↵
741        ↵
742        ↵
743        ↵
744        ↵
745        ↵
746        ↵
747        ↵
748        ↵
749        ↵
750        ↵
751        ↵
752        ↵
753        ↵
754        ↵
755        ↵
756        ↵
757        ↵
758        ↵
759        ↵
760        ↵
761        ↵
762        ↵
763        ↵
764        ↵
765        ↵
766        ↵
767        ↵
768        ↵
769        ↵
770        ↵
771        ↵
772        ↵
773        ↵
774        ↵
775        ↵
776        ↵
777        ↵
778        ↵
779        ↵
780        ↵
781        ↵
782        ↵
783        ↵
784        ↵
785        ↵
786        ↵
787        ↵
788        ↵
789        ↵
790        ↵
791        ↵
792        ↵
793        ↵
794        ↵
795        ↵
796        ↵
797        ↵
798        ↵
799        ↵
800        ↵
801        ↵
802        ↵
803        ↵
804        ↵
805        ↵
806        ↵
807        ↵
808        ↵
809        ↵
810        ↵
811        ↵
812        ↵
813        ↵
814        ↵
815        ↵
816        ↵
817        ↵
818        ↵
819        ↵
820        ↵
821        ↵
822        ↵
823        ↵
824        ↵
825        ↵
826        ↵
827        ↵
828        ↵
829        ↵
830        ↵
831        ↵
832        ↵
833        ↵
834        ↵
835        ↵
836        ↵
837        ↵
838        ↵
839        ↵
840        ↵
841        ↵
842        ↵
843        ↵
844        ↵
845        ↵
846        ↵
847        ↵
848        ↵
849        ↵
850        ↵
851        ↵
852        ↵
853        ↵
854        ↵
855        ↵
856        ↵
857        ↵
858        ↵
859        ↵
860        ↵
861        ↵
862        ↵
863        ↵
864        ↵
865        ↵
866        ↵
867        ↵
868        ↵
869        ↵
870        ↵
871        ↵
872        ↵
873        ↵
874        ↵
875        ↵
876        ↵
877        ↵
878        ↵
879        ↵
880        ↵
881        ↵
882        ↵
883        ↵
884        ↵
885        ↵
886        ↵
887        ↵
888        ↵
889        ↵
890        ↵
891        ↵
892        ↵
893        ↵
894        ↵
895        ↵
896        ↵
897        ↵
898        ↵
899        ↵
900        ↵
901        ↵
902        ↵
903        ↵
904        ↵
905        ↵
906        ↵
907        ↵
908        ↵
909        ↵
910        ↵
911        ↵
912        ↵
913        ↵
914        ↵
915        ↵
916        ↵
917        ↵
918        ↵
919        ↵
920        ↵
921        ↵
922        ↵
923        ↵
924        ↵
925        ↵
926        ↵
927        ↵
928        ↵
929        ↵
930        ↵
931        ↵
932        ↵
933        ↵
934        ↵
935        ↵
936        ↵
937        ↵
938        ↵
939        ↵
940        ↵
941        ↵
942        ↵
943        ↵
944        ↵
945        ↵
946        ↵
947        ↵
948        ↵
949        ↵
950        ↵
951        ↵
952        ↵
953        ↵
954        ↵
955        ↵
956        ↵
957        ↵
958        ↵
959        ↵
960        ↵
961        ↵
962        ↵
963        ↵
964        ↵
965        ↵
966        ↵
967        ↵
968        ↵
969        ↵
970        ↵
971        ↵
972        ↵
973        ↵
974        ↵
975        ↵
976        ↵
977        ↵
978        ↵
979        ↵
980        ↵
981        ↵
982        ↵
983        ↵
984        ↵
985        ↵
986        ↵
987        ↵
988        ↵
989        ↵
990        ↵
991        ↵
992        ↵
993        ↵
994        ↵
995        ↵
996        ↵
997        ↵
998        ↵
999        ↵
1000       ↵

```

Listing 31: Consultando límites del sistema

Ejemplo Integrador: Un Mini Shell

Para consolidar todos estos conceptos, implementaremos un mini shell que puede ejecutar comandos con argumentos y manejar tuberías simples.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/wait.h>
6
7  #define MAX_INPUT 1024
8  #define MAX_ARGS 64
9
10 void parsear_comando(char *input, char **args) {
11     int i = 0;
12     args[i] = strtok(input, " \t\n");
13     while (args[i] != NULL && i < MAX_ARGS - 1) {
14         i++;
15         args[i] = strtok(NULL, " \t\n");
16     }
17     args[i] = NULL;
18 }
19
20 int main() {
21     char input[MAX_INPUT];
22     char *args[MAX_ARGS];
23     pid_t pid;
24     int background;
25

```

```

26     printf("Mini Shell v1.0\n");
27     printf("Comandos: exit para salir\n");
28
29     while (1) {
30         printf("msh> ");
31         fflush(stdout);
32
33         if (fgets(input, MAX_INPUT, stdin) == NULL) {
34             break; // EOF
35         }
36
37         // Eliminar salto de linea al final
38         input[strcspn(input, "\n")] = '\0';
39
40         // Comando vacio
41         if (strlen(input) == 0) {
42             continue;
43         }
44
45         // Comando exit
46         if (strcmp(input, "exit") == 0) {
47             break;
48         }
49
50         // Verificar si es background (termina con &)
51         background = 0;
52         if (input[strlen(input) - 1] == '&') {
53             background = 1;
54             input[strlen(input) - 1] = '\0'; // Eliminar ←
55             el &
56         }
57
58         parsear_comando(input, args);
59
60         if (args[0] == NULL) {
61             continue;
62         }
63
64         // Ejecutar comando
65         pid = fork();
66
67         if (pid == 0) {
68             // Proceso hijo
69             execvp(args[0], args);
70
71             // Si llegamos aqui, execvp fallo
72             fprintf(stderr, "msh: comando no encontrado: ←
             %s\n", args[0]);
             exit(1);

```



```
73     }
74     else if (pid > 0) {
75         // Proceso padre
76         if (!background) {
77             // Esperar si no es background
78             int status;
79             waitpid(pid, &status, 0);
80         }
81         else {
82             printf("[Proceso en background: %d]\n", ←
83                    pid);
84         }
85     }
86     else {
87         perror("fork fallo");
88     }
89 }
90 printf("Saliendo del shell...\n");
91 return 0;
92 }
```

Listing 32: Mini Shell en C

Resumen de la Unidad XI

Hemos explorado las formas en que un programa en C puede interactuar directamente con el sistema operativo para gestionar recursos:

- **Llamadas al sistema:** Interfaces directas al kernel, específicas del SO.
- **Gestión de procesos:**
 - `fork()`: Crear nuevos procesos.
 - `exec()`: Ejecutar nuevos programas.
 - `wait()`, `waitpid()`: Sincronización con procesos hijos.
- **Señales:**
 - `signal()`, `sigaction()`: Capturar señales.
 - `kill()`: Enviar señales.
 - `alarm()`, `setitimer()`: Temporizadores.
- **Memoria avanzada:**
 - `mmap()`, `munmap()`: Mapeo de memoria (para memoria compartida o archivos mapeados).

■ **Comunicación entre procesos (IPC):**

- `pipe()`: Tuberías sin nombre para procesos relacionados.
- `mkfifo()`: Tuberías con nombre para procesos no relacionados.

■ **Tiempo de alta resolución:**

- `gettimeofday()`: Medición precisa de intervalos.

■ **Límites del sistema:**

- `getrlimit()`, `setrlimit()`: Consultar y establecer límites.

Estas herramientas nos permiten crear programas mucho más poderosos y flexibles, desde shells hasta servidores y sistemas complejos que aprovechan al máximo los recursos del sistema operativo.

Unidad XII: Profesores Particulares de Algoritmos y Datos - Apoyo Académico

A lo largo de este tratado, hemos recorrido un largo camino: desde los conceptos más básicos de algoritmos y programación en C, hasta la gestión de recursos del sistema y una introducción a la programación orientada a objetos. Hemos explorado arrays, punteros, estructuras de control, funciones, archivos, algoritmos de ordenamiento y búsqueda, y mucho más.

Sin embargo, aprender a programar es un viaje que a veces requiere orientación adicional. Ya sea que estés cursando el CBC, la secundaria, la universidad, o simplemente aprendiendo por tu cuenta, contar con un buen profesor particular puede marcar la diferencia. Esta unidad está dedicada a proporcionarte recursos de apoyo: una lista de profesores particulares especializados en algoritmos, programación en C, y materias afines, así como enlaces a canales de YouTube que pueden ayudarte a reforzar los conceptos de esta materia.

Profesores Particulares Recomendados

A continuación, presentamos una lista de docentes y estudiantes avanzados que ofrecen clases particulares de algoritmos, programación en C, matemática y física. Todos ellos tienen experiencia en la enseñanza y pueden ayudarte a superar tus dificultades académicas.

Nombre	Especialidad	Contacto	Niveles
Nahuel Villafañe <i>(Estudiante avanzado)</i>	Matemática y Física Clases personalizadas, enfoque práctico	IG: <code>clasesconnahu</code> (consultar disponibilidad)	Primario, Secundario, CBC
Martín Cardozo <i>(Docente responsable de la materia)</i>	Algoritmos y Estructuras de Datos, Programación C/C++ Amplia experiencia en la cátedra Algoritmos y Programación I	Universitario, Terciario Tel: 11-1234-5678	
Laura González <i>(Licenciada en Ciencias de la Computación)</i>	Matemática (Análisis, Álgebra), Física, Programación C Clases de apoyo para CBC y primeros años	Secundario, CBC, Universitario Tel: 11-2345-6789	
Carlos Rodríguez <i>(Ingeniero en Sistemas)</i>	Algoritmos, Programación C++, Estructuras de Datos Especialista en ordenamiento y búsqueda, memoria dinámica	Universitario, Terciario Tel: 11-3456-7890	
Ana Martínez <i>(Profesora de Matemática y Computación)</i>	Introducción a la Programación, Lógica, Matemática Clases lúdicas y didácticas para principiantes	Primario, Secundario Tel: 11-4567-8901	
Javier López <i>(Estudiante avanzado)</i>	Programación en C, Punteros, Memoria Dinámica, Archivos Ayuda con trabajos prácticos y preparación de parciales	Universitario Tel: 11-5678-9012	

Recomendaciones para elegir profesor particular

- **Define tus necesidades:** ¿Necesitas ayuda con un tema específico (punteros, recursividad, archivos) o con la materia en general?
- **Consulta disponibilidad:** Los horarios de los profesores pueden variar, especialmente en épocas de parciales.
- **Pregunta por su método de enseñanza:** Algunos profesores se enfocan en la

teoría, otros en la práctica con ejercicios, y otros en la resolución de trabajos prácticos.

- **Considera clases grupales:** Si tienes compañeros con las mismas dificultades, las clases grupales suelen ser más económicas.

Recursos en YouTube para Aprender Algoritmos y Programación en C

Internet está lleno de recursos gratuitos. YouTube, en particular, tiene canales excelentes que explican los conceptos de programación de manera visual y práctica. Aquí tienes una selección de los mejores canales y videos para complementar tu estudio.

Canales Recomendados (Español)

Canal	Descripción	Enlace
Programación ATS	Uno de los canales más completos en español sobre programación en C y C++. Explicaciones claras y ejemplos paso a paso.	https://www.youtube.com/@ProgramacionATS
Código Facilito	Tutoriales de programación en diversos lenguajes, incluyendo C. Muy didáctico.	https://www.youtube.com/@codigofacilito
La Geekipedia De Ernesto	Videos sobre algoritmos, estructuras de datos y programación en C con ejemplos prácticos.	https://www.youtube.com/@LaGeekipediaDeErnesto
Mitos y Leyendas de la Programación	Contenido variado sobre programación, incluyendo series completas sobre C y algoritmos.	https://www.youtube.com/@MitosProgramacion
BettaTech	Tutoriales de programación en C, C++, y otros lenguajes. Explicaciones claras y concisas.	https://www.youtube.com/@BettaTech

Videos Recomendados por Tema

Introducción a la Programación y Algoritmos

- "Qué es un algoritmo Programación ATS: <https://www.youtube.com/watch?v=U3CGMyjz1vM>
- "Lógica de programación: Algoritmos Código Facilito: <https://www.youtube.com/watch?v=QdPcH80R5Zo>

- **Introducción a la programación - Conceptos básicos** La Geekipedia De Ernesto: <https://www.youtube.com/watch?v=Kp4Mvapo9kc>

Lenguaje C (Unidades I-III)

- **Curso de Programación en C Completo Programación ATS** (Lista de reproducción): <https://www.youtube.com/playlist?list=PLWtYZ2ejMVJkXw7tV6l7vRg7A6hU7wYkF>
- **"Punteros en C - Explicación completa Programación ATS:** <https://www.youtube.com/watch?v=s8T7cPnYhHk>
- **.Arrays en C Código Facilito:** <https://www.youtube.com/watch?v=HXwBvFQk1xA>
- **"Memoria dinámica en C (malloc, calloc, realloc, free) Programación ATS:** https://www.youtube.com/watch?v=U_nP5Z5VzZc
- **"Punteros a punteros en C La Geekipedia De Ernesto:** https://www.youtube.com/watch?v=HZnQ7p60_vk

Estructuras de Control (Unidad II)

- **.Estructuras de control en C - if, else, switch Programación ATS:** <https://www.youtube.com/watch?v=W8KJg3E6H3w>
- **"Bucles en C - for, while, do while Programación ATS:** <https://www.youtube.com/watch?v=3F8A7Q7q7qI>
- **"Sentencias break y continue en C Código Facilito:** <https://www.youtube.com/watch?v=G3YrLQ6Z2qI>

Funciones (Unidad V)

- **"Funciones en C - Conceptos básicos Programación ATS:** <https://www.youtube.com/watch?v=W8KJg3E6H3w>
- **"Paso por valor y paso por referencia en C Programación ATS:** <https://www.youtube.com/watch?v=Qq1C1XvZzYQ>
- **Recursividad en C - Factorial y Fibonacci Programación ATS:** <https://www.youtube.com/watch?v=R6S5X2Z3F7I>

Archivos (Unidad VIII)

- **"Manejo de archivos en C - fopen, fclose, fread, fwrite Programación ATS:** <https://www.youtube.com/watch?v=Z2U6JqXq3qM>
- **.Archivos de texto vs binarios en C La Geekipedia De Ernesto:** <https://www.youtube.com/watch?v=Y3LrQ6Z2qI>

Algoritmos de Ordenamiento y Búsqueda (Unidad IX)

- "Búsqueda secuencial y binaria en C Programación ATS: <https://www.youtube.com/watch?v=Kp4Mvapo9kc>
- .ºrdenamiento por inserción en C Programación ATS: <https://www.youtube.com/watch?v=U3CGMyjz1vM>
- .ºrdenamiento por selección en C Programación ATS: <https://www.youtube.com/watch?v=QdPcH80R5Zo>
- .ºrdenamiento Shell en C Programación ATS: <https://www.youtube.com/watch?v=HXwBvFQk1xA>
- "Merge Sort en C Programación ATS: <https://www.youtube.com/watch?v=R6S5X2Z3F7I>
- "Quick Sort en C Programación ATS: <https://www.youtube.com/watch?v=Z2U6JqXq3qM>

Programación Orientada a Objetos (Unidad X)

- Introducción a la POO en C++ Programación ATS: <https://www.youtube.com/watch?v=W8KJg3E6H3w>
- Clases y objetos en C++ Programación ATS: <https://www.youtube.com/watch?v=Qq1C1XvZzYQ>
- Constructores y destructores en C++ Programación ATS: <https://www.youtube.com/watch?v=R6S5X2Z3F7I>
- "Sobrecarga de operadores en C++ Programación ATS: <https://www.youtube.com/watch?v=Z2U6JqXq3qM>

Manejo de Recursos del Sistema (Unidad XI)

- "Procesos en C - fork, exec, wait Programación ATS: <https://www.youtube.com/watch?v=Y3LrQ6Z2qI>
- "Señales en C - signal, kill, alarm Programación ATS: <https://www.youtube.com/watch?v=HXwBvFQk1xA>
- "Tuberías (pipes) en C Programación ATS: https://www.youtube.com/watch?v=U_nP5Z5VzZc

Plataformas de Ejercicios Prácticos

Además de los videos, practicar es fundamental. Estas plataformas ofrecen ejercicios de programación con verificación automática:

- **Codecademy:** <https://www.codecademy.com/learn/learn-c> - Curso interactivo de C.
- **HackerRank:** <https://www.hackerrank.com/domains/c> - Ejercicios de C clasificados por dificultad.
- **LeetCode:** <https://leetcode.com/problemset/algorithms/> - Problemas de algoritmos (muchos en C).
- **Programiz:** <https://www.programiz.com/c-programming> - Tutoriales y ejemplos.
- **OmegaUp:** <https://omegaup.com/> - Plataforma en español con problemas de programación competitiva.

Consejos Finales para el Estudio

1. **No memorices, comprende:** La programación no se trata de memorizar sintaxis, sino de entender conceptos y saber aplicarlos.
2. **Programa todos los días:** La práctica constante es la clave. Dedicar al menos 30 minutos diarios a escribir código.
3. **Depura:** Cuando un programa no funciona, no te frustres. Usa el depurador (gdb) o imprime variables para entender qué está pasando.
4. **Enseña:** Explicarle un concepto a alguien más es la mejor forma de afianzarlo.
5. **Consulta:** No dudes en preguntar a tus profesores, compañeros o en foros como Stack Overflow.

¡Mucho éxito en tu viaje por el mundo de la programación!

TRATADO DE ALGORITMO Y DATOS II

Nahuel Villafañe

Estudiante avanzado de Física

@clasesconnahu

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
2026

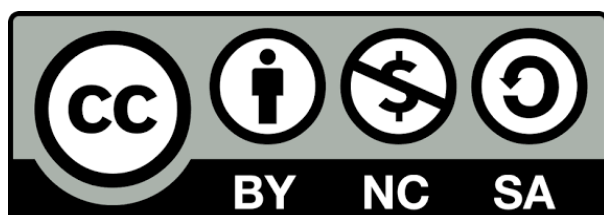


Figura 1: Enter Caption

Capítulo 1

Unidad I: Introducción al Análisis de Algoritmos

1.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Definir formalmente un algoritmo y comprender su rol como tecnología fundamental en la resolución de problemas computacionales.
- Analizar rigurosamente el tiempo de ejecución de un algoritmo en el modelo RAM.
- Utilizar con soltura la notación asintótica (O , Ω , Θ , o , ω) para describir el crecimiento de funciones y comparar eficiencias.
- Diferenciar y aplicar el análisis del mejor caso, peor caso y caso promedio.
- Identificar y aplicar las técnicas básicas de análisis de algoritmos iterativos y recursivos, incluyendo la resolución de recurrencias mediante el Teorema Maestro.

1.2. ¿Qué es un Algoritmo? Fundamentos y Definiciones

Un algoritmo es un concepto central en computación. Informalmente, es un procedimiento computacional bien definido que toma un conjunto de valores como **entrada** y produce un conjunto de valores como **salida**. Es una secuencia de pasos computacionales que transforma la entrada en la salida.

1.2.1. Definición Formal

Podemos definir un algoritmo como una secuencia finita de instrucciones precisas, no ambiguas y ejecutables que, partiendo de un estado inicial (la entrada), culmina en un estado final (la salida). La palabra "algoritmo" deriva del nombre del matemático persa Al-Khwārizmī.

1.2.2. Propiedades de un Algoritmo

Un algoritmo debe cumplir con las siguientes propiedades:

- **Entrada:** Recibe cero o más valores de entrada.
- **Salida:** Produce al menos un valor de salida.
- **Definitud:** Cada paso debe estar definido con precisión y sin ambigüedades.
- **Finitud:** El algoritmo debe terminar después de un número finito de pasos.
- **Efectividad:** Todas las operaciones deben ser lo suficientemente básicas como para poder ser realizadas, en principio, por un ser humano con lápiz y papel o por una computadora.

1.2.3. Ejemplo Clásico: El Problema de la Ordenación

El problema de ordenar una secuencia de números es un ejemplo paradigmático.

- **Entrada:** Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.
- **Salida:** Una permutación (reordenamiento) de la secuencia de entrada $\langle a'_1, a'_2, \dots, a'_n \rangle$ tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Un algoritmo para este problema, como el de inserción (Insertion Sort), es un procedimiento que especifica cómo reordenar los números.

1.2.4. Algoritmos como Tecnología

La eficiencia de un algoritmo es una medida crucial. Consideremos dos algoritmos para el mismo problema: Insertion Sort ($\Theta(n^2)$) y Merge Sort ($\Theta(n \log n)$). Para $n = 10^6$:

- Insertion Sort: $\approx 10^{12}$ operaciones.
- Merge Sort: $\approx 20 \times 10^6$ operaciones.

La diferencia es abismal. Incluso si la computadora que ejecuta Insertion Sort es 100 veces más rápida, Merge Sort será más rápido para n grande. El *tiempo de ejecución* es nuestro recurso principal de análisis.

1.3. El Modelo RAM (Random-Access Machine)

Para analizar el tiempo de ejecución de manera rigurosa, necesitamos un modelo computacional. Usaremos el modelo RAM, que es una abstracción simple pero poderosa de una computadora real.

1.3.1. Supuestos del Modelo RAM

- **Instrucciones secuenciales:** Las instrucciones se ejecutan una tras otra, sin concurrencia.
- **Tiempo constante:** Las instrucciones comunes (aritméticas: suma, resta, multiplicación, división; de datos: carga, almacenamiento, copia; de control: saltos condicionales e incondicionales, llamadas a subrutinas) toman un tiempo constante.
- **Acceso aleatorio:** Se accede a la memoria en tiempo constante, independientemente de la dirección.
- **Tamaño de palabra:** Los números se representan con un tamaño de palabra fijo (e.g., $c \log n$ bits para un problema de tamaño n), lo que nos permite indexar arreglos y realizar operaciones aritméticas en tiempo constante.

Este modelo es lo suficientemente simple para el análisis y lo suficientemente realista para predecir el comportamiento en computadoras reales.

1.3.2. Limitaciones del Modelo

El modelo RAM ignora la jerarquía de memoria (cachés, memoria virtual) y la paralelización, aspectos importantes en la práctica pero que complican el análisis. Sin embargo, para la mayoría de los algoritmos, el modelo RAM es un excelente predictor de rendimiento.

1.4. Análisis del Tiempo de Ejecución

El tiempo de ejecución de un algoritmo depende de varios factores.

1.4.1. Tamaño de la Entrada

La medida más natural depende del problema.

- **Ordenación:** n , el número de elementos.
- **Multiplicación de enteros:** El número total de bits de los operandos.
- **Algoritmos de grafos:** $|V|$ (número de vértices) y $|E|$ (número de aristas).

En notación asintótica, usaremos n para denotar la medida del tamaño de la entrada, a menos que se especifique lo contrario.

1.4.2. Tiempo de Ejecución como Función

Definimos el tiempo de ejecución $T(n)$ como el número de **operaciones primitivas** o **pasos** ejecutados por el algoritmo para una entrada de tamaño n . Asumimos que cada línea de pseudocódigo toma un tiempo constante c_i , aunque diferentes líneas pueden tener diferentes constantes.

1.4.3. Mejor Caso, Peor Caso y Caso Promedio

Dado que el tiempo puede variar para diferentes entradas del mismo tamaño, analicemos:

- **Mejor caso:** El tiempo mínimo para cualquier entrada de tamaño n .
- **Peor caso:** El tiempo máximo para cualquier entrada de tamaño n . Es el más utilizado porque proporciona una garantía de que el algoritmo nunca tardará más.
- **Caso promedio:** El tiempo esperado, promediado sobre todas las entradas de tamaño n bajo una distribución de probabilidad dada.

1.4.4. Análisis Detallado del Insertion Sort

Consideremos el pseudocódigo de Insertion Sort. Analizaremos cada línea, asignando un costo c_i y contando su ejecución.

```

1 INSERTION-SORT(A)
2   for j = 2 to A.length
3     key = A[j]
4     // Inserta A[j] en la secuencia ordenada A[1..j-1]
5     i = j - 1
6     while i > 0 and A[i] > key
7       A[i+1] = A[i]
8       i = i - 1
9     A[i+1] = key

```

Listing 1.1: Insertion Sort (CLRS)

Línea	Costo	Vec. Ejec.
1	c_1	n
2	c_2	$n - 1$
3	c_3	$n - 1$ (comentario, 0)
4	c_4	$n - 1$
5	c_5	$\sum_{j=2}^n t_j$
6	c_6	$\sum_{j=2}^n (t_j - 1)$
7	c_7	$\sum_{j=2}^n (t_j - 1)$
8	c_8	$n - 1$

Donde t_j es el número de veces que se ejecuta la prueba del bucle **while** para un valor de j . El tiempo total es:

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

- **Mejor caso:** El arreglo ya está ordenado. Entonces $t_j = 1$ para cada j .

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

Es una función lineal: $T(n) = an + b = \Theta(n)$.

- **Peor caso:** El arreglo está en orden inverso. Entonces $t_j = j$ para cada j .

$$\sum_{j=2}^n t_j = \sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^n (t_j - 1) = \sum_{j=2}^n (j - 1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

Sustituyendo, obtenemos una función cuadrática: $T(n) = \frac{c_5+c_6+c_7}{2}n^2 + \dots = \Theta(n^2)$.

1.5. Notación Asintótica: El Lenguaje del Crecimiento

Para simplificar, ignoramos las constantes y los términos de menor orden, enfocándonos en la **tasa de crecimiento** de la función. Esto nos permite comparar algoritmos independientemente de la máquina o de las constantes de implementación.

1.5.1. Notación Θ (Theta) - Cota Ajustada Asintóticamente

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2, n_0 > 0 \text{ tal que } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0\}$$

$f(n) = \Theta(g(n))$ significa que $f(n)$ está .atrapada.entre dos múltiplos constantes de $g(n)$ para n suficientemente grande.

Ejemplo

Demostrar que $\frac{1}{2}n^2 - 3n = \Theta(n^2)$. Buscamos c_1, c_2, n_0 tales que $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$ para $n \geq n_0$. Dividiendo por n^2 : $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$. Para $n \geq 1$, $\frac{1}{2} - \frac{3}{n} \leq \frac{1}{2}$, así que podemos elegir $c_2 = 1/2$. Para $n \geq 12$, $\frac{1}{2} - \frac{3}{n} \geq \frac{1}{4}$, así que podemos elegir $c_1 = 1/4$ y $n_0 = 12$.

1.5.2. Notación O (Big-O) - Cota Superior Asintótica

$$O(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq f(n) \leq c g(n) \text{ para todo } n \geq n_0\}$$

Proporciona un límite superior. Si $f(n) = O(g(n))$, entonces $g(n)$ crece al menos tan rápido como $f(n)$.

Ejemplo

$2n = O(n^2)$ porque para $c = 2, n_0 = 1$, $2n \leq 2n^2$. $2n = O(n)$ también es cierto.

1.5.3. Notación Ω (Omega) - Cota Inferior Asintótica

$$\Omega(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$$

Proporciona un límite inferior. Si $f(n) = \Omega(g(n))$, entonces $f(n)$ crece al menos tan rápido como $g(n)$.

Ejemplo

$\sqrt{n} = \Omega(\lg n)$ porque para $c = 1, n_0 = 16$, $\sqrt{16} = 4 \geq \lg 16 = 4$, y para $n > 16$ la desigualdad se mantiene.

1.5.4. Notación o (small-o) - Cota Superior no Ajustada

$$o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq f(n) < cg(n) \text{ para todo } n \geq n_0\}$$

Indica que $f(n)$ es asintóticamente despreciable frente a $g(n)$. Equivalente a $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.

Ejemplo

$$n^{1,9999} = o(n^2).$$

1.5.5. Notación ω (small-omega) - Cota Inferior no Ajustada

$$\omega(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq cg(n) < f(n) \text{ para todo } n \geq n_0\}$$

Indica que $f(n)$ es asintóticamente dominante frente a $g(n)$. Equivalente a $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Ejemplo

$$n^{2,0001} = \omega(n^2).$$

1.5.6. Teorema Fundamental

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ y } f(n) = \Omega(g(n))$$

Este teorema nos permite obtener cotas ajustadas combinando cotas superiores e inferiores.

1.5.7. Relaciones y Analogías

Podemos pensar en las notaciones asintóticas como análogas a las comparaciones entre números reales:

- $f(n) = O(g(n))$ como $a \leq b$
- $f(n) = \Omega(g(n))$ como $a \geq b$
- $f(n) = \Theta(g(n))$ como $a = b$
- $f(n) = o(g(n))$ como $a < b$
- $f(n) = \omega(g(n))$ como $a > b$

Advertencia: No todas las funciones son comparables. Por ejemplo, $n^{1+\sin n}$ oscila y no es comparable con n .

1.6. Propiedades y Manipulación de la Notación Asintótica

1.6.1. Transitividad

- $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
- Lo mismo aplica para O , Ω , o , ω .

1.6.2. Reflexividad

- $f(n) = \Theta(f(n))$, $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$.

1.6.3. Simetría

- $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$.

1.6.4. Simetría Transpuesta

- $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$.
- $f(n) = o(g(n)) \iff g(n) = \omega(f(n))$.

1.6.5. Operaciones con Notación Asintótica

Podemos usar la notación asintótica en expresiones algebraicas:

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ significa que existe una función $f(n) \in \Theta(n)$ tal que la expresión es igual a $2n^2 + f(n)$.
- $2n^2 + \Theta(n) = \Theta(n^2)$ significa que para cualquier $f(n) \in \Theta(n)$, existe $g(n) \in \Theta(n^2)$ tal que $2n^2 + f(n) = g(n)$.

1.7. Análisis de Algoritmos Recursivos y Recurrencias

Los algoritmos recursivos se analizan mediante **recurrencias**. Una recurrencia es una ecuación que define una función en términos de sus valores para entradas más pequeñas.

1.7.1. Ejemplo: Merge Sort

Para Merge Sort, el algoritmo divide el problema en dos mitades, las ordena recursivamente y luego las combina en tiempo lineal.

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$

1.7.2. Teorema Maestro

El Teorema Maestro proporciona una solución directa para recurrencias de la forma $T(n) = aT(n/b) + f(n)$, donde $a \geq 1$, $b > 1$ y $f(n)$ es una función asintóticamente positiva.

Teorema 1.1 (Teorema Maestro). Sean $a \geq 1$, $b > 1$ constantes, $f(n)$ una función y $T(n)$ definida sobre los enteros no negativos por:

$$T(n) = aT(n/b) + f(n)$$

Entonces:

1. Si $f(n) = O(n^{\log_b a - \epsilon})$ para alguna constante $\epsilon > 0$, entonces $T(n) = \Theta(n^{\log_b a})$.
2. Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ para alguna constante $k \geq 0$, entonces $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
3. Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ para alguna constante $\epsilon > 0$, y si $af(n/b) \leq cf(n)$ para alguna constante $c < 1$ y todo n suficientemente grande, entonces $T(n) = \Theta(f(n))$.

Ejemplo

[Caso 1] $T(n) = 9T(n/3) + n$. Aquí $a = 9$, $b = 3$, $f(n) = n$, $\log_b a = \log_3 9 = 2$. $f(n) = n = O(n^{2-1})$ con $\epsilon = 1$. Por lo tanto, $T(n) = \Theta(n^2)$.

Ejemplo

[Caso 2] $T(n) = T(2n/3) + 1$. Aquí $a = 1$, $b = 3/2$, $f(n) = 1$, $\log_b a = \log_{3/2} 1 = 0$. $f(n) = 1 = \Theta(n^0 \log^0 n)$. Por lo tanto, $T(n) = \Theta(\log n)$.

Ejemplo

[Caso 3] $T(n) = 3T(n/4) + n \log n$. Aquí $a = 3$, $b = 4$, $\log_b a = \log_4 3 \approx 0,792$. $f(n) = n \log n = \Omega(n^{0,792+0,2})$. Verificamos la condición de regularidad: $af(n/b) = 3(n/4) \log(n/4) = \frac{3}{4}n(\log n - 2) \leq \frac{3}{4}n \log n$. Con $c = 3/4$, se cumple. Por lo tanto, $T(n) = \Theta(n \log n)$.

1.8. Ejercicios Resueltos (Ampliados)

Ejercicio

[Ordenamiento por Inserción con Inversiones] Demuestre que el número de comparaciones realizadas por Insertion Sort en el peor caso es $\frac{n(n-1)}{2}$ y que este es el número de inversiones en una permutación inversa.

Solución

En el peor caso, el arreglo está en orden inverso. Para cada j (de 2 a n), la clave se compara con todos los $j-1$ elementos anteriores. El número total de comparaciones es:

$$\sum_{j=2}^n (j-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

Una **inversión** es un par (i, j) con $i < j$ y $A[i] > A[j]$. En una permutación inversa, cada par (i, j) es una inversión. Hay $\binom{n}{2} = \frac{n(n-1)}{2}$ inversiones. Por lo tanto, el número de comparaciones en el peor caso es exactamente el número de inversiones.

Ejercicio

[Análisis de un Bucle Triple] Analice el tiempo de ejecución del siguiente fragmento de código:

```

1 for i = 1 to n
2   for j = 1 to i
3     for k = 1 to j
4       x = x + 1
5
```

Solución

El número de veces que se ejecuta la operación $x = x + 1$ es:

$$\sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} = \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right)$$

Usando $\sum i = n(n+1)/2$ y $\sum i^2 = n(n+1)(2n+1)/6$:

$$\begin{aligned}
&= \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right) = \frac{n(n+1)}{4} \left(\frac{2n+1}{3} + 1 \right) \\
&= \frac{n(n+1)}{4} \cdot \frac{2n+4}{3} = \frac{n(n+1)(n+2)}{6} = \Theta(n^3)
\end{aligned}$$

Ejercicio

[Demostración con Definición de O] Demuestre formalmente que $3n^2 + 2n + 1 = O(n^2)$ usando la definición.

Solución

Debemos encontrar $c > 0$ y $n_0 > 0$ tales que $3n^2 + 2n + 1 \leq cn^2$ para todo $n \geq n_0$. Para $n \geq 1$, tenemos $2n \leq 2n^2$ y $1 \leq n^2$. Entonces:

$$3n^2 + 2n + 1 \leq 3n^2 + 2n^2 + n^2 = 6n^2$$

Por lo tanto, podemos elegir $c = 6$ y $n_0 = 1$.

Ejercicio

[Demostración con Definición de Ω] Demuestre formalmente que $3n^2 + 2n + 1 = \Omega(n^2)$ usando la definición.

Solución

Debemos encontrar $c > 0$ y $n_0 > 0$ tales que $3n^2 + 2n + 1 \geq cn^2$ para todo $n \geq n_0$. Para $n \geq 1$, tenemos $3n^2 + 2n + 1 \geq 3n^2$. Por lo tanto, podemos elegir $c = 3$ y $n_0 = 1$.

Ejercicio

[Demostración de Θ combinando O y Ω] Demuestre que $3n^2 + 2n + 1 = \Theta(n^2)$.

Solución

Por los dos ejercicios anteriores, tenemos $3n^2 + 2n + 1 = O(n^2)$ y $3n^2 + 2n + 1 = \Omega(n^2)$. Por el teorema fundamental, $3n^2 + 2n + 1 = \Theta(n^2)$.

Ejercicio

[Crecimiento de Logaritmos] Demuestre que $\log(n!) = \Theta(n \log n)$.

Solución

Cota inferior: $n! \geq (n/2)^{n/2}$. Tomando logaritmo:

$$\log(n!) \geq \frac{n}{2} \log \left(\frac{n}{2} \right) = \frac{n}{2} (\log n - 1) = \Omega(n \log n)$$

Cota superior: $n! \leq n^n$. Tomando logaritmo:

$$\log(n!) \leq n \log n = O(n \log n)$$

Por lo tanto, $\log(n!) = \Theta(n \log n)$.

Ejercicio

[Comparación de Funciones] Ordene las siguientes funciones por orden de crecimiento asintótico: $n^{3/2}$, $n \log n$, 2^n , $n!$, $\log n$, n , $n^{1/2}$.

Solución

Orden de menor a mayor crecimiento:

$$\log n, \quad n^{1/2}, \quad n, \quad n \log n, \quad n^{3/2}, \quad 2^n, \quad n!$$

- $\log n$ crece más lentamente que cualquier potencia positiva de n .
- $n^{1/2} = \sqrt{n}$ es menor que n .
- n es lineal.
- $n \log n$ es mayor que lineal pero menor que cualquier potencia n^c con $c > 1$.
- $n^{3/2}$ es una potencia > 1 .
- 2^n es exponencial.
- $n!$ es mayor que 2^n .

Ejercicio

[Resolución de Recurrencia con Teorema Maestro] Resuelva $T(n) = 16T(n/4) + n^2$.

Solución

$a = 16$, $b = 4$, $\log_b a = \log_4 16 = 2$. $f(n) = n^2 = \Theta(n^2)$. Es el Caso 2 con $k = 0$. Por lo tanto:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n^2 \log n)$$

Ejercicio

[Resolución de Recurrencia con Teorema Maestro] Resuelva $T(n) = 7T(n/2) + n^3$.

Solución

$a = 7$, $b = 2$, $\log_b a = \log_2 7 \approx 2,807$. $f(n) = n^3 = \Omega(n^{2,807+0,1})$ con $\epsilon = 0,1$. Verificamos la condición de regularidad:

$$af(n/b) = 7(n/2)^3 = 7n^3/8 = \frac{7}{8}n^3$$

Elegimos $c = 7/8 < 1$. Se cumple. Por lo tanto, $T(n) = \Theta(n^3)$.

Ejercicio

[Recurrencia que no aplica al Teorema Maestro] Resuelva $T(n) = 2T(n/2) + n \log n$.

Solución

Aquí $a = 2$, $b = 2$, $\log_b a = 1$. $f(n) = n \log n = \Theta(n \log^1 n)$. Esto parece el Caso 2 con $k = 1$, pero el Teorema Maestro requiere $f(n) = \Theta(n^{\log_b a} \log^k n)$. Aquí $n^{\log_b a} = n$. $f(n) = n \log n$ es exactamente de esa forma con $k = 1$. Por lo tanto:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n) = \Theta(n \log^2 n)$$

Nota: Algunas formulaciones del Teorema Maestro incluyen este caso explícitamente.

Ejercicio

[Análisis de Búsqueda Binaria] Escriba la recurrencia para la búsqueda binaria y resuélvala.

Solución

La búsqueda binaria divide el problema en una mitad (o $n/2$) y realiza una comparación constante.

$$T(n) = T(n/2) + O(1)$$

Aplicando el Teorema Maestro: $a = 1$, $b = 2$, $\log_b a = 0$, $f(n) = O(1) = O(n^0)$. Es el Caso 2 con $k = 0$. Por lo tanto:

$$T(n) = \Theta(\log n)$$

Ejercicio

[Análisis de un Algoritmo de División Desigual] Resuelva $T(n) = T(n/3) + T(2n/3) + n$.

Solución

Este es un caso clásico de divide y vencerás con subproblemas de tamaño desigual. Podemos usar un árbol de recursión. La altura máxima es $h = \log_{3/2} n$ (cuando siempre tomamos el subproblema más grande). En cada nivel, la suma de los tamaños de los subproblemas es n , y el trabajo combinado es n . Por lo tanto, el costo total es $O(n \log n)$. Un análisis más cuidadoso muestra que $T(n) = \Theta(n \log n)$. El Teorema Maestro no aplica directamente porque los subproblemas no son del mismo tamaño.

Ejercicio

[Algoritmo de Strassen] El algoritmo de Strassen para multiplicación de matrices tiene recurrencia $T(n) = 7T(n/2) + O(n^2)$. Resuélvala.

Solución

$a = 7$, $b = 2$, $\log_b a = \log_2 7 \approx 2,807$. $f(n) = O(n^2) = O(n^{2,807-\epsilon})$ con $\epsilon = 0,807$. Es el Caso 1. Por lo tanto:

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$$

Esto es mejor que el algoritmo clásico $O(n^3)$.

1.9. Ejercicios Propuestos (Ampliados)

1. Analice el tiempo de ejecución de un algoritmo que cuenta el número de pares (i, j) en un arreglo de tamaño n tales que $i < j$ y $A[i] > A[j]$ (inversiones).

2. Determine si las siguientes afirmaciones son verdaderas o falsas:

a) $n^2 = O(n^3)$

b) $n^2 = \Omega(n^3)$

c) $n^2 = o(n^3)$

d) $n^2 = \omega(n^3)$

e) $n^2 = \Theta(n^3)$

f) $2^{n+1} = \Theta(2^n)$

g) $2^{2n} = \Theta(2^n)$

h) $\log(n^2) = \Theta(\log n)$

3. Demuestre formalmente que $5n^3 + 10n^2 + 3n = \Theta(n^3)$.

4. Demuestre que $n^{1,5} = \omega(n \log n)$.

5. Resuelva las siguientes recurrencias usando el Teorema Maestro:

a) $T(n) = 4T(n/2) + n$

b) $T(n) = 4T(n/2) + n^2$

c) $T(n) = 4T(n/2) + n^3$

d) $T(n) = 2T(n/2) + n \log n$

e) $T(n) = 3T(n/3) + n \log n$

f) $T(n) = 2T(n/2) + \sqrt{n}$

g) $T(n) = T(n/2) + n$

6. Implemente una versión iterativa de la búsqueda binaria y analice su tiempo de ejecución.

7. Demuestre que si $f(n) = O(g(n))$ entonces $f(n) + g(n) = \Theta(g(n))$.

8. Encuentre el número de operaciones (en términos de n) que realiza el siguiente fragmento de código:

```
1 for i = 1 to n
2   for j = i to n
3     for k = i to j
4       print(i*j*k)
5
```

9. Ordene las siguientes funciones por orden de crecimiento asintótico: n^3 , 2^n , $n \log^2 n$, $n^{0.5}$, $n^{2.5}$, $\log n$, $n!$, $n \log n$, n .
10. Resuelva $T(n) = T(n-1) + 1/n$. (Pista: ¿Qué serie conocida aparece?)
11. Un algoritmo tiene un tiempo de ejecución $T(n) = 3T(n/2) + n \log n$. ¿Qué nos dice el Teorema Maestro sobre su complejidad?
12. Explique por qué el análisis del peor caso es más utilizado que el análisis del mejor caso. Dé un ejemplo donde el mejor caso no sea representativo.
13. ¿Es correcto afirmar que $f(n) = O(g(n))$ implica $2^{f(n)} = O(2^{g(n)})$? Justifique.
14. Diseñe un algoritmo recursivo para encontrar el máximo de un arreglo. Escriba su recurrencia y resuélvala.
15. El algoritmo de multiplicación de Karatsuba tiene recurrencia $T(n) = 3T(n/2) + O(n)$. ¿Cuál es su complejidad asintótica?

Capítulo 2

Unidad II: Notación Asintótica: O , Ω , Θ , o y ω

2.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la necesidad de la notación asintótica para abstraer detalles de implementación y comparar algoritmos.
- Definir formalmente las notaciones O , Ω , Θ , o y ω .
- Demostrar relaciones asintóticas entre funciones utilizando las definiciones formales.
- Aplicar las propiedades de transitividad, reflexividad, simetría y simetría transpuesta.
- Manipular expresiones que contienen notación asintótica en ecuaciones.
- Clasificar funciones por su orden de crecimiento y comparar eficiencias de algoritmos.

2.2. Introducción: ¿Por qué necesitamos la notación asintótica?

Cuando analizamos un algoritmo, a menudo obtenemos expresiones exactas para el tiempo de ejecución, como:

$$T(n) = 3n^2 + 5n + 2$$

Sin embargo, esta expresión depende de constantes que varían según la máquina, el lenguaje de programación y la implementación. Lo que realmente nos interesa es cómo crece $T(n)$ a medida que n se hace grande. La **notación asintótica** nos permite:

- Ignorar constantes multiplicativas y términos de menor orden.
- Centrarnos en la **tasa de crecimiento** de la función.
- Comparar algoritmos independientemente de la plataforma.

Decimos que $3n^2 + 5n + 2$ crece como n^2 , y lo escribimos como $3n^2 + 5n + 2 = \Theta(n^2)$.

2.3. Notación Θ (Theta): Cota Ajustada Asintóticamente

La notación Θ se utiliza para acotar una función tanto superior como inferiormente, dentro de un factor constante. Es la más fuerte de las notaciones porque establece que dos funciones tienen el mismo orden de crecimiento.

2.3.1. Definición Formal

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2, n_0 > 0 \text{ tal que } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ para todo } n \geq n_0\}$$

Se lee: " $f(n)$ es theta de $g(n)$ ". Intuitivamente, $f(n)$ está .atrapada. entre dos múltiplos constantes de $g(n)$ para n suficientemente grande.

Ejemplo

Demostrar que $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.

- Buscamos c_1, c_2, n_0 tales que $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$ para $n \geq n_0$.
- Dividiendo por n^2 : $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$.
- Para $n \geq 1$, $\frac{1}{2} - \frac{3}{n} \leq \frac{1}{2}$, así que elegimos $c_2 = \frac{1}{2}$.
- Para $n \geq 12$, $\frac{1}{2} - \frac{3}{n} \geq \frac{1}{4}$, así que elegimos $c_1 = \frac{1}{4}$ y $n_0 = 12$.

Por lo tanto, $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.

2.3.2. Teorema Fundamental de la Notación Asintótica

Teorema 2.1. $f(n) = \Theta(g(n))$ si y solo si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

Este teorema nos permite obtener cotas ajustadas combinando cotas superiores e inferiores.

2.4. Notación O (Big-O): Cota Superior Asintótica

La notación O proporciona un límite superior. Si $f(n) = O(g(n))$, entonces $g(n)$ crece al menos tan rápido como $f(n)$.

2.4.1. Definición Formal

$$O(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq f(n) \leq c g(n) \text{ para todo } n \geq n_0\}$$

Se lee: " $f(n)$ es O de $g(n)$ ". Indica que $f(n)$ no crece más rápido que $g(n)$.

Ejemplo

Demostrar que $3n^2 + 2n + 1 = O(n^2)$. Para $n \geq 1$, tenemos $2n \leq 2n^2$ y $1 \leq n^2$. Entonces:

$$3n^2 + 2n + 1 \leq 3n^2 + 2n^2 + n^2 = 6n^2$$

Con $c = 6$ y $n_0 = 1$, se cumple la definición.

Ejemplo

Demostrar que $2^n = O(3^n)$. Para $n \geq 0$, $2^n \leq 1 \cdot 3^n$, con $c = 1$, $n_0 = 0$. Por lo tanto, $2^n = O(3^n)$.

2.4.2. Observaciones sobre O

- O proporciona un límite superior, no necesariamente ajustado. Por ejemplo, $n = O(n^2)$ es cierto, aunque n crece más lentamente.
- Si $f(n) = \Theta(g(n))$, entonces $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

2.5. Notación Ω (Omega): Cota Inferior Asintótica

La notación Ω proporciona un límite inferior. Si $f(n) = \Omega(g(n))$, entonces $f(n)$ crece al menos tan rápido como $g(n)$.

2.5.1. Definición Formal

$$\Omega(g(n)) = \{f(n) : \exists c, n_0 > 0 \text{ tal que } 0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$$

Se lee: " $f(n)$ es omega de $g(n)$ ". Indica que $f(n)$ no crece más lentamente que $g(n)$.

Ejemplo

Demostrar que $3n^2 + 2n + 1 = \Omega(n^2)$. Para $n \geq 1$, $3n^2 + 2n + 1 \geq 3n^2$. Con $c = 3$ y $n_0 = 1$, se cumple.

Ejemplo

Demostrar que $\sqrt{n} = \Omega(\log n)$. Para $n \geq 16$, $\sqrt{n} \geq \log n$. Con $c = 1$ y $n_0 = 16$, se cumple.

2.6. Notación o (Small-o): Cota Superior no Ajustada

La notación o indica que $f(n)$ es asintóticamente despreciable frente a $g(n)$. Es una cota superior estricta.

2.6.1. Definición Formal

$$o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq f(n) < cg(n) \text{ para todo } n \geq n_0\}$$

Se lee: " $f(n)$ es pequeña-o de $g(n)$ ". Equivalentemente, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

Ejemplo

Demostrar que $n^{1,9999} = o(n^2)$.

$$\lim_{n \rightarrow \infty} \frac{n^{1,9999}}{n^2} = \lim_{n \rightarrow \infty} \frac{1}{n^{0,0001}} = 0$$

Por lo tanto, $n^{1,9999} = o(n^2)$.

Ejemplo

$n^2 \neq o(n^2)$ porque el límite es 1, no 0.

2.7. Notación ω (Small-omega): Cota Inferior no Ajustada

La notación ω indica que $f(n)$ es asintóticamente dominante frente a $g(n)$. Es una cota inferior estricta.

2.7.1. Definición Formal

$$\omega(g(n)) = \{f(n) : \forall c > 0, \exists n_0 > 0 \text{ tal que } 0 \leq cg(n) < f(n) \text{ para todo } n \geq n_0\}$$

Se lee: " $f(n)$ es pequeña-omega de $g(n)$ ". Equivalentemente, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.

Ejemplo

Demostrar que $n^{2,0001} = \omega(n^2)$.

$$\lim_{n \rightarrow \infty} \frac{n^{2,0001}}{n^2} = \lim_{n \rightarrow \infty} n^{0,0001} = \infty$$

Por lo tanto, $n^{2,0001} = \omega(n^2)$.

2.8. Propiedades de las Notaciones Asintóticas

2.8.1. Transitividad

$$\blacksquare f(n) = \Theta(g(n)) \text{ y } g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$$

- $f(n) = O(g(n))$ y $g(n) = O(h(n)) \implies f(n) = O(h(n))$
- $f(n) = \Omega(g(n))$ y $g(n) = \Omega(h(n)) \implies f(n) = \Omega(h(n))$
- $f(n) = o(g(n))$ y $g(n) = o(h(n)) \implies f(n) = o(h(n))$
- $f(n) = \omega(g(n))$ y $g(n) = \omega(h(n)) \implies f(n) = \omega(h(n))$

2.8.2. Reflexividad

- $f(n) = \Theta(f(n))$
- $f(n) = O(f(n))$
- $f(n) = \Omega(f(n))$

Nota: o y ω no son reflexivas porque $f(n) \neq o(f(n))$ (el límite sería 1, no 0).

2.8.3. Simetría

- $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$

2.8.4. Simetría Transpuesta

- $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$
- $f(n) = o(g(n)) \iff g(n) = \omega(f(n))$

2.8.5. Relaciones de Comparación

Podemos establecer una analogía con la comparación de números reales:

Notación Asintótica	Comparación Numérica
$f(n) = O(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

Advertencia: No todas las funciones son comparables. Por ejemplo, $n^{1+\sin n}$ y n no se pueden comparar asintóticamente porque la razón oscila entre $n^0 = 1$ y n^2 .

2.9. Manipulación de Expresiones con Notación Asintótica

2.9.1. Notación en Ecuaciones

Cuando la notación asintótica aparece en una expresión, representa una función anónima de esa clase.

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ significa que existe $f(n) \in \Theta(n)$ tal que $2n^2 + 3n + 1 = 2n^2 + f(n)$.
- $2n^2 + \Theta(n) = \Theta(n^2)$ significa que para cualquier $f(n) \in \Theta(n)$, existe $g(n) \in \Theta(n^2)$ tal que $2n^2 + f(n) = g(n)$.

2.9.2. Sumas y Productos

- $\sum_{i=1}^n O(i) = O(n^2)$ porque cada término es $O(i)$ y la suma es $O(\sum i) = O(n^2)$.
- $O(f(n)) \cdot O(g(n)) = O(f(n)g(n))$.

2.10. Clasificación de Funciones por Orden de Crecimiento

2.10.1. Funciones Polinomiales

Para $p(n) = a_d n^d + a_{d-1} n^{d-1} + \dots + a_0$ con $a_d > 0$:

$$p(n) = \Theta(n^d)$$

2.10.2. Funciones Logarítmicas

Para cualquier $a > 0$, $\log^a n = o(n^c)$ para cualquier $c > 0$. El logaritmo crece más lentamente que cualquier potencia positiva de n .

2.10.3. Funciones Exponenciales

Para $a > 1$, $n^c = o(a^n)$ para cualquier c . La exponencial crece más rápido que cualquier polinomio.

2.10.4. Funciones Factoriales

$$n! = \omega(2^n) \text{ y } \log(n!) = \Theta(n \log n).$$

2.10.5. Jerarquía Común de Funciones

Orden de menor a mayor crecimiento:

$$1, \log \log n, \log n, \sqrt{n}, n, n \log n, n^2, n^3, \dots, 2^n, n!, 2^{n^2}$$

2.11. Ejercicios Resueltos (Ampliados)

Ejercicio

Demuestre que $n^2 + 100n = \Theta(n^2)$.

Solución

Buscamos c_1, c_2, n_0 tales que $c_1 n^2 \leq n^2 + 100n \leq c_2 n^2$.

- **Cota superior:** $n^2 + 100n \leq n^2 + n^2 = 2n^2$ para $n \geq 100$. Con $c_2 = 2$, $n_0 = 100$.
- **Cota inferior:** $n^2 + 100n \geq n^2$ para $n \geq 0$. Con $c_1 = 1$, $n_0 = 0$.

Por lo tanto, $n^2 + 100n = \Theta(n^2)$.

Ejercicio

Demuestre que $n^2 \neq O(n)$.

Solución

Supongamos que existe c, n_0 tal que $n^2 \leq cn$ para todo $n \geq n_0$. Esto implica $n \leq c$ para todo $n \geq n_0$, lo cual es falso porque n crece sin cota. Por lo tanto, $n^2 \neq O(n)$.

Ejercicio

Demuestre que $2^{n+1} = O(2^n)$.

Solución

$2^{n+1} = 2 \cdot 2^n$. Con $c = 2$ y $n_0 = 0$, se cumple $2^{n+1} \leq 2 \cdot 2^n$.

Ejercicio

Demuestre que $2^{2n} \neq O(2^n)$.

Solución

$2^{2n} = (2^2)^n = 4^n$. Supongamos que existe c, n_0 tal que $4^n \leq c2^n$ para $n \geq n_0$. Esto es $2^n \leq c$, lo cual es falso para n grande.

Ejercicio

Demuestre que $\log(n!) = \Theta(n \log n)$.

Solución

Cota inferior: $n! \geq (n/2)^{n/2}$. Tomando logaritmo:

$$\log(n!) \geq \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2}(\log n - 1) = \Theta(n \log n)$$

Cota superior: $n! \leq n^n$. Tomando logaritmo:

$$\log(n!) \leq n \log n = O(n \log n)$$

Por lo tanto, $\log(n!) = \Theta(n \log n)$.

Ejercicio

Demuestre que $\sqrt{n} = \omega(\log n)$.

Solución

Calculamos el límite:

$$\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{\log n} = \infty$$

Por lo tanto, $\sqrt{n} = \omega(\log n)$.

Ejercicio

Demuestre que $n \log n = o(n^2)$.

Solución

$$\lim_{n \rightarrow \infty} \frac{n \log n}{n^2} = \lim_{n \rightarrow \infty} \frac{\log n}{n} = 0$$

Por lo tanto, $n \log n = o(n^2)$.

Ejercicio

Demuestre que $f(n) = \Theta(g(n))$ si y solo si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, con $0 < c < \infty$.

Solución

(Si): Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, entonces para $\epsilon = c/2$, existe n_0 tal que para $n \geq n_0$:

$$c - \epsilon < \frac{f(n)}{g(n)} < c + \epsilon \implies \frac{c}{2}g(n) < f(n) < \frac{3c}{2}g(n)$$

Por lo tanto, $f(n) = \Theta(g(n))$. **(Solo si):** Si $f(n) = \Theta(g(n))$, entonces existen c_1, c_2, n_0 tales que $c_1g(n) \leq f(n) \leq c_2g(n)$. Esto implica $\frac{f(n)}{g(n)}$ está acotada entre c_1 y c_2 . Sin embargo, el límite puede no existir si la función oscila. Por ejemplo, $f(n) = (2 + \sin n)n$, $g(n) = n$, $f(n) = \Theta(g(n))$ pero el límite no existe.

Ejercicio

Ordene las siguientes funciones por orden de crecimiento asintótico: n^3 , 2^n , $n \log^2 n$, $n^{0,5}$, $n^{2,5}$, $\log n$, $n!$, $n \log n$, n , $n^2 \log n$.

Solución

Orden de menor a mayor:

$$\log n, \quad n^{0,5}, \quad n, \quad n \log n, \quad n \log^2 n, \quad n^2 \log n, \quad n^{2,5}, \quad n^3, \quad 2^n, \quad n!$$

Justificación:

- $\log n$ crece más lentamente que cualquier potencia positiva.
- $n^{0,5} = \sqrt{n}$ es menor que n .
- n es lineal.
- $n \log n$ es mayor que lineal pero menor que $n^{1+\epsilon}$.
- $n \log^2 n$ es mayor que $n \log n$.
- $n^2 \log n$ es mayor que n^2 .
- $n^{2,5}$ es mayor que $n^2 \log n$ porque $n^{0,5}$ crece más rápido que $\log n$.
- n^3 es una potencia mayor.
- 2^n es exponencial.
- $n!$ es mayor que 2^n .

Ejercicio

Determine si las siguientes afirmaciones son verdaderas o falsas:

1. $n^2 = O(n^3)$
2. $n^2 = \Omega(n^3)$
3. $n^2 = o(n^3)$
4. $n^2 = \omega(n^3)$
5. $n^2 = \Theta(n^3)$
6. $2^{n+1} = \Theta(2^n)$
7. $2^{2n} = \Theta(2^n)$
8. $\log(n^2) = \Theta(\log n)$
9. $n \log n = \Theta(n)$

Solución

1. **Verdadero:** $n^2 \leq 1 \cdot n^3$ para $n \geq 1$.
2. **Falso:** $n^2/n^3 = 1/n \rightarrow 0$, no hay constante c tal que $cn^3 \leq n^2$ para n grande.
3. **Verdadero:** $\lim_{n \rightarrow \infty} n^2/n^3 = 0$.
4. **Falso:** El límite es 0, no ∞ .
5. **Falso:** No es Θ porque n^2 crece más lentamente que n^3 .
6. **Verdadero:** $2^{n+1} = 2 \cdot 2^n$, por lo que es $\Theta(2^n)$.
7. **Falso:** $2^{2n} = 4^n$, que crece más rápido que 2^n .
8. **Verdadero:** $\log(n^2) = 2 \log n = \Theta(\log n)$.
9. **Falso:** $n \log n$ crece más rápido que n por un factor $\log n$.

Ejercicio

Demuestre que $f(n) = O(g(n))$ y $g(n) = O(h(n))$ implica $f(n) = O(h(n))$.

Solución

Por definición, existen c_1, n_1 tales que $f(n) \leq c_1 g(n)$ para $n \geq n_1$, y existen c_2, n_2 tales que $g(n) \leq c_2 h(n)$ para $n \geq n_2$. Tomando $n_0 = \max(n_1, n_2)$ y $c = c_1 c_2$, tenemos:

$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) = c h(n)$$

Por lo tanto, $f(n) = O(h(n))$.

Ejercicio

Demuestre que $f(n) = O(g(n))$ implica $g(n) = \Omega(f(n))$.

Solución

Si $f(n) = O(g(n))$, existen c, n_0 tales que $f(n) \leq c g(n)$ para $n \geq n_0$. Esto equivale a $g(n) \geq \frac{1}{c} f(n)$, que es la definición de $g(n) = \Omega(f(n))$.

Ejercicio

Demuestre que $f(n) = o(g(n))$ implica $f(n) = O(g(n))$, pero el recíproco no es cierto.

Solución

Si $f(n) = o(g(n))$, entonces $\lim f(n)/g(n) = 0$, lo que implica que existe n_0 tal que para $n \geq n_0$, $f(n) \leq 1 \cdot g(n)$. Por lo tanto, $f(n) = O(g(n))$. El recíproco no es cierto porque $n = O(n)$ pero $n \neq o(n)$ (el límite es 1, no 0).

Ejercicio

Demuestre que si $f(n) = \Theta(g(n))$, entonces $\log f(n) = \Theta(\log g(n))$, asumiendo que $f(n), g(n) \geq 2$ para n grande.

Solución

Si $f(n) = \Theta(g(n))$, existen c_1, c_2, n_0 tales que $c_1 g(n) \leq f(n) \leq c_2 g(n)$. Tomando logaritmo:

$$\log(c_1) + \log g(n) \leq \log f(n) \leq \log(c_2) + \log g(n)$$

Para n grande, $\log(c_1)$ y $\log(c_2)$ son constantes despreciables frente a $\log g(n)$. Por lo tanto, $\log f(n) = \Theta(\log g(n))$.

Ejercicio

Demuestre que $n^c = \omega(\log^d n)$ para cualquier $c > 0$, $d > 0$.

Solución

$$\lim_{n \rightarrow \infty} \frac{n^c}{\log^d n} = \infty$$

Esto se debe a que cualquier potencia positiva de n crece más rápido que cualquier potencia del logaritmo. Por lo tanto, $n^c = \omega(\log^d n)$.

Ejercicio

Demuestre que $2^n = \omega(n^k)$ para cualquier k constante.

Solución

$$\lim_{n \rightarrow \infty} \frac{2^n}{n^k} = \infty$$

Las exponenciales crecen más rápido que cualquier polinomio. Por lo tanto, $2^n = \omega(n^k)$.

Ejercicio

Determine la relación asintótica entre $f(n) = n^{1+\sin n}$ y $g(n) = n$.

Solución

La función $\sin n$ oscila entre -1 y 1. Por lo tanto, $f(n) = n^{1+\sin n}$ oscila entre $n^0 = 1$ y n^2 . No existe una relación asintótica fija: a veces $f(n)$ es mucho menor que $g(n)$ (cuando $\sin n = -1$), a veces mucho mayor (cuando $\sin n = 1$). Por lo tanto, ni $f(n) = O(g(n))$ ni $f(n) = \Omega(g(n))$ se cumplen.

Ejercicio

Demuestre que $n^{\log n} = \omega(n^k)$ para cualquier k constante.

Solución

Tomando logaritmos:

$$\log(n^{\log n}) = \log n \cdot \log n = \log^2 n$$

$$\log(n^k) = k \log n$$

Dado que $\log^2 n = \omega(\log n)$, tenemos $n^{\log n} = \omega(n^k)$.

Ejercicio

Demuestre que $\log(n!) = \Theta(n \log n)$.

Solución

Cota inferior: $n! \geq (n/2)^{n/2}$. Tomando logaritmo:

$$\log(n!) \geq \frac{n}{2} \log \left(\frac{n}{2} \right) = \frac{n}{2} (\log n - 1) = \Omega(n \log n)$$

Cota superior: $n! \leq n^n$. Tomando logaritmo:

$$\log(n!) \leq n \log n = O(n \log n)$$

Por lo tanto, $\log(n!) = \Theta(n \log n)$.

2.12. Ejercicios Propuestos (Ampliados)

1. Demuestre que $5n^3 + 10n^2 + 3n = \Theta(n^3)$ usando la definición.
2. Demuestre que $n \log n = O(n^2)$ pero $n \log n \neq \Omega(n^2)$.
3. Demuestre que $n^{1.5} = \omega(n \log n)$.
4. Determine si $2^n = \Theta(3^n)$.
5. Demuestre que $\log n = o(n^\epsilon)$ para cualquier $\epsilon > 0$.
6. Demuestre que $n^2 = \omega(n \log n)$.
7. Ordene las siguientes funciones por orden de crecimiento asintótico: $n^{2.5}$, $n \log n$, 2^n , $n!$, $\log n$, n , $n^{1/2}$, $n^{3/2}$, $n^2 \log n$, n^3 .
8. Demuestre que $f(n) = O(g(n))$ implica $f(n) + g(n) = \Theta(g(n))$.
9. Demuestre que $f(n) = o(g(n))$ implica $f(n) + g(n) = \Theta(g(n))$.
10. Demuestre que $f(n) = \Theta(g(n))$ si y solo si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

11. Demuestre que $f(n) = o(g(n))$ si y solo si $g(n) = \omega(f(n))$.
12. Demuestre que la relación $f(n) = O(g(n))$ es reflexiva y transitiva, pero no simétrica.
13. Determine si las siguientes afirmaciones son verdaderas o falsas:
 - a) $n^{0,5} = O(n^{0,4})$
 - b) $n^{0,5} = \Omega(n^{0,4})$
 - c) $n^{0,5} = \Theta(n^{0,4})$
 - d) $\log n = O(1)$
 - e) $\log n = \Omega(1)$
 - f) $2^{\log n} = \Theta(n)$
 - g) $2^{\log^2 n} = O(2^{\log n})$
14. Demuestre que $n^{\log n} = o(2^n)$.
15. Demuestre que $\log \log n = o(\log n)$.
16. Demuestre que $n^{1/2} = o(n^{2/3})$.
17. Demuestre que $n^{1/2} = \omega(\log n)$.
18. Demuestre que si $f(n) = \Theta(g(n))$, entonces $f(n)^k = \Theta(g(n)^k)$ para cualquier $k > 0$.
19. Demuestre que $\sum_{i=1}^n i = \Theta(n^2)$.
20. Demuestre que $\sum_{i=1}^n i^2 = \Theta(n^3)$.
21. Demuestre que $\sum_{i=1}^n \log i = \Theta(n \log n)$.
22. Demuestre que $\sum_{i=1}^n 2^i = \Theta(2^n)$.

Capítulo 3

Unidad III: Análisis de Casos: Mejor, Peor y Promedio

3.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la diferencia fundamental entre el análisis del mejor caso, peor caso y caso promedio.
- Identificar en qué situaciones cada tipo de análisis es relevante.
- Calcular el tiempo de ejecución esperado de algoritmos utilizando técnicas de probabilidad básica.
- Utilizar variables aleatorias indicadoras para simplificar el análisis de casos promedio.
- Aplicar el análisis probabilístico a algoritmos como la búsqueda lineal, la contratación y otros.
- Distinguir entre análisis probabilístico (sobre la entrada) y algoritmos aleatorizados (sobre elecciones internas).

3.2. Introducción: ¿Por qué analizar diferentes casos?

Cuando analizamos un algoritmo, el tiempo de ejecución no es el mismo para todas las entradas de un tamaño dado. Por ejemplo, Insertion Sort puede ser mucho más rápido si el arreglo ya está ordenado que si está en orden inverso. Por lo tanto, necesitamos diferentes tipos de análisis:

- **Mejor caso:** El tiempo mínimo para cualquier entrada de tamaño n . Es útil para entender el límite inferior del algoritmo, pero rara vez es representativo.
- **Peor caso:** El tiempo máximo para cualquier entrada de tamaño n . Proporciona una garantía de que el algoritmo nunca tardará más. Es el más utilizado en la práctica.

- **Caso promedio:** El tiempo esperado, promediado sobre todas las entradas de tamaño n bajo una distribución de probabilidad. Es útil para entender el comportamiento típico.

3.3. El Problema de la Contratación (Hiring Problem)

Para introducir el análisis de casos, estudiaremos un problema clásico que modela la búsqueda de un máximo en una secuencia.

3.3.1. Descripción del Problema

Una agencia de empleo envía un candidato por día. Después de entrevistar a cada candidato, debemos decidir si contratarlo inmediatamente. Solo contratamos si es mejor que todos los anteriores. El costo es c_i por entrevista y c_h por contratación ($c_h > c_i$). Queremos analizar el costo esperado.

3.3.2. Pseudocódigo

```

1 HIRE-ASSISTANT(n)
2   best = 0    // candidato ficticio peor que todos
3   for i = 1 to n
4     entrevistar al candidato i
5     if candidato i es mejor que best
6       best = i
7   contratar candidato i

```

Listing 3.1: HIRE-ASSISTANT

3.3.3. Análisis del Peor Caso

En el peor caso, los candidatos vienen en orden de calidad creciente. Entonces contratamos a todos los n candidatos. El costo total es:

$$\text{Costo} = n \cdot c_i + n \cdot c_h = \Theta(nc_h)$$

Este es un costo alto, pero ¿es realista esperar que siempre ocurra?

3.3.4. Análisis del Caso Promedio

Supongamos que los candidatos llegan en orden aleatorio. ¿Cuál es el número esperado de contrataciones?

Variables Aleatorias Indicadoras

Una técnica poderosa para simplificar el análisis probabilístico es usar **variables aleatorias indicadoras**. Para un evento A , definimos:

$$I\{A\} = \begin{cases} 1 & \text{si } A \text{ ocurre} \\ 0 & \text{en caso contrario} \end{cases}$$

Lema 3.1 (Lema 5.1 de CLRS). *Para un evento A , $E[I\{A\}] = \Pr\{A\}$.*

Aplicación al Problema de Contratación

Definimos $X_i = I\{\text{el candidato } i \text{ es contratado}\}$. Entonces el número total de contrataciones es $X = \sum_{i=1}^n X_i$.

¿Cuál es la probabilidad de que el candidato i sea contratado? Esto ocurre si es mejor que todos los candidatos $1, 2, \dots, i-1$. Como todos los órdenes son igualmente probables, la probabilidad de que el mejor de los primeros i esté en la posición i es $1/i$.

Por lo tanto:

$$E[X_i] = \Pr\{\text{candidato } i \text{ contratado}\} = \frac{1}{i}$$

Ahora, por linealidad de la expectativa:

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{i} = H_n$$

donde H_n es el n -ésimo número armónico.

Sabemos que $H_n = \ln n + \gamma + O(1/n)$, con $\gamma \approx 0,577$. Por lo tanto:

$$E[X] = \Theta(\log n)$$

El costo esperado es:

$$\text{Costo esperado} = n \cdot c_i + c_h \cdot H_n = \Theta(n + c_h \log n)$$

¡Mucho mejor que el peor caso!

3.4. Análisis de Búsqueda Lineal

3.4.1. Descripción

Dado un arreglo $A[1..n]$ y un valor v , queremos encontrar un índice i tal que $A[i] = v$, o devolver NIL si no existe.

3.4.2. Pseudocódigo

```

1 LINEAR-SEARCH(A, v)
2   for i = 1 to n
3     if A[i] == v
4       return i
5   return NIL

```

Listing 3.2: LINEAR-SEARCH

3.4.3. Análisis del Peor Caso

El peor caso ocurre cuando v no está en el arreglo, o está en la última posición. Se realizan n comparaciones:

$$T_{\text{peor}}(n) = \Theta(n)$$

3.4.4. Análisis del Mejor Caso

El mejor caso ocurre cuando v está en la primera posición. Se realiza 1 comparación:

$$T_{\text{mejor}}(n) = \Theta(1)$$

3.4.5. Análisis del Caso Promedio

Supongamos que v está en el arreglo con probabilidad p , y que si está, es igualmente probable que esté en cualquier posición. Además, supongamos que las búsquedas exitosas e infructuosas tienen diferentes distribuciones.

Búsqueda Exitosa

Si v está en el arreglo, la posición es uniforme entre 1 y n . El número de comparaciones es igual a la posición. El valor esperado es:

$$E[\text{comparaciones} \mid \text{exitosa}] = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2} = \Theta(n)$$

Búsqueda Infructuosa

Si v no está en el arreglo, se realizan n comparaciones. El valor esperado es n .

Caso Promedio General

Si la probabilidad de que v esté en el arreglo es p , entonces:

$$E[\text{comparaciones}] = p \cdot \frac{n+1}{2} + (1-p) \cdot n$$

Si $p = 1/2$, por ejemplo:

$$E = \frac{1}{2} \cdot \frac{n+1}{2} + \frac{1}{2} \cdot n = \frac{n+1}{4} + \frac{n}{2} = \frac{3n+1}{4} = \Theta(n)$$

3.5. Análisis de Búsqueda Binaria

3.5.1. Descripción

La búsqueda binaria trabaja sobre un arreglo ordenado, comparando con el elemento medio y reduciendo el espacio de búsqueda a la mitad.

3.5.2. Pseudocódigo

```

1 BINARY-SEARCH(A, low, high, v)
2   if low > high
3   return NIL
4   mid = floor((low+high)/2)
5   if A[mid] == v
6   return mid
7   else if A[mid] > v

```

```

8 7      return BINARY-SEARCH(A, low, mid-1, v)
9 8  else
10 9      return BINARY-SEARCH(A, mid+1, high, v)

```

Listing 3.3: BINARY-SEARCH

3.5.3. Análisis del Peor Caso

En el peor caso, v no está en el arreglo o está en la última comparación. Cada comparación reduce el tamaño del problema a la mitad. La recurrencia es $T(n) = T(n/2) + \Theta(1)$, cuya solución es $T(n) = \Theta(\log n)$.

3.5.4. Análisis del Mejor Caso

El mejor caso ocurre cuando v es el elemento medio. Se realiza 1 comparación: $T_{\text{mejor}}(n) = \Theta(1)$.

3.5.5. Análisis del Caso Promedio

El análisis del caso promedio es más complejo. Para búsqueda binaria, el número esperado de comparaciones es aproximadamente $\log n$, pero la constante es ligeramente menor. Se puede demostrar que:

$$E[\text{comparaciones}] \approx \log_2 n - 1$$

para búsqueda exitosa, y aproximadamente $\log_2 n$ para búsqueda infructuosa.

3.6. El Problema de la Contratación en Línea (Online Hiring)

3.6.1. Descripción

Ahora supongamos que queremos contratar al mejor candidato, pero solo podemos contratar una vez. Decidimos entrevistar a los primeros k candidatos (para tener una referencia) y luego contratar al primer candidato que sea mejor que todos los vistos hasta ese momento.

3.6.2. Pseudocódigo

```

1 ON-LINE-MAXIMUM(k, n)
2   best = -infty
3   for i = 1 to k
4       if score(i) > best
5           best = score(i)
6   for i = k+1 to n
7       if score(i) > best
8           return i
9   return n

```

Listing 3.4: ON-LINE-MAXIMUM

3.6.3. Análisis de Probabilidad de Éxito

Queremos maximizar la probabilidad de contratar al mejor candidato.

Teorema 3.1. *La probabilidad de éxito con la estrategia de rechazar los primeros k candidatos es aproximadamente $\frac{k}{n} \ln \left(\frac{n}{k} \right)$.*

3.6.4. Demostración

Sea S_i el evento de que el mejor candidato está en la posición i y es contratado. Para que esto ocurra:

1. El mejor candidato debe estar en la posición i (probabilidad $1/n$).
2. Ningún candidato entre $k+1$ y $i-1$ debe ser mejor que el máximo de los primeros k . Esto ocurre si el máximo de los primeros $i-1$ está entre los primeros k (probabilidad $k/(i-1)$).

Entonces:

$$\Pr\{S_i\} = \frac{1}{n} \cdot \frac{k}{i-1}$$

La probabilidad total de éxito es:

$$\Pr\{\text{éxito}\} = \sum_{i=k+1}^n \frac{k}{n(i-1)} = \frac{k}{n} \sum_{j=k}^{n-1} \frac{1}{j} \approx \frac{k}{n} \ln \left(\frac{n}{k} \right)$$

3.6.5. Optimización

Para maximizar esta probabilidad, derivamos respecto a k :

$$\frac{d}{dk} \left(\frac{k}{n} \ln \left(\frac{n}{k} \right) \right) = \frac{1}{n} \ln \left(\frac{n}{k} \right) - \frac{1}{n} = 0 \implies \ln \left(\frac{n}{k} \right) = 1 \implies k = \frac{n}{e}$$

La probabilidad máxima es aproximadamente $1/e \approx 0,368$.

3.7. Algoritmos Aleatorizados vs. Análisis Probabilístico

3.7.1. Diferencia Fundamental

- **Análisis Probabilístico:** Asumimos una distribución sobre las entradas (ej. todas las permutaciones igualmente probables). El algoritmo es determinista.
- **Algoritmos Aleatorizados:** El algoritmo toma decisiones aleatorias (usa un generador de números aleatorios). No se asume nada sobre la entrada.

3.7.2. Ejemplo: Quicksort Aleatorizado

El Quicksort determinista tiene peor caso $O(n^2)$ cuando la entrada está ordenada. El Quicksort aleatorizado elige el pivote al azar, evitando que una entrada particular cause el peor caso. El tiempo esperado es $O(n \log n)$ para cualquier entrada.

3.7.3. Pseudocódigo de Quicksort Aleatorizado

```

1 RANDOMIZED-QUICKSORT(A, p, r)
2   1   if p < r
3     2       q = RANDOMIZED-PARTITION(A, p, r)
4     3       RANDOMIZED-QUICKSORT(A, p, q-1)
5     4       RANDOMIZED-QUICKSORT(A, q+1, r)
6
7 RANDOMIZED-PARTITION(A, p, r)
8   1   i = RANDOM(p, r)
9   2   swap(A[r], A[i])
10  3   return PARTITION(A, p, r)

```

Listing 3.5: RANDOMIZED-QUICKSORT

3.8. Variables Aleatorias Indicadoras: Un Caso de Estudio

3.8.1. Definición Formal

Una variable aleatoria indicadora $I\{A\}$ es 1 si el evento A ocurre, 0 en caso contrario. Su valor esperado es $\Pr\{A\}$.

3.8.2. Ejemplo: Número Esperado de Caras en n Lanzamientos

Sean $X_i = I\{\text{lanzamiento } i \text{ es cara}\}$. Entonces $X = \sum_{i=1}^n X_i$ es el número total de caras.

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{2} = \frac{n}{2}$$

3.8.3. Ejemplo: Número Esperado de Inversiones en una Permutación Aleatoria

Una inversión es un par (i, j) con $i < j$ y $A[i] > A[j]$. Para cada par, definimos $X_{ij} = I\{A[i] > A[j]\}$. En una permutación aleatoria, $\Pr\{A[i] > A[j]\} = 1/2$.

$$E[X] = \sum_{1 \leq i < j \leq n} E[X_{ij}] = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}$$

3.9. Ejercicios Resueltos (Ampliados)

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar exactamente una vez?

Solución

Contratar exactamente una vez significa que solo se contrata al primer candidato. Esto ocurre si el primer candidato es el mejor de todos. La probabilidad de que el mejor de n candidatos esté en la primera posición es $1/n$.

$$\Pr\{\text{una contratación}\} = \frac{1}{n}$$

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar exactamente dos veces?

Solución

Para contratar exactamente dos veces, el candidato 1 debe tener algún rango r , y el mejor candidato debe estar después de él, y todos los candidatos entre el 1 y el mejor deben tener rango menor que r .

$$\Pr\{\text{dos contrataciones}\} = \frac{1}{n} \sum_{r=1}^{n-1} \frac{1}{n-r} = \frac{H_{n-1}}{n}$$

donde H_{n-1} es el $(n-1)$ -ésimo número armónico.

Ejercicio

En búsqueda lineal, si v está en el arreglo con probabilidad p y es uniforme, ¿cuál es el número esperado de comparaciones?

Solución

Sea X el número de comparaciones. Condicionamos:

$$E[X] = \Pr\{\text{exitosa}\} \cdot E[X \mid \text{exitosa}] + \Pr\{\text{infructuosa}\} \cdot E[X \mid \text{infructuosa}]$$

$$E[X] = p \cdot \frac{n+1}{2} + (1-p) \cdot n$$

Si $p = 1$, $E[X] = (n+1)/2 = \Theta(n)$. Si $p = 0$, $E[X] = n = \Theta(n)$. En cualquier caso, es $\Theta(n)$.

Ejercicio

En el problema de contratación en línea, ¿cuál es la probabilidad de éxito si $k = n/2$?

Solución

$$\Pr\{\text{éxito}\} \approx \frac{k}{n} \ln\left(\frac{n}{k}\right) = \frac{1}{2} \ln(2) \approx 0,3466$$

Ejercicio

Demuestre que el número esperado de caras en n lanzamientos de una moneda justa es $n/2$ usando variables indicadoras.

Solución

Sea $X_i = I\{\text{lanzamiento } i \text{ es cara}\}$. $E[X_i] = 1/2$. Entonces $X = \sum X_i$ y $E[X] = \sum E[X_i] = n \cdot 1/2 = n/2$.

Ejercicio

En una permutación aleatoria de n elementos, ¿cuál es el número esperado de puntos fijos (elementos que están en su posición original)?

Solución

Sea $X_i = I\{A[i] = i\}$. $\Pr\{A[i] = i\} = 1/n$. Entonces $E[X] = \sum_{i=1}^n 1/n = 1$. ¡En promedio, hay exactamente un punto fijo!

Ejercicio

En una permutación aleatoria de n elementos, ¿cuál es la probabilidad de que un elemento dado esté en su posición original? ¿Y de que dos elementos específicos estén en sus posiciones?

Solución

Para un elemento dado, $\Pr\{A[i] = i\} = 1/n$. Para dos elementos i y j , $\Pr\{A[i] = i \text{ y } A[j] = j\} = \frac{1}{n} \cdot \frac{1}{n-1}$, porque después de fijar i , quedan $n - 1$ posiciones para j .

Ejercicio

En el problema de contratación, ¿cuál es el número esperado de contrataciones si los candidatos llegan en orden aleatorio?

Solución

Ya calculamos: $E[X] = H_n = \ln n + \gamma + O(1/n) = \Theta(\log n)$.

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de que el mejor candidato sea contratado?

Solución

El mejor candidato siempre es contratado porque es mejor que todos los anteriores. ¡La probabilidad es 1!

Ejercicio

En búsqueda binaria, ¿cuál es el número de comparaciones en el peor caso para $n = 1000$?

Solución

El número de comparaciones es $\lceil \log_2 n \rceil = \lceil \log_2 1000 \rceil = \lceil 9,97 \rceil = 10$.

Ejercicio

Demuestre que en una permutación aleatoria, el número esperado de inversiones es $n(n-1)/4$.

Solución

Usando variables indicadoras $X_{ij} = I\{A[i] > A[j]\}$ para $i < j$. $\Pr\{A[i] > A[j]\} = 1/2$. Hay $\binom{n}{2}$ pares. Entonces:

$$E[X] = \sum_{i < j} \frac{1}{2} = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}$$

Ejercicio

En el problema de la contratación en línea, ¿cuál es la probabilidad de éxito si $k = n/3$?

Solución

$$\Pr\{\text{éxito}\} \approx \frac{k}{n} \ln\left(\frac{n}{k}\right) = \frac{1}{3} \ln(3) \approx 0,3662$$

Ejercicio

Demuestre que la probabilidad de éxito en el problema de contratación en línea es máxima cuando $k \approx n/e$.

Solución

Derivamos $f(k) = \frac{k}{n} \ln(n/k) = \frac{k}{n} (\ln n - \ln k)$. Derivando respecto a k :

$$f'(k) = \frac{1}{n} (\ln n - \ln k) - \frac{k}{n} \cdot \frac{1}{k} = \frac{1}{n} (\ln n - \ln k - 1)$$

Igualando a 0: $\ln n - \ln k = 1 \implies \ln(n/k) = 1 \implies n/k = e \implies k = n/e$.

Ejercicio

En búsqueda lineal, si $p = 0,5$, ¿cuál es el número esperado de comparaciones?

Solución

$$E[X] = 0,5 \cdot \frac{n+1}{2} + 0,5 \cdot n = \frac{n+1}{4} + \frac{n}{2} = \frac{3n+1}{4}$$

Ejercicio

En búsqueda lineal, si $p = 0$, ¿cuál es el número esperado de comparaciones?

Solución

$$E[X] = 0 \cdot \frac{n+1}{2} + 1 \cdot n = n$$

Ejercicio

En búsqueda lineal, si $p = 1$, ¿cuál es el número esperado de comparaciones?

Solución

$$E[X] = 1 \cdot \frac{n+1}{2} + 0 \cdot n = \frac{n+1}{2}$$

Ejercicio

Demuestre que en una permutación aleatoria, la probabilidad de que el primer elemento sea el máximo es $1/n$.

Solución

El máximo es igualmente probable que esté en cualquier posición. La probabilidad de que esté en la primera es $1/n$.

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar n veces?

Solución

Contratar n veces significa que cada candidato es mejor que todos los anteriores. Esto ocurre solo si los candidatos vienen en orden estrictamente creciente. La probabilidad es $1/n!$.

Ejercicio

En el problema de contratación, ¿cuál es la probabilidad de contratar al candidato i ?

Solución

$\Pr\{\text{contratar } i\} = 1/i.$

Ejercicio

En el problema de contratación en línea, ¿cuál es la probabilidad de contratar al mejor candidato si $k = 0$?

Solución

Si $k = 0$, contratamos al primer candidato. La probabilidad de que sea el mejor es $1/n$.

Ejercicio

En el problema de contratación en línea, ¿cuál es la probabilidad de contratar al mejor candidato si $k = n - 1$?

Solución

Si $k = n - 1$, entrevistamos a todos menos al último, y contratamos al último. La probabilidad de que el último sea el mejor es $1/n$.

3.10. Ejercicios Propuestos (Ampliados)

1. En el problema de contratación, calcule el número esperado de contrataciones para $n = 10$ usando la fórmula H_n .
2. En una permutación aleatoria de $n = 5$ elementos, calcule el número esperado de puntos fijos.
3. Demuestre que $\Pr\{\text{una inversión específica ocurra}\} = 1/2$ en una permutación aleatoria.
4. En búsqueda lineal, si $p = 0,2$, ¿cuál es el número esperado de comparaciones para $n = 100$?
5. En búsqueda binaria, ¿cuántas comparaciones se hacen en el peor caso para $n = 1,000,000$?
6. En el problema de contratación en línea, calcule la probabilidad de éxito para $n = 100$ y $k = 37$.
7. Demuestre que $\sum_{i=1}^n 1/i = \Theta(\log n)$.
8. En una permutación aleatoria, ¿cuál es el número esperado de elementos que son mayores que todos los anteriores?
9. En el problema de contratación, ¿cuál es la varianza del número de contrataciones? (Pista: use que $\text{Var}(X) = E[X^2] - E[X]^2$ y que $E[X_i X_j] = E[X_i]E[X_j]$ para $i \neq j$ porque son independientes.)

10. Diseñe un algoritmo aleatorizado que decida si un número está en un arreglo, y analice su tiempo esperado.
11. En el problema de la búsqueda de un máximo, ¿cuál es la probabilidad de que el máximo esté en la primera mitad?
12. En el problema de la contratación en línea, ¿cuál es la probabilidad de éxito si $k = n/4$?
13. Demuestre que $E[X^2] \geq E[X]^2$ para cualquier variable aleatoria X (desigualdad de Jensen).
14. En una permutación aleatoria, ¿cuál es la probabilidad de que el elemento i sea un punto fijo y el elemento j también?
15. En el problema de contratación, ¿cuál es la probabilidad de contratar exactamente dos veces para $n = 3$?
16. En búsqueda lineal, si $p = 0,8$, ¿cuál es el número esperado de comparaciones para $n = 50$?
17. Demuestre que la probabilidad de que el máximo esté en la última posición en una permutación aleatoria es $1/n$.
18. En el problema de contratación en línea, demuestre que la probabilidad de éxito es exactamente $\frac{k}{n} \sum_{j=k}^{n-1} \frac{1}{j}$.
19. En una permutación aleatoria, ¿cuál es el número esperado de elementos que son menores que todos los siguientes?
20. En el problema de contratación, ¿cuál es la probabilidad de que el candidato 1 sea contratado?

Capítulo 4

Unidad IV: Estructuras de Datos Avanzadas - Árboles Balanceados

4.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

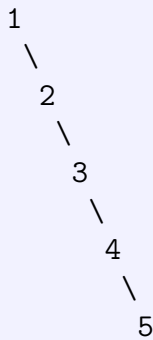
- Comprender la necesidad de árboles balanceados para garantizar operaciones $O(\log n)$ en el peor caso.
- Definir y caracterizar árboles AVL: factor de equilibrio, altura, rotaciones.
- Implementar las operaciones de inserción y eliminación en árboles AVL.
- Comprender los conceptos básicos de árboles rojo-negro: propiedades, coloración, invariantes.
- Analizar la complejidad de las operaciones en árboles AVL y rojo-negro.
- Comparar ambas estructuras y entender sus aplicaciones.

4.2. Introducción: La Necesidad de Árboles Balanceados

Los árboles de búsqueda binaria (BST) son estructuras eficientes en promedio, pero pueden degenerar en una lista enlazada si los elementos se insertan en orden. Esto produce operaciones $O(n)$ en el peor caso.

Ejemplo

[Degeneración de un BST] Insertar las claves 1, 2, 3, 4, 5 en un BST vacío produce:



La búsqueda de 5 requiere 5 comparaciones, $O(n)$ en lugar de $O(\log n)$.

Para garantizar un rendimiento $O(\log n)$ en el peor caso, necesitamos **árboles balanceados**. Los más conocidos son:

- **Árboles AVL** (Adelson-Velsky y Landis, 1962)
- **Árboles Rojo-Negro** (Bayer, 1972)

4.3. Árboles AVL: Definición y Propiedades

4.3.1. Definición

Un árbol AVL es un árbol de búsqueda binaria que satisface la **propiedad de equilibrio AVL**: para cada nodo x , las alturas de sus subárboles izquierdo y derecho difieren en a lo sumo 1.

El **factor de equilibrio** de un nodo x se define como:

$$FE(x) = \text{altura}(x.\text{izquierdo}) - \text{altura}(x.\text{derecho})$$

En un árbol AVL, $FE(x) \in \{-1, 0, 1\}$ para todo nodo x .

4.3.2. Altura de un Árbol AVL

Teorema 4.1. *Un árbol AVL de n nodos tiene altura $h \leq 1,44 \log_2(n + 1)$.*

Demostración. Sea N_h el número mínimo de nodos en un árbol AVL de altura h .

- $N_0 = 1$ (un nodo)
- $N_1 = 2$ (raíz con un hijo)
- Para $h \geq 2$, el árbol tiene raíz con subárbol izquierdo de altura $h - 1$ y subárbol derecho de altura $h - 2$ (el mínimo ocurre cuando la diferencia es máxima).

Por lo tanto:

$$N_h = N_{h-1} + N_{h-2} + 1$$

Esta recurrencia es similar a los números de Fibonacci. Se puede demostrar que $N_h \geq F_{h+2} - 1$, donde F_k es el k -ésimo número de Fibonacci. Como $F_k \approx \phi^k / \sqrt{5}$ con $\phi = (1 + \sqrt{5})/2 \approx 1,618$, tenemos:

$$N_h \geq \phi^{h+2} / \sqrt{5} - 1 \implies h \leq \log_{\phi}(N_h + 1) \approx 1,44 \log_2(n + 1)$$

□

4.3.3. Estructura de un Nodo AVL

Cada nodo almacena:

- **key**: valor de la clave
- **left**: puntero al hijo izquierdo
- **right**: puntero al hijo derecho
- **parent**: puntero al padre (opcional)
- **height**: altura del subárbol enraizado en el nodo

4.4. Rotaciones en Árboles AVL

Las rotaciones son operaciones locales que reestructuran el árbol manteniendo la propiedad de BST. Son la herramienta fundamental para restaurar el equilibrio después de inserciones o eliminaciones.

4.4.1. Rotación a la Derecha (Right Rotate)

La rotación a la derecha se aplica cuando un nodo tiene un hijo izquierdo desbalanceado.

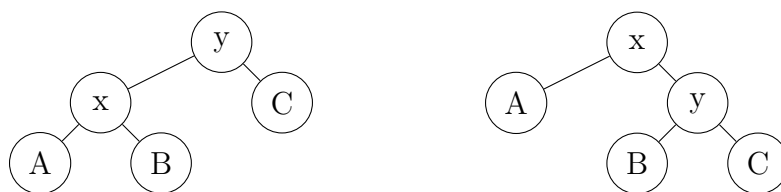


Figura 4.1: Rotación a la derecha: antes y después

```

1 RIGHT-ROTATE(y)
2   x = y.left
3   y.left = x.right
4   if x.right != NIL
5       x.right.parent = y
6   x.right = y
7   x.parent = y.parent
8   if y.parent == NIL
9       root = x
10  else if y == y.parent.left
11      y.parent.left = x

```

```

12     else
13         y.parent.right = x
14     y.parent = x
15     // Actualizar alturas
16     update-height(y)
17     update-height(x)

```

Listing 4.1: Rotación a la derecha

4.4.2. Rotación a la Izquierda (Left Rotate)

La rotación a la izquierda es simétrica.

```

1 LEFT-ROTATE(x)
2     y = x.right
3     x.right = y.left
4     if y.left != NIL
5         y.left.parent = x
6     y.left = x
7     y.parent = x.parent
8     if x.parent == NIL
9         root = y
10    else if x == x.parent.left
11        x.parent.left = y
12    else
13        x.parent.right = y
14    x.parent = y
15    // Actualizar alturas
16    update-height(x)
17    update-height(y)

```

Listing 4.2: Rotación a la izquierda

4.4.3. Actualización de Alturas

La altura de un nodo se calcula como:

$$\text{height}(x) = 1 + \max(\text{height}(x.\text{left}), \text{height}(x.\text{right}))$$

con $\text{height}(\text{NIL}) = -1$ (o 0, según convención).

```

1 UPDATE-HEIGHT(x)
2     x.height = 1 + max(height(x.left), height(x.right))

```

Listing 4.3: Actualizar altura

4.5. Inserción en Árboles AVL

4.5.1. Pasos de la Inserción

1. Insertar el nuevo nodo como en un BST normal.
2. Actualizar las alturas de los ancestros.
3. Verificar el factor de equilibrio en cada ancestro.
4. Si algún nodo está desbalanceado ($|\text{FE}| > 1$), aplicar la rotación correspondiente.

4.5.2. Casos de Desbalance

Hay cuatro casos de desbalance después de una inserción:

1. **Caso 1: Izquierda-Izquierda (LL):** El desbalance ocurre en el hijo izquierdo del hijo izquierdo.
2. **Caso 2: Izquierda-Derecha (LR):** El desbalance ocurre en el hijo derecho del hijo izquierdo.
3. **Caso 3: Derecha-Derecha (RR):** El desbalance ocurre en el hijo derecho del hijo derecho.
4. **Caso 4: Derecha-Izquierda (RL):** El desbalance ocurre en el hijo izquierdo del hijo derecho.

4.5.3. Rotaciones para los Casos

- **Caso LL:** Rotación a la derecha sobre el nodo desbalanceado.
- **Caso RR:** Rotación a la izquierda sobre el nodo desbalanceado.
- **Caso LR:** Rotación a la izquierda sobre el hijo izquierdo, luego rotación a la derecha sobre el nodo desbalanceado.
- **Caso RL:** Rotación a la derecha sobre el hijo derecho, luego rotación a la izquierda sobre el nodo desbalanceado.

4.5.4. Pseudocódigo de Inserción

```

1 AVL-INSERT(root, key)
2   root = AVL-INSERT-REC(root, key)
3   return root
4
5 AVL-INSERT-REC(node, key)
6   if node == NIL
7     return new Node(key)
8   if key < node.key
9     node.left = AVL-INSERT-REC(node.left, key)
10    node.left.parent = node
11  else if key > node.key
12    node.right = AVL-INSERT-REC(node.right, key)
13    node.right.parent = node
14  else
15    return node // clave duplicada, no insertar
16
17  // Actualizar altura
18  UPDATE-HEIGHT(node)
19
20  // Verificar balance
21  balance = height(node.left) - height(node.right)
22
23  // Caso LL
24  if balance > 1 and key < node.left.key
25    return RIGHT-ROTATE(node)

```

```

26
27 // Caso RR
28 if balance < -1 and key > node.right.key
29     return LEFT-ROTATE(node)
30
31 // Caso LR
32 if balance > 1 and key > node.left.key
33     node.left = LEFT-ROTATE(node.left)
34     return RIGHT-ROTATE(node)
35
36 // Caso RL
37 if balance < -1 and key < node.right.key
38     node.right = RIGHT-ROTATE(node.right)
39     return LEFT-ROTATE(node)
40
41 return node

```

Listing 4.4: Inserción en AVL

4.5.5. Ejemplo de Inserción

Insertar las claves: 10, 20, 30, 40, 50, 25.

1. Insertar 10: árbol balanceado.
2. Insertar 20: balanceado.
3. Insertar 30: desbalance en 10 (factor -2, caso RR). Rotación izquierda sobre 10.
4. Insertar 40: balanceado.
5. Insertar 50: desbalance en 20 (factor -2, caso RR). Rotación izquierda sobre 20.
6. Insertar 25: desbalance en 30 (factor 2, caso LR). Rotación izquierda sobre 25, luego derecha sobre 30.

4.6. Eliminación en Árboles AVL

4.6.1. Pasos de la Eliminación

1. Eliminar el nodo como en un BST normal.
2. Actualizar las alturas de los ancestros.
3. Verificar el factor de equilibrio en cada ancestro.
4. Si algún nodo está desbalanceado, aplicar la rotación correspondiente.

4.6.2. Casos de Desbalance en Eliminación

Los casos son similares a la inserción, pero con una sutileza: después de una rotación, puede ser necesario continuar verificando hacia arriba.

```

1 AVL-DELETE(root, key)
2   root = AVL-DELETE-REC(root, key)
3   return root
4
5 AVL-DELETE-REC(node, key)
6   if node == NIL
7     return NIL
8
9   if key < node.key
10    node.left = AVL-DELETE-REC(node.left, key)
11    if node.left != NIL
12      node.left.parent = node
13  else if key > node.key
14    node.right = AVL-DELETE-REC(node.right, key)
15    if node.right != NIL
16      node.right.parent = node
17  else
18    // Nodo encontrado
19    if node.left == NIL or node.right == NIL
20      // Caso 0 o 1 hijo
21      child = node.left if node.left != NIL else node.right
22      if child != NIL
23        child.parent = node.parent
24      return child
25    else
26      // Dos hijos: encontrar sucesor
27      successor = MINIMUM(node.right)
28      node.key = successor.key
29      node.right = AVL-DELETE-REC(node.right, successor.key)
30      if node.right != NIL
31        node.right.parent = node
32
33    // Actualizar altura
34    UPDATE-HEIGHT(node)
35
36    // Verificar balance
37    balance = height(node.left) - height(node.right)
38
39    // Caso LL
40    if balance > 1 and height(node.left.left) >= height(node.left.right)
41    )
42      return RIGHT-ROTATE(node)
43
44    // Caso LR
45    if balance > 1 and height(node.left.left) < height(node.left.right)
46      node.left = LEFT-ROTATE(node.left)
47      return RIGHT-ROTATE(node)
48
49    // Caso RR
50    if balance < -1 and height(node.right.right) >= height(node.right.
51    left)
52      return LEFT-ROTATE(node)
53
54    // Caso RL
55    if balance < -1 and height(node.right.right) < height(node.right.
56    left)
57      node.right = RIGHT-ROTATE(node.right)
58      return LEFT-ROTATE(node)

```

```

56
57     return node

```

Listing 4.5: Eliminación en AVL

4.7. Árboles Rojo-Negro: Conceptos Básicos

4.7.1. Definición

Un árbol rojo-negro es un árbol de búsqueda binaria con un bit extra de color (rojo o negro) por nodo que satisface las siguientes propiedades:

1. Cada nodo es rojo o negro.
2. La raíz es negra.
3. Cada hoja (NIL) es negra.
4. Si un nodo es rojo, ambos hijos son negros.
5. Para cada nodo, todos los caminos desde el nodo a las hojas descendientes contienen el mismo número de nodos negros (altura negra).

4.7.2. Estructura de un Nodo Rojo-Negro

```

1 struct Node {
2     int key;
3     Node *left, *right, *parent;
4     bool color; // true = rojo, false = negro
5 };

```

4.7.3. Propiedad Fundamental

Teorema 4.2. *Un árbol rojo-negro con n nodos internos tiene altura $h \leq 2 \log_2(n + 1)$.*

Demostración. Sea $bh(x)$ la altura negra de un nodo x . Por propiedad 4, al menos la mitad de los nodos en cualquier camino desde la raíz hasta una hoja son negros, por lo que $bh(\text{raíz}) \geq h/2$. Además, el número mínimo de nodos internos en un árbol con altura negra b es al menos $2^b - 1$. Por lo tanto:

$$n \geq 2^{bh(\text{raíz})-1} \geq 2^{h/2-1} \implies h \leq 2 \log_2(n+1)$$

□

4.7.4. Rotaciones en Árboles Rojo-Negro

Las rotaciones son idénticas a las de AVL, pero se usan junto con recoloraciones para mantener las propiedades.

4.7.5. Inserción en Árboles Rojo-Negro

La inserción tiene dos fases:

1. Insertar el nuevo nodo como rojo (similar a BST).
2. Corregir posibles violaciones de propiedades mediante recoloraciones y rotaciones.

Casos de Corrección

Hay tres casos principales cuando el padre del nuevo nodo es rojo:

1. **Caso 1: El tío es rojo.** Se recoloran padre, tío y abuelo. El abuelo se convierte en el nuevo nodo a verificar.
2. **Caso 2: El tío es negro y el nodo es hijo derecho.** Se rota a la izquierda sobre el padre, transformando al caso 3.
3. **Caso 3: El tío es negro y el nodo es hijo izquierdo.** Se rota a la derecha sobre el abuelo y se recoloran.

```

1 RB-INSERT(root, key)
2   z = new Node(key)
3   z.color = RED
4   // Insertar como en BST
5   BST-INSERT(root, z)
6   // Corregir violaciones
7   while z != root and z.parent.color == RED
8       if z.parent == z.parent.parent.left
9           uncle = z.parent.parent.right
10          if uncle.color == RED // Caso 1
11              z.parent.color = BLACK
12              uncle.color = BLACK
13              z.parent.parent.color = RED
14              z = z.parent.parent
15          else
16              if z == z.parent.right // Caso 2
17                  z = z.parent
18                  LEFT-ROTATE(z)
19              // Caso 3
20              z.parent.color = BLACK
21              z.parent.parent.color = RED
22              RIGHT-ROTATE(z.parent.parent)
23      else // Sim trico
24          // ... c digo an logo con izquierda/derecha
25          intercambiadas
26      root.color = BLACK

```

Listing 4.6: Inserción en árbol rojo-negro (simplificada)

4.7.6. Eliminación en Árboles Rojo-Negro

La eliminación es más compleja. Cuando se elimina un nodo negro, se crea un "negro extra" que debe ser manejado. Hay varios casos para restaurar las propiedades.

4.8. Comparación: AVL vs. Rojo-Negro

Característica	AVL	Rojo-Negro
Altura máxima	$\approx 1,44 \log n$	$\approx 2 \log n$
Búsqueda	Más rápida (más balanceado)	Ligeramente más lenta
Inserción	Más rotaciones	Menos rotaciones
Eliminación	Más rotaciones	Menos rotaciones
Aplicaciones	Búsquedas frecuentes	Inserciones/eliminaciones frecuentes

4.9. Ejercicios Resueltos (Ampliados)

Ejercicio

Construya un árbol AVL insertando las claves 10, 20, 30, 40, 50, 25 en ese orden.

Solución

1. Insertar 10: árbol con raíz 10.
2. Insertar 20: 10 (der) = 20. Alturas: 10(1), 20(0). Balanceados.
3. Insertar 30: 10 (der) = 20 (der) = 30. Factor en 10 = $0-2 = -2$ (desbalance). Caso RR: rotación izquierda sobre 10. Resultado: raíz 20, izquierdo 10, derecho 30.
4. Insertar 40: 20 (der) = 30 (der) = 40. Factor en 30 = $0-1 = -1$, en 20 = $0-2 = -2$. Caso RR sobre 20: rotación izquierda sobre 20. Resultado: raíz 30, izquierdo 20, derecho 40; 20 tiene izquierdo 10.
5. Insertar 50: 30 (der) = 40 (der) = 50. Factor en 40 = -1 , en 30 = -2 . Caso RR sobre 30: rotación izquierda sobre 30. Resultado: raíz 40, izquierdo 30, derecho 50; 30 tiene izquierdo 20, derecho?; 20 tiene izquierdo 10.
6. Insertar 25: 40 (izq) = 30 (izq) = 20 (der) = 25. Verificar balances: 20 tiene factor 1 (izq 10, der 25), 30 tiene factor 1 (izq 20, der ?), 40 tiene factor -1 . En 30, balance ¿1 y 25 ¿20, caso LR: rotación izquierda sobre 20 (20-25), luego derecha sobre 30.

Árbol final: raíz 30, izquierdo 20, derecho 40; 20 tiene izquierdo 10, derecho 25; 40 tiene derecho 50.

Ejercicio

Demuestre que un árbol AVL de altura h tiene al menos F_{h+2} nodos, donde F es la sucesión de Fibonacci.

Solución

Sea N_h el mínimo número de nodos en un AVL de altura h .

- $N_0 = 1 = F_2$
- $N_1 = 2 = F_3$
- Para $h \geq 2$, $N_h = N_{h-1} + N_{h-2} + 1 \geq F_{h+1} + F_h + 1 = F_{h+2} + 1 - 1$? En realidad $F_{h+2} = F_{h+1} + F_h$, entonces $N_h \geq F_{h+2} + 1$? Verificar: $N_2 = 3$, $F_4 = 3$, entonces $N_2 \geq F_4$.

Por inducción, $N_h \geq F_{h+2}$.

Ejercicio

¿Cuál es la altura máxima de un árbol AVL con 100 nodos?

Solución

$h \leq 1,44 \log_2(101) \approx 1,44 \times 6,66 \approx 9,6$. Por lo tanto, $h \leq 9$.

Ejercicio

Explique por qué en un árbol rojo-negro, la altura máxima es $2 \log_2(n + 1)$.

Solución

Por propiedad 4, al menos la mitad de los nodos en cualquier camino son negros. Si la altura negra es b , entonces $b \geq h/2$. Por propiedad 5, el número de nodos $n \geq 2^b - 1 \geq 2^{h/2} - 1$, por lo que $h \leq 2 \log_2(n + 1)$.

Ejercicio

Realice una rotación a la derecha sobre el nodo 30 en el árbol: 50(izq=30, der=70); 30(izq=20, der=40).

Solución

Después de rotación derecha sobre 30:

- 20 se convierte en hijo izquierdo de 50.
- 30 se convierte en hijo derecho de 20.
- El subárbol derecho de 20 (que estaba vacío) se convierte en hijo izquierdo de 30.

Resultado: raíz 50, izquierdo 20; 20 tiene derecho 30; 30 tiene izquierdo ?, derecho 40.

Ejercicio

Inserte la clave 15 en un árbol AVL que tiene raíz 10 con derecho 20, y luego verifique el balance.

Solución

Insertar 15: 10 (der) = 20 (izq) = 15. Ahora 20 tiene factor -1 (izq 15, der ?), 10 tiene factor -2 (izq ?, der 20). Es caso LR: rotación izquierda sobre 15, luego derecha sobre 10.

Ejercicio

¿Cuál es la altura negra de un árbol rojo-negro con raíz negra y todos los nodos negros?

Solución

Si todos los nodos son negros, la altura negra es igual a la altura del árbol. Para n nodos, $n \geq 2^h - 1$, por lo que $h \leq \log_2(n + 1)$.

Ejercicio

Demuestre que en un árbol rojo-negro, el camino más largo es a lo sumo el doble del camino más corto.

Solución

El camino más corto tiene solo nodos negros. El camino más largo alterna rojo-negro, por lo que tiene a lo sumo el doble de nodos.

Ejercicio

¿Qué sucede si insertamos una clave en un árbol AVL y después de la inserción el factor de equilibrio de la raíz es 2?

Solución

Indica que el subárbol izquierdo es más alto. Depende de dónde se insertó:

- Si la inserción fue en el subárbol izquierdo del hijo izquierdo (LL), rotación derecha.
- Si fue en el subárbol derecho del hijo izquierdo (LR), rotación izquierda-derecha.

Ejercicio

En un árbol rojo-negro, ¿puede un nodo rojo tener un hijo rojo?

Solución

No, por propiedad 4, si un nodo es rojo, ambos hijos deben ser negros.

Ejercicio

Construya un árbol rojo-negro insertando las claves 1, 2, 3, 4, 5 en ese orden.

Solución

1. Insertar 1 (raíz negra).
2. Insertar 2 (rojo, hijo derecho de 1). Árbol balanceado.
3. Insertar 3: 1(der=2(der=3)). Violación: tío de 3 es NIL (negro), caso 3. Rotación izquierda sobre 1, recolorar. Resultado: raíz 2 (negro), izquierdo 1 (rojo), derecho 3 (rojo).
4. Insertar 4: 2(der=3(der=4)). Tío de 4 es 1 (rojo), caso 1. Recolorar: 3 a negro, 1 a negro, 2 a rojo. Como 2 es raíz, se recolora a negro.
5. Insertar 5: 2(der=3(der=4(der=5))). Tío de 5 es NIL (negro), caso 3. Rotación izquierda sobre 4, luego izquierda sobre 3, recolorar.

Ejercicio

¿Cuál es la diferencia principal entre la inserción en AVL y en rojo-negro?

Solución

AVL usa rotaciones inmediatas para mantener el factor de equilibrio estricto, mientras que rojo-negro usa recoloraciones primero y rotaciones solo cuando es necesario. AVL es más estricto (más balanceado), rojo-negro tiene menos rotaciones.

Ejercicio

Demuestre que un árbol AVL con $n = 7$ puede tener altura 3.

Solución

El árbol AVL mínimo con altura 3 tiene $N_3 = N_2 + N_1 + 1 = 4 + 2 + 1 = 7$ nodos. Por lo tanto, es posible.

Ejercicio

¿Qué es una rotación doble y cuándo se usa?

Solución

Una rotación doble consiste en dos rotaciones simples. Se usa en los casos LR y RL, cuando el desbalance ocurre en el "nieto" del nodo desbalanceado.

Ejercicio

En un árbol rojo-negro, si la raíz es roja, ¿se viola alguna propiedad?

Solución

Sí, propiedad 2: la raíz debe ser negra.

Ejercicio

¿Cuántas rotaciones puede requerir una inserción en un árbol AVL en el peor caso?

Solución

En el peor caso, puede requerir $O(\log n)$ rotaciones, pero en la práctica solo una o dos rotaciones son comunes.

Ejercicio

¿Cuántas rotaciones puede requerir una inserción en un árbol rojo-negro?

Solución

Como máximo dos rotaciones, porque los casos 2 y 3 implican una rotación cada uno, y el caso 1 solo recoloraciones.

4.10. Ejercicios Propuestos (Ampliados)

1. Construya un árbol AVL insertando las claves: 50, 40, 30, 20, 10, 25, 35.
2. Construya un árbol AVL insertando las claves: 5, 3, 7, 2, 4, 6, 8, 1.
3. Elimine la clave 30 del árbol AVL construido en el ejercicio 1.
4. Demuestre que $N_h \geq F_{h+2}$ por inducción.
5. ¿Cuál es la altura máxima de un árbol AVL con 1000 nodos?
6. Construya un árbol rojo-negro insertando las claves: 10, 20, 30, 40, 50, 60.
7. Construya un árbol rojo-negro insertando las claves: 1, 2, 3, 4, 5, 6, 7.
8. Explique por qué en un árbol rojo-negro, un nodo rojo no puede tener un hijo rojo.
9. ¿Qué sucede si eliminamos un nodo rojo de un árbol rojo-negro? ¿Y si es negro?
10. Implemente una función que verifique si un árbol es AVL.
11. Implemente una función que verifique si un árbol es rojo-negro.
12. Compare el número de rotaciones necesarias para insertar 1000 elementos aleatorios en un AVL vs. rojo-negro.

13. ¿Cuál es la complejidad en el peor caso de búsqueda en un AVL? ¿Y en un rojo-negro?
14. Dibuje un árbol AVL de altura 3 con el mínimo número de nodos.
15. Dibuje un árbol rojo-negro de altura 4 con el máximo número de nodos.
16. Demuestre que la altura negra de un árbol rojo-negro es al menos $h/2$.
17. ¿Puede un árbol rojo-negro tener todos los nodos rojos? ¿Por qué?
18. ¿Qué ventaja tiene el rojo-negro sobre el AVL en aplicaciones con muchas inserciones y eliminaciones?
19. Diseñe un algoritmo para encontrar el k -ésimo elemento más pequeño en un árbol AVL en $O(\log n)$.
20. Demuestre que la altura de un árbol AVL es $\Theta(\log n)$.

Capítulo 5

Unidad V: Árboles B y B+ - Estructuras para Almacenamiento Secundario

5.1. Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la problemática del almacenamiento secundario (discos) y la necesidad de estructuras optimizadas para E/S.
- Definir la estructura y propiedades de los árboles B.
- Implementar las operaciones de búsqueda, inserción y eliminación en árboles B.
- Comprender la variante B+ y sus ventajas en sistemas de bases de datos.
- Analizar la complejidad de las operaciones en términos de accesos a disco.
- Aplicar árboles B en contextos reales como índices de bases de datos y sistemas de archivos.

5.2. Introducción: El Problema del Almacenamiento Secundario

5.2.1. Jerarquía de Memoria

Los sistemas computacionales tienen una jerarquía de memoria:

- **Registros:** nanosegundos, capacidad muy limitada.
- **Caché:** nanosegundos, capacidad limitada.
- **Memoria RAM:** microsegundos, capacidad moderada.
- **Disco magnético:** milisegundos, capacidad enorme.

5.2.2. Tiempos de Acceso

Nivel	Tiempo de acceso típico
Registros	¡1 ns
Caché L1	1-2 ns
Caché L2	5-10 ns
Memoria RAM	50-100 ns
Disco (búsqueda)	5-10 ms

La diferencia es de 5-6 órdenes de magnitud: ¡un acceso a disco equivale a millones de accesos a RAM!

5.2.3. Principio de Localidad

Para mitigar la lentitud del disco, se utilizan **páginas**: bloques contiguos de datos (típicamente 4-16 KB). Leer una página cuesta lo mismo que leer un byte, por lo que conviene traer más datos de los necesarios.

5.2.4. Objetivo de los Árboles B

Minimizar el número de accesos a disco. Para ello:

- Cada nodo del árbol ocupa una página de disco.
- El factor de ramificación (número de hijos por nodo) es grande (cientos o miles).
- La altura del árbol es muy pequeña ($\log_{1000} n$).

5.3. Definición de Árbol B

5.3.1. Propiedades Fundamentales

Un árbol B de grado mínimo $t \geq 2$ (también llamado orden $m = 2t$) satisface:

1. Cada nodo x tiene los siguientes campos:
 - $n[x]$: número de claves almacenadas en x .
 - Las $n[x]$ claves almacenadas en orden no decreciente: $k_1[x] \leq k_2[x] \leq \dots \leq k_{n[x]}[x]$.
 - $hoja[x]$: booleano que indica si x es hoja.
2. Cada nodo interno x tiene $n[x] + 1$ punteros $c_1[x], c_2[x], \dots, c_{n[x]+1}[x]$ a sus hijos.
3. Las claves separan los rangos de los subárboles: para cualquier clave k en el subárbol $c_i[x]$, se cumple:

$$k_{i-1}[x] \leq k \leq k_i[x]$$

(con $k_0 = -\infty$ y $k_{n[x]+1} = \infty$).

4. Todas las hojas tienen la misma profundidad (el árbol es perfectamente balanceado).

5. Límites en el número de claves:

- Cada nodo (excepto la raíz) tiene al menos $t - 1$ claves.
- Cada nodo tiene a lo sumo $2t - 1$ claves (está **lleno** si tiene $2t - 1$ claves).

5.3.2. Altura de un Árbol B

Teorema 5.1. *Un árbol B de altura h con n claves satisface:*

$$h \leq \log_t \left(\frac{n+1}{2} \right)$$

Demostración. El número mínimo de claves en un árbol B de altura h :

- Raíz: 1 clave
- Nivel 1: al menos 2 nodos (raíz con t hijos)
- Nivel 2: al menos $2t$ nodos
- Nivel i : al menos $2t^{i-1}$ nodos

Cada nodo tiene al menos $t - 1$ claves, excepto la raíz. Por lo tanto:

$$\begin{aligned} n &\geq 1 + (t-1)(2 + 2t + 2t^2 + \dots + 2t^{h-1}) \\ n &\geq 1 + 2(t-1)(t^{h-1} - 1)/(t-1) = 2t^{h-1} - 1 \\ t^{h-1} &\leq (n+1)/2 \implies h \leq \log_t((n+1)/2) + 1 \end{aligned}$$

□

5.3.3. Ejemplo Numérico

Con $t = 200$ (factor de ramificación 200-400) y $n = 10^9$:

$$h \leq \log_{200}(5 \times 10^8) \approx \frac{\log_{10}(5 \times 10^8)}{\log_{10}(200)} \approx \frac{8,7}{2,3} \approx 3,8$$

¡Solo 4 niveles! Es decir, 4 accesos a disco para encontrar cualquier clave.

5.4. Representación de un Árbol B

5.4.1. Estructura de Nodo

```

1 struct BNode {
2     int n;                // n mero de claves
3     int keys[2*t-1];      // arreglo de claves
4     BNode* children[2*t]; // arreglo de punteros a hijos
5     bool leaf;            // es hoja?
6 };

```

Listing 5.1: Estructura de un nodo de árbol B

5.4.2. Operaciones de Disco

En pseudocódigo, usamos:

- **DISK-READ(x)**: lee la página del nodo x del disco a memoria.
- **DISK-WRITE(x)**: escribe la página del nodo x de vuelta al disco.

Asumimos que un nodo cabe en una página de disco.

5.5. Búsqueda en Árbol B

5.5.1. Pseudocódigo

```

1 B-TREE-SEARCH(x, k)
2   i = 1
3   while i <= x.n and k > x.keys[i]
4     i = i + 1
5   if i <= x.n and k == x.keys[i]
6     return (x, i)           // encontrado
7   if x.leaf
8     return NIL              // no encontrado
9   else
10    DISK-READ(x.children[i])
11    return B-TREE-SEARCH(x.children[i], k)

```

Listing 5.2: Búsqueda en árbol B

5.5.2. Análisis de Complejidad

- **Accesos a disco:** $O(h) = O(\log_t n)$.
- **Comparaciones:** $O(t \cdot h) = O(t \log_t n)$.

Para t grande, las comparaciones dominan, pero son en memoria, por lo que el cuello de botella son los accesos a disco.

5.6. Inserción en Árbol B

5.6.1. Estrategia General

1. Buscar la hoja donde debe insertarse la clave.
2. Si la hoja tiene espacio (menos de $2t - 1$ claves), insertar y listo.
3. Si la hoja está llena, **dividirla** en dos nodos con $t - 1$ claves cada uno, y subir la clave mediana al padre.
4. Si el padre está lleno, repetir la división hacia arriba.
5. Si la raíz se divide, se crea una nueva raíz (el árbol crece en altura).

5.6.2. Operación de División

```

1 B-TREE-SPLIT-CHILD(x, i, y)
2   // y = x.children[i] est  lleno (2t-1 claves)
3   z = new BNode()
4   z.leaf = y.leaf
5   z.n = t - 1
6
7   // Copiar las t-1 claves m s grandes de y a z
8   for j = 1 to t-1
9       z.keys[j] = y.keys[j + t]
10
11   if not y.leaf
12       // Copiar los t hijos m s grandes de y a z
13       for j = 1 to t
14           z.children[j] = y.children[j + t]
15
16   y.n = t - 1
17
18   // Insertar z como hijo de x
19   for j = x.n+1 downto i+1
20       x.children[j+1] = x.children[j]
21   x.children[i+1] = z
22
23   // Subir la clave mediana a x
24   for j = x.n downto i
25       x.keys[j+1] = x.keys[j]
26   x.keys[i] = y.keys[t]
27   x.n = x.n + 1
28
29   DISK-WRITE(y)
30   DISK-WRITE(z)
31   DISK-WRITE(x)

```

Listing 5.3: Dividir un nodo

5.6.3. Inserción Principal

```

1 B-TREE-INSERT(T, k)
2   r = T.root
3   if r.n == 2*t - 1  // ra z llena
4       s = new BNode()
5       s.leaf = false
6       s.n = 0
7       s.children[1] = r
8       T.root = s
9       B-TREE-SPLIT-CHILD(s, 1, r)
10      B-TREE-INSERT-NONFULL(s, k)
11   else
12       B-TREE-INSERT-NONFULL(r, k)
13
14 B-TREE-INSERT-NONFULL(x, k)
15   i = x.n
16   if x.leaf
17       // Insertar en la hoja
18       while i >= 1 and k < x.keys[i]
19           x.keys[i+1] = x.keys[i]

```

```

20         i = i - 1
21         x.keys[i+1] = k
22         x.n = x.n + 1
23         DISK-WRITE(x)
24     else
25         // Buscar el hijo adecuado
26         while i >= 1 and k < x.keys[i]
27             i = i - 1
28         i = i + 1
29         DISK-READ(x.children[i])
30         if x.children[i].n == 2*t - 1
31             B-TREE-SPLIT-CHILD(x, i, x.children[i])
32             if k > x.keys[i]
33                 i = i + 1
34         B-TREE-INSERT-NONFULL(x.children[i], k)

```

Listing 5.4: Inserción en árbol B

5.7. Eliminación en Árbol B

5.7.1. Estrategia General

La eliminación es más compleja. Hay tres casos principales:

1. **Caso 1:** La clave está en una hoja. Se elimina directamente.
2. **Caso 2:** La clave está en un nodo interno. Se reemplaza por su predecesor o sucesor (que están en hojas), y se elimina recursivamente.
3. **Caso 3:** La clave no está en el nodo actual. Se desciende al hijo correspondiente, pero si ese hijo tiene $t - 1$ claves (mínimo), se **presta** una clave de un hermano o se **fusiona** con un hermano.

5.7.2. Prevención de Subdesbordamiento

Antes de descender a un hijo, si ese hijo tiene el mínimo de claves:

- Si un hermano tiene más de $t - 1$ claves, se **presta** una clave: se baja una clave del padre al hijo y se sube una clave del hermano al padre.
- Si ambos hermanos tienen $t - 1$ claves, se **fusiona** el hijo con un hermano, y se baja una clave del padre.

5.7.3. Pseudocódigo Simplificado

```

1 B-TREE-DELETE(x, k)
2     i = 1
3     while i <= x.n and k > x.keys[i]
4         i = i + 1
5
6     if i <= x.n and k == x.keys[i]
7         // Caso 1: clave en nodo actual
8         if x.leaf

```

```

9      // Eliminar directamente
10     for j = i to x.n-1
11         x.keys[j] = x.keys[j+1]
12     x.n = x.n - 1
13     DISK-WRITE(x)
14 else
15     // Caso 2: clave en nodo interno
16     y = x.children[i] // hijo izquierdo
17     z = x.children[i+1] // hijo derecho
18     if y.n >= t
19         // Usar predecesor
20         pred = B-TREE-FIND-PREDECESSOR(y)
21         x.keys[i] = pred.key
22         B-TREE-DELETE(y, pred.key)
23     else if z.n >= t
24         // Usar sucesor
25         succ = B-TREE-FIND-SUCCESSOR(z)
26         x.keys[i] = succ.key
27         B-TREE-DELETE(z, succ.key)
28     else
29         // Fusionar y, clave i, y z
30         B-TREE-MERGE(x, i, y, z)
31         B-TREE-DELETE(y, k)
32 else
33     // Caso 3: clave en sub rbol hijo
34     if x.leaf
35         return NIL // no encontrada
36     child = x.children[i]
37     DISK-READ(child)
38     if child.n == t-1
39         // Necesita rebalance
40         B-TREE-REBALANCE(x, i, child)
41     B-TREE-DELETE(child, k)

```

Listing 5.5: Eliminación en árbol B (esquema)

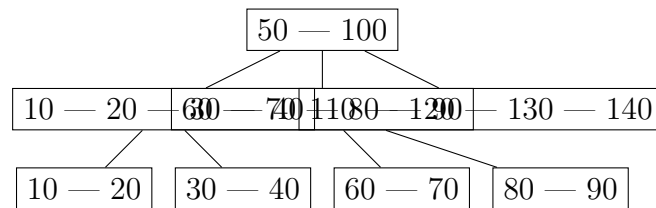
5.8. Árboles B+

5.8.1. Diferencias con Árbol B

El árbol B+ es una variante optimizada para bases de datos:

1. **Solo las hojas almacenan datos.** Los nodos internos solo almacenan claves y punteros.
2. **Las hojas están enlazadas.** Forman una lista enlazada para recorridos secuenciales eficientes.
3. **Todas las claves aparecen en las hojas.** Las claves en los nodos internos son copias que guían la búsqueda.

5.8.2. Estructura de un Árbol B+



5.8.3. Ventajas del Árbol B+

- **Mayor factor de ramificación:** Los nodos internos solo almacenan claves, no datos, por lo que caben más claves por página.
- **Recorridos secuenciales eficientes:** Las hojas enlazadas permiten escanear rangos sin retroceder en el árbol.
- **Uniformidad:** Todas las búsquedas terminan en hojas, por lo que el tiempo es constante para todas las claves.

5.8.4. Búsqueda en Árbol B+

```

1 BPLUS-TREE-SEARCH(x, k)
2   // Bajar hasta encontrar la hoja
3   while not x.leaf
4       i = 1
5       while i <= x.n and k > x.keys[i]
6           i = i + 1
7       DISK-READ(x.children[i])
8       x = x.children[i]
9
10  // Buscar en la hoja
11  i = 1
12  while i <= x.n and k > x.keys[i]
13      i = i + 1
14  if i <= x.n and k == x.keys[i]
15      return (x, i)    // encontrado, con datos en hoja
16  else
17      return NIL

```

Listing 5.6: Búsqueda en árbol B+

5.8.5. Inserción en Árbol B+

La inserción es similar al árbol B, pero:

- Siempre se inserta en la hoja.
- Cuando se divide una hoja, la clave mediana sube al padre, pero **permanece también en la hoja derecha** (para mantener todas las claves en hojas).
- Las hojas se enlazan entre sí.

5.9. Aplicaciones en Bases de Datos y Sistemas de Archivos

5.9.1. Índices en Bases de Datos

- **Índice primario:** Ordenado por la clave primaria. Las hojas contienen los registros completos o punteros a ellos.
- **Índice secundario:** Ordenado por otra clave. Las hojas contienen punteros a los registros.
- **Índice agrupado (clustered):** El orden físico de los datos sigue el orden del índice (generalmente B+).

5.9.2. Sistemas de Archivos

- **Ext4:** Usa árboles Htree (variante de B-tree) para directorios grandes.
- **NTFS:** Usa árboles B+ para el MFT (Master File Table).
- **ZFS:** Usa árboles B+ para la estructura de datos principal.

5.9.3. Optimizaciones Comunes

- **Pre-fetching:** Leer páginas anticipadamente basado en patrones de acceso.
- **Caché de nodos:** Mantener nodos frecuentemente accedidos en memoria.
- **Compresión de claves:** Reducir el espacio ocupado para aumentar el factor de ramificación.

5.10. Ejercicios Resueltos (Ampliados)

Ejercicio

Un árbol B tiene grado mínimo $t = 100$. ¿Cuántos accesos a disco se necesitan en el peor caso para buscar una clave entre 10^9 claves?

Solución

Altura máxima: $h \leq \log_t((n+1)/2) + 1 = \log_{100}(5 \times 10^8) + 1$.

$$\log_{100}(5 \times 10^8) = \frac{\ln(5 \times 10^8)}{\ln 100} \approx \frac{20,03}{4,605} \approx 4,35$$

Por lo tanto, $h \leq 5,35$, es decir, como máximo 5-6 accesos a disco.

Ejercicio

¿Cuál es el número máximo de claves en un árbol B de altura 2 con grado mínimo $t = 10$?

Solución

Máximo: raíz con $2t - 1 = 19$ claves, cada uno de los 20 hijos con $2t - 1 = 19$ claves.
 Total: $19 + 20 \times 19 = 19 + 380 = 399$ claves.

Ejercicio

Inserte las claves 10, 20, 30, 40, 50, 60, 70 en un árbol B con $t = 2$ (árbol 2-3-4).

Solución

1. Insertar 10,20,30: raíz [10,20,30] (llena).
2. Insertar 40: dividir raíz. Nueva raíz [20], hijos [10] y [30,40].
3. Insertar 50: hoja [30,40,50] llena, dividir. Subir 40. Raíz [20,40], hijos [10], [30], [50].
4. Insertar 60: hoja [50,60].
5. Insertar 70: hoja [50,60,70] llena, dividir. Subir 60. Raíz [20,40,60], hijos [10], [30], [50], [70].

Ejercicio

En un árbol B+ con $t = 100$, ¿cuántas claves caben en un nodo hoja si cada clave ocupa 8 bytes y cada puntero 8 bytes, y la página es de 4 KB?

Solución

Tamaño de página: 4096 bytes. En hoja: n claves + n punteros a datos + puntero a siguiente hoja.

$$8n + 8n + 8 = 16n + 8 \leq 4096 \implies 16n \leq 4088 \implies n \leq 255,5$$

Por lo tanto, $n \leq 255$ claves por hoja.

Ejercicio

¿Cuál es la altura máxima de un árbol B con $n = 10^6$ claves y $t = 50$?

Solución

$$h \leq \log_{50}((10^6 + 1)/2) + 1 \approx \log_{50}(5 \times 10^5) + 1. \log_{50}(5 \times 10^5) = \frac{\ln(5 \times 10^5)}{\ln 50} \approx \frac{13,12}{3,91} \approx 3,35. \text{ Por lo tanto, } h \leq 4,35, \text{ es decir, 4-5 niveles.}$$

Ejercicio

Explique por qué los árboles B+ son preferidos sobre los árboles B en bases de datos.

Solución

1. Mayor factor de ramificación en nodos internos (sin datos).
2. Recorridos secuenciales eficientes (hojas enlazadas).
3. Uniformidad en tiempo de búsqueda (siempre hasta hoja).
4. Mejor localidad de referencia para datos.

Ejercicio

En un árbol B con $t = 3$, ¿cuál es el número mínimo de claves en un nodo que no es raíz?

Solución

Mínimo: $t - 1 = 2$ claves.

Ejercicio

¿Qué sucede cuando se inserta una clave en un árbol B y la raíz está llena?

Solución

Se crea una nueva raíz vacía, se divide la raíz vieja en dos nodos, y la clave mediana sube a la nueva raíz. El árbol aumenta en altura.

Ejercicio

En un árbol B+, ¿dónde se almacenan realmente los datos?

Solución

Los datos se almacenan solo en las hojas. Los nodos internos contienen solo claves y punteros.

Ejercicio

¿Cuántos accesos a disco se necesitan para buscar una clave en un árbol B+ de altura 3?

Solución

Se necesita acceder a cada nivel: raíz (nivel 0), nodo interno (nivel 1), nodo interno (nivel 2), hoja (nivel 3). Total: 4 accesos a disco.

Ejercicio

Un árbol B tiene $t = 100$ y altura 3. ¿Cuál es el número máximo de claves que puede almacenar?

Solución

Nivel 0: $2t - 1 = 199$ claves, $2t = 200$ hijos. Nivel 1: $200 \times 199 = 39,800$ claves, $200 \times 200 = 40,000$ hijos. Nivel 2 (hojas): $40,000 \times 199 = 7,960,000$ claves. Total: $199 + 39,800 + 7,960,000 = 8,000,000$ aproximadamente.

Ejercicio

¿Qué ventaja tiene un árbol B sobre un árbol AVL en sistemas con discos?

Solución

El árbol B tiene altura mucho menor (logaritmo en base t vs. logaritmo en base 2), por lo que requiere menos accesos a disco. Además, utiliza mejor el principio de localidad.

Ejercicio

En un árbol B, ¿qué significa que un nodo está "lleno"?

Solución

Un nodo está lleno cuando tiene $2t - 1$ claves, el máximo permitido.

Ejercicio

¿Por qué los árboles B tienen un factor de ramificación grande?

Solución

Para reducir la altura del árbol y, por lo tanto, el número de accesos a disco. Cada acceso a disco es muy costoso en tiempo.

Ejercicio

En un árbol B+, ¿cómo se realiza una consulta de rango (ej. todas las claves entre 50 y 100)?

Solución

Se busca la clave 50 en el árbol, se encuentra la hoja, y se recorre la lista enlazada de hojas hasta llegar a la clave 100.

Ejercicio

¿Cuál es la diferencia entre un índice primario y uno secundario en bases de datos?

Solución

Índice primario: Ordenado por la clave primaria, las hojas contienen los registros completos. **Índice secundario:** Ordenado por otra clave, las hojas contienen punteros a los registros.

Ejercicio

En un árbol B con $t = 5$, ¿cuántas claves puede tener un nodo como máximo?

Solución

Máximo: $2t - 1 = 9$ claves.

Ejercicio

¿Qué es la altura negra en árboles rojo-negro y cómo se relaciona con árboles B?

Solución

La altura negra es el número de nodos negros en un camino. Un árbol rojo-negro de altura negra b es equivalente a un árbol B de orden b si se fusionan nodos rojos con sus padres negros.

Ejercicio

En un árbol B+, ¿qué contiene la lista enlazada de hojas?

Solución

Contiene todas las claves en orden ascendente, con punteros a los datos asociados.

Ejercicio

¿Cuántas operaciones de escritura a disco se requieren para insertar una clave en un árbol B de altura 3 si no se produce ninguna división?

Solución

Se leen los 3 niveles (raíz, interno, hoja) y se escribe la hoja modificada. Total: 1 escritura.

5.11. Ejercicios Propuestos (Ampliados)

1. Construya un árbol B con $t = 2$ insertando las claves: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
2. Construya un árbol B con $t = 3$ insertando las claves: 10, 20, 30, 40, 50, 60, 70, 80, 90.
3. Elimine la clave 30 del árbol construido en el ejercicio 2.

4. Calcule la altura máxima de un árbol B con $n = 10^8$ y $t = 200$.
5. Diseñe un árbol B+ para almacenar 1 millón de registros con claves de 8 bytes y punteros de 8 bytes, usando páginas de 4 KB. ¿Cuántas claves por hoja? ¿Cuál es la altura?
6. Explique el algoritmo de división de un nodo en árbol B.
7. ¿Qué sucede si intentamos eliminar una clave de un árbol B y el nodo queda con menos de $t - 1$ claves?
8. Compare árboles B y B+ en términos de eficiencia de búsqueda y recorrido secuencial.
9. ¿Por qué los sistemas de archivos modernos usan árboles B+?
10. Implemente en pseudocódigo la función de búsqueda en árbol B+.
11. ¿Qué es un "índice agrupado" cómo se implementa con árboles B+?
12. ¿Cuántos accesos a disco se necesitan para insertar 1000 claves en un árbol B de altura 3?
13. Explique la diferencia entre el grado mínimo t y el orden m en un árbol B.
14. ¿Qué es un "árbol B*" en qué se diferencia del árbol B estándar?
15. Diseñe un algoritmo para encontrar el k -ésimo elemento más pequeño en un árbol B.
16. ¿Cómo afecta el tamaño de página al rendimiento de un árbol B?
17. ¿Qué es la "compresión de claves" cómo ayuda en árboles B?
18. Explique cómo se manejan las claves duplicadas en un árbol B+.
19. ¿Qué ventaja tiene un árbol B+ sobre un árbol B para consultas de rango?
20. Investigue: ¿qué estructura de índice usa PostgreSQL por defecto?

Unidad VI: Heaps (Montículos) - Implementación con Arreglos

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la estructura de datos heap y su representación eficiente mediante arreglos.
- Implementar las operaciones fundamentales: MAX-HEAPIFY, BUILD-MAX-HEAP, HEAPSORT.
- Implementar operaciones de cola de prioridad: INSERT, EXTRACT-MAX, INCREASE-KEY.
- Analizar la complejidad temporal de cada operación.
- Aplicar heaps en algoritmos como Heapsort y en la implementación de colas de prioridad.

Introducción: ¿Qué es un Heap?

Definición

Un **heap** (montículo) es un árbol binario casi completo que satisface la **propiedad de heap**:

- **Max-heap:** Para todo nodo i (excepto la raíz), $A[\text{Padre}(i)] \geq A[i]$. El máximo está en la raíz.
- **Min-heap:** Para todo nodo i (excepto la raíz), $A[\text{Padre}(i)] \leq A[i]$. El mínimo está en la raíz.

Propiedades Clave

- Es un árbol binario **casi completo**: todas las filas están completas excepto posiblemente la última, que se llena de izquierda a derecha.
- Se puede representar eficientemente con un **arreglo** sin necesidad de punteros.
- Altura: $\lfloor \log_2 n \rfloor$, donde n es el número de nodos.

Aplicaciones

- **Heapsort:** Algoritmo de ordenación $O(n \log n)$ in-place.
- **Colas de prioridad:** Operaciones de inserción y extracción de máximo/mínimo en $O(\log n)$.
- **Algoritmos de grafos:** Dijkstra (caminos mínimos) y Prim (árbol de expansión mínimo).

Representación con Arreglos

Indexación

Dado un arreglo A de tamaño n (1-indexado), las relaciones son:

- **Raíz:** $A[1]$
- **Padre de i :** $\lfloor i/2 \rfloor$
- **Hijo izquierdo de i :** $2i$
- **Hijo derecho de i :** $2i + 1$

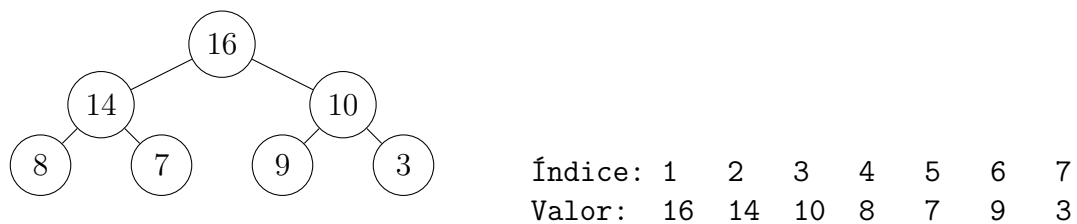


Figura 2: Un max-heap y su representación como arreglo

Ventajas de la Representación con Arreglos

- **Eficiencia espacial:** Sin punteros, solo el arreglo.
- **Localidad de referencia:** Acceso secuencial en memoria.
- **Cálculo rápido:** Relaciones padre-hijo con operaciones aritméticas simples.

Atributos

```

1 struct Heap {
2     int* array;           // arreglo de elementos
3     int size;             // n mero actual de elementos
4     int capacity;        // capacidad m xima
5 };

```

Listing 1: Estructura de un heap

Operación Fundamental: MAX-HEAPIFY

Descripción

MAX-HEAPIFY mantiene la propiedad de max-heap. Asume que los subárboles izquierdo y derecho del nodo i son max-heaps, pero $A[i]$ puede ser menor que sus hijos.

Pseudocódigo

```

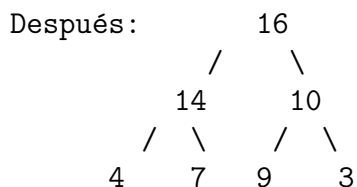
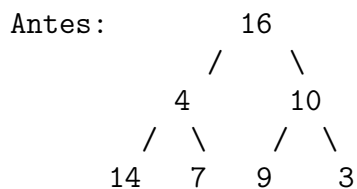
1 MAX-HEAPIFY(A, i, n)
2   l = 2*i                // hijo izquierdo
3   r = 2*i + 1            // hijo derecho
4   largest = i
5
6   if l <= n and A[l] > A[largest]
7       largest = l
8   if r <= n and A[r] > A[largest]
9       largest = r
10
11  if largest != i
12      swap(A[i], A[largest])
13      MAX-HEAPIFY(A, largest, n)

```

Listing 2: MAX-HEAPIFY

Ejemplo Visual

Consideremos el arreglo $[16, 4, 10, 14, 7, 9, 3]$ en el nodo $i = 2$ (valor 4):



Análisis de Complejidad

- Cada llamada recursiva desciende un nivel.
- Altura del heap: $\lfloor \log_2 n \rfloor$.
- Tiempo: $O(\log n)$.

Construcción de un Heap: BUILD-MAX-HEAP

Descripción

Convierte un arreglo desordenado en un max-heap. Aprovecha que las hojas (índices $\lfloor n/2 \rfloor + 1$ hasta n) ya son max-heaps triviales.

Pseudocódigo

```

1 BUILD-MAX-HEAP(A, n)
2   for i = floor(n/2) downto 1
3     MAX-HEAPIFY(A, i, n)

```

Listing 3: BUILD-MAX-HEAP

Ejemplo

Construir max-heap desde $[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$:

1. $n = 10$, $\lfloor 10/2 \rfloor = 5$. Procesar $i = 5$ (valor 16): ya es heap.
2. $i = 4$ (valor 2): MAX-HEAPIFY intercambia con 14. Resultado: $[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$.
3. $i = 3$ (valor 3): intercambia con 10. $[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$.
4. $i = 2$ (valor 1): intercambia con 16. $[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$, luego con 14. $[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$. Revisar.
5. $i = 1$ (valor 4): intercambia con 16. $[16, 4, 10, 14, 1, 9, 3, 2, 8, 7]$, luego con 14. $[16, 14, 10, 4, 1, 9, 3, 2, 8, 7]$.

Análisis de Complejidad

- Análisis simple: $O(n \log n)$ (n llamadas a MAX-HEAPIFY).
- Análisis ajustado: $O(n)$ porque la mayoría de los nodos están en la parte baja del árbol.

Teorema 0.1. *BUILD-MAX-HEAP tiene complejidad $O(n)$.*

Demostración. La altura de un nodo i es h_i . MAX-HEAPIFY en un nodo de altura h toma $O(h)$. El número de nodos de altura h es a lo sumo $\lceil n/2^{h+1} \rceil$.

$$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^{h+1}}\right) = O(n)$$

La suma converge a 2.

□

Algoritmo HEAPSORT

Descripción

Heapsort ordena un arreglo en tres pasos:

1. Construir un max-heap con BUILD-MAX-HEAP.
2. Intercambiar la raíz (máximo) con el último elemento.
3. Reducir el tamaño del heap y llamar MAX-HEAPIFY en la nueva raíz.
4. Repetir hasta que el heap tenga un solo elemento.

Pseudocódigo

```
1 HEAPSORT(A, n)
2   BUILD-MAX-HEAP(A, n)
3   for i = n downto 2
4       swap(A[1], A[i])
5       MAX-HEAPIFY(A, 1, i-1)
```

Listing 4: HEAPSORT

Ejemplo

Ordenar [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]:

1. Construir max-heap: [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]
2. Intercambiar 16 con 1: [1, 14, 10, 8, 7, 9, 3, 2, 4, 16]
3. MAX-HEAPIFY en raíz: [14, 8, 10, 4, 7, 9, 3, 2, 1, 16]
4. Intercambiar 14 con 1: [1, 8, 10, 4, 7, 9, 3, 2, 14, 16]
5. ... continuar hasta obtener [1, 2, 3, 4, 7, 8, 9, 10, 14, 16]

Análisis de Complejidad

- BUILD-MAX-HEAP: $O(n)$.
- Bucle: $n - 1$ iteraciones.
- Cada iteración: $O(\log n)$ por MAX-HEAPIFY.
- Total: $O(n \log n)$.

Colas de Prioridad con Heaps

Operaciones Básicas

Un max-heap puede implementar una cola de prioridad máxima con:

- **MAXIMUM:** Devuelve la raíz. $O(1)$.
- **EXTRACT-MAX:** Elimina y devuelve el máximo. $O(\log n)$.
- **INCREASE-KEY:** Aumenta la clave de un elemento. $O(\log n)$.
- **INSERT:** Inserta un nuevo elemento. $O(\log n)$.

HEAP-MAXIMUM

```
1 HEAP-MAXIMUM(A)
2   return A[1]
```

Listing 5: HEAP-MAXIMUM

HEAP-EXTRACT-MAX

```
1 HEAP-EXTRACT-MAX(A, n)
2   if n < 1
3       error "heap underflow"
4   max = A[1]
5   A[1] = A[n]
6   n = n - 1
7   MAX-HEAPIFY(A, 1, n)
8   return max
```

Listing 6: HEAP-EXTRACT-MAX

HEAP-INCREASE-KEY

```
1 HEAP-INCREASE-KEY(A, i, key)
2   if key < A[i]
3       error "new key is smaller"
4   A[i] = key
5   while i > 1 and A[parent(i)] < A[i]
6       swap(A[i], A[parent(i)])
7       i = parent(i)
```

Listing 7: HEAP-INCREASE-KEY

MAX-HEAP-INSERT

```
1 MAX-HEAP-INSERT(A, n, key)
2   n = n + 1
3   A[n] = -infinity
4   HEAP-INCREASE-KEY(A, n, key)
```

Listing 8: MAX-HEAP-INSERT

Min-Heaps

Definición

Un min-heap es análogo al max-heap pero con la propiedad invertida: $A[\text{Padre}(i)] \leq A[i]$. El mínimo está en la raíz.

Operaciones Simétricas

- **MIN-HEAPIFY:** Similar a MAX-HEAPIFY pero con comparación $\leq$.
- **BUILD-MIN-HEAP:** Idéntico a BUILD-MAX-HEAP pero usando MIN-HEAPIFY.
- **HEAP-MINIMUM:** Devuelve $A[1]$.
- **HEAP-EXTRACT-MIN:** Extrae la raíz.
- **HEAP-DECREASE-KEY:** Disminuye una clave.

Pseudocódigo de MIN-HEAPIFY

```

1 MIN-HEAPIFY(A, i, n)
2   l = 2*i
3   r = 2*i + 1
4   smallest = i
5
6   if l <= n and A[l] < A[smallest]
7       smallest = l
8   if r <= n and A[r] < A[smallest]
9       smallest = r
10
11  if smallest != i
12      swap(A[i], A[smallest])
13      MIN-HEAPIFY(A, smallest, n)

```

Listing 9: MIN-HEAPIFY

Aplicaciones Prácticas

Heapsort en la Práctica

- Es un algoritmo de ordenación $O(n \log n)$ **in-place**.
- No es estable (no preserva el orden relativo de elementos iguales).
- En la práctica, Quicksort suele ser más rápido, pero Heapsort tiene mejor comportamiento en el peor caso.

Colas de Prioridad en Sistemas Operativos

- **Planificación de procesos:** Los procesos con mayor prioridad se ejecutan primero.
- **Manejo de interrupciones:** Las interrupciones con mayor prioridad se atienden primero.

Algoritmos de Grafos

- **Dijkstra:** Encuentra caminos más cortos usando una cola de prioridad.
- **Prim:** Encuentra el árbol de expansión mínimo.

Ejercicios Resueltos (Ampliados)

Ejercicio

Dado el arreglo $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, verifique si es un max-heap.

Solución

Verificamos cada nodo (excepto hojas):

- $i = 1$: $16 \geq 14$ y $16 \geq 10$
- $i = 2$: $14 \geq 8$ y $14 \geq 7$
- $i = 3$: $10 \geq 9$ y $10 \geq 3$
- $i = 4$: $8 \geq 2$ y $8 \geq 4$
- $i = 5$: $7 \geq 1$

Es un max-heap válido.

Ejercicio

Aplique MAX-HEAPIFY al arreglo $[3, 10, 5, 8, 7, 6, 9]$ en $i = 1$.

Solución

1. $i = 1$, $l = 2$ (10), $r = 3$ (5). Mayor es 10 (índice 2).
2. Intercambiar $A[1]$ y $A[2]$: $[10, 3, 5, 8, 7, 6, 9]$.
3. Llamar MAX-HEAPIFY en $i = 2$ (valor 3).
4. $i = 2$, $l = 4$ (8), $r = 5$ (7). Mayor es 8 (índice 4).
5. Intercambiar $A[2]$ y $A[4]$: $[10, 8, 5, 3, 7, 6, 9]$.
6. Llamar MAX-HEAPIFY en $i = 4$ (valor 3). Sus hijos son 8 y 9 (índices 8 y 9, fuera de rango). Fin.

Resultado: $[10, 8, 5, 3, 7, 6, 9]$.

Ejercicio

Construya un max-heap desde $[5, 3, 17, 10, 84, 19, 6, 22, 9]$.

Solución

$n = 9$, $\lfloor 9/2 \rfloor = 4$.

1. $i = 4$ (10): hijos 8 (22) y 9 (9). Mayor es 22, intercambiar: [5, 3, 17, 22, 84, 19, 6, 10, 9].
2. $i = 3$ (17): hijos 6 (19) y 7 (6). Mayor es 19, intercambiar: [5, 3, 19, 22, 84, 17, 6, 10, 9].
3. $i = 2$ (3): hijos 4 (22) y 5 (84). Mayor es 84, intercambiar con 3: [5, 84, 19, 22, 3, 17, 6, 10, 9]. Luego verificar $i = 5$ (3): hijos 10 y 11 fuera.
4. $i = 1$ (5): hijos 2 (84) y 3 (19). Mayor es 84, intercambiar: [84, 5, 19, 22, 3, 17, 6, 10, 9]. Luego MAX-HEAPIFY en $i = 2$ (5): hijos 4 (22) y 5 (3). Mayor 22, intercambiar: [84, 22, 19, 5, 3, 17, 6, 10, 9]. Luego MAX-HEAPIFY en $i = 4$ (5): hijos 8 (10) y 9 (9). Mayor 10, intercambiar: [84, 22, 19, 10, 3, 17, 6, 5, 9].

Resultado: [84, 22, 19, 10, 3, 17, 6, 5, 9].

Ejercicio

Ordene [5, 13, 2, 25, 7, 17, 20, 8, 4] usando Heapsort.

Solución

1. Construir max-heap: proceso similar al ejercicio anterior.
2. Intercambiar raíz con último, reducir tamaño, MAX-HEAPIFY.
3. Repetir hasta ordenar.

Resultado final: [2, 4, 5, 7, 8, 13, 17, 20, 25].

Ejercicio

Demuestre que en un max-heap, la raíz es el elemento máximo.

Solución

Por la propiedad de max-heap, $A[\text{Padre}(i)] \geq A[i]$ para todo i . Por transitividad, la raíz $A[1]$ es mayor o igual que cualquier otro nodo.

Ejercicio

¿Cuál es la altura de un heap con 100 elementos?

Solución

Altura = $\lfloor \log_2 100 \rfloor = \lfloor 6,64 \rfloor = 6$.

Ejercicio

Demuestre que BUILD-MAX-HEAP tiene complejidad $O(n)$.

Solución

La suma $\sum_{h=0}^{\lceil \log n \rceil} \lceil n/2^{h+1} \rceil \cdot O(h) \leq O(n \sum_{h=0}^{\infty} h/2^h) = O(n)$ porque $\sum_{h=0}^{\infty} h/2^h = 2$.

Ejercicio

Implemente HEAP-DELETE para eliminar un elemento en posición i .

Solución

```

1  HEAP-DELETE(A, n, i)
2      if i == n
3          n = n - 1
4          return
5      A[i] = A[n]
6      n = n - 1
7      if A[i] > A[parent(i)]
8          HEAP-INCREASE-KEY(A, i, A[i])
9      else
10         MAX-HEAPIFY(A, i, n)

```

Ejercicio

¿Cuál es la diferencia entre un max-heap y un min-heap?

Solución

En max-heap, $A[Padre] \geq A[Hijo]$; en min-heap, $A[Padre] \leq A[Hijo]$. El máximo está en la raíz del max-heap; el mínimo en la raíz del min-heap.

Ejercicio

¿Cuántos nodos tiene un heap completo de altura h ?

Solución

Un heap completo de altura h tiene $2^{h+1} - 1$ nodos.

Ejercicio

En un max-heap, ¿dónde puede estar el elemento mínimo?

Solución

En las hojas (último nivel). Puede estar en cualquier hoja.

Ejercicio

¿Qué operaciones de heap se usan en Heapsort?

Solución

BUILD-MAX-HEAP y MAX-HEAPIFY.

Ejercicio

¿Cuál es la complejidad de HEAP-EXTRACT-MAX?

Solución

$O(\log n)$ por la llamada a MAX-HEAPIFY.

Ejercicio

¿Qué sucede si llamamos MAX-HEAPIFY en un nodo hoja?

Solución

No hace nada porque los hijos no existen, y termina inmediatamente.

Ejercicio

¿Cuántas comparaciones se hacen en el peor caso en MAX-HEAPIFY?

Solución

En cada nivel, se comparan dos hijos (2 comparaciones). La profundidad es $\log n$, total $O(\log n)$ comparaciones.

Ejercicio

¿Puede un heap estar representado como un árbol binario no completo?

Solución

No, por definición es un árbol binario casi completo. La representación con arreglos requiere que los nodos estén en posiciones contiguas.

Ejercicio

Demuestre que en un heap de n nodos, hay $\lceil n/2 \rceil$ hojas.

Solución

Las hojas son los nodos desde $\lfloor n/2 \rfloor + 1$ hasta n . El número es $n - \lfloor n/2 \rfloor = \lceil n/2 \rceil$.

Ejercicio

¿Cuál es la complejidad de HEAP-INCREASE-KEY en el mejor caso?

Solución

$O(1)$, si la clave ya es mayor que su padre.

Ejercicio

Implemente HEAP-MERGE para fusionar dos heaps.

Solución

```

1 HEAP-MERGE(A, n1, B, n2)
2   C = array of size n1 + n2
3   copy A and B into C
4   BUILD-MAX-HEAP(C, n1 + n2)
5   return C

```

Complejidad: $O(n_1 + n_2)$.

Ejercicio

¿Por qué Heapsort no es un algoritmo estable?

Solución

Porque el intercambio de elementos puede alterar el orden relativo de elementos iguales.

Ejercicio

¿Cuál es la ventaja de Heapsort sobre Merge Sort?

Solución

Heapsort ordena in-place (no requiere memoria adicional), mientras que Merge Sort requiere $O(n)$ espacio auxiliar.

Ejercicios Propuestos (Ampliados)

1. Construya un max-heap desde [10, 20, 15, 30, 40, 50, 25, 35, 45, 5].
2. Construya un min-heap desde [8, 3, 5, 1, 4, 9, 2, 7, 6].
3. Ordene [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5] usando Heapsort.
4. Demuestre que la altura de un heap de n nodos es $\lfloor \log_2 n \rfloor$.
5. Implemente HEAP-DELETE para eliminar el elemento en posición i .

6. ¿Cuál es la complejidad de construir un heap insertando elementos uno a uno (con INSERT)?
7. Explique cómo usar un min-heap para implementar una cola de prioridad mínima.
8. ¿Qué es un "heap binario" y cómo se diferencia de un "heap binomial"?
9. Demuestre que el número de hojas en un heap de n nodos es $\lceil n/2 \rceil$.
10. Implemente HEAP-EXTRACT-MAX iterativamente (sin recursión).
11. ¿Cuántos nodos tiene un heap completo de altura h ? ¿Y uno casi completo?
12. ¿Puede un heap tener nodos con el mismo valor? ¿Cómo afecta esto a la propiedad?
13. Implemente HEAP-SORT para ordenar en orden descendente usando un min-heap.
14. ¿Cuál es el peor caso de Heapsort? ¿Y el mejor caso?
15. Demuestre que BUILD-MAX-HEAP es $O(n)$ usando el método de agregación.
16. ¿Qué es la "propiedad de heap" y por qué es importante?
17. Implemente una cola de prioridad usando heaps en C o Python.
18. ¿Cómo se puede encontrar el k -ésimo elemento más pequeño usando un heap?
19. Explique la relación entre heaps y árboles de torneo.
20. ¿Qué es un "d-ary heap" y cuándo es útil?

Unidad VII: Hashing (Tablas Hash)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el concepto de tablas hash y su aplicación en diccionarios.
- Diseñar y analizar funciones hash efectivas.
- Implementar resolución de colisiones mediante encadenamiento y direccionamiento abierto.
- Analizar el rendimiento esperado de las operaciones de hash.
- Aplicar hashing en problemas reales como búsqueda de elementos y conteo de frecuencias.

Introducción: El Problema del Acceso Directo

El Ideal: Direccionamiento Directo

Si tenemos un universo U de claves pequeño (ej. $\{0, 1, \dots, m-1\}$), podemos usar una **tabla de direccionamiento directo**:

- Arreglo $T[0..m-1]$.
- Insertar clave k en $T[k]$.
- Buscar clave k en $T[k]$.
- Eliminar clave k de $T[k]$.

Todas las operaciones: $O(1)$ en el peor caso.

El Problema

- ¿Qué pasa si U es enorme (ej. $U = \{0, 1, \dots, 2^{64} - 1\}$)?
- No podemos asignar un arreglo de tamaño $|U|$.
- Necesitamos una función que **comprima** el universo a un rango manejable.

Solución: Tablas Hash

Una **tabla hash** es un arreglo $T[0..m-1]$ donde m es mucho menor que $|U|$. Una **función hash** $h : U \rightarrow \{0, 1, \dots, m-1\}$ asigna cada clave a una posición.

k_1	$T[0]$
k_2	$T[1]$
k_3	$T[2]$
k_4	$T[3]$
k_5	$T[4]$
	$T[5]$

El Problema de las Colisiones

Como $|U| > m$, por el principio del palomar, existen al menos dos claves $k_1 \neq k_2$ con $h(k_1) = h(k_2)$. Esto se llama **colisión**. Necesitamos métodos para resolverlas.

Funciones Hash

¿Qué hace una buena función hash?

- **Hash uniforme simple:** Cada clave tiene la misma probabilidad de hashear a cualquier slot, independientemente de otras claves.
- **Determinista:** $h(k)$ debe ser el mismo cada vez.
- **Rápida de calcular:** Idealmente $O(1)$.
- **Dispersión:** Claves similares deben hashear a slots diferentes.

Interpretar Claves como Números Naturales

La mayoría de las funciones hash asumen que las claves son números naturales. Si no lo son, debemos convertirlas:

- **Cadenas:** Interpretar como número en base 128 (ASCII) o 256 (bytes).
- **Floats:** Interpretar su representación binaria como entero.
- **Objetos:** Combinar hashes de sus componentes.

Método de la División

$$h(k) = k \bmod m$$

- **Ventaja:** Muy rápida (una operación de módulo).

- **Desventaja:** Requiere cuidado al elegir m .
- **Elección de m :**
 - Evitar potencias de 2 ($m = 2^p$): $h(k)$ son los p bits menos significativos.
 - Evitar $m = 2^p - 1$ si k es cadena en base 2^p : permutaciones de caracteres tienen el mismo hash.
 - **Bueno:** m primo no cercano a potencia de 2.

Ejemplo

Para $m = 701$ (primo), $h(100) = 100 \bmod 701 = 100$, $h(702) = 702 \bmod 701 = 1$.

Método de la Multiplicación

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

donde $0 < A < 1$ es una constante.

- **Ventaja:** El valor de m no es crítico.
- **Elección de A :** Knuth sugiere $A \approx (\sqrt{5} - 1)/2 \approx 0,6180339887$.
- **Implementación eficiente:** Si $m = 2^p$, usar aritmética de enteros.

Ejemplo

Para $k = 123456$, $p = 14$, $m = 2^{14} = 16384$, $A = 2654435769/2^{32}$:

- $k \cdot A \approx 0,17612864$
- $m \cdot (kA \bmod 1) = 16384 \times 0,17612864 \approx 2886,6$
- $h(k) = 2886$

Hash Universal

Definición 0.1. Una colección $\mathcal{H}$ de funciones hash es **universal** si para cualquier par de claves distintas k, l :

$$\Pr_{h \in \mathcal{H}} [h(k) = h(l)] \leq \frac{1}{m}$$

Construcción Universal

Sea p un primo mayor que cualquier clave. Definimos:

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

donde $a \in \{1, 2, \dots, p-1\}$, $b \in \{0, 1, \dots, p-1\}$.

Teorema 0.2. La familia $\mathcal{H} = \{h_{a,b} : 1 \leq a \leq p-1, 0 \leq b \leq p-1\}$ es universal.

Esbozo. Para $k \neq l$, $(ak + b) \bmod p$ y $(al + b) \bmod p$ son independientes y uniformes. La probabilidad de que difieran en un múltiplo de m es a lo sumo $1/m$. $\square$

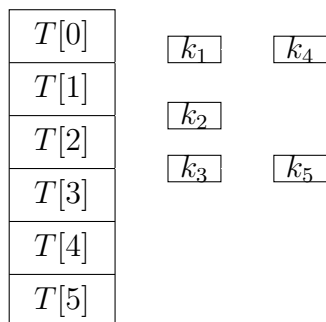
Otras Funciones Hash

- **Hash de cadenas (Polinomial):** $h(s) = \sum_{i=0}^{n-1} s[i] \cdot B^{n-1-i} \text{ mód } m$, con B primo.
- **Hash criptográfico:** MD5, SHA-1, SHA-256 (más lentos, pero resistentes a colisiones maliciosas).
- **Hash con desplazamiento:** $h(k) = (k \oplus \text{constante}) \text{ mód } m$ (rápido, pero menos disperso).

Resolución de Colisiones por Encadenamiento

Descripción

Cada slot $T[j]$ contiene una lista enlazada de todos los elementos que hashean a j .



Operaciones

```

1 CHAINED-HASH-INSERT(T, x)
2     // insertar al inicio de la lista
3     x.next = T[h(x.key)]
4     T[h(x.key)] = x
5
6 CHAINED-HASH-SEARCH(T, k)
7     x = T[h(k)]
8     while x != NIL and x.key != k
9         x = x.next
10    return x
11
12 CHAINED-HASH-DELETE(T, x)
13    // asumiendo lista doblemente enlazada
14    if x.prev != NIL
15        x.prev.next = x.next
16    else
17        T[h(x.key)] = x.next
18    if x.next != NIL
19        x.next.prev = x.prev

```

Listing 10: Encadenamiento

Análisis de Rendimiento

Sea $\alpha = n/m$ el **factor de carga** (número promedio de elementos por lista).

Teorema 0.3. *En una tabla hash con encadenamiento y hash uniforme simple:*

- Una búsqueda fallida toma tiempo esperado $\Theta(1 + \alpha)$.
- Una búsqueda exitosa toma tiempo esperado $\Theta(1 + \alpha)$.

Esbozo. Para búsqueda fallida, el número esperado de elementos en la lista es α . Para búsqueda exitosa, se examina $1 + (\text{número de elementos insertados después})$: esperado $\alpha/2$. $\square$

Si $n = O(m)$, entonces $\alpha = O(1)$ y las operaciones son $O(1)$ esperado.

Resolución de Colisiones por Direccionamiento Abierto

Descripción

Todos los elementos se almacenan en la propia tabla. Cada slot contiene un elemento o NIL. Para insertar, se sondean slots hasta encontrar uno vacío.

Secuencia de Sondas

La función hash se extiende a $h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$. Para cada clave k , la secuencia $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ es una permutación de $\{0, \dots, m-1\}$.

Sondeo Lineal

$$h(k, i) = (h'(k) + i) \text{ mód } m$$

- **Ventaja:** Simple, buena localidad de referencia.
- **Desventaja: Agrupamiento primario:** largas secuencias de slots ocupados se forman, aumentando el tiempo de búsqueda.

Sondeo Cuadrático

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \text{ mód } m$$

- **Ventaja:** Mejor dispersión que lineal.
- **Desventaja: Agrupamiento secundario:** si dos claves tienen el mismo $h'(k)$, sus secuencias son iguales.

Doble Hashing

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- **Ventaja:** Excelente dispersión, se aproxima al hash uniforme.
- **Requisito:** $h_2(k)$ debe ser relativamente primo con m (para que la secuencia sea una permutación).

Ejemplo

$m = 13$, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$. Para $k = 14$: $h_1 = 1$, $h_2 = 1 + 3 = 4$. Secuencia: 1, 5, 9, 0, 4, 8, 12, 3, 7, 11, 2, 6, 10.

Operaciones

```

1 HASH-INSERT(T, k)
2   i = 0
3   repeat
4       j = h(k, i)
5       if T[j] == NIL
6           T[j] = k
7       return j
8       i = i + 1
9   until i == m
10  error "table overflow"
11
12 HASH-SEARCH(T, k)
13   i = 0
14   repeat
15       j = h(k, i)
16       if T[j] == k
17           return j
18       i = i + 1
19   until T[j] == NIL or i == m
20  return NIL

```

Listing 11: Direccionamiento Abierto

Análisis de Rendimiento

Teorema 0.4. En una tabla hash con direccionamiento abierto y hash uniforme, con factor de carga $\alpha < 1$:

- Número esperado de sondas en búsqueda fallida: $\leq \frac{1}{1-\alpha}$.
- Número esperado de sondas en búsqueda exitosa: $\leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

α	Búsqueda fallida	Búsqueda exitosa
0.50	2.00	1.39
0.75	4.00	2.08
0.90	10.00	2.56
0.95	20.00	3.15

Eliminación en Direccionamiento Abierto

La eliminación es problemática porque si marcamos un slot como NIL, se pueden perder claves posteriores en la secuencia de sonda.

Solución: Marcador DELETED

- Usar un valor especial DELETED.
- HASH-SEARCH trata DELETED como ocupado (continúa).
- HASH-INSERT trata DELETED como vacío (puede insertar).
- Desventaja: El rendimiento se degrada con muchas eliminaciones.

Hash Perfecto

Definición

Para conjuntos **estáticos** (sin inserciones ni eliminaciones), podemos construir un hash perfecto:

- **Hash perfecto:** Sin colisiones.
- **Hash mínimo perfecto:** Sin colisiones y $m = n$.

Estrategia de Dos Niveles

1. Primera función hash h a $m = n$ slots.
2. Para cada slot j con n_j elementos, usar segunda función hash h_j a $m_j = n_j^2$ slots.
3. Con probabilidad alta, no hay colisiones en el segundo nivel.
4. Espacio total esperado: $O(n)$.

Ejercicios Resueltos (Ampliados)

Ejercicio

Sea $m = 11$ y $h(k) = k \bmod 11$. Inserte las claves 10, 22, 31, 4, 15, 28, 17, 88, 59 en una tabla con encadenamiento.

Solución

- $h(10) = 10, h(22) = 0, h(31) = 9, h(4) = 4, h(15) = 4, h(28) = 6, h(17) = 6, h(88) = 0, h(59) = 4.$
- Slot 0: 22, 88
- Slot 4: 4, 15, 59
- Slot 6: 28, 17
- Slot 9: 31
- Slot 10: 10

Ejercicio

Con $m = 10$, $h(k) = k \bmod 10$, inserte 5, 15, 25, 35, 45. ¿Qué problema observa?

Solución

Todas hashean a 5. Es un mal ejemplo porque m es par y todas las claves son múltiplos de 5. m debería ser primo.

Ejercicio

Use el método de la multiplicación con $A = 0,618$, $m = 1000$, $k = 123456$. Calcule $h(k)$.

Solución

$kA = 123456 \times 0,618 \approx 76300,608$. $kA \bmod 1 \approx 0,608$. $m \times 0,608 \approx 608$. $h(k) = 608$.

Ejercicio

Construya una familia universal de funciones hash para claves en $\{0, \dots, 1000\}$ con $m = 100$.

Solución

Elija $p = 1009$ (primo $\nmid 1000$). Para $a \in \{1, \dots, 1008\}$, $b \in \{0, \dots, 1008\}$, $h_{a,b}(k) = ((ak + b) \bmod 1009) \bmod 100$.

Ejercicio

Demuestre que el número esperado de colisiones en una tabla hash con encadenamiento es $\binom{n}{2}/m$.

Solución

Sea $X_{ij} = I\{h(k_i) = h(k_j)\}$. $E[X_{ij}] = 1/m$. Hay $\binom{n}{2}$ pares. $E[\text{colisiones}] = \binom{n}{2}/m$.

Ejercicio

En direccionamiento abierto con $\alpha = 0,75$, ¿cuál es el número esperado de sondas para búsqueda fallida?

Solución

$1/(1 - \alpha) = 1/0,25 = 4$ sondas esperadas.

Ejercicio

¿Por qué el doble hashing requiere que $h_2(k)$ sea relativamente primo con m ?

Solución

Si $d = \gcd(h_2(k), m) > 1$, la secuencia solo visita m/d slots, no todos.

Ejercicio

¿Qué es el factor de carga y por qué debe ser menor que 1 en direccionamiento abierto?

Solución

$\alpha = n/m$. En direccionamiento abierto, la tabla se llena completamente cuando $n = m$, por lo que α debe ser < 1 para que haya espacio para insertar.

Ejercicio

Calcule la probabilidad de que dos claves colisionen en una tabla de tamaño m si h es uniforme.

Solución

$1/m$.

Ejercicio

¿Cuál es la complejidad esperada de búsqueda en una tabla hash con encadenamiento si $n = 2m$?

Solución

$\alpha = 2$, esperado $\Theta(1 + \alpha) = \Theta(3) = O(1)$.

Ejercicio

Diseñe una función hash para cadenas usando el método polinomial con $B = 131$.

Solución

```

1 hash_string(s, m)
2     h = 0
3     for each char c in s
4         h = (h * 131 + c) mod m
5     return h

```

Ejercicio

¿Qué es el ^agrupamiento primario.^{en} sondeo lineal?

Solución

Es la tendencia a formar largas secuencias contiguas de slots ocupados, que aumentan el tiempo de búsqueda.

Ejercicio

Demuestre que el método de la división con $m = 2^p$ es malo porque $h(k)$ ignora los bits más significativos.

Solución

$k \bmod 2^p$ son los p bits menos significativos. Claves que difieren solo en bits altos colisionan.

Ejercicio

¿Cuántas funciones hay en la familia universal $\mathcal{H}_{p,m}$?

Solución

$p(p-1)$ funciones (p opciones para b , $p-1$ para a).

Ejercicio

En doble hashing con $m = 13$, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$, encuentre la secuencia para $k = 42$.

Solución

$h_1 = 42 \bmod 13 = 3$, $h_2 = 1 + (42 \bmod 11) = 1 + 9 = 10$. Secuencia: 3, 0, 10, 7, 4, 1, 11, 8, 5, 2, 12, 9, 6.

Ejercicio

¿Qué es el hash perfecto y cuándo se usa?

Solución

Hash sin colisiones. Se usa para conjuntos estáticos (ej. palabras reservadas de un lenguaje, tablas de símbolos fijas).

Ejercicio

Calcule el número esperado de sondas para búsqueda exitosa con $\alpha = 0,9$ en direccionamiento abierto.

Solución

$$\frac{1}{\alpha} \ln \frac{1}{1-\alpha} = \frac{1}{0,9} \ln \frac{1}{0,1} \approx 1,111 \times 2,303 \approx 2,56.$$

Ejercicio

¿Por qué el sondeo cuadrático no visita todos los slots si m no es primo?

Solución

Puede haber ciclos cortos. Con $m = 2^p$, la secuencia es periódica con período $m/2$.

Ejercicio

¿Cuál es la probabilidad de que en una tabla de tamaño m con n claves, un slot específico esté vacío?

Solución

$$(1 - 1/m)^n.$$

Ejercicios Propuestos (Ampliados)

1. Inserte las claves 5, 28, 19, 15, 20, 33, 12, 17, 10 en una tabla hash con $m = 9$ y $h(k) = k \bmod 9$ usando encadenamiento.
2. Inserte las mismas claves usando direccionamiento abierto con sondeo lineal.
3. Inserte las mismas claves usando doble hashing con $h_1(k) = k \bmod 9$ y $h_2(k) = 1 + (k \bmod 8)$.
4. Diseñe una función hash universal para claves de 4 bytes.
5. Demuestre que el número esperado de colisiones es $\binom{n}{2}/m$.
6. ¿Cuál es el factor de carga óptimo para una tabla hash con encadenamiento?
7. Implemente una tabla hash con encadenamiento en C.
8. Analice el rendimiento de una tabla hash cuando todas las claves hashlean al mismo slot.

9. ¿Qué es un "hash criptográfico" y en qué se diferencia de un hash de tabla?
10. Calcule la probabilidad de que al menos dos claves colisionen en una tabla de tamaño m con n claves (paradoja del cumpleaños).
11. Explique cómo manejar claves duplicadas en una tabla hash.
12. ¿Qué es el rehashing y cuándo es necesario?
13. Diseñe una función hash para números de punto flotante.
14. Implemente un diccionario con tablas hash en Python.
15. ¿Cuál es la complejidad esperada de INSERT en una tabla hash con encadenamiento?
16. ¿Qué es el "hash con direccionamiento abierto" y por qué es más eficiente en espacio?
17. Compare las ventajas y desventajas de encadenamiento vs. direccionamiento abierto.
18. ¿Qué es el "hash perfecto mínimo" y cómo se construye?
19. Investigue cómo se implementan los diccionarios en Python (CPython).
20. ¿Cómo afecta el tamaño de la tabla al rendimiento del hash?

Unidad VIII: Métodos de Resolución de Colisiones

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la naturaleza y causas de las colisiones en tablas hash.
- Implementar y analizar la resolución de colisiones por encadenamiento.
- Implementar y analizar las variantes de direccionamiento abierto: lineal, cuadrático y doble hashing.
- Comparar el rendimiento de los diferentes métodos bajo distintas condiciones.
- Seleccionar el método apropiado según los requisitos de la aplicación.

Introducción: El Problema de las Colisiones

¿Por qué ocurren las colisiones?

Dado que el universo U de claves es mucho mayor que el tamaño m de la tabla, por el **principio del palomar**, existen al menos dos claves distintas $k_1 \neq k_2$ tales que $h(k_1) = h(k_2)$. Esto es inevitable.

Enfoques Fundamentales

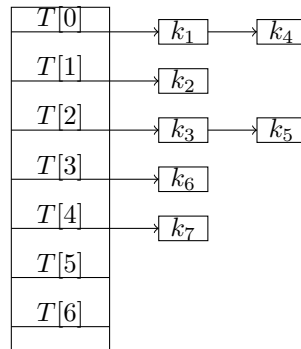
Hay dos grandes familias de métodos para resolver colisiones:

1. **Encadenamiento (Chaining):** Cada slot contiene una estructura (lista, árbol) que almacena múltiples elementos.
2. **Direccionamiento Abierto (Open Addressing):** Los elementos se almacenan en la propia tabla, buscando slots alternativos.

Resolución por Encadenamiento

Descripción General

En el encadenamiento, cada posición de la tabla $T[j]$ contiene un puntero al inicio de una lista enlazada (o cualquier otra estructura) que almacena todas las claves que hashan a j .



Implementación Detallada

Estructura de Nodo

```

1 struct Node {
2     int key;
3     void* data;
4     struct Node* next;
5     struct Node* prev; // para eliminaci n O(1)
6 };
7
8 struct HashTable {
9     struct Node** table; // arreglo de punteros a listas
10    int size;           // m
11    int count;          // n
12 };
  
```

Listing 1: Estructura de nodo para encadenamiento

Inserción

```

1 void insert(HashTable* ht, int key, void* data) {
2     int index = hash(key, ht->size);
3     Node* new_node = malloc(sizeof(Node));
4     new_node->key = key;
5     new_node->data = data;
6     new_node->next = ht->table[index];
  
```

```
7     new_node->prev = NULL;
8
9     if (ht->table[index] != NULL)
10         ht->table[index]->prev = new_node;
11
12     ht->table[index] = new_node;
13     ht->count++;
14 }
```

Listing 2: Inserción con encadenamiento

Búsqueda

```
1 Node* search(HashTable* ht, int key) {
2     int index = hash(key, ht->size);
3     Node* current = ht->table[index];
4
5     while (current != NULL && current->key != key)
6         current = current->next;
7
8     return current;
9 }
```

Listing 3: Búsqueda con encadenamiento

Eliminación

```
1 void delete(HashTable* ht, int key) {
2     int index = hash(key, ht->size);
3     Node* current = ht->table[index];
4
5     while (current != NULL && current->key != key)
6         current = current->next;
7
8     if (current == NULL) return; // no encontrado
9
10    // Reajustar punteros
11    if (current->prev != NULL)
12        current->prev->next = current->next;
13    else
14        ht->table[index] = current->next;
15
16    if (current->next != NULL)
17        current->next->prev = current->prev;
18
19    free(current);
20    ht->count--;
21 }
```

 Listing 4: Eliminación con encadenamiento

Variantes de Encadenamiento

- **Listas ordenadas:** Búsqueda más rápida (se puede detener antes), pero inserción más lenta.
- **Árboles balanceados:** Peor caso $O(\log n)$ en lugar de $O(n)$, útil para tablas muy cargadas.
- **Listas con cabecera:** Simplifica la inserción al inicio.

Análisis Detallado

Factor de Carga

$$\alpha = \frac{n}{m}$$

Tiempo Esperado de Búsqueda

Teorema 0.0.1. *Bajo hash uniforme simple:*

- *Búsqueda fallida:* $E[T] = \Theta(1 + \alpha)$
- *Búsqueda exitosa:* $E[T] = \Theta(1 + \alpha)$

Demostración detallada. **Búsqueda fallida:** La clave k hasha a cualquier slot con probabilidad $1/m$. El número esperado de elementos en ese slot es $n/m = \alpha$. Por lo tanto:

$$E[\text{sondas}] = 1 + \alpha$$

(1 por el slot en sí, más el recorrido de la lista).

Búsqueda exitosa: Sea X_i la variable indicadora de que el i -ésimo elemento insertado está antes que el elemento buscado en su lista. Por linealidad:

$$E[\text{elementos examinados}] = 1 + \sum_{i=1}^n \Pr(\text{elemento } i \text{ está antes})$$

La probabilidad de que el elemento i esté antes es $1/(\text{número de elementos en el slot cuando se insertó})$. Promediando:

$$E[T] = 1 + \frac{1}{n} \sum_{j=1}^n \frac{j-1}{m} = 1 + \frac{1}{nm} \cdot \frac{n(n-1)}{2} = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

□

Ventajas y Desventajas del Encadenamiento

■ Ventajas:

- Simple de implementar.
- Eliminación fácil.
- Rendimiento predecible.
- α puede ser mayor que 1.

■ Desventajas:

- Uso de memoria adicional para punteros.
- Mala localidad de referencia.
- Peor caso $O(n)$ si la función hash es mala.

Direccionamiento Abierto

Descripción General

En direccionamiento abierto, todos los elementos se almacenan en la propia tabla. Cada slot contiene un elemento o NIL. Cuando hay colisión, se busca un slot alternativo mediante una **secuencia de sonda**.

$T[0]$	$T[1]$	$T[2]$	$T[3]$	$T[4]$	$T[5]$	$T[6]$	$T[7]$
15	7	22	NIL	8	31	NIL	NIL
3	11	NIL	NIL	NIL	NIL	NIL	NIL
NIL	NIL	NIL	NIL	NIL	NIL	NIL	NIL
NIL	NIL	NIL	NIL	NIL	NIL	NIL	NIL

Sondeo Lineal

Definición

$$h(k, i) = (h'(k) + i) \text{ mód } m$$

donde h' es la función hash auxiliar.

Implementación

```

1 int linear_probe(HashTable* ht, int key, int i) {
2     return (hash(key, ht->size) + i) % ht->size;
3 }
4
5 void insert_linear(HashTable* ht, int key, void* data) {
6     int i = 0;
7     int index;
8     do {
9         index = linear_probe(ht, key, i);
10        if (ht->table[index] == NULL || ht->table[index] ==
11            DELETED) {
12            ht->table[index] = create_node(key, data);
13            ht->count++;
14            return;
15        }
16        i++;
17    } while (i < ht->size);
18    error("table overflow");
19 }

```

Listing 5: Sondeo lineal

Análisis del Sondeo Lineal

- **Ventajas:** Muy simple, buena localidad de referencia.
- **Desventajas:** **Agrupamiento primario** - se forman largas secuencias de slots ocupados.

Teorema 0.0.2 (Agrupamiento primario). *En sondeo lineal, la longitud esperada de la secuencia de slots ocupados que contiene un slot dado es $\Theta(1/(1-\alpha)^2)$.*

Sondeo Cuadrático

Definición

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

donde c_1 y c_2 son constantes no nulas.

Implementación

```

1 int quadratic_probe(HashTable* ht, int key, int i) {
2     int h = hash(key, ht->size);
3     return (h + c1*i + c2*i*i) % ht->size;
4 }

```

Listing 6: Sondeo cuadrático

Análisis del Sondeo Cuadrático

- **Ventajas:** Mejor dispersión que lineal, evita agrupamiento primario.
- **Desventajas:** **Agrupamiento secundario** - si $h'(k_1) = h'(k_2)$, las secuencias son iguales.
- **Restricciones:** m debe ser primo para garantizar que se visiten todos los slots.

Doble Hashing

Definición

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \text{ mód } m$$

donde h_1 y h_2 son funciones hash auxiliares.

Implementación

```
1 int double_hash(HashTable* ht, int key, int i) {
2     int h1 = hash1(key, ht->size);
3     int h2 = hash2(key, ht->size);
4     return (h1 + i * h2) % ht->size;
5 }
6
7 void insert_double(HashTable* ht, int key, void* data) {
8     int i = 0;
9     int index;
10    do {
11        index = double_hash(ht, key, i);
12        if (ht->table[index] == NULL || ht->table[index] ==
13            DELETED) {
14            ht->table[index] = create_node(key, data);
15            ht->count++;
16            return;
17        }
18        i++;
19    } while (i < ht->size);
20    error("table overflow");
21 }
```

Listing 7: Doble hashing

Condición de Relativa Primalidad

Para que la secuencia de sonda sea una permutación completa, $h_2(k)$ debe ser relativamente primo con m . Formas de garantizarlo:

- Elegir m potencia de 2 y $h_2(k)$ impar.
- Elegir m primo y $1 \leq h_2(k) < m$.

Análisis del Doble Hashing

- **Ventajas:** Excelente dispersión, se aproxima al hash uniforme.
- **Desventajas:** Más lento por dos funciones hash.
- **Teóricamente óptimo:** Proporciona $\Theta(m^2)$ secuencias diferentes.

Comparativa de Métodos

Tabla Comparativa

Característica	Encadenamiento	Sondeo Lineal	Doble Hashing
Memoria adicional	Alta (punteros)	Baja	Baja
Localidad de referencia	Mala	Excelente	Buena
Eliminación	Fácil	Difícil (marcador)	Difícil (marcador)
α permitido	≥ 1	< 1	< 1
Peor caso búsqueda	$O(n)$	$O(n)$	$O(n)$
Esperado búsqueda	$1 + \alpha/2$	$\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$

Gráfica de Rendimiento



Eliminación en Direcccionamiento Abierto

El Problema

Si simplemente marcamos un slot como NIL, se pueden perder claves que estaban después en la secuencia de sonda.

Solución con Marcador DELETED

```
1 #define DELETED -1
2
3 void delete_open(HashTable* ht, int key) {
4     int i = 0;
5     int index;
6     do {
7         index = probe(ht, key, i);
8         if (ht->table[index] != NULL && ht->table[index]->
9             key == key) {
10             free(ht->table[index]);
11             ht->table[index] = DELETED; // marcador
12             ht->count--;
13             return;
14         }
15         i++;
16     } while (i < ht->size && ht->table[index] != NULL);
17 }
```

Listing 8: Eliminación con marcador

Inserción con DELETED

```
1 void insert_open(HashTable* ht, int key, void* data) {
2     int i = 0;
3     int index;
4     do {
5         index = probe(ht, key, i);
6         if (ht->table[index] == NULL || ht->table[index] ==
7             DELETED) {
8             ht->table[index] = create_node(key, data);
9             ht->count++;
10             return;
11         }
12         i++;
13     } while (i < ht->size);
14     error("table overflow");
15 }
```

Listing 9: Inserción con marcador

Consecuencias del Uso de DELETED

- **Ventaja:** Permite eliminación sin reorganización.
- **Desventaja:** Degrada el rendimiento con muchas eliminaciones.
- **Solución:** Rehashing periódico.

Rehashing

Cuándo Rehashear

- Cuando α supera un umbral (ej. 0.75).
- Cuando se acumulan muchos marcadores DELETED.
- Cuando el rendimiento se degrada.

Algoritmo de Rehashing

```
1 void rehash(HashTable* ht) {
2     int old_size = ht->size;
3     Node** old_table = ht->table;
4
5     // Duplicar tama o (o elegir nuevo primo)
6     ht->size = next_prime(2 * old_size);
7     ht->table = calloc(ht->size, sizeof(Node*));
8     ht->count = 0;
9
10    // Reinserter todos los elementos
11    for (int i = 0; i < old_size; i++) {
12        if (old_table[i] != NULL && old_table[i] != DELETED)
13        {
14            insert(ht, old_table[i]->key, old_table[i]->data);
15            free(old_table[i]);
16        }
17    }
18    free(old_table);
19 }
```

Listing 10: Rehashing

Análisis Amortizado

El costo de rehashing se amortiza entre las inserciones. Si duplicamos el tamaño cuando la tabla está llena, el costo amortizado por inserción es $O(1)$.

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.1. *Inserte las claves 10, 22, 31, 4, 15, 28, 17, 88, 59 en una tabla con $m = 11$ usando encadenamiento y $h(k) = k \bmod 11$.*

Solución:

- $h(10) = 10, h(22) = 0, h(31) = 9, h(4) = 4, h(15) = 4, h(28) = 6, h(17) = 6, h(88) = 0, h(59) = 4.$
- Slot 0: $22 \rightarrow 88$
- Slot 4: $4 \rightarrow 15 \rightarrow 59$
- Slot 6: $28 \rightarrow 17$
- Slot 9: 31
- Slot 10: 10

Ejercicio 0.0.2. *Inserte las mismas claves usando sondeo lineal.*

Solución:

1. 10: slot 10
2. 22: slot 0
3. 31: slot 9
4. 4: slot 4
5. 15: slot 4 ocupado $\rightarrow$ slot 5
6. 28: slot 6
7. 17: slot 6 ocupado $\rightarrow$ slot 7
8. 88: slot 0 ocupado $\rightarrow$ slot 1
9. 59: slot 4 ocupado $\rightarrow$ slot 5 ocupado $\rightarrow$ slot 6 ocupado $\rightarrow$ slot 7 ocupado $\rightarrow$ slot 8

Tabla final: [22, 88, NIL, NIL, 4, 15, 28, 17, 59, 31, 10]

Ejercicio 0.0.3. *Inserte las mismas claves usando doble hashing con $h_1(k) = k \bmod 11, h_2(k) = 1 + (k \bmod 10)$.*

Solución:

1. 10: $h_1 = 10, h_2 = 1 + (10 \bmod 10) = 1$, slot 10

2. 22: $h_1 = 0$, $h_2 = 1 + (22 \bmod 10) = 3$, slot 0
3. 31: $h_1 = 9$, $h_2 = 1 + (31 \bmod 10) = 2$, slot 9
4. 4: $h_1 = 4$, $h_2 = 1 + (4 \bmod 10) = 5$, slot 4
5. 15: $h_1 = 4$, $h_2 = 1 + 5 = 6$, slot 4 ocupado $\rightarrow$ slot 10 (4+6) ocupado $\rightarrow$ slot 5 (4+12)
6. 28: $h_1 = 6$, $h_2 = 1 + 8 = 9$, slot 6
7. 17: $h_1 = 6$, $h_2 = 1 + 7 = 8$, slot 6 ocupado $\rightarrow$ slot 3 (6+8)
8. 88: $h_1 = 0$, $h_2 = 1 + 8 = 9$, slot 0 ocupado $\rightarrow$ slot 9 ocupado $\rightarrow$ slot 7 (0+18)
9. 59: $h_1 = 4$, $h_2 = 1 + 9 = 10$, slot 4 ocupado $\rightarrow$ slot 3 (4+10) ocupado $\rightarrow$ slot 2 (4+20)

Ejercicio 0.0.4. Demuestre que en sondeo lineal, la longitud esperada de la secuencia de slots ocupados es $\Theta(1/(1-\alpha)^2)$.

Solución:

La probabilidad de que una secuencia de k slots consecutivos esté ocupada es α^k . La longitud esperada es $\sum_{k=1}^{\infty} k \alpha^k = \alpha/(1-\alpha)$. Pero esto es para un slot específico. La longitud esperada de una ejecución es $\Theta(1/(1-\alpha)^2)$ por propiedades de las ejecuciones en cadenas de Bernoulli.

Ejercicio 0.0.5. ¿Por qué en doble hashing se requiere que $h_2(k)$ sea relativamente primo con m ?

Solución:

Si $d = \gcd(h_2(k), m) > 1$, la secuencia $(h_1 + ih_2) \bmod m$ solo visita m/d slots distintos, no todos.

Ejercicio 0.0.6. Calcule el número esperado de sondas para búsqueda exitosa en direccionamiento abierto con $\alpha = 0.8$ usando la fórmula $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

Solución:

$$\frac{1}{0.8} \ln \frac{1}{0.2} = 1.25 \times \ln 5 \approx 1.25 \times 1.609 = 2.011 \text{ sondas.}$$

Ejercicio 0.0.7. ¿Qué ventaja tiene el encadenamiento sobre direccionamiento abierto?

Solución:

- Permite $\alpha > 1$.
- Eliminación más sencilla.

- No requiere marcadores DELETED.
- Mejor rendimiento en el peor caso con buena función hash.

Ejercicio 0.0.8. *¿Qué ventaja tiene direccionamiento abierto sobre encadenamiento?*

Solución:

- Mejor localidad de referencia (todo en un arreglo).
- Menor uso de memoria (sin punteros).
- Puede ser más rápido para tablas pequeñas.

Ejercicio 0.0.9. *¿Cuándo es preferible usar sondeo lineal?*

Solución:

Cuando la tabla es pequeña, la función hash es buena, y se espera baja carga. Por su simplicidad y localidad.

Ejercicio 0.0.10. *¿Cuándo es preferible usar doble hashing?*

Solución:

Cuando se necesita la mejor dispersión posible y se puede tolerar el costo extra de dos funciones hash.

Ejercicio 0.0.11. *Simule la inserción de 100 elementos en una tabla de tamaño 100 con sondeo lineal. ¿Cuál es el número esperado de colisiones?*

Solución:

Con $\alpha = 1$, el número esperado de sondas para búsqueda fallida es $1/(1-1) = \infty$, indicando que es probable que ocurran muchas colisiones.

Ejercicios Propuestos (Ampliados)

1. Inserte las claves 5, 28, 19, 15, 20, 33, 12, 17, 10 en una tabla con $m = 9$ usando encadenamiento.
2. Inserte las mismas claves usando sondeo lineal.
3. Inserte las mismas claves usando sondeo cuadrático con $c_1 = 1$, $c_2 = 3$.
4. Inserte las mismas claves usando doble hashing con $h_1(k) = k \bmod 9$, $h_2(k) = 1 + (k \bmod 8)$.
5. Implemente una tabla hash con encadenamiento en Python.

6. Implemente una tabla hash con sondeo lineal en C.
7. Analice el rendimiento de sondeo lineal con $\alpha = 0.5, 0.75, 0.9$.
8. Demuestre que el número esperado de sondas en búsqueda exitosa para direccionamiento abierto es $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.
9. ¿Qué es el "agrupamiento secundario" y qué métodos lo sufren?
10. ¿Cómo afecta la elección de m al rendimiento en sondeo cuadrático?
11. Diseñe una estrategia de rehashing que mantenga α entre 0.25 y 0.75.
12. Compare el uso de memoria de encadenamiento vs. direccionamiento abierto.
13. ¿Qué sucede si en direccionamiento abierto no usamos marcador DELETED y simplemente marcamos NIL?
14. Implemente búsqueda en una tabla con sondeo cuadrático.
15. ¿Cuál es la complejidad esperada de inserción en una tabla con encadenamiento?
16. ¿Por qué el doble hashing se considera el mejor método de direccionamiento abierto?
17. Simule la inserción de 50 elementos aleatorios en una tabla de tamaño 100 usando sondeo lineal. Calcule la longitud promedio de las ejecuciones.
18. ¿Cómo se puede adaptar el encadenamiento para usar árboles balanceados en lugar de listas?
19. ¿Qué es el "hash perfecto" y cómo se relaciona con la resolución de colisiones?
20. Investigue cómo manejan las colisiones los diccionarios en Python (CPython).

Unidad IX: Análisis de Rendimiento de Tablas Hash

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los fundamentos probabilísticos del análisis de tablas hash.
- Analizar rigurosamente el rendimiento esperado de encadenamiento y direccionamiento abierto.
- Aplicar técnicas de variables aleatorias indicadoras en el análisis.
- Derivar y utilizar las fórmulas de complejidad esperada para búsquedas exitosas y fallidas.
- Comprender la importancia del factor de carga y cómo afecta al rendimiento.

Introducción: El Marco Probabilístico

Supuestos Fundamentales

Para analizar el rendimiento de tablas hash, realizamos los siguientes supuestos:

1. **Hash uniforme simple:** Cada clave tiene la misma probabilidad de hashear a cualquier slot, independientemente de otras claves.
2. **Independencia:** Los valores hash de diferentes claves son independientes.
3. **Tiempo constante:** Calcular $h(k)$ toma $O(1)$.

Variables Aleatorias Fundamentales

Definimos:

- n : número de claves almacenadas.
- m : tamaño de la tabla.
- $\alpha = n/m$: factor de carga.
- n_j : número de claves que hashlean al slot j .

Análisis de Encadenamiento

Distribución del Número de Claves por Slot

Distribución Binomial

Para un slot fijo j , n_j sigue una distribución binomial:

$$\Pr(n_j = k) = \binom{n}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{n-k}$$

Momento de Primer Orden

$$E[n_j] = \frac{n}{m} = \alpha$$

Momento de Segundo Orden

$$E[n_j^2] = \frac{n}{m} + \frac{n(n-1)}{m^2} = \alpha + \alpha^2 - \frac{\alpha}{n}$$

Búsqueda Fallida

Definición

Una búsqueda fallida ocurre cuando la clave k no está en la tabla. Se debe examinar toda la lista $T[h(k)]$ hasta encontrar NIL.

Análisis

Teorema 0.0.3. *El número esperado de elementos examinados en una búsqueda fallida es $1 + \alpha$.*

Demostración. Sea X el número de elementos examinados. El primer elemento examinado es el slot $h(k)$. Luego se recorre la lista. El número esperado de elementos en la lista es $E[n_{h(k)}] = \alpha$. Por lo tanto:

$$E[X] = 1 + \alpha$$

El 1 corresponde al slot mismo, y α al recorrido de la lista. □

Búsqueda Exitosa

Definición

Una búsqueda exitosa ocurre cuando la clave k está en la tabla. Se debe examinar la lista hasta encontrar k .

Análisis con Variables Indicadoras

Teorema 0.0.4. *El número esperado de elementos examinados en una búsqueda exitosa es $1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$.*

Demostración. Sean las claves $k_1, k_2, \dots, k_n$ en orden de inserción. Para cada par (i, j) con $i < j$, definimos:

$$X_{ij} = I\{h(k_i) = h(k_j)\}$$

La probabilidad de colisión es $1/m$, por lo que $E[X_{ij}] = 1/m$.

El número de elementos que preceden a k_i en su lista es $\sum_{j=1}^{i-1} X_{ji}$. Por lo tanto, el número esperado de elementos examinados al buscar k_i es:

$$E[\text{elementos}] = 1 + \sum_{j=1}^{i-1} E[X_{ji}] = 1 + \frac{i-1}{m}$$

Promediando sobre todos los i :

$$E[T] = \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{i-1}{m}\right) = 1 + \frac{1}{nm} \sum_{i=1}^n (i-1) = 1 + \frac{n(n-1)}{2nm} = 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

□

Varianza del Tiempo de Búsqueda

Teorema 0.0.5. *La varianza del número de elementos examinados en una búsqueda exitosa es $O(\alpha)$.*

Esbozo. Usando que $\text{Var}(X) = E[X^2] - E[X]^2$ y que $E[X^2]$ se puede calcular similarmente a $E[X]$, se obtiene el resultado. □

Análisis de Direccionamiento Abierto

Supuestos Adicionales

- **Hash uniforme:** Cada clave tiene igual probabilidad de tener cualquier secuencia de sonda.
- $\alpha < 1$ (la tabla no se llena completamente).

Búsqueda Fallida

Definición

Una búsqueda fallida examina slots hasta encontrar uno vacío.

Análisis

Teorema 0.0.6. *El número esperado de sondas en una búsqueda fallida es $\frac{1}{1-\alpha}$.*

Demostración. Sea X el número de sondas. Entonces:

$$\Pr(X \geq i) = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdots \frac{n-i+2}{m-i+2}$$

Para i fijo, $\Pr(X \geq i) \leq \alpha^{i-1}$ porque $(n-j)/(m-j) \leq n/m$ para $j \geq 0$.

Por lo tanto:

$$E[X] = \sum_{i=1}^{\infty} \Pr(X \geq i) \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}$$

□

Búsqueda Exitosa

Análisis

Teorema 0.0.7. *El número esperado de sondas en una búsqueda exitosa es $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.*

Demostración. Consideremos la inserción de la $(i+1)$ -ésima clave. En ese momento, el factor de carga es i/m . El número esperado de sondas para esa inserción es $\frac{1}{1-i/m} = \frac{m}{m-i}$.

Promediando sobre todas las claves:

$$\begin{aligned} E[T] &= \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} \\ &= \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} = \frac{1}{\alpha} (H_m - H_{m-n}) \end{aligned}$$

donde H_k es el k -ésimo número armónico.

Usando la aproximación integral:

$$H_m - H_{m-n} \approx \int_{m-n}^m \frac{1}{x} dx = \ln \frac{m}{m-n} = \ln \frac{1}{1-\alpha}$$

Por lo tanto:

$$E[T] \approx \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

□

Comparación de Métodos de Sonda

Sondeo Lineal

Teorema 0.0.8. *En sondeo lineal, el número esperado de sondas es:*

$$\text{Fallida: } \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right)$$

$$\text{Exitosa: } \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right)$$

Sondeo Cuadrático y Doble Hashing

Ambos tienen rendimiento aproximado al hash uniforme:

$$\text{Fallida: } \frac{1}{1-\alpha}$$

$$\text{Exitosa: } \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

Análisis de Rehashing

Costo Amortizado

Teorema 0.0.9. *El costo amortizado de inserción en una tabla hash que duplica su tamaño cuando está llena es $O(1)$.*

Demostración. Supongamos que duplicamos el tamaño cuando $n = m$. El costo de rehashing es $O(n)$ (reinsertar n elementos). El número de inserciones entre rehashes es n . Por lo tanto, el costo amortizado por inserción es:

$$\frac{O(n)}{n} + O(1) = O(1)$$

□

Estrategias de Crecimiento

- **Doblado:** $m \leftarrow 2m$. Amortizado $O(1)$.
- **Crecimiento lineal:** $m \leftarrow m + c$. Amortizado $O(n)$.
- **Próximo primo:** $m \leftarrow \text{next_prime}(2m)$. Mantiene propiedades de primalidad.

Análisis de Casos Extremos

Peor Caso

Teorema 0.0.10. *Existe una secuencia de n operaciones que hace que el tiempo de búsqueda sea $\Theta(n)$ para cualquier función hash determinista.*

Demostración. El adversario puede elegir n claves que todas hashéen al mismo slot. La función hash es fija, por lo que puede hacerlo conociéndola. $\square$

Hash Universal

Teorema 0.0.11. *Con hash universal, el tiempo esperado de búsqueda es $O(1)$ para cualquier secuencia de operaciones.*

Demostración. La probabilidad de colisión para cualquier par de claves es $\leq 1/m$. Por linealidad, el número esperado de colisiones es $O(1)$. $\square$

Análisis de la Paradoja del Cumpleaños

Relación con Hashing

La probabilidad de que dos claves colisionen después de n inserciones es:

$$\Pr(\text{colisión}) \approx 1 - e^{-n^2/(2m)}$$

Punto de Colisión Esperado

El número esperado de inserciones hasta la primera colisión es $\sqrt{\pi m/2}$.

Aplicación

Si $n = \sqrt{m}$, la probabilidad de colisión es aproximadamente $1 - e^{-1/2} \approx 0.393$.

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.12. *Una tabla hash tiene $m = 1000$ y $n = 500$ claves. Calcule α y el número esperado de sondas para búsqueda fallida en encadenamiento.*

Solución:

$$\alpha = 500/1000 = 0.5. \text{ Número esperado de sondas} = 1 + \alpha = 1.5.$$

Ejercicio 0.0.13. *Misma tabla, calcule el número esperado de sondas para búsqueda exitosa en encadenamiento.*

Solución:

$$1 + \alpha/2 - \alpha/(2n) = 1 + 0.25 - 0.0005 = 1.2495.$$

Ejercicio 0.0.14. Una tabla con direccionamiento abierto tiene $m = 1000$ y $n = 800$. Calcule el número esperado de sondas para búsqueda fallida.

Solución:

$$\alpha = 0.8. \text{ Sondas} = 1/(1 - 0.8) = 5.$$

Ejercicio 0.0.15. Misma tabla, calcule el número esperado de sondas para búsqueda exitosa.

Solución:

$$\frac{1}{0.8} \ln \frac{1}{0.2} = 1.25 \times 1.609 = 2.011.$$

Ejercicio 0.0.16. Demuestre que en encadenamiento, $E[n_j^2] = \alpha + \alpha^2 - \alpha/n$.

Solución:

$$E[n_j^2] = \text{Var}(n_j) + E[n_j]^2. \text{Var}(n_j) = n \cdot \frac{1}{m} \cdot (1 - \frac{1}{m}) = \alpha(1 - 1/m). \text{Entonces:}$$

$$E[n_j^2] = \alpha(1 - 1/m) + \alpha^2 = \alpha + \alpha^2 - \alpha/m.$$

Ejercicio 0.0.17. ¿Cuál es la probabilidad de que un slot específico esté vacío después de n inserciones?

Solución:

$$(1 - 1/m)^n \approx e^{-n/m} = e^{-\alpha}.$$

Ejercicio 0.0.18. ¿Cuál es el número esperado de slots vacíos?

Solución:

$$m \cdot (1 - 1/m)^n \approx me^{-\alpha}.$$

Ejercicio 0.0.19. Demuestre que $\sum_{i=1}^{\infty} \alpha^{i-1} = 1/(1 - \alpha)$.

Solución:

$$\text{Es una serie geométrica con razón } \alpha < 1: \sum_{i=0}^{\infty} \alpha^i = 1/(1 - \alpha).$$

Ejercicio 0.0.20. Para $\alpha = 0.9$, calcule las sondas esperadas en búsqueda exitosa y fallida en direccionamiento abierto.

Solución:

$$\text{Fallida: } 1/(1 - 0.9) = 10. \text{ Exitosa: } (1/0.9) \ln(1/0.1) = 1.111 \times 2.3026 = 2.558.$$

Ejercicio 0.0.21. ¿Cuántas inserciones se necesitan en promedio para que ocurra la primera colisión en una tabla de tamaño $m = 1000$?

Solución:

$$\sqrt{\pi m/2} = \sqrt{3.1416 \times 500} = \sqrt{1570.8} \approx 39.6 \text{ inserciones.}$$

Ejercicio 0.0.22. Demuestre que $\frac{1}{\alpha} \ln \frac{1}{1-\alpha} \leq \frac{1}{1-\alpha}$ para $0 < \alpha < 1$.

Solución:

La desigualdad $\ln(1/(1-\alpha)) \leq \alpha/(1-\alpha)$ es equivalente a $\ln(1+x) \leq x$ con $x = \alpha/(1-\alpha)$. Esto es cierto porque $\ln(1+x) \leq x$ para $x > -1$.

Ejercicio 0.0.23. Una tabla hash con encadenamiento tiene $m = 100$ y $n = 200$. ¿Cuál es el número esperado de sondas para búsqueda exitosa?

Solución:

$$\alpha = 2. \text{ Fórmula: } 1 + \alpha/2 - \alpha/(2n) = 1 + 1 - 0.005 = 1.995.$$

Ejercicio 0.0.24. Demuestre que la varianza del número de claves por slot en encadenamiento es $\alpha(1 - 1/m)$.

Solución:

$$\text{Var}(n_j) = n \cdot \frac{1}{m} \cdot \left(1 - \frac{1}{m}\right) = \alpha(1 - 1/m).$$

Ejercicio 0.0.25. Para $\alpha = 0.5$, compare las sondas esperadas en búsqueda exitosa entre encadenamiento y direccionamiento abierto.

Solución:

Encadenamiento: 1.25. Direccionamiento abierto: $(1/0.5) \ln(1/0.5) = 2 \times 0.693 = 1.386$. El encadenamiento es ligeramente mejor.

Ejercicio 0.0.26. ¿Cuál es el número esperado de colisiones totales en una tabla hash con n claves y m slots?

Solución:

$$E[\text{colisiones}] = \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m}.$$

Ejercicio 0.0.27. Demuestre que $\Pr(\text{slot } j \text{ vacío}) \approx e^{-\alpha}$.

Solución:

$$\Pr = (1 - 1/m)^n. \text{ Usando } (1 - 1/m)^n \approx e^{-n/m} = e^{-\alpha}.$$

Ejercicios Propuestos (Ampliados)

1. Calcule α para $n = 1000$, $m = 500$. ¿Es aceptable para direccionamiento abierto?
2. Para $\alpha = 0.75$, calcule sondas esperadas en búsqueda exitosa y fallida en direccionamiento abierto.

3. Para $\alpha = 0.75$, calcule sondas esperadas en búsqueda exitosa y fallida en encadenamiento.
4. Demuestre que $\sum_{i=0}^{n-1} \frac{m}{m-i} = m(H_m - H_{m-n})$.
5. ¿Cuál es el número esperado de slots con exactamente una clave?
6. Derive la fórmula para el número esperado de sondas en búsqueda fallida para sondeo lineal.
7. ¿Cómo cambia el rendimiento si m es primo vs. potencia de 2?
8. Demuestre que la probabilidad de que no haya colisiones después de n inserciones es $\prod_{i=0}^{n-1} (1 - i/m)$.
9. Calcule la probabilidad de que no haya colisiones para $n = 10$, $m = 100$.
10. ¿Cuál es el valor de α que minimiza el número esperado de sondas en búsqueda exitosa?
11. Analice el rendimiento de una tabla hash cuando todas las claves tienen el mismo hash.
12. Demuestre que el costo amortizado de rehashing es $O(1)$ usando el método potencial.
13. ¿Cuál es la relación entre α y la probabilidad de que un slot tenga más de k claves?
14. Para $\alpha = 2$, ¿cuál es el número esperado de sondas en búsqueda fallida en encadenamiento?
15. Demuestre que $H_m - H_{m-n} \approx \ln(1/(1 - \alpha))$.
16. ¿Cómo afecta la varianza del número de claves por slot al rendimiento?
17. Analice el rendimiento de una tabla hash con α cercano a 1.
18. ¿Cuál es la complejidad esperada de inserción en una tabla con direccionamiento abierto?
19. Derive la fórmula para el número esperado de sondas en búsqueda exitosa para sondeo cuadrático.
20. Compare el rendimiento de encadenamiento vs. direccionamiento abierto para diferentes valores de α .

Unidad X: Grafos (Introducción) - Representaciones y Recorridos

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la definición formal de grafos y sus tipos.
- Implementar y elegir entre representaciones de grafos (matrices y listas de adyacencia).
- Implementar y analizar los algoritmos de búsqueda en profundidad (DFS) y amplitud (BFS).
- Aplicar recorridos para resolver problemas como orden topológico y detección de ciclos.
- Analizar la complejidad de los algoritmos de grafos.

Definiciones Fundamentales

¿Qué es un Grafo?

Un **grafo** $G = (V, E)$ consiste en:

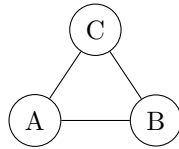
- V : conjunto finito de **vértices** (o nodos).
- E : conjunto de **aristas** (pares de vértices).

Tipos de Grafos

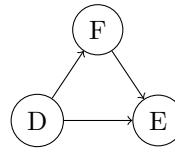
- **Grafo no dirigido:** Las aristas son pares no ordenados $\{u, v\}$.
- **Grafo dirigido (digrafo):** Las aristas son pares ordenados (u, v) .
- **Grafo ponderado:** Cada arista tiene un peso asociado.

Terminología Básica

- **Grado:** Número de aristas incidentes a un vértice.
- **Grado de entrada (indegree):** En grafos dirigidos, número de aristas que entran.
- **Grado de salida (outdegree):** En grafos dirigidos, número de aristas que salen.
- **Camino:** Secuencia de vértices conectados por aristas.
- **Ciclo:** Camino que comienza y termina en el mismo vértice.
- **Grafo conexo:** Existe un camino entre cualquier par de vértices.



(a) Grafo no dirigido



(b) Grafo dirigido

Representaciones de Grafos

Matriz de Adyacencia

Definición

Para un grafo con n vértices, la matriz de adyacencia A es una matriz $n \times n$ donde:

$$A[i][j] = \begin{cases} 1 & \text{si } (i, j) \in E \\ 0 & \text{en caso contrario} \end{cases}$$

Ejemplo

Para el grafo no dirigido con vértices $\{1, 2, 3\}$ y aristas $(1, 2)$, $(1, 3)$, $(2, 3)$:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

Propiedades

- Para grafos no dirigidos: A es simétrica ($A[i][j] = A[j][i]$).
- Espacio: $\Theta(V^2)$.
- Tiempo para verificar si $(u, v) \in E$: $O(1)$.
- Tiempo para listar vecinos de u : $\Theta(V)$.

Ventajas y Desventajas

- **Ventajas:** Acceso $O(1)$ a aristas, simple de implementar.
- **Desventajas:** Ineficiente en espacio para grafos dispersos ($|E| \ll |V|^2$).

Listas de Adyacencia

Definición

Para cada vértice u , mantenemos una lista de todos los vértices v tales que $(u, v) \in E$.

Ejemplo

Para el mismo grafo:

```
1: 2, 3
2: 1, 3
3: 1, 2
```

Implementación

```
1 struct Graph {
2     int V; // n mero de v rtices
3     struct Node** adj; // arreglo de listas
4     int* visited; // para recorridos
5 };
6
7 struct Node {
8     int vertex;
9     struct Node* next;
10 };
```

Listing 11: Estructura de listas de adyacencia

Propiedades

- Espacio: $\Theta(V + E)$.
- Tiempo para verificar si $(u, v) \in E$: $O(\text{grado}(u))$.
- Tiempo para listar vecinos de u : $\Theta(\text{grado}(u))$.

Ventajas y Desventajas

- **Ventajas:** Eficiente en espacio para grafos dispersos, iteración rápida sobre vecinos.
- **Desventajas:** Verificación de existencia de arista más lenta.

Comparativa

Característica	Matriz	Listas
Espacio	$\Theta(V^2)$	$\Theta(V + E)$
Verificar arista	$O(1)$	$O(\text{grado}(u))$
Listar vecinos	$\Theta(V)$	$\Theta(\text{grado}(u))$
Añadir arista	$O(1)$	$O(1)$
Eliminar arista	$O(1)$	$O(\text{grado}(u))$

Búsqueda en Amplitud (BFS)**Descripción**

BFS explora el grafo nivel por nivel, comenzando desde un vértice fuente. Primero visita todos los vértices a distancia 1, luego distancia 2, etc.

Algoritmo

```

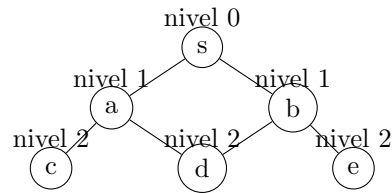
1 BFS(G, s)
2   for each u in G.V
3       u.color = WHITE
4       u.d = INFINITY
5       u.parent = NIL
6   s.color = GRAY
7   s.d = 0
8   s.parent = NIL
9   Q = empty queue
10  enqueue(Q, s)
11  while Q not empty
12      u = dequeue(Q)
13      for each v in G.adj[u]
14          if v.color == WHITE
15              v.color = GRAY

```

```
16         v.d = u.d + 1
17         v.parent = u
18         enqueue(Q, v)
19     u.color = BLACK
```

Listing 12: BFS

Ejemplo Visual



Análisis de Complejidad

- Cada vértice se pone en cola una vez: $O(V)$.
- Cada arista se examina una vez (dos en no dirigido): $O(E)$.
- Total: $O(V + E)$.

Propiedades de BFS

Teorema 0.0.12. *BFS calcula correctamente la distancia mínima (número de aristas) desde s a cada vértice alcanzable.*

Aplicaciones de BFS

- Caminos más cortos en grafos no ponderados.
- Componentes conexas.
- Verificación de bipartitud.
- Encontrar el diámetro de un árbol.

Búsqueda en Profundidad (DFS)

Descripción

DFS explora tan profundo como sea posible antes de retroceder. Es un algoritmo recursivo que puede generar un bosque de árboles.

Algoritmo

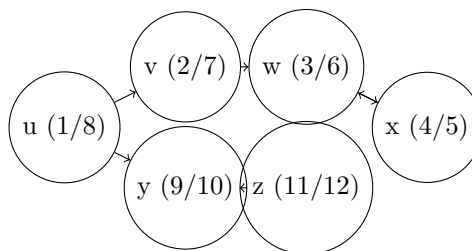
```

1 DFS(G)
2   for each u in G.V
3     u.color = WHITE
4     u.parent = NIL
5   time = 0
6   for each u in G.V
7     if u.color == WHITE
8       DFS-VISIT(u)
9
10  DFS-VISIT(u)
11    u.color = GRAY
12    time = time + 1
13    u.d = time           // tiempo de descubrimiento
14    for each v in G.adj[u]
15      if v.color == WHITE
16        v.parent = u
17        DFS-VISIT(v)
18    u.color = BLACK
19    time = time + 1
20    u.f = time           // tiempo de finalizaci n

```

Listing 13: DFS

Ejemplo Visual



Análisis de Complejidad

- Cada vértice se visita una vez: $O(V)$.
- Cada arista se examina una vez: $O(E)$.
- Total: $O(V + E)$.

Propiedades de DFS

Teorema del Paréntesis

Teorema 0.0.13. *Para dos vértices u y v , los intervalos $[d[u], f[u]]$ y $[d[v], f[v]]$ son disjuntos o uno está contenido en el otro.*

Clasificación de Aristas

- **Aristas de árbol:** (u, v) donde v fue descubierto desde u .
- **Aristas hacia atrás:** (u, v) donde v es ancestro de u .
- **Aristas hacia adelante:** (u, v) donde v es descendiente de u (no árbol).
- **Aristas cruzadas:** Las demás.

Aplicaciones de DFS

- Detección de ciclos.
- Orden topológico.
- Componentes fuertemente conexas.
- Puntos de articulación y puentes.

Orden Topológico

Definición

Un **orden topológico** de un grafo acíclico dirigido (DAG) es un ordenamiento lineal de sus vértices tal que para toda arista (u, v) , u aparece antes que v .

Aplicaciones

- Planificación de tareas con dependencias.
- Resolución de expresiones.
- Compilación de código.

Algoritmo basado en DFS

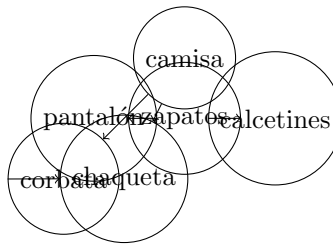
```

1 TOPOLOGICAL-SORT(G)
2   call DFS(G) to compute finishing times f[v]
3   as each vertex is finished, insert it at front of linked
   list
4   return linked list

```

Listing 14: Orden topológico con DFS

Ejemplo



Teorema de Correctitud

Teorema 0.0.14. *TOPOLOGICAL-SORT produce un orden topológico de un DAG.*

Demostración. Para cualquier arista (u, v) , en DFS $f[v] < f[u]$, por lo que u aparece antes que v en la lista ordenada por tiempos de finalización decrecientes. $\square$

Detección de Ciclos

Ciclos en Grafos No Dirigidos

```

1 HAS-CYCLE-UNDIRECTED(G)
2   for each u in G.V
3     u.color = WHITE
4   for each u in G.V
5     if u.color == WHITE
6       if DFS-CYCLE(u, NIL)
7         return TRUE
8   return FALSE
9
10 DFS-CYCLE(u, parent)
11   u.color = GRAY
12   for each v in G.adj[u]
13     if v.color == WHITE

```

```

14         if DFS-CYCLE(v, u)
15             return TRUE
16     else if v != parent and v.color == GRAY
17         return TRUE
18     u.color = BLACK
19     return FALSE

```

Listing 15: Detección de ciclos en grafos no dirigidos

Ciclos en Grafos Dirigidos

```

1 HAS-CYCLE-DIRECTED(G)
2     for each u in G.V
3         u.color = WHITE
4     for each u in G.V
5         if u.color == WHITE
6             if DFS-CYCLE-DIRECTED(u)
7                 return TRUE
8     return FALSE
9
10 DFS-CYCLE-DIRECTED(u)
11     u.color = GRAY
12     for each v in G.adj[u]
13         if v.color == WHITE
14             if DFS-CYCLE-DIRECTED(v)
15                 return TRUE
16         else if v.color == GRAY
17             return TRUE
18     u.color = BLACK
19     return FALSE

```

Listing 16: Detección de ciclos en grafos dirigidos

Teorema de Detección de Ciclos

Teorema 0.0.15. *Un grafo dirigido tiene un ciclo si y solo si DFS encuentra una arista hacia atrás.*

Demostración. Si hay una arista hacia atrás (u, v) , entonces v es ancestro de u , formando un ciclo. Recíprocamente, si hay un ciclo, el primer vértice descubierto en el ciclo tiene una arista hacia atrás desde el último vértice. $\square$

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.28. *Represente el grafo con vértices $\{1, 2, 3, 4\}$ y aristas $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 4)$ mediante matriz y listas de adyacencia.*

Solución:

Matriz:

$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

Listas:

- 1: 2, 3
- 2: 1, 4
- 3: 1, 4
- 4: 2, 3

Ejercicio 0.0.29. *Ejecute BFS desde el vértice 1 en el grafo anterior.*

Solución:

1. Inicial: $d[1] = 0$, cola: [1]
2. Procesar 1: visitar 2 y 3, $d[2] = 1$, $d[3] = 1$, cola: [2,3]
3. Procesar 2: visitar 4, $d[4] = 2$, cola: [3,4]
4. Procesar 3: 4 ya visitado, cola: [4]
5. Procesar 4: fin

Distancias: $d[1] = 0$, $d[2] = 1$, $d[3] = 1$, $d[4] = 2$.

Ejercicio 0.0.30. *Ejecute DFS en el mismo grafo.*

Solución:

1. Desde 1: visitar 2, desde 2 visitar 4, desde 4 no más, volver a 2, volver a 1.
2. Luego visitar 3 desde 1.

Tiempos: $d[1] = 1$, $f[1] = 6$; $d[2] = 2$, $f[2] = 5$; $d[4] = 3$, $f[4] = 4$; $d[3] = 7$, $f[3] = 8$.

Ejercicio 0.0.31. *Determine si el grafo dirigido con vértices $\{1, 2, 3, 4\}$ y aristas $(1, 2)$, $(2, 3)$, $(3, 1)$, $(2, 4)$ tiene ciclos.*

Solución:

Tiene ciclo $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. DFS detectará una arista hacia atrás cuando explore $(3, 1)$.

Ejercicio 0.0.32. *Ordene topológicamente el DAG: $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 4)$.*

Solución:

Un orden posible: 1, 2, 3, 4 o 1, 3, 2, 4.

Ejercicio 0.0.33. *¿Cuál es la complejidad de BFS si el grafo se representa con matriz de adyacencia?*

Solución:

$O(V^2)$, porque para cada vértice se recorre toda la fila de la matriz.

Ejercicio 0.0.34. *¿Cuál es la complejidad de DFS si el grafo se representa con listas de adyacencia?*

Solución:

$O(V + E)$.

Ejercicio 0.0.35. *Demuestre que en un grafo no dirigido, BFS desde un vértice visita todos los vértices de su componente conexa.*

Solución:

Por inducción: si v está en la componente, hay un camino de longitud k desde s . BFS visitará todos los vértices a distancia k después de los de distancia $k - 1$.

Ejercicio 0.0.36. *¿Qué es una arista hacia atrás en DFS?*

Solución:

Es una arista (u, v) donde v es ancestro de u en el árbol DFS. Indica la presencia de un ciclo.

Ejercicio 0.0.37. *¿Puede un grafo no dirigido tener aristas cruzadas en DFS?*

Solución:

No. En grafos no dirigidos, solo hay aristas de árbol y hacia atrás.

Ejercicio 0.0.38. *¿Cuál es la utilidad del orden topológico?*

Solución:

Permite ordenar tareas con dependencias, resolver sistemas de ecuaciones, optimizar compilación.

Ejercicio 0.0.39. *¿Qué sucede si aplicamos orden topológico a un grafo con ciclos?*

Solución:

No es posible obtener un orden topológico porque existiría una dependencia circular.

Ejercicio 0.0.40. En BFS, ¿cómo se pueden encontrar las distancias mínimas?

Solución:

Se almacena $d[v]$, que es la distancia desde la fuente. Se actualiza cuando se descubre un vértice blanco.

Ejercicio 0.0.41. En DFS, ¿para qué sirven los tiempos de descubrimiento y finalización?

Solución:

Permiten clasificar aristas, detectar ciclos, y son útiles para orden topológico y componentes fuertemente conexas.

Ejercicios Propuestos (Ampliados)

1. Represente el grafo con vértices $\{A, B, C, D, E\}$ y aristas (A, B) , (A, C) , (B, D) , (C, D) , (D, E) mediante matriz y listas.
2. Ejecute BFS desde A en el grafo anterior.
3. Ejecute DFS en el mismo grafo.
4. Determine si el grafo con aristas $(1, 2)$, $(2, 3)$, $(3, 4)$, $(4, 1)$ tiene ciclos.
5. Ordene topológicamente: $(1, 2)$, $(1, 3)$, $(2, 4)$, $(3, 5)$, $(4, 6)$, $(5, 6)$.
6. ¿Qué diferencia hay entre BFS y DFS en términos de orden de visita?
7. Demuestre que BFS calcula la distancia mínima.
8. ¿Cómo se detectan componentes conexas con BFS?
9. ¿Cómo se detectan componentes fuertemente conexas?
10. Implemente BFS en C.
11. Implemente DFS recursivo y no recursivo.
12. ¿Qué es un grafo bipartito y cómo se detecta con BFS?
13. ¿Cuál es la complejidad de verificar si un grafo es bipartito?
14. ¿Cómo se encuentra el diámetro de un árbol con BFS?
15. ¿Qué es un puente en un grafo?
16. ¿Cómo se pueden encontrar puentes con DFS?
17. ¿Qué es un punto de articulación?
18. ¿Cómo se relaciona DFS con el orden topológico?
19. ¿Qué es un grafo acíclico dirigido (DAG)?
20. Investigue aplicaciones de grafos en redes sociales.

Unidad XI: Estrategias de Diseño de Algoritmos (Parte I)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

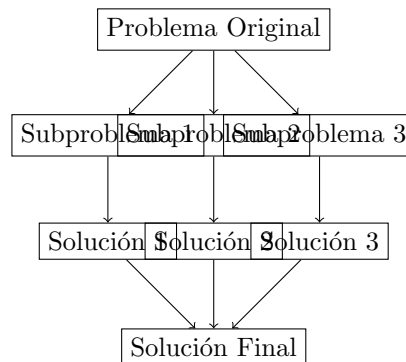
- Comprender el paradigma de divide y vencerás y su aplicación en problemas de ordenamiento.
- Analizar recurrencias utilizando el Teorema Maestro.
- Implementar y analizar Merge Sort y Quick Sort.
- Comprender el concepto de algoritmos greedy y la propiedad de elección local óptima.
- Implementar y analizar el algoritmo de Huffman para compresión de datos.
- Implementar y analizar los algoritmos de Kruskal y Prim para árboles de cubrimiento mínimo.

Parte I: Divide y Vencerás (Divide and Conquer)

Definición y Filosofía

El paradigma de **divide y vencerás** consiste en tres pasos fundamentales:

1. **Dividir:** Descomponer el problema en subproblemas más pequeños, independientes y de la misma naturaleza.
2. **Conquistar:** Resolver los subproblemas recursivamente (si son suficientemente pequeños, resolver directamente).
3. **Combinar:** Integrar las soluciones de los subproblemas para obtener la solución del problema original.



Análisis de Recurrencias

Forma General

La recurrencia típica de divide y vencerás es:

$$T(n) = aT(n/b) + f(n)$$

donde:

- a : número de subproblemas
- n/b : tamaño de cada subproblema
- $f(n)$: costo de dividir y combinar

Teorema Maestro

Teorema 0.0.16 (Teorema Maestro). Sean $a \geq 1$, $b > 1$ constantes, $f(n)$ una función, y $T(n)$ definida por:

$$T(n) = aT(n/b) + f(n)$$

Entonces:

1. Si $f(n) = O(n^{\log_b a - \epsilon})$ para algún $\epsilon > 0$, entonces $T(n) = \Theta(n^{\log_b a})$.
2. Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ para algún $k \geq 0$, entonces $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
3. Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ para algún $\epsilon > 0$, y si $af(n/b) \leq cf(n)$ para algún $c < 1$ y n suficientemente grande, entonces $T(n) = \Theta(f(n))$.

Merge Sort

Descripción

Merge Sort es un algoritmo de ordenamiento basado en divide y vencerás que garantiza $O(n \log n)$ en el peor caso.

Pseudocódigo

```

1 MERGE-SORT(A, p, r)
2   if p < r
3       q = floor((p + r) / 2)
4       MERGE-SORT(A, p, q)
5       MERGE-SORT(A, q+1, r)
6       MERGE(A, p, q, r)
7
8 MERGE(A, p, q, r)
9   n1 = q - p + 1
10  n2 = r - q
11  L[1..n1+1], R[1..n2+1]
12  for i = 1 to n1
13      L[i] = A[p + i - 1]
14  for j = 1 to n2
15      R[j] = A[q + j]
16  L[n1+1] = INFINITY
17  R[n2+1] = INFINITY
18  i = 1, j = 1
19  for k = p to r
20      if L[i] <= R[j]
21          A[k] = L[i]
22          i = i + 1
23      else
24          A[k] = R[j]
25          j = j + 1

```

Listing 17: Merge Sort

Ejemplo Visual

```

[38, 27, 43, 3, 9, 82, 10]
      /      \
[38, 27, 43, 3]  [9, 82, 10]
    /  \      /  \
[38,27] [43,3] [9,82] [10]
  /  \  /  \  /  \  |
[38] [27] [43] [3] [9] [82] [10]
  \  /  \  /  \  /  |
[27,38] [3,43] [9,82] [10]
    \      /      \  /
[3,27,38,43]      [9,10,82]
          \      /
[3,9,10,27,38,43,82]

```

Análisis de Complejidad

- Recurrencia: $T(n) = 2T(n/2) + \Theta(n)$
- Por Teorema Maestro: $\log_b a = \log_2 2 = 1$, $f(n) = \Theta(n) = \Theta(n^1)$
- Caso 2 con $k = 0$: $T(n) = \Theta(n \log n)$

Quick Sort

Descripción

Quick Sort es un algoritmo de ordenamiento que elige un pivote y particiona el arreglo alrededor de él. Su rendimiento promedio es $O(n \log n)$ pero puede ser $O(n^2)$ en el peor caso.

Pseudocódigo

```

1 QUICKSORT(A, p, r)
2   if p < r
3       q = PARTITION(A, p, r)
4       QUICKSORT(A, p, q-1)
5       QUICKSORT(A, q+1, r)
6
7 PARTITION(A, p, r)
8   x = A[r]                // pivote
9   i = p - 1
10  for j = p to r-1
11      if A[j] <= x
12          i = i + 1
13          swap(A[i], A[j])
14  swap(A[i+1], A[r])
15  return i + 1

```

Listing 18: Quick Sort

Ejemplo Visual

```

[5, 3, 8, 4, 2, 7, 1, 6]  (pivote = 6)
[5, 3, 4, 2, 1, 6, 8, 7]  (después de partición)
    /           \
[5, 3, 4, 2, 1]    [8, 7]
    |               |
[1, 3, 4, 2, 5]    [7, 8]
    /   \
[1, 3, 4, 2]    []
    |
[1, 2, 3, 4]

```

Variantes de Quick Sort

Quick Sort Aleatorizado

```

1 RANDOMIZED-PARTITION(A, p, r)
2   i = RANDOM(p, r)
3   swap(A[r], A[i])
4   return PARTITION(A, p, r)

```

Listing 19: Quick Sort Aleatorizado

La aleatorización elimina la posibilidad de que una entrada específica cause el peor caso.

Mediana de Tres Se elige el pivote como la mediana de tres elementos (primero, medio, último) para mejorar el equilibrio.

Análisis de Complejidad

Teorema 0.0.17. *El tiempo esperado de Quick Sort aleatorizado es $O(n \log n)$.*

Esbozo. Sean X_{ij} variables indicadoras de que z_i y z_j se comparan. La probabilidad es $2/(j - i + 1)$. Sumando:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} = O(n \log n)$$

□

Comparación: Merge Sort vs. Quick Sort

Característica	Merge Sort	Quick Sort
Peor caso	$O(n \log n)$	$O(n^2)$
Caso promedio	$O(n \log n)$	$O(n \log n)$
Mejor caso	$O(n \log n)$	$O(n \log n)$
In-place	No	Sí
Estable	Sí	No
Memoria extra	$O(n)$	$O(\log n)$

Parte II: Algoritmos Greedy (Voraces)

Definición y Filosofía

Un **algoritmo greedy** toma la decisión que parece mejor en el momento (elección localmente óptima), esperando que esto conduzca a una solución globalmente óptima.

Propiedades Fundamentales

1. **Propiedad de elección greedy:** Una solución óptima global puede ser alcanzada tomando elecciones localmente óptimas.
2. **Subestructura óptima:** Una solución óptima contiene dentro de sí soluciones óptimas a los subproblemas.

Código de Huffman para Compresión

Descripción

El algoritmo de Huffman construye un código de prefijo óptimo para compresión de datos, asignando códigos más cortos a caracteres más frecuentes.

Pseudocódigo

```

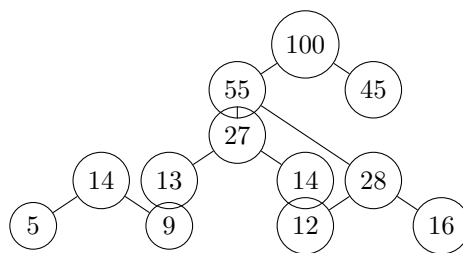
1 HUFFMAN(C)
2   n = |C|
3   Q = C // cola de prioridad mínima
4   for i = 1 to n-1
5       z = new node
6       z.left = x = EXTRACT-MIN(Q)
7       z.right = y = EXTRACT-MIN(Q)
8       z.freq = x.freq + y.freq
9       INSERT(Q, z)
10  return EXTRACT-MIN(Q) // ra z del rbol

```

Listing 20: Algoritmo de Huffman

Ejemplo

Frecuencias: a:45, b:13, c:12, d:16, e:9, f:5



Códigos resultantes:

- a: 0
- b: 101

- c: 100
- d: 111
- e: 1101
- f: 1100

Análisis de Complejidad

- Construir cola de prioridad: $O(n)$
- $n - 1$ extracciones e inserciones: $O(n \log n)$
- Total: $O(n \log n)$

Algoritmo de Kruskal para Árbol de Cubrimiento Mínimo

Descripción

Kruskal construye un MST agregando aristas en orden creciente de peso, siempre que no formen ciclos.

Pseudocódigo

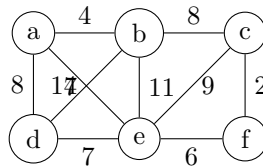
```

1 KRUSKAL(G, w)
2   A = empty set
3   for each vertex v in G.V
4     MAKE-SET(v)
5   sort edges of G.E by weight w
6   for each edge (u,v) in sorted order
7     if FIND-SET(u) != FIND-SET(v)
8       A = A ∪ {(u,v)}
9       UNION(u,v)
10  return A

```

Listing 21: Algoritmo de Kruskal

Ejemplo



Orden de adición: (c,f) peso 2, (a,b) peso 4, (d,e) peso 7, (a,e) peso 7, (b,c) peso 8, (e,f) peso 6? (e,f) no porque forma ciclo.

Análisis de Complejidad

- Ordenar aristas: $O(E \log E)$
- Operaciones de conjuntos disjuntos: $O(E\alpha(V))$
- Total: $O(E \log E) = O(E \log V)$

Algoritmo de Prim para Árbol de Cubrimiento Mínimo

Descripción

Prim construye un MST creciendo un árbol desde un vértice inicial, agregando siempre la arista de menor peso que conecta el árbol con un vértice externo.

Pseudocódigo

```

1 PRIM(G, w, r)
2   for each u in G.V
3       u.key = INFINITY
4       u.parent = NIL
5   r.key = 0
6   Q = G.V // cola de prioridad mínima
7   while Q not empty
8       u = EXTRACT-MIN(Q)
9       for each v in G.adj[u]
10          if v in Q and w(u,v) < v.key
11              v.parent = u
12              v.key = w(u,v)
13          DECREASE-KEY(Q, v, v.key)

```

Listing 22: Algoritmo de Prim

Ejemplo

Mismo grafo, comenzando desde a:

1. a: key=0, visitar a: actualizar b(4), d(8), e(7)
2. Extraer b(4): visitar b: actualizar c(8), e(7? menor que 7? no)
3. Extraer e(7): visitar e: actualizar d(7? igual), f(6)
4. Extraer f(6): visitar f: actualizar c(2)
5. Extraer c(2): visitar c: fin
6. Extraer d(7): fin

Análisis de Complejidad

- Con montículo binario: $O(E \log V)$
- Con montículo de Fibonacci: $O(E + V \log V)$

Comparación: Kruskal vs. Prim

Característica	Kruskal	Prim
Estructura	Basado en aristas	Basado en vértices
Complejidad	$O(E \log E)$	$O(E \log V)$
Adecuado para	Grafos dispersos	Grafos densos
Estructura de datos	Conjuntos disjuntos	Cola de prioridad

Ejercicios Resueltos (Ampliados)

Ejercicio 0.0.42. Resuelva la recurrencia $T(n) = 3T(n/4) + n^2$ usando el Teorema Maestro.

Solución:

$a = 3$, $b = 4$, $\log_b a = \log_4 3 \approx 0.792$, $f(n) = n^2 = \Omega(n^{0.792+1.2})$. Verificar regularidad: $3(n/4)^2 = 3n^2/16 \leq cn^2$ con $c = 3/16 < 1$. Caso 3: $T(n) = \Theta(n^2)$.

Ejercicio 0.0.43. Resuelva $T(n) = 4T(n/2) + n^2$.

Solución:

$a = 4$, $b = 2$, $\log_b a = 2$, $f(n) = n^2 = \Theta(n^2)$. Caso 2 con $k = 0$: $T(n) = \Theta(n^2 \log n)$.

Ejercicio 0.0.44. Aplique Merge Sort a $[3, 41, 52, 26, 38, 57, 9, 49]$.

Solución:

1. Dividir: $[3, 41, 52, 26]$ y $[38, 57, 9, 49]$
2. Dividir: $[3, 41]$, $[52, 26]$, $[38, 57]$, $[9, 49]$
3. Dividir hasta un elemento, luego combinar ordenadamente.

Resultado: $[3, 9, 26, 38, 41, 49, 52, 57]$.

Ejercicio 0.0.45. Aplique Quick Sort a $[5, 3, 8, 4, 2, 7, 1, 6]$ con pivote último.

Solución:

1. Pivote=6: partición $[5, 3, 4, 2, 1, 6, 8, 7]$
2. Izquierdo $[5, 3, 4, 2, 1]$: pivote=1 $\rightarrow [1, 3, 4, 2, 5]$
3. Derecho $[8, 7]$: pivote=7 $\rightarrow [7, 8]$

- Continuar recursivamente.

Ejercicio 0.0.46. Construya el código de Huffman para frecuencias: $a:10$, $b:15$, $c:12$, $d:3$, $e:4$, $f:13$.

Solución:

- Combinar $d(3)$ y $e(4) \rightarrow 7$
- Combinar 7 y $c(12) \rightarrow 19$? No, combinar $a(10)$ y $7 \rightarrow 17$
- Combinar $b(15)$ y $f(13) \rightarrow 28$
- Combinar 17 y $c(12) \rightarrow 29$
- Combinar 28 y $29 \rightarrow 57$

Ejercicio 0.0.47. Aplique Kruskal al grafo de 6 vértices con aristas: $(a,b,4)$, $(a,e,7)$, $(b,c,8)$, $(b,e,11)$, $(c,f,2)$, $(d,e,7)$, $(d,b,14)$, $(e,f,6)$.

Solución:

Orden: $(c,f,2)$, $(a,b,4)$, $(a,e,7)$, $(d,e,7)$, $(e,f,6)$, $(b,c,8)$, $(b,e,11)$, $(d,b,14)$. Se seleccionan: (c,f) , (a,b) , (a,e) , (d,e) , (e,f) ? (e,f) forma ciclo con (c,f) y (a,e) ? No, (e,f) con (c,f) y (a,e) forma ciclo? $(a,e)-(e,f)-(c,f)-(a,b)$ no conecta, $(a,e)-(e,f)-(c,f)$ no cierra. En realidad (e,f) forma ciclo con $(a,e)-(a,b)-(b,c)-(c,f)$? Sí, hay camino. Se selecciona $(b,c,8)$ en su lugar.

Ejercicio 0.0.48. Aplique Prim al mismo grafo comenzando desde a .

Solución:

- a : visitar $b(4)$, $e(7)$
- Extraer $b(4)$: visitar $c(8)$, $e(7)$? igual
- Extraer $e(7)$: visitar $d(7)$, $f(6)$
- Extraer $f(6)$: visitar $c(2)$
- Extraer $c(2)$: visitar? fin
- Extraer $d(7)$: fin

Aristas seleccionadas: (a,b) , (a,e) , (e,f) , (f,c) , (e,d) .

Ejercicio 0.0.49. Demuestre que Merge Sort es estable.

Solución:

En MERGE, cuando $L[i] = R[j]$, se toma de L primero, manteniendo el orden original.

Ejercicio 0.0.50. *¿Por qué Quick Sort puede tener peor caso $O(n^2)$?*

Solución:

Cuando el pivote es siempre el mínimo o máximo, una partición queda vacía y la otra con $n - 1$ elementos, resultando $T(n) = T(n - 1) + O(n) = O(n^2)$.

Ejercicio 0.0.51. *¿Cuál es la propiedad de elección greedy en Huffman?*

Solución:

Elegir los dos caracteres de menor frecuencia para fusionarlos primero lleva a un código óptimo.

Ejercicio 0.0.52. *¿Qué sucede si en Kruskal se eligen aristas en orden aleatorio?*

Solución:

El resultado no sería necesariamente el MST, ya que podría seleccionar aristas de mayor peso antes.

Ejercicio 0.0.53. *¿Cómo se implementa la cola de prioridad en Prim con montículo binario?*

Solución:

Se usa EXTRACT-MIN para obtener el vértice con menor clave, y DECREASE-KEY para actualizar claves de vecinos.

Ejercicios Propuestos (Ampliados)

1. Resuelva $T(n) = 9T(n/3) + n$.
2. Resuelva $T(n) = 2T(n/2) + n \log n$.
3. Resuelva $T(n) = 8T(n/2) + n^3$.
4. Implemente Merge Sort en C.
5. Implemente Quick Sort con pivote aleatorio.
6. Compare el rendimiento de Merge Sort y Quick Sort en arreglos ordenados y aleatorios.
7. Construya el código de Huffman para "algoritmos y estructuras de datos".
8. Aplique Kruskal al grafo con vértices A,B,C,D,E y aristas: (A,B,1), (A,C,4), (B,C,2), (B,D,5), (C,D,3), (C,E,6), (D,E,7).
9. Aplique Prim al mismo grafo.

56 UNIDAD XI: ESTRATEGIAS DE DISEÑO DE ALGORITMOS (PARTE I)

10. Demuestre que el algoritmo de Huffman produce un código de prefijo óptimo.
11. ¿Qué es un código de prefijo y por qué es importante?
12. ¿Cómo se detectan ciclos en Kruskal?
13. ¿Qué ventaja tiene Prim sobre Kruskal en grafos densos?
14. ¿Qué es la "mediana de tres" en Quick Sort?
15. Demuestre que el tiempo esperado de Quick Sort aleatorizado es $O(n \log n)$.
16. ¿Cuál es la complejidad espacial de Merge Sort? ¿Y de Quick Sort?
17. ¿Qué es la propiedad de subestructura óptima en algoritmos greedy?
18. ¿Cómo se puede mejorar Quick Sort para evitar el peor caso?
19. ¿Qué es un árbol de cubrimiento mínimo y por qué es importante?
20. Investigue aplicaciones de Huffman en compresión de archivos (ZIP, JPEG).

Unidad XII: Recursos Complementarios y Apoyo Académico

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante tendrá acceso a:

- Una bibliografía completa y actualizada sobre algoritmos y estructuras de datos.
- Canales de YouTube de calidad para profundizar en los temas.
- Profesores particulares especializados en la materia.
- Información de contacto para apoyo académico personalizado.

Bibliografía Fundamental

Bibliografía Principal

- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
 - **Comentario:** El "CLRS" es la biblia de los algoritmos. Cubre desde fundamentos hasta temas avanzados con rigor matemático. Es la referencia principal de este tratado.
 - **Capítulos clave:** 1-5 (Fundamentos), 6-9 (Ordenamiento), 10-14 (Estructuras de datos), 15-17 (Técnicas de diseño), 22-26 (Grafos).
- **Sedgewick, R., & Wayne, K. (2011).** *Algorithms* (4th ed.). Addison-Wesley.
 - **Comentario:** Excelente complemento al CLRS con un enfoque más práctico e implementaciones en Java. Incluye numerosos ejemplos y visualizaciones.

- **Capítulos clave:** 1 (Fundamentos), 2 (Ordenamiento), 3 (Búsqueda), 4 (Grafos), 5 (Cadenas).
- **Knuth, D. E. (1997-1998).** *The Art of Computer Programming* (Volúmenes 1-3, 3rd ed.). Addison-Wesley.
 - **Comentario:** La obra magna de Knuth. Profundidad sin igual en análisis de algoritmos. Recomendado para estudiantes avanzados que quieran comprender los fundamentos matemáticos.
 - **Volumen 1:** Algoritmos fundamentales.
 - **Volumen 2:** Matemáticas seminuméricas.
 - **Volumen 3:** Ordenamiento y búsqueda.

Bibliografía Complementaria

- **Kleinberg, J., & Tardos, É. (2006).** *Algorithm Design*. Addison-Wesley.
 - **Comentario:** Excelente para diseño de algoritmos y problemas de optimización. Enfoque en la construcción de soluciones.
- **Skiena, S. S. (2020).** *The Algorithm Design Manual* (3rd ed.). Springer.
 - **Comentario:** Muy práctico, incluye un catálogo de problemas y cómo resolverlos con las estructuras adecuadas.
- **Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008).** *Algorithms*. McGraw-Hill.
 - **Comentario:** Texto conciso y elegante. Buen equilibrio entre teoría y práctica.
- **Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983).** *Data Structures and Algorithms*. Addison-Wesley.
 - **Comentario:** Clásico que sentó las bases de la enseñanza de algoritmos. Aún vigente en muchos conceptos fundamentales.

Bibliografía para Profundización

- **Tarjan, R. E. (1983).** *Data Structures and Network Algorithms*. SIAM.
 - **Comentario:** Imprescindible para grafos y estructuras de datos avanzadas. Del creador de los árboles splay y el análisis amortizado.
- **Okasaki, C. (1998).** *Purely Functional Data Structures*. Cambridge University Press.
 - **Comentario:** Para quienes quieran explorar estructuras de datos en programación funcional.

- **Mehlhorn, K., & Sanders, P. (2008).** *Algorithms and Data Structures: The Basic Toolbox*. Springer.
 - **Comentario:** Enfoque moderno con implementaciones eficientes en C++.

Canales de YouTube Recomendados

Canales en Español

- **Mitos y Leyendas de la Programación (Andrés)**
 - **URL:** youtube.com/c/MitosyLeyendasdeProgramacion
 - **Contenido:** Excelentes explicaciones de algoritmos y estructuras de datos en español. Enfoque didáctico y claro.
 - **Recomendado para:** Algoritmos de ordenamiento, grafos, programación dinámica.
- **Programación ATS**
 - **URL:** youtube.com/c/ProgramacionATS
 - **Contenido:** Tutoriales prácticos de estructuras de datos en C y Java.
 - **Recomendado para:** Implementaciones prácticas de pilas, colas, listas, árboles.
- **Deusto Ciencia**
 - **URL:** youtube.com/c/DeustoCiencia
 - **Contenido:** Explicaciones rigurosas de algoritmos y complejidad computacional.
 - **Recomendado para:** Notación asintótica, recurrencias, teoría de la computación.
- **Algoritmos y Programación (UBA)**
 - **URL:** youtube.com/c/AlgoritmosyProgramacionUBA
 - **Contenido:** Material de las cátedras de la UBA, incluyendo Algo2 y Algo3.
 - **Recomendado para:** Seguir la cursada con material oficial.

Canales en Inglés

- **freeCodeCamp**
 - **URL:** youtube.com/c/freecodecamp

- **Contenido:** Cursos completos de algoritmos y estructuras de datos. Excelente producción.
- **Recomendado para:** Cursos intensivos de varias horas sobre temas específicos.

■ **Abdul Bari**

- **URL:** youtube.com/c/AbdulBari
- **Contenido:** Explicaciones claras y detalladas de algoritmos. Muy popular en la comunidad.
- **Recomendado para:** Algoritmos de grafos, programación dinámica, árboles.

■ **Michael Sambol**

- **URL:** youtube.com/c/MichaelSambol
- **Contenido:** Visualizaciones animadas de algoritmos. Breves y claras.
- **Recomendado para:** Entender visualmente cómo funcionan los algoritmos.

■ **CS Dojo**

- **URL:** youtube.com/c/CSDojo
- **Contenido:** Explicaciones amigables de conceptos de programación y algoritmos.
- **Recomendado para:** Introducción a estructuras de datos.

■ **Reducible**

- **URL:** youtube.com/c/Reducible
- **Contenido:** Animaciones de alta calidad explicando algoritmos complejos.
- **Recomendado para:** Algoritmos de grafos, teoría de la complejidad.

■ **Back To Back SWE**

- **URL:** youtube.com/c/BackToBackSWE
- **Contenido:** Enfoque en entrevistas técnicas y problemas de algoritmos.
- **Recomendado para:** Preparación para entrevistas y problemas específicos.

Canales de Divulgación Matemática

- **3Blue1Brown**
 - **URL:** youtube.com/c/3blue1brown
 - **Contenido:** Visualizaciones matemáticas impresionantes. Útil para entender análisis asintótico y complejidad.
- **Numberphile**
 - **URL:** youtube.com/c/numberphile
 - **Contenido:** Matemáticas recreativas. Útil para entender números de Fibonacci, primos y teoría de números.

Profesores Particulares Especializados

Profesores Destacados

- **Nahuel Villafañe (@clasesconnahu)**
 - **Especialidad:** Física y Matemáticas (UBA Exactas).
 - **Experiencia:** Estudiante avanzado de Física en la UBA.
 - **Modalidad:** Clases particulares presenciales (CABA) y virtuales por Zoom/Meet.
 - **Contacto:**
 - Instagram: @clasesconnahu
 - **Promoción:** Primera clase de diagnóstico gratuita. Material incluido.
- **Martín Diéguez (@martindieguez)**
 - **Especialidad:** Algoritmos y Programación (UBA).
 - **Experiencia:** Docente de Algoritmos y Programación en la UBA. Amplia experiencia en la enseñanza de fundamentos de programación y estructuras de datos.
 - **Contacto:** Instagram @martindieguez
- **Facundo Palavecino**
 - **Especialidad:** Matemática discreta, Algoritmos y Complejidad.
 - **Experiencia:** Ayudante en materias de matemática de Exactas. Especialista en demostraciones y fundamentos teóricos.
 - **Contacto:** Disponible por consulta en el Departamento de Computación.

Plataformas de Tutorías

■ Clases Particulares UBA

- **URL:** clasesparticularesuba.com.ar
- **Contenido:** Buscador de profesores particulares especializados en materias de Exactas.

■ Superprof Argentina

- **URL:** superprof.com.ar
- **Contenido:** Plataforma de tutores con reseñas y perfiles detallados.

Recursos Adicionales

Sitios Web Educativos

■ VisuAlgo

- **URL:** visualgo.net
- **Contenido:** Visualizaciones interactivas de algoritmos. Excelente para entender el funcionamiento paso a paso.

■ GeeksforGeeks

- **URL:** geeksforgeeks.org
- **Contenido:** Artículos, implementaciones y ejercicios de algoritmos. Muy completo.

■ LeetCode

- **URL:** leetcode.com
- **Contenido:** Plataforma de práctica con problemas de algoritmos. Ideal para preparación de entrevistas.

■ HackerRank

- **URL:** hackerrank.com
- **Contenido:** Desafíos de algoritmos con sistemas de evaluación automática.

■ Coursera

- **URL:** coursera.org
- **Contenido:** Cursos de algoritmos de universidades como Princeton, Stanford y MIT.

Grupos de Estudio y Comunidades

- **Discord: Algoritmos UBA**
 - **Contenido:** Comunidad de estudiantes de Exactas para consultas y estudio colaborativo.
 - **Link:** Disponible por invitación en los grupos de la materia.
- **Telegram: Algo2 - Algo3 UBA**
 - **Contenido:** Grupos de discusión de las materias, con material compartido y consultas.
- **Reddit: r/algorithms**
 - **URL:** reddit.com/r/algorithms
 - **Contenido:** Discusión global sobre algoritmos y estructuras de datos.
- **Stack Overflow**
 - **URL:** stackoverflow.com
 - **Contenido:** Consultas específicas sobre implementaciones y conceptos.

Consejos para el Estudio de Algoritmos

Estrategias Efectivas

1. **Entender antes de implementar:** Lee la descripción del algoritmo, analiza su invariante y comprende por qué funciona antes de escribir código.
2. **Visualizar:** Usa herramientas como VisuAlgo o dibuja manualmente la ejecución paso a paso.
3. **Implementar desde cero:** Evita copiar código; escríbelo vos mismo para internalizar los detalles.
4. **Analizar complejidad:** Para cada algoritmo que implementes, calcula su complejidad temporal y espacial.
5. **Resolver ejercicios variados:** No te quedes solo con los ejemplos del libro. Practica con problemas de LeetCode, HackerRank o parciales viejos.
6. **Estudiar en grupo:** Explicar un algoritmo a otros es la mejor forma de asegurarte de que lo entendiste.
7. **Repasar demostraciones:** En Algo2 y Algo3, las demostraciones de corrección son parte fundamental de la evaluación.

Preparación para Parciales y Finales

- **Parciales:** Enfocarse en ejercicios de implementación y análisis de complejidad. Practicar con parciales viejos de la cátedra.
- **Finales:** Preparar demostraciones de corrección de los algoritmos principales (MST, Dijkstra, Floyd-Warshall, etc.). Conocer los teoremas fundamentales (Teorema Maestro, propiedades de hashing, etc.).
- **Recursos:** Consultar las guías de ejercicios de la materia y los apuntes de cátedra disponibles en el Departamento de Computación.

Sobre el Autor del Tratado

[colback=blue!5!white, colframe=blue!75!black, title=**Nahuel Villafañe**]
Estudiante avanzado de Ciencias de la Computación

Universidad de Buenos Aires (UBA) - Facultad de Ciencias Exactas y Naturales

Experiencia:

- Estudiante de Algoritmos y Estructuras de Datos II y III
- Ayudante ad-honorem en materias del Departamento de Computación
- Desarrollador de material didáctico para estudiantes de Exactas
- Autor de este tratado: "*Tratado de Algoritmos y Datos II*"

Clases Particulares:

- Especialidad: Física y Matemáticas

Contacto:

- Instagram: @clasesconnahu

Epílogo: El Camino del Algoritmo

Querido estudiante,

Has completado este tratado, pero tu camino en el mundo de los algoritmos apenas comienza. Los conceptos que has aprendido —desde el análisis asintótico hasta los árboles de cubrimiento mínimo— son las herramientas fundamentales que te permitirán resolver problemas cada vez más complejos.

Recuerda que un buen programador no es quien sabe más algoritmos de memoria, sino quien sabe identificar cuál es el algoritmo adecuado para cada problema y adaptarlo a las necesidades específicas. La práctica constante, la

curiosidad intelectual y la humildad para aprender de los errores son tus mejores aliados.

Este tratado fue escrito con la esperanza de acompañarte en ese camino, ofreciéndote no solo conocimiento, sino también la confianza para enfrentar los desafíos que vendrán en Algoritmos II, Algoritmos III y más allá.

Que el análisis de complejidad te acompañe, que tus invariantes de bucle sean siempre correctos, y que tus árboles nunca se desbalancen.

Con cariño,

Nahuel Villafañe

Buenos Aires, 2026



Figura 1: Enter Caption

Licencia

CC BY-NC-SA 4.0

Atribución-NoComercial-CompartirIgual 4.0 Internacional



Este documento, “*Tratado de Algoritmos y Datos III*”, fue creado por **Nahuel Villafañe** y se distribuye bajo una licencia **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional**.

Puede **compartir** (copiar y redistribuir el material en cualquier medio o formato) y **adaptar** (remezclar, transformar y construir a partir del material), bajo las siguientes condiciones:

- **Atribución (BY)** — Debe dar crédito de manera adecuada, brindar un enlace a la licencia e indicar si se han realizado cambios.
- **No Comercial (NC)** — No puede hacer uso del material con fines comerciales.
- **Compartir Igual (SA)** — Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia que el original.

Cita sugerida:

Villafañe, N. (2026). *Tratado de Algoritmos y Datos III*. Buenos Aires, Argentina.

Contacto del autor: @clasesconpahu (Instagram)

Licencia completa disponible en: <https://creativecommons.org/licenses/by-nc-sa/4.0/>

TRATADO DE ALGORITMOS Y DATOS III

Nahuel Villafañe

Año 2026

Resumen de la Unidad	118
Bibliografía de la Unidad	118
Unidad X: Algoritmos sobre Números	119
Objetivos de la Unidad	119
Introducción a la Aritmética Modular	119
Definiciones Fundamentales	119
Propiedades de la Aritmética Modular	119
Inverso Modular	120
Algoritmo de Euclides para el Máximo Común Divisor	120
Descripción	120
Pseudocódigo	120
Ejemplo	120
Análisis de Complejidad	120
Demostración de Correctitud	120
Algoritmo de Euclides Extendido	121
Descripción	121
Pseudocódigo	121
Ejemplo	121
Aplicaciones del Euclides Extendido	121
Cálculo del Inverso Modular	122
Exponenciación Modular	122
Descripción	122
Algoritmo de Exponenciación Rápida	122
Ejemplo	122
Análisis de Complejidad	122
Introducción a la Criptografía RSA	123
Fundamentos Matemáticos	123
Generación de Claves	123
Cifrado y Descifrado	123
Ejemplo Simple	123
Implementación Básica de RSA	124
Seguridad de RSA	124
Algoritmos Adicionales sobre Números	124
Criba de Eratóstenes	124
Test de Primalidad de Miller-Rabin	124
Ejercicios Resueltos (20 Ejercicios)	125
Ejercicios Propuestos (20 Ejercicios)	126
Resumen de la Unidad	127
Bibliografía de la Unidad	127

Unidad XI: Aritmética Modular	129
Objetivos de la Unidad	129
Introducción a la Aritmética Modular	129
Definición de Congruencia	129
Propiedades Básicas	130
Operaciones en Aritmética Modular	130
Conjunto de Residuos	130
Operaciones Aritméticas Modulares	130
Suma Modular	130
Resta Modular	130
Multiplicación Modular	130
Exponenciación Modular	131
Inverso Modular	131
Definición	131
Existencia del Inverso Modular	131
Cálculo del Inverso Modular con Euclides Extendido	131
Ejemplos de Inversos Modulares	131
Teorema Chino del Resto (CRT)	132
Enunciado del Teorema	132
Algoritmo de Solución	132
Ejemplo	132
Teorema de Fermat y Euler	132
Teorema de Fermat	132
Función de Euler	133
Propiedades de	133
Teorema de Euler	133
Aplicaciones de la Aritmética Modular	133
Verificación de Números Primos	133
Criptografía RSA	133
Códigos de Verificación	133
Hashing	133
Ejercicios Resueltos (20 Ejercicios)	134
Ejercicios Propuestos (20 Ejercicios)	134
Resumen de la Unidad	135
Bibliografía de la Unidad	135
Unidad XII: Algoritmo Extendido de Euclides	137
Objetivos de la Unidad	137
Algoritmo de Euclides Estándar (Repaso)	137
Definición del MCD	137
Propiedad Fundamental	137

Algoritmo de Euclides	138
Ejemplo Visual	138
Algoritmo de Euclides Extendido	138
Motivación	138
Fundamento Matemático	138
Derivación del Algoritmo	138
Implementación Recursiva	139
Ejemplo Detallado	139
Implementación Iterativa	139
Aplicaciones del Algoritmo Extendido	140
Cálculo del Inverso Modular	140
Ecuaciones Diofánticas Lineales	140
Aplicación en RSA	140
Fracciones Continuas	141
Demostración de Correctitud	141
Análisis de Complejidad	141
Comparación de Implementaciones	141
Ejercicios Resueltos (20 Ejercicios)	142
Ejercicios Propuestos (20 Ejercicios)	143
Resumen de la Unidad	144
Bibliografía de la Unidad	144

Unidad I: Complejidad Computacional y NP-Complejidad

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la diferencia fundamental entre problemas tratables e intratables.
- Definir formalmente las clases de complejidad P, NP, NP-completo y NP-hard.
- Realizar reducciones polinomiales entre problemas.
- Demostrar que un problema es NP-completo.
- Identificar problemas NP-completos clásicos en aplicaciones reales.
- Aplicar estrategias para enfrentar problemas NP-completos en la práctica.

Introducción: ¿Qué hace que un problema sea "difícil"?

La Pregunta Fundamental

A lo largo de Algoritmos y Datos II, hemos estudiado algoritmos eficientes que resuelven problemas en tiempo polinomial: $O(n)$, $O(n \log n)$, $O(n^2)$, etc. Pero existen problemas para los cuales no se conoce ningún algoritmo de tiempo polinomial. La pregunta central de la teoría de la complejidad computacional es:

¿P = NP?

Esta es una de las preguntas abiertas más importantes de la computación, con un premio de \$1,000,000 del Instituto Clay de Matemáticas.

Problemas Fáciles vs. Difíciles

Problemas "Fáciles"	Problemas "Difíciles"
Ordenar una lista	Problema del viajante (TSP)
Buscar en un arreglo	Problema de la mochila (0/1)
Caminos más cortos	Coloreo de grafos
Árbol de cubrimiento mínimo	Clique máximo
Multiplicación de matrices	Programación lineal entera

Crecimiento de Funciones

Para entender la diferencia, recordemos el crecimiento de funciones:

n	$\log n$	n	n^2	2^n	$n!$
10	3.3	10	100	1,024	3,628,800
20	4.3	20	400	1,048,576	2.4×10^{18}
30	4.9	30	900	10^9	2.6×10^{32}
50	5.6	50	2,500	10^{15}	3.0×10^{64}
100	6.6	100	10,000	10^{30}	9.3×10^{157}

Mientras que n^2 es manejable para $n = 100$, 2^n es imposible incluso para $n = 100$ (requeriría más tiempo que la edad del universo).

Clase P: Problemas Tratables

Definición Formal

La clase **P** (Polynomial time) es el conjunto de todos los problemas de decisión que pueden ser resueltos por un algoritmo determinista en tiempo polinomial en el tamaño de la entrada.

Características de P

- Existe un algoritmo que resuelve el problema en $O(n^k)$ para alguna constante k .
- El algoritmo es determinista (no usa aleatoriedad).
- El tiempo de ejecución es "razonable" para entradas de tamaño práctico.

Ejemplos de Problemas en P

- ORDENAMIENTO: Dado un arreglo de n números, ¿está ordenado?
- CONEXIDAD: Dado un grafo, ¿es conexo?
- CAMINO MÁS CORTO: Dado un grafo, dos vértices s y t , y un entero k , ¿existe un camino de s a t con longitud $\leq k$?
- ÁRBOL DE CUBRIMIENTO: Dado un grafo y un entero k , ¿existe un árbol de cubrimiento de peso total $\leq k$?

Clase NP: Problemas Verificables Rápidamente

Definición Formal

La clase **NP** (Nondeterministic Polynomial time) es el conjunto de problemas de decisión para los cuales existe un **certificado** que puede ser verificado en tiempo polinomial por un algoritmo determinista.

Interpretación

Un problema está en NP si:

1. Dada una solución candidata (certificado), podemos verificar su corrección rápidamente.
2. No necesariamente podemos **encontrar** la solución rápidamente.

Ejemplo: SAT (Satisfacibilidad Booleana)

- **Problema:** Dada una fórmula booleana (ej. $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$), ¿existe una asignación de valores verdadero/falso que haga la fórmula verdadera?
- **Certificado:** Una asignación de valores a las variables.
- **Verificación:** Sustituir los valores y evaluar la fórmula. Esto toma tiempo lineal en el tamaño de la fórmula.
- **Complejidad de encontrar la solución:** Se desconoce si es polinomial.

Ejemplos de Problemas en NP

- SAT: Dada una fórmula booleana, ¿es satisfacible?
- CLIQUE: Dado un grafo y un entero k , ¿existe un subconjunto de k vértices completamente conectado?
- VERTEX COVER: Dado un grafo y un entero k , ¿existe un conjunto de k vértices que cubra todas las aristas?
- TSP: Dado un conjunto de ciudades y distancias, ¿existe un recorrido de longitud $\leq k$ que visite todas las ciudades?
- MOCHILA (0/1): Dados objetos con peso y valor, y una capacidad, ¿existe un subconjunto con valor $\geq k$?

Relación P vs. NP

$P \subseteq NP$ Si un problema se puede resolver en tiempo polinomial, podemos verificar cualquier solución simplemente resolviendo el problema. El certificado puede ser vacío.

La pregunta abierta es si $P = NP$ o $P \subset NP$. La mayoría de los expertos cree que $P \neq NP$.

[scale=0.8] (0,0) circle (2cm); (0,0) circle (1.2cm); at (0,1.6) NP; at (0,0.6) P;

NP-Compleitud: Los Problemas Más Difíciles de NP

Definición de Reducción Polinomial

Un problema A se reduce polinomialmente a un problema B (escrito $A \leq_p B$) si existe una función f computable en tiempo polinomial que transforma instancias de A en instancias de B tal que:

$$x \text{ es un "sí" de } A \iff f(x) \text{ es un "sí" de } B$$

(A) at (0,0) Instancia de A; (B) at (4,0) Instancia de B; [-] (A) – node[above] f (polinomial) (B); [-] (A) – node[below] sí/no (0,-1); [-] (B) – node[below] sí/no (4,-1);

Propiedades de las Reducciones

- **Transitividad:** Si $A \leq_p B$ y $B \leq_p C$, entonces $A \leq_p C$.
- Si $A \leq_p B$ y $B \in P$, entonces $A \in P$.
- Si $A \leq_p B$ y A es NP-completo, entonces B es NP-hard.

Definición de NP-Completo

Un problema L es **NP-completo** si:

1. $L \in NP$ (está en NP)
2. Para todo $L' \in NP$, $L' \leq_p L$ (es NP-difícil)

[scale=0.8] (0,0) circle (2.5cm); (0,0) circle (1.5cm); at (0,2) NP; at (0,1) P;
at (0.8,1.8) NP-completo;

Propiedades de NP-Completo

- Si cualquier problema NP-completo está en P, entonces $P = NP$.
- Si cualquier problema NP-completo no está en P, entonces todos los NP-completos no están en P.
- Los problemas NP-completos son "los más difíciles" de NP.

NP-Hard

Un problema L es **NP-hard** si para todo $L' \in NP$, $L' \leq_p L$. No se requiere que $L \in NP$.

Los problemas NP-hard pueden ser incluso más difíciles que los NP-completos (pueden no ser de decisión o no estar en NP).

El Primer Problema NP-Completo: SAT

Teorema de Cook-Levin

[Cook-Levin, 1971] El problema SAT (Satisfacibilidad Booleana) es NP-completo.

Demostración (Esbozo)

1. **SAT está en NP:** Dada una asignación, verificarla es lineal.
2. **SAT es NP-difícil:** Se puede simular cualquier máquina de Turing no determinista con una fórmula booleana.

Consecuencias del Teorema de Cook-Levin

- Proporciona un "primer" problema NP-completo.
- Para demostrar que otro problema X es NP-completo, basta con mostrar:
 1. $X \in NP$
 2. $SAT \leq_p X$ (o cualquier otro NP-completo conocido)

Problemas NP-Completos Clásicos

SAT y sus Variantes

- **3-SAT:** SAT con cláusulas de exactamente 3 literales. También NP-completo.
- **Circuit-SAT:** Dado un circuito booleano, ¿existe una entrada que haga la salida verdadera?

Problemas de Grafos

CLIQUE

- **Entrada:** Grafo $G = (V, E)$, entero k
- **Pregunta:** ¿Existe un subconjunto $C \subseteq V$ de tamaño k tal que todos los vértices en C están conectados entre sí?

[scale=0.6] [circle, draw] (1) at (0,1) 1; [circle, draw] (2) at (2,1) 2; [circle, draw] (3) at (1,2) 3; [circle, draw] (4) at (0.5,0) 4; [circle, draw] (5) at (1.5,0) 5;
 (1) – (2); (1) – (3); (2) – (3); (1) – (4); (2) – (5);

Clique de tamaño 3: $\{1, 2, 3\}$.

VERTEX COVER

- **Entrada:** Grafo $G = (V, E)$, entero k
- **Pregunta:** ¿Existe un subconjunto $C \subseteq V$ de tamaño k tal que cada arista tiene al menos un extremo en C ?

[scale=0.6] [circle, draw, fill=gray] (a) at (0,0) a; [circle, draw] (b) at (2,0) b; [circle, draw] (c) at (1,1.5) c; [circle, draw] (d) at (3,1.5) d;
(a) – (b); (a) – (c); (b) – (c); (b) – (d); (c) – (d);

Vertex cover de tamaño 2: $\{a, b\}$ cubre todas las aristas.

HAMILTONIAN CYCLE

- **Entrada:** Grafo $G = (V, E)$
- **Pregunta:** ¿Existe un ciclo que visite cada vértice exactamente una vez?

GRAPH COLORING

- **Entrada:** Grafo $G = (V, E)$, entero k
- **Pregunta:** ¿Se pueden colorear los vértices con k colores de modo que no haya dos vértices adyacentes con el mismo color?

Problemas de Números**SUBSET SUM**

- **Entrada:** Conjunto de números enteros $\{a_1, \dots, a_n\}$, entero t
- **Pregunta:** ¿Existe un subconjunto cuya suma sea exactamente t ?

PARTITION

- **Entrada:** Conjunto de números enteros
- **Pregunta:** ¿Se pueden dividir en dos subconjuntos con la misma suma?

KNAPSACK (0/1)

- **Entrada:** Objetos con peso w_i y valor v_i , capacidad W , entero k
- **Pregunta:** ¿Existe un subconjunto de peso total $\leq W$ con valor total $\geq k$?

Demostración de NP-Compleitud: Metodología

Pasos para Demostrar NP-Compleitud

1. **Demostrar que $L \in NP$:**
 - Mostrar un certificado y un verificador polinomial.
2. **Seleccionar un problema NP-completo conocido L' :**
 - SAT, 3-SAT, CLIQUE, VERTEX COVER, etc.
3. **Construir una reducción $L' \leq_p L$:**
 - Definir una función f que transforme instancias de L' en instancias de L .
 - Demostrar que f es computable en tiempo polinomial.
 - Demostrar que x es un "sí" de $L' \iff f(x)$ es un "sí" de L .

Ejemplo: Reducción de 3-SAT a CLIQUE

3-SAT $\leq_p$ CLIQUE

[Esbozo de la reducción] Dada una fórmula 3-SAT con m cláusulas:

1. Crear un vértice por cada literal en cada cláusula.
2. Conectar vértices que están en diferentes cláusulas y no son contradictorios (x y $\neg x$ no se conectan).
3. Preguntar si existe un clique de tamaño m .

Qué Hacer Cuando Enfrentamos un Problema NP-Completo

Estrategias Prácticas

1. **Buscar restricciones del problema:** Muchos problemas NP-completos se vuelven tratables bajo ciertas condiciones (grafos planares, árboles, etc.).
2. **Usar algoritmos de aproximación:** Encontrar soluciones cercanas a la óptima.
3. **Usar algoritmos heurísticos:** No garantizan optimalidad pero funcionan bien en la práctica.
4. **Usar algoritmos exponenciales pero optimizados:** Backtracking, branch-and-bound, programación dinámica con máscaras de bits.
5. **Usar algoritmos de parámetros fijos (FPT):** Si el parámetro es pequeño, el algoritmo puede ser práctico.
6. **Usar programación lineal entera:** Resolver con solvers comerciales.

Algoritmos de Aproximación

Un algoritmo de aproximación garantiza una solución cuyo valor está dentro de un factor constante del óptimo.

Para VERTEX COVER, el algoritmo greedy que toma todas las aristas y agrega ambos extremos da una aproximación 2.

Ejercicios Resueltos (20 Ejercicios)

¿Cuál es la diferencia fundamental entre las clases P y NP? **P** (Polynomial time) son problemas que pueden ser **resueltos** en tiempo polinomial por una máquina determinista.

NP (Nondeterministic Polynomial time) son problemas cuyas soluciones pueden ser **verificadas** en tiempo polinomial.

La diferencia clave es que para problemas en P podemos encontrar la solución rápidamente, mientras que para problemas en NP solo podemos verificar rápidamente una solución propuesta. Se cree (pero no se ha probado) que $P \neq NP$, lo que significa que hay problemas en NP que no pueden resolverse en tiempo polinomial.

22 UNIDAD I: COMPLEJIDAD COMPUTACIONAL Y NP-COMPLETITUD

Dado el problema SUBSET SUM: ¿Existe un subconjunto de $\{1, 4, 9, 14, 23\}$ que sume 28? ¿Qué tipo de problema es? Probamos combinaciones:

- $23 + 4 + 1 = 28$
- $23 + 4 + 1 = 28$ funciona.

Existe solución: $\{23, 4, 1\}$.

SUBSET SUM es un problema NP-completo. No se conoce algoritmo polinomial para el caso general, pero para este caso particular encontramos una solución por búsqueda exhaustiva.

Demuestre que si $P = NP$, entonces todos los problemas en NP se pueden resolver en tiempo polinomial. Si $P = NP$, entonces por definición todos los problemas en NP están en P. Esto significa que para cualquier problema en NP existe un algoritmo determinista que lo resuelve en tiempo polinomial.

En particular, los problemas NP-completos (que son un subconjunto de NP) también estarían en P, por lo que podríamos resolver SAT, CLIQUE, TSP, etc., en tiempo polinomial.

Esta es la razón por la que la pregunta $P = NP$ es tan importante: si la respuesta es afirmativa, muchos problemas considerados "difíciles" tendrían soluciones eficientes.

¿Cuál es la complejidad temporal de verificar una solución propuesta para el problema CLIQUE? Dado un grafo $G = (V, E)$ y un subconjunto $C \subseteq V$ propuesto como clique:

1. Verificar que $|C| = k$: $O(1)$
2. Verificar que para todo par $u, v \in C$, $(u, v) \in E$: hay k^2 pares, cada verificación $O(1)$.

La verificación toma $O(k^2)$ tiempo, que es polinomial en el tamaño de la entrada. Por lo tanto, CLIQUE está en NP.

Muestre una reducción de VERTEX COVER a CLIQUE. Dado un grafo $G = (V, E)$ y un entero k , construimos el grafo complemento $\overline{G} = (V, \overline{E})$ donde $\overline{E}$ contiene todas las aristas que no están en E .

La propiedad clave es: C es un vertex cover de G de tamaño k si y solo si $V \setminus C$ es un clique de $\overline{G}$ de tamaño $|V| - k$.

Reducción: Para una instancia (G, k) de VERTEX COVER, creamos la instancia $(\overline{G}, |V| - k)$ de CLIQUE.

Corrección:

- Si C es vertex cover de G de tamaño k , entonces cualquier arista de $\overline{G}$ tiene al menos un extremo fuera de C , por lo que $V \setminus C$ es clique en $\overline{G}$.

- Si S es clique en $\overline{G}$ de tamaño $|V| - k$, entonces $V \setminus S$ es vertex cover de G de tamaño k .

¿Por qué SAT es importante en la teoría de NP-completitud? SAT fue el primer problema demostrado NP-completo (Teorema de Cook-Levin, 1971). Esto es fundamental porque:

1. Proporciona un "primer" problema NP-completo a partir del cual reducir otros problemas.
2. Demuestra que la clase NP tiene problemas "completos".
3. Abre la puerta para demostrar NP-completitud de cientos de otros problemas.

Para demostrar que un problema X es NP-completo, basta con mostrar que $X \in NP$ y que $SAT \leq_p X$ (o reducir desde cualquier otro NP-completo conocido).

¿Cuál es la diferencia entre NP-completo y NP-hard?

- **NP-completo:** Un problema que está en NP y es NP-difícil (todos los problemas de NP se reducen a él).
- **NP-hard:** Un problema tal que todos los problemas de NP se reducen a él, pero no necesariamente está en NP.

Los problemas NP-hard pueden ser más "difíciles" que los NP-completos. Por ejemplo, el problema de la parada (Halting Problem) es NP-hard pero no está en NP (es indecidible).

Todo problema NP-completo es NP-hard, pero no todo problema NP-hard es NP-completo.

Dada la fórmula $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$, ¿es satisfacible? Probamos asignaciones:

- $x_1 = V, x_2 = V$: primera cláusula V , segunda? $\neg V \vee x_3 = F \vee x_3 = x_3$, tercera? $\neg V \vee \neg x_3 = F \vee \neg x_3 = \neg x_3$. Necesitamos $x_3 = V$ y $x_3 = F$, imposible.
- $x_1 = V, x_2 = F$: primera V , segunda $\neg V \vee x_3 = x_3$, tercera $\neg F \vee \neg x_3 = V \vee \neg x_3 = V$. Basta con $x_3 = V$.

Asignación: $x_1 = V, x_2 = F, x_3 = V$ satisface todas las cláusulas. Es satisfacible.

¿Cuál es la complejidad de 2-SAT (SAT con cláusulas de exactamente 2 literales)? 2-SAT está en P. Se puede resolver en tiempo $O(n^2)$ usando

implicaciones y componentes fuertemente conexas. Esto muestra que la restricción a cláusulas de tamaño 2 hace que el problema sea tratable, mientras que 3-SAT ya es NP-completo.

Muestre que VERTEX COVER está en NP. Para demostrar que VERTEX COVER está en NP, debemos exhibir un certificado y un verificador polinomial:

- **Certificado:** Un subconjunto $C \subseteq V$ de k vértices.
- **Verificador:**
 1. Verificar que $|C| = k$: $O(1)$
 2. Para cada arista $(u, v) \in E$, verificar que $u \in C$ o $v \in C$: $O(|E|)$

El verificador corre en tiempo $O(|E|)$, que es polinomial en el tamaño de la entrada. Por lo tanto, VERTEX COVER $\in NP$.

¿Qué significa que un problema sea "NP-completo bajo reducciones de Cook"? Una **reducción de Cook** (o reducción de Turing) permite usar el problema B como una caja negra (oráculo) que resuelve instancias en tiempo unitario, y se permite hacer múltiples consultas. Es más general que la reducción de Karp (reducción many-one) que usa una sola consulta.

La mayoría de las demostraciones de NP-completitud usan reducciones de Karp, que son un caso especial de reducciones de Cook.

¿Es el problema de ordenamiento NP-completo? Justifique. No. El problema de ordenamiento está en P (existen algoritmos $O(n \log n)$ como Merge Sort). Por lo tanto, no puede ser NP-completo a menos que $P = NP$.

Para que un problema sea NP-completo, debe estar en NP y ser NP-difícil. El ordenamiento está en P, pero no se conoce que sea NP-difícil (y probablemente no lo sea).

Dado el problema de la mochila (0/1) con $n = 4$ objetos: pesos (2, 3, 4, 5), valores (3, 4, 5, 6), capacidad $W = 7$, encuentre la solución óptima. Probamos combinaciones:

- Objetos 1 y 2: peso $2 + 3 = 5$, valor $3 + 4 = 7$
- Objetos 1 y 3: peso $2 + 4 = 6$, valor $3 + 5 = 8$
- Objetos 1 y 4: peso $2 + 5 = 7$, valor $3 + 6 = 9$
- Objetos 2 y 3: peso $3 + 4 = 7$, valor $4 + 5 = 9$
- Objetos 2 y 4: peso $3 + 5 = 8 > 7$, no
- Objetos 3 y 4: peso $4 + 5 = 9 > 7$, no

- Objetos 1,2,3: peso $2 + 3 + 4 = 9 > 7$, no

Soluciones óptimas: $\{1, 4\}$ con valor 9, o $\{2, 3\}$ con valor 9.

¿Cuál es la complejidad de la búsqueda exhaustiva para un problema NP-completo con $n = 50$? Para muchos problemas NP-completos (como SAT o SUBSET SUM), la búsqueda exhaustiva tiene complejidad $O(2^n)$. Para $n = 50$:

$$2^{50} \approx 1.12 \times 10^{15} \text{ operaciones}$$

Suponiendo 10^9 operaciones por segundo, tomaría aproximadamente 10^6 segundos, ¡unos 11.5 días! Esto muestra por qué los problemas NP-completos son intratables para instancias grandes.

Muestre que si un problema NP-completo se puede resolver en tiempo $O(n^{100})$, entonces $P = NP$. Si existe un algoritmo polinomial para un problema NP-completo L , entonces para cualquier $L' \in NP$ tenemos $L' \leq_p L$. Como la reducción es polinomial y el algoritmo para L es polinomial, la composición es polinomial. Por lo tanto, todo problema en NP se puede resolver en tiempo polinomial, lo que implica $P = NP$.

Esto es cierto independientemente del exponente, incluso si es n^{100} . Lo que importa es que sea polinomial.

¿Qué es el problema HAMILTONIAN CYCLE? ¿Es NP-completo? **HAMILTONIAN CYCLE:** Dado un grafo $G = (V, E)$, ¿existe un ciclo que visite cada vértice exactamente una vez?

Sí, HAMILTONIAN CYCLE es NP-completo. Se puede reducir desde VERTEX COVER o desde SAT. La versión para grafos dirigidos también es NP-completa.

La versión para grafos completos con pesos (TSP) también es NP-completa.

¿Cuál es la relación entre P, NP, NP-completo y NP-hard? Dibuje un diagrama de Venn. Si asumimos $P \neq NP$:

[scale=0.7] (0,0) circle (3cm); (0,0) circle (1.5cm); (-1.5,1.5) circle (1.2cm);
at (0,2.5) NP; at (0,1) P; at (-1.8,2.2) NP-completo; at (2.5,0) NP-hard;

- $P \subseteq NP$
- NP-completo $\subset$ NP
- NP-hard contiene a NP-completo y se extiende fuera de NP

¿Es posible que $P = NP$? ¿Qué implicaría? Es posible, pero la mayoría de los expertos cree que es improbable. Si $P = NP$:

- Todos los problemas NP-completos tendrían algoritmos polinomiales.

- La criptografía de clave pública (RSA, etc.) sería vulnerable, ya que factorización está en NP.
- Muchos problemas de optimización serían resolubles eficientemente.
- Sería uno de los descubrimientos más importantes de la computación.

El consenso general es que $P \neq NP$, pero no hay demostración.

Reduzca VERTEX COVER a SUBSET SUM. **Reducción:** Dado un grafo $G = (V, E)$ con $V = \{1, \dots, n\}$ y $E = \{e_1, \dots, e_m\}$, construimos:

- Un número a_i por cada vértice i : representación en base grande con dígitos para las aristas incidentes.
- Un número t como objetivo.

Cada vértice se representa como un número en base $B = m + 1$: el j -ésimo dígito es 1 si la arista e_j es incidente al vértice i .

El objetivo t es un número con todos los dígitos en 1 (para que cada arista sea cubierta al menos una vez).

Un subconjunto de vértices suma t si y solo si es un vertex cover.

Explique por qué el problema del viajante (TSP) es NP-difícil. TSP (Traveling Salesman Problem): Dadas n ciudades y distancias, encontrar el recorrido más corto que visita todas exactamente una vez.

La versión de decisión de TSP ("¿existe un recorrido de longitud $\leq k$?") es NP-completa porque:

1. Está en NP (verificar un recorrido es lineal).
2. Se puede reducir HAMILTONIAN CYCLE a TSP (asignar peso 1 a las aristas del grafo, 2 a las que no existen).

TSP es NP-hard porque la versión de optimización es al menos tan difícil como la versión de decisión.

¿Qué es el problema de la clique máxima? ¿Es NP-completo? **CLIQUE:** Dado un grafo G y un entero k , ¿existe un subconjunto de k vértices completamente conectado?

La versión de decisión de CLIQUE es NP-completa. Se puede reducir desde 3-SAT construyendo un grafo donde los vértices representan literales y se conectan si son compatibles (no son contradictorios y están en diferentes cláusulas).

La versión de optimización (encontrar la clique máxima) es NP-hard.

¿Qué son los problemas co-NP? Dé un ejemplo. **co-NP** es la clase de problemas cuyo complemento está en NP. Es decir, un problema L está en co-NP si $\bar{L} \in NP$.

Ejemplo: UNSAT (verificar que una fórmula no es satisfacible) está en co-NP. Dado un certificado de que no existe asignación, no es fácil verificarlo (a diferencia de SAT, donde una asignación es un certificado fácil).

Se cree que $NP \neq co-NP$, y que los problemas NP-completos no están en co-NP (a menos que $P = NP$).

Ejercicios Propuestos (20 Ejercicios)

1. Demuestre que SUBSET SUM está en NP.
2. ¿Es el problema de encontrar el elemento máximo en un arreglo NP-completo? ¿Por qué?
3. Muestre una reducción de SAT a 3-SAT.
4. ¿Cuál es la diferencia entre reducción de Karp y reducción de Cook?
5. Demuestre que si $A \leq_p B$ y $B \in P$, entonces $A \in P$.
6. ¿Es el problema de ordenamiento topológico NP-completo?
7. Muestre que VERTEX COVER en grafos bipartitos está en P.
8. ¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es polinomial?
9. Demuestre que 3-SAT es NP-completo (esbozo).
10. ¿Qué implica que un problema sea NP-completo para la búsqueda de algoritmos?
11. Muestre una reducción de CLIQUE a VERTEX COVER.
12. ¿Cuál es la complejidad de la programación dinámica para SUBSET SUM?
13. ¿Qué es el problema de la partición? ¿Es NP-completo?
14. Demuestre que PARTITION $\leq_p$ SUBSET SUM.
15. ¿Qué es un algoritmo de aproximación? Dé un ejemplo.
16. ¿Cuál es la complejidad de verificar una solución para TSP?
17. ¿Qué es el problema del circuito de Hamilton en grafos dirigidos?

18. ¿Cómo se relaciona P con co-NP?
19. ¿Qué es el problema de la mochila fraccionaria? ¿Es NP-completo?
20. Investigue el problema INDEPENDENT SET y su relación con VERTEX COVER.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La diferencia fundamental entre problemas tratables (P) e intratables (NP-completos).
- Las definiciones formales de P, NP, NP-completo y NP-hard.
- El concepto de reducción polinomial y su importancia para demostrar NP-completitud.
- El Teorema de Cook-Levin y la importancia de SAT como primer problema NP-completo.
- Los problemas NP-completos clásicos y cómo reconocerlos.
- Estrategias prácticas para enfrentar problemas NP-completos en aplicaciones reales.

Bibliografía de la Unidad

- Arora, S., & Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 34.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

Unidad III: Introducción a Problemas NP-hard

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la definición y características de los problemas NP-hard.
- Distinguir entre problemas NP-completos y NP-hard.
- Identificar problemas NP-hard clásicos en diferentes dominios.
- Aplicar estrategias para enfrentar problemas NP-hard en la práctica.
- Comprender la jerarquía de complejidad más allá de NP.

Definición de NP-hard

Definición Formal

Un problema L es **NP-hard** (NP-difícil) si para todo problema $L' \in NP$, existe una reducción polinomial $L' \leq_p L$.

A diferencia de los problemas NP-completos, un problema NP-hard **no necesita estar en NP**.

[scale=0.8] (0,0) ellipse (3.5cm and 2.5cm); (0,0) circle (2cm); (0,0) circle
(1cm);
at (0,2.2) NP; at (0,1) P; at (2.5,1.5) NP-completo; at (3.5,0) NP-hard;
[-ı, thick] (2,1.2) – (3,1.8);

Relación entre NP-completo y NP-hard

Todo problema NP-completo es NP-hard. Por definición, un problema NP-completo L satisface:

1. $L \in NP$
2. Para todo $L' \in NP$, $L' \leq_p L$

La segunda condición es exactamente la definición de NP-hard.

No todo problema NP-hard es NP-completo. [Contraejemplo] El **Problema de la Parada** (Halting Problem) es NP-hard pero no está en NP (es indecidible). Otros ejemplos incluyen problemas de optimización cuya versión de decisión está en NP.

Problemas NP-hard Clásicos

Problemas de Optimización

TSP (Traveling Salesman Problem) - Versión de Optimización

TSP (Optimización): Dadas n ciudades y distancias $d(i, j)$ entre cada par, encontrar el recorrido más corto que visita cada ciudad exactamente una vez y regresa al punto de partida.

La versión de decisión de TSP ("¿existe un recorrido de longitud $\leq k$?") es NP-completa. La versión de optimización es NP-hard.

KNAPSACK - Versión de Optimización

KNAPSACK (Optimización): Dados n objetos con pesos w_i y valores v_i , y una capacidad W , encontrar el subconjunto que maximiza el valor total sin exceder la capacidad.

VERTEX COVER - Versión de Optimización

VERTEX COVER (Optimización): Dado un grafo $G = (V, E)$, encontrar el conjunto de vértices más pequeño que cubre todas las aristas.

Problemas de Conteo

#SAT (SAT counting): Dada una fórmula booleana, contar el número de asignaciones que la satisfacen.

#SAT es P-completo, una clase de complejidad aún más difícil que NP.

Clase P

#P es la clase de problemas de conteo asociados a problemas en NP. Dado un problema de decisión en NP, contar el número de soluciones.

- **#SAT** es P-completo
- **#P** es al menos tan difícil como NP

Estrategias para Enfrentar Problemas NP-hard

Algoritmos de Aproximación

Un **algoritmo de aproximación** es un algoritmo que produce una solución cercana a la óptima, con una garantía de calidad.

[Vertex Cover] El algoritmo greedy que elige todos los vértices de todas las aristas es una aproximación 2.

Algoritmos Heurísticos

- **Búsqueda Local:** Comenzar con una solución y mejorarla mediante pequeños cambios.
- **Algoritmos Genéticos:** Basados en evolución y selección natural.
- **Recocido Simulado:** Inspirado en el enfriamiento de metales.
- **Búsqueda Tabú:** Evita ciclos manteniendo una lista de movimientos recientes.

Algoritmos de Parámetros Fijos (FPT)

Un problema es **FPT** (Fixed-Parameter Tractable) si se puede resolver en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro del problema.

VERTEX COVER se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación.

Algoritmos Exactos Exponenciales Optimizados

- **Meet-in-the-Middle:** Dividir el problema en dos mitades y combinar.
- **Branch and Bound:** Poda de ramas que no pueden mejorar la mejor solución encontrada.

- **Programación Dinámica con Máscaras de Bits:** $O(2^n n^2)$ para TSP.

Programación Lineal Entera (ILP)

- Formular el problema como un programa lineal entero.
- Usar solvers comerciales (CPLEX, Gurobi, SCIP).
- Útil para instancias de tamaño moderado.

Problemas NP-hard Notables

Problemas de Grafos

COLORACIÓN DE GRAFOS

Dado un grafo G , encontrar el número cromático $\chi(G)$ (mínimo número de colores necesarios para colorear los vértices sin que dos adyacentes compartan color).

- La versión de decisión ("¿se puede colorear con k colores?") es NP-completa para $k \geq 3$.
- La versión de optimización (encontrar $\chi(G)$) es NP-hard.
- Para $k = 2$, es polinomial (verificar bipartitud).

CLIQUE MÁXIMA

Dado un grafo G , encontrar el clique de tamaño máximo.

CAMINO MÁS LARGO

Dado un grafo G y dos vértices s, t , encontrar el camino más largo de s a t (sin repetir vértices).

Problemas de Números

FACTORIZACIÓN DE ENTEROS

Dado un entero N , encontrar su factorización en números primos.

No se sabe si la factorización es NP-completa, pero se cree que está en la intersección de NP y co-NP. Es la base de la criptografía RSA.

LOGARITMO DISCRETO

Dados g , h y un primo p , encontrar x tal que $g^x \equiv h \pmod{p}$.

Problemas de Planificación**JOB SHOP SCHEDULING**

Dadas m máquinas y n trabajos, cada trabajo consiste en una secuencia de operaciones en máquinas específicas, minimizar el tiempo total (makespan).

BIN PACKING (Optimización)

Dados objetos con tamaños s_i , encontrar el mínimo número de contenedores de capacidad C necesarios para empaquetarlos.

Problemas de Cadenas**ALINEAMIENTO DE SECUENCIAS MÚLTIPLE**

Dadas múltiples cadenas, encontrar un alineamiento que maximice una puntuación de similitud.

Problemas de Satisfacibilidad Cuantificada (QBF)

Dada una fórmula booleana con cuantificadores $\forall$ y $\exists$, determinar si es verdadera.

QBF es PSPACE-completo, considerado más difícil que los problemas NP-completos.

Técnicas de Enfrentamiento Práctico**Algoritmos de Aproximación para Problemas Clásicos****Aproximación para VERTEX COVER**

[language=C, caption=Aproximación 2 para Vertex Cover] APROX-VERTEX-COVER(G)
 $C = \text{empty set}$
 for each edge (u,v) in E if u not in C and v not in C add u and v to C
 return C

Aproximación para TSP Métrico

Para TSP métrico (desigualdad triangular), el algoritmo de árbol de expansión mínimo da una aproximación 2.

[language=C, caption=Aproximación 2 para TSP Métrico] APROX-TSP(G)
 T = MST(G) H = double each edge of T // recorrido de Euler return short-cut(H) // eliminar vértices repetidos

Aproximación para KNAPSACK

[language=C, caption=Esquema de Aproximación para KNAPSACK] APROX-KNAPSACK(items, W, epsilon) // Escalar valores para hacer el problema más pequeño scale = epsilon * $\max_v \text{value}_i / n$ foreach item v_i $v'_i = \text{floor}(v_i / \text{scale})$ // Resolver KNAPSACK

Algoritmos Exactos Exponenciales**FPT para VERTEX COVER**

[language=C, caption=FPT para Vertex Cover] FPT-VERTEX-COVER(G, k) if no edges: return empty set if k == 0: return no solution choose an edge (u,v) // Ramificar: u en la solución o v en la solución C1 = u FPT-VERTEX-COVER(G - u, k-1) C2 = v FPT-VERTEX-COVER(G - v, k-1) return mejor de C1 y C2 Complejidad: $O(2^k \cdot n)$.

Meet-in-the-Middle para SUBSET SUM

[language=C, caption=Meet-in-the-Middle para Subset Sum] MITM-SUBSET-SUM(S, t) n = |S| A = primeros n/2 elementos B = últimos n/2 elementos // Generar todas las sumas de subconjuntos de A sumsA = todas las sumas de subconjuntos de A // Para cada suma en B, buscar complemento en sumsA for each subset sum s of B if t - s in sumsA return True return False Complejidad: $O(2^{n/2})$.

Ejercicios Resueltos (20 Ejercicios)

¿Cuál es la diferencia fundamental entre NP-completo y NP-hard?

- **NP-completo:** Un problema que está en NP y es NP-hard.
- **NP-hard:** Un problema tal que todo problema en NP se reduce a él, pero no necesariamente está en NP.

Es decir, NP-hard es una condición más débil (solo requiere la propiedad de reducción), mientras que NP-completo requiere además que el problema pertenezca a NP.

Ejemplo: TSP (optimización) es NP-hard pero no necesariamente NP-completo porque la versión de optimización no es un problema de decisión.

¿Puede un problema indecidible ser NP-hard? Justifique. Sí. El Problema de la Parada (Halting Problem) es indecidible y es NP-hard. Para demostrar que es NP-hard, se reduce SAT al Problema de la Parada:

- Dada una fórmula ϕ , construir un programa que pruebe todas las asignaciones y termine si encuentra una que satisfaga ϕ .
- El programa termina si y solo si ϕ es satisfacible.

Esto muestra que $\text{SAT} \leq_p \text{HALTING}$, por lo que HALTING es NP-hard.

¿Qué es un algoritmo de aproximación? Dé un ejemplo. Un algoritmo de aproximación es un algoritmo que encuentra una solución cercana a la óptima, con una garantía de calidad demostrable.

Ejemplo: Vertex Cover tiene un algoritmo de aproximación 2:

1. Tomar todas las aristas del grafo.
2. Por cada arista, agregar ambos extremos al cover.
3. El cover resultante tiene tamaño a lo sumo el doble del óptimo.

La garantía es que $|C| \leq 2 \cdot |C_{\text{opt}}|$.

¿Qué significa que un algoritmo sea FPT (Fixed-Parameter Tractable)? Un algoritmo FPT resuelve un problema parametrizado en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro (como el tamaño de la solución) y f es una función arbitraria (típicamente exponencial en k).

Ejemplo: Vertex Cover se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación. Para k pequeño (ej. $k \leq 30$), el algoritmo es práctico.

La ventaja de FPT es que separa la dificultad exponencial al parámetro k del tamaño n del problema.

¿Cuál es la relación entre P, NP, NP-hard y PSPACE?

- $P \subseteq NP \subseteq PSPACE$
- $\text{NP-hard} \not\subseteq \text{NP}$ (problemas NP-hard pueden estar fuera de NP)
- $\text{NP-completo} = \text{NP} \cap \text{NP-hard}$

Se cree que todas estas inclusiones son estrictas: $P \subset NP \subset PSPACE$, pero no está demostrado.

Un problema PSPACE-completo (como QBF) es más difícil que cualquier problema NP-completo (a menos que $NP = PSPACE$).

¿Por qué el problema de la factorización es importante en criptografía? La seguridad del sistema RSA se basa en la dificultad de factorizar grandes números compuestos. Si existiera un algoritmo polinomial para factorización, RSA sería inseguro.

Factorización está en NP (el certificado son los factores primos) y en co-NP (el certificado de no primalidad). No se sabe si es NP-completo, y muchos creen que no lo es. Está en la intersección $NP \cap \text{co-NP}$.

El algoritmo más rápido conocido para factorización es el General Number Field Sieve (GNFS), que es subexponencial pero no polinomial.

Demuestre que el problema de optimización de VERTEX COVER es NP-hard. La versión de decisión de VERTEX COVER ("¿existe un vertex cover de tamaño $\leq k$?") es NP-completa. Para demostrar que la versión de optimización es NP-hard, observamos que:

1. Si tuviéramos un algoritmo que encuentra el vertex cover mínimo, podríamos responder la pregunta de decisión comparando el tamaño del cover encontrado con k .
2. Por lo tanto, la versión de optimización es al menos tan difícil como la versión de decisión.

Como la versión de decisión es NP-completa, la versión de optimización es NP-hard.

¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es NP-completo? La coloración de grafos pregunta si se pueden colorear los vértices de un grafo con k colores de modo que vértices adyacentes tengan colores diferentes.

- Para $k = 1$: trivial, solo funciona si el grafo no tiene aristas.
- Para $k = 2$: polinomial (verificar si el grafo es bipartito con BFS).
- Para $k \geq 3$: NP-completo.

La versión de optimización (encontrar el número cromático) es NP-hard.

Describe el algoritmo de aproximación para TSP métrico y su factor de aproximación. **Algoritmo:**

1. Calcular el árbol de expansión mínimo (MST) del grafo.

2. Duplicar cada arista del MST para obtener un grafo euleriano.
3. Encontrar un recorrido de Euler.
4. Aplicar atajos (shortcut) para eliminar vértices repetidos, obteniendo un ciclo Hamiltoniano.

Factor de aproximación: 2. Es decir, $C_{approx} \leq 2 \cdot C_{opt}$.

Demostración: El MST tiene peso $\leq C_{opt}$. Duplicar las aristas da peso $2 \cdot C_{opt}$. Los atajos no aumentan el peso por la desigualdad triangular.

¿Qué es el problema SAT y por qué es más difícil que SAT? #SAT es el problema de contar el número de asignaciones que satisfacen una fórmula booleana.

Es más difícil que SAT porque:

1. Resolver SAT solo requiere saber si existe al menos una asignación.
2. Contar todas las asignaciones requiere explorar el espacio de soluciones más exhaustivamente.
3. #SAT es #P-completo, y se cree que #P es más difícil que NP.

Si se pudiera resolver #SAT en tiempo polinomial, entonces $P = NP$ (porque se podría responder SAT viendo si el conteo es > 0).

¿Qué es el problema de la programación lineal entera (ILP) y por qué es NP-hard? ILP (Integer Linear Programming) es el problema de optimizar una función lineal sujeta a restricciones lineales, con la condición de que las variables deben ser enteras.

Es NP-hard porque muchos problemas NP-completos (como KNAPSACK, VERTEX COVER) pueden formularse como ILP. Por ejemplo, KNAPSACK se formula como:

$$\text{Maximizar } \sum v_i x_i \text{ sujeta a } \sum w_i x_i \leq W, x_i \in \{0, 1\}$$

La versión sin restricción de integralidad (programación lineal) está en P. ¿Qué diferencia hay entre un algoritmo de aproximación y una heurística?

- **Algoritmo de aproximación:** Tiene una garantía demostrable sobre la calidad de la solución (ej. "nunca es peor que el doble del óptimo").
- **Heurística:** No tiene garantía teórica; funciona bien en la práctica pero puede fallar catastróficamente en algunos casos.

Ejemplo: La heurística del vecino más cercano para TSP no tiene factor de aproximación constante (puede ser arbitrariamente mala), mientras que el algoritmo basado en MST garantiza factor 2.

¿Qué es el problema de la mochila múltiple (Multiple Knapsack)? El problema de la mochila múltiple es una generalización donde hay m mochilas, cada una con capacidad W_j , y se deben asignar objetos a las mochilas maximizando el valor total.

Es NP-hard incluso para $m = 2$. Se puede reducir KNAPSACK a Multiple Knapsack con $m = 2$ (usando una mochila dummy).

Tiene aplicaciones en:

- Carga de contenedores
- Asignación de recursos
- Publicidad en línea (asignación de anuncios a espacios)

Demuestre que TSP es NP-hard. Para demostrar que TSP es NP-hard, reducimos HAMILTONIAN CYCLE a TSP:

1. Dado un grafo $G = (V, E)$ de HAMILTONIAN CYCLE.
2. Construir instancia de TSP con n ciudades.
3. Definir distancias: $d(i, j) = 1$ si $(i, j) \in E$, $d(i, j) = 2$ si no.
4. Preguntar si existe un recorrido de longitud $\leq n$.

Existe un ciclo Hamiltoniano en G si y solo si existe un recorrido TSP de longitud n .

Como HAMILTONIAN CYCLE es NP-completo, TSP es NP-hard.

¿Qué es el problema de la planificación (Job Shop Scheduling) y por qué es difícil? Job Shop Scheduling es el problema de asignar trabajos a máquinas en el tiempo. Cada trabajo consiste en una secuencia de operaciones, cada operación requiere una máquina específica por un tiempo determinado.

Objetivo: minimizar el makespan (tiempo total).

Es NP-hard incluso para 2 máquinas y casos muy restringidos. La dificultad surge de:

- Secuenciación de operaciones
- Asignación de recursos
- Conflictos por máquinas compartidas

Es un problema fundamental en ingeniería de producción y logística.

¿Qué es un esquema de aproximación (PTAS) y un esquema de aproximación totalmente polinomial (FPTAS)?

- **PTAS** (Polynomial-Time Approximation Scheme): Un algoritmo que para cualquier $\epsilon > 0$ produce una solución con factor $1 + \epsilon$ (o $1 - \epsilon$ para maximización) en tiempo polinomial en n (pero puede ser exponencial en $1/\epsilon$).
- **FPTAS** (Fully PTAS): El tiempo también es polinomial en $1/\epsilon$.

KNAPSACK tiene FPTAS, mientras que TSP no tiene PTAS a menos que $P = NP$.

¿Qué es el problema de la parada (Halting Problem) y por qué es indecidible? El Problema de la Parada pregunta: Dado un programa P y una entrada x , ¿termina P al ejecutarse con entrada x ?

Turing demostró que es indecidible (no existe algoritmo que lo resuelva para todos los casos). La demostración usa diagonalización:

1. Supongamos que existe $H(P, x)$ que decide si P termina con entrada x .
2. Construimos $D(P)$ que llama a $H(P, P)$ y entra en bucle si H dice que termina.
3. Evaluar $D(D)$ lleva a una contradicción.

Es NP-hard porque podemos reducir SAT a él (como vimos antes).

¿Qué es el problema de la correspondencia de Post (PCP)? PCP (Post Correspondence Problem): Dadas n fichas, cada una con una cadena superior u_i y una cadena inferior v_i , ¿existe una secuencia de fichas (con repetición) tal que la concatenación de las cadenas superiores es igual a la concatenación de las cadenas inferiores?

Es indecidible. Es un ejemplo clásico de problema no computable que se usa para demostrar indecidibilidad de otros problemas.

Aunque es indecidible, hay algoritmos que funcionan para instancias pequeñas.

¿Qué es un algoritmo de búsqueda local y cuándo es útil? Un algoritmo de búsqueda local comienza con una solución candidata y la mejora iterativamente mediante pequeños cambios (movimientos locales).

Ejemplo para TSP:

1. Comenzar con un recorrido aleatorio.

2. Aplicar movimientos 2-opt (intercambiar dos aristas) para mejorar.
3. Repetir hasta no encontrar mejoras.

Es útil cuando:

- El espacio de búsqueda es muy grande
- Se necesita una solución rápida (no necesariamente óptima)
- Se pueden definir movimientos locales efectivos

¿Qué es el problema de la búsqueda de cliques máximos y por qué es difícil? El problema de la clique máxima es encontrar el subconjunto de vértices completamente conectado más grande en un grafo.

Es NP-hard porque:

1. La versión de decisión ("¿existe un clique de tamaño $\geq k$?") es NP-completa.
2. La versión de optimización es al menos tan difícil.

No se conoce algoritmo de aproximación con factor constante para clique máxima (es inaproximable a menos que $P = NP$).

En la práctica, se usan algoritmos de branch-and-bound con poda.

Ejercicios Propuestos (20 Ejercicios)

1. ¿Cuál es la diferencia entre un problema NP-hard y un problema NP-completo? Dé ejemplos.
2. Demuestre que la versión de optimización de KNAPSACK es NP-hard.
3. ¿Qué es el algoritmo de aproximación para VERTEX COVER? ¿Cuál es su factor?
4. ¿Qué es un algoritmo FPT? Dé un ejemplo.
5. ¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es polinomial?
6. Demuestre que TSP (optimización) es NP-hard.
7. ¿Qué es el problema de la factorización y por qué es importante en criptografía?

8. ¿Qué es el problema de la planificación (Job Shop Scheduling)?
9. ¿Qué es un PTAS y un FPTAS? Dé ejemplos.
10. ¿Qué es el problema de la parada y por qué es indecidible?
11. ¿Qué es el problema de la correspondencia de Post?
12. ¿Qué es un algoritmo de búsqueda local? Dé un ejemplo.
13. ¿Qué es el problema de la búsqueda de cliques máximos?
14. ¿Qué es el problema de la mochila múltiple?
15. ¿Qué es el problema de la programación lineal entera?
16. ¿Qué es el problema de la alineamiento de secuencias múltiples?
17. ¿Qué es el problema QBF y cómo se relaciona con PSPACE?
18. ¿Qué es el problema SAT y por qué es más difícil que SAT?
19. ¿Qué es el algoritmo de meet-in-the-middle para SUBSET SUM?
20. Investigue el problema de la ruta más larga en grafos.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La definición formal de NP-hard y su relación con NP-completo.
- Que los problemas NP-hard no necesitan estar en NP, pueden ser incluso más difíciles (indecidibles).
- La jerarquía de complejidad más allá de NP: PSPACE, EXP, P.
- Estrategias prácticas para enfrentar problemas NP-hard: aproximación, heurísticas, FPT, algoritmos exactos exponenciales.
- Problemas NP-hard clásicos en grafos, números, planificación y cadenas.
- La importancia de estos problemas en aplicaciones reales.

Bibliografía de la Unidad

- Arora, S., & Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 34-35.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1), 230-265.

Unidad III: Introducción a Problemas NP-hard

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la definición y características de los problemas NP-hard.
- Distinguir entre problemas NP-completos y NP-hard.
- Identificar problemas NP-hard clásicos en diferentes dominios.
- Aplicar estrategias para enfrentar problemas NP-hard en la práctica.
- Comprender la jerarquía de complejidad más allá de NP.

Definición de NP-hard

Definición Formal

Un problema L es **NP-hard** (NP-difícil) si para todo problema $L' \in NP$, existe una reducción polinomial $L' \leq_p L$.

A diferencia de los problemas NP-completos, un problema NP-hard **no necesita estar en NP**.

[scale=0.8] (0,0) ellipse (3.5cm and 2.5cm); (0,0) circle (2cm); (0,0) circle
(1cm);
at (0,2.2) NP; at (0,1) P; at (2.5,1.5) NP-completo; at (3.5,0) NP-hard;
[-ı, thick] (2,1.2) – (3,1.8);

Relación entre NP-completo y NP-hard

Todo problema NP-completo es NP-hard. Por definición, un problema NP-completo L satisface:

1. $L \in NP$
2. Para todo $L' \in NP$, $L' \leq_p L$

La segunda condición es exactamente la definición de NP-hard.

No todo problema NP-hard es NP-completo. [Contraejemplo] El **Problema de la Parada** (Halting Problem) es NP-hard pero no está en NP (es indecidible). Otros ejemplos incluyen problemas de optimización cuya versión de decisión está en NP.

Problemas NP-hard Clásicos

Problemas de Optimización

TSP (Traveling Salesman Problem) - Versión de Optimización

TSP (Optimización): Dadas n ciudades y distancias $d(i, j)$ entre cada par, encontrar el recorrido más corto que visita cada ciudad exactamente una vez y regresa al punto de partida.

La versión de decisión de TSP ("¿existe un recorrido de longitud $\leq k$?") es NP-completa. La versión de optimización es NP-hard.

KNAPSACK - Versión de Optimización

KNAPSACK (Optimización): Dados n objetos con pesos w_i y valores v_i , y una capacidad W , encontrar el subconjunto que maximiza el valor total sin exceder la capacidad.

VERTEX COVER - Versión de Optimización

VERTEX COVER (Optimización): Dado un grafo $G = (V, E)$, encontrar el conjunto de vértices más pequeño que cubre todas las aristas.

Problemas de Conteo

#SAT (SAT counting): Dada una fórmula booleana, contar el número de asignaciones que la satisfacen.

#SAT es P-completo, una clase de complejidad aún más difícil que NP.

Clase P

#P es la clase de problemas de conteo asociados a problemas en NP. Dado un problema de decisión en NP, contar el número de soluciones.

- **#SAT** es P-completo
- **#P** es al menos tan difícil como NP

Estrategias para Enfrentar Problemas NP-hard

Algoritmos de Aproximación

Un **algoritmo de aproximación** es un algoritmo que produce una solución cercana a la óptima, con una garantía de calidad.

[Vertex Cover] El algoritmo greedy que elige todos los vértices de todas las aristas es una aproximación 2.

Algoritmos Heurísticos

- **Búsqueda Local:** Comenzar con una solución y mejorarla mediante pequeños cambios.
- **Algoritmos Genéticos:** Basados en evolución y selección natural.
- **Recocido Simulado:** Inspirado en el enfriamiento de metales.
- **Búsqueda Tabú:** Evita ciclos manteniendo una lista de movimientos recientes.

Algoritmos de Parámetros Fijos (FPT)

Un problema es **FPT** (Fixed-Parameter Tractable) si se puede resolver en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro del problema.

VERTEX COVER se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación.

Algoritmos Exactos Exponenciales Optimizados

- **Meet-in-the-Middle:** Dividir el problema en dos mitades y combinar.
- **Branch and Bound:** Poda de ramas que no pueden mejorar la mejor solución encontrada.

- **Programación Dinámica con Máscaras de Bits:** $O(2^n n^2)$ para TSP.

Programación Lineal Entera (ILP)

- Formular el problema como un programa lineal entero.
- Usar solvers comerciales (CPLEX, Gurobi, SCIP).
- Útil para instancias de tamaño moderado.

Problemas NP-hard Notables

Problemas de Grafos

COLORACIÓN DE GRAFOS

Dado un grafo G , encontrar el número cromático $\chi(G)$ (mínimo número de colores necesarios para colorear los vértices sin que dos adyacentes compartan color).

- La versión de decisión ("¿se puede colorear con k colores?") es NP-completa para $k \geq 3$.
- La versión de optimización (encontrar $\chi(G)$) es NP-hard.
- Para $k = 2$, es polinomial (verificar bipartitud).

CLIQUE MÁXIMA

Dado un grafo G , encontrar el clique de tamaño máximo.

CAMINO MÁS LARGO

Dado un grafo G y dos vértices s, t , encontrar el camino más largo de s a t (sin repetir vértices).

Problemas de Números

FACTORIZACIÓN DE ENTEROS

Dado un entero N , encontrar su factorización en números primos.

No se sabe si la factorización es NP-completa, pero se cree que está en la intersección de NP y co-NP. Es la base de la criptografía RSA.

LOGARITMO DISCRETO

Dados g , h y un primo p , encontrar x tal que $g^x \equiv h \pmod{p}$.

Problemas de Planificación**JOB SHOP SCHEDULING**

Dadas m máquinas y n trabajos, cada trabajo consiste en una secuencia de operaciones en máquinas específicas, minimizar el tiempo total (makespan).

BIN PACKING (Optimización)

Dados objetos con tamaños s_i , encontrar el mínimo número de contenedores de capacidad C necesarios para empaquetarlos.

Problemas de Cadenas**ALINEAMIENTO DE SECUENCIAS MÚLTIPLE**

Dadas múltiples cadenas, encontrar un alineamiento que maximice una puntuación de similitud.

Problemas de Satisfacibilidad Cuantificada (QBF)

Dada una fórmula booleana con cuantificadores $\forall$ y $\exists$, determinar si es verdadera.

QBF es PSPACE-completo, considerado más difícil que los problemas NP-completos.

Técnicas de Enfrentamiento Práctico**Algoritmos de Aproximación para Problemas Clásicos****Aproximación para VERTEX COVER**

[language=C, caption=Aproximación 2 para Vertex Cover] APROX-VERTEX-COVER(G)
 $C = \text{empty set}$
 for each edge (u,v) in E if u not in C and v not in C add u and v to C
 return C

Aproximación para TSP Métrico

Para TSP métrico (desigualdad triangular), el algoritmo de árbol de expansión mínimo da una aproximación 2.

[language=C, caption=Aproximación 2 para TSP Métrico] APROX-TSP(G)
 T = MST(G) H = double each edge of T // recorrido de Euler return short-cut(H) // eliminar vértices repetidos

Aproximación para KNAPSACK

[language=C, caption=Esquema de Aproximación para KNAPSACK] APROX-KNAPSACK(items, W, epsilon) // Escalar valores para hacer el problema más pequeño scale = epsilon * $\max_v \text{value}_i / n$ foreach item v_i $v'_i = \text{floor}(v_i / \text{scale})$ // Resolver KNAPSACK

Algoritmos Exactos Exponenciales**FPT para VERTEX COVER**

[language=C, caption=FPT para Vertex Cover] FPT-VERTEX-COVER(G, k) if no edges: return empty set if k == 0: return no solution choose an edge (u,v) // Ramificar: u en la solución o v en la solución C1 = u FPT-VERTEX-COVER(G - u, k-1) C2 = v FPT-VERTEX-COVER(G - v, k-1) return mejor de C1 y C2 Complejidad: $O(2^k \cdot n)$.

Meet-in-the-Middle para SUBSET SUM

[language=C, caption=Meet-in-the-Middle para Subset Sum] MITM-SUBSET-SUM(S, t) n = |S| A = primeros n/2 elementos B = últimos n/2 elementos // Generar todas las sumas de subconjuntos de A sumsA = todas las sumas de subconjuntos de A // Para cada suma en B, buscar complemento en sumsA for each subset sum s of B if t - s in sumsA return True return False Complejidad: $O(2^{n/2})$.

Ejercicios Resueltos (20 Ejercicios)

¿Cuál es la diferencia fundamental entre NP-completo y NP-hard?

- **NP-completo:** Un problema que está en NP y es NP-hard.
- **NP-hard:** Un problema tal que todo problema en NP se reduce a él, pero no necesariamente está en NP.

Es decir, NP-hard es una condición más débil (solo requiere la propiedad de reducción), mientras que NP-completo requiere además que el problema pertenezca a NP.

Ejemplo: TSP (optimización) es NP-hard pero no necesariamente NP-completo porque la versión de optimización no es un problema de decisión.

¿Puede un problema indecidible ser NP-hard? Justifique. Sí. El Problema de la Parada (Halting Problem) es indecidible y es NP-hard. Para demostrar que es NP-hard, se reduce SAT al Problema de la Parada:

- Dada una fórmula ϕ , construir un programa que pruebe todas las asignaciones y termine si encuentra una que satisfaga ϕ .
- El programa termina si y solo si ϕ es satisfacible.

Esto muestra que $\text{SAT} \leq_p \text{HALTING}$, por lo que HALTING es NP-hard.

¿Qué es un algoritmo de aproximación? Dé un ejemplo. Un algoritmo de aproximación es un algoritmo que encuentra una solución cercana a la óptima, con una garantía de calidad demostrable.

Ejemplo: Vertex Cover tiene un algoritmo de aproximación 2:

1. Tomar todas las aristas del grafo.
2. Por cada arista, agregar ambos extremos al cover.
3. El cover resultante tiene tamaño a lo sumo el doble del óptimo.

La garantía es que $|C| \leq 2 \cdot |C_{\text{opt}}|$.

¿Qué significa que un algoritmo sea FPT (Fixed-Parameter Tractable)? Un algoritmo FPT resuelve un problema parametrizado en tiempo $f(k) \cdot n^{O(1)}$, donde k es un parámetro (como el tamaño de la solución) y f es una función arbitraria (típicamente exponencial en k).

Ejemplo: Vertex Cover se puede resolver en $O(2^k \cdot n)$ usando búsqueda de ramificación. Para k pequeño (ej. $k \leq 30$), el algoritmo es práctico.

La ventaja de FPT es que separa la dificultad exponencial al parámetro k del tamaño n del problema.

¿Cuál es la relación entre P, NP, NP-hard y PSPACE?

- $P \subseteq NP \subseteq PSPACE$
- $\text{NP-hard} \not\subseteq \text{NP}$ (problemas NP-hard pueden estar fuera de NP)
- $\text{NP-completo} = \text{NP} \cap \text{NP-hard}$

Se cree que todas estas inclusiones son estrictas: $P \subset NP \subset PSPACE$, pero no está demostrado.

Un problema PSPACE-completo (como QBF) es más difícil que cualquier problema NP-completo (a menos que $NP = PSPACE$).

¿Por qué el problema de la factorización es importante en criptografía? La seguridad del sistema RSA se basa en la dificultad de factorizar grandes números compuestos. Si existiera un algoritmo polinomial para factorización, RSA sería inseguro.

Factorización está en NP (el certificado son los factores primos) y en co-NP (el certificado de no primalidad). No se sabe si es NP-completo, y muchos creen que no lo es. Está en la intersección $NP \cap \text{co-NP}$.

El algoritmo más rápido conocido para factorización es el General Number Field Sieve (GNFS), que es subexponencial pero no polinomial.

Demuestre que el problema de optimización de VERTEX COVER es NP-hard. La versión de decisión de VERTEX COVER ("¿existe un vertex cover de tamaño $\leq k$?") es NP-completa. Para demostrar que la versión de optimización es NP-hard, observamos que:

1. Si tuviéramos un algoritmo que encuentra el vertex cover mínimo, podríamos responder la pregunta de decisión comparando el tamaño del cover encontrado con k .
2. Por lo tanto, la versión de optimización es al menos tan difícil como la versión de decisión.

Como la versión de decisión es NP-completa, la versión de optimización es NP-hard.

¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es NP-completo? La coloración de grafos pregunta si se pueden colorear los vértices de un grafo con k colores de modo que vértices adyacentes tengan colores diferentes.

- Para $k = 1$: trivial, solo funciona si el grafo no tiene aristas.
- Para $k = 2$: polinomial (verificar si el grafo es bipartito con BFS).
- Para $k \geq 3$: NP-completo.

La versión de optimización (encontrar el número cromático) es NP-hard.

Describe el algoritmo de aproximación para TSP métrico y su factor de aproximación. **Algoritmo:**

1. Calcular el árbol de expansión mínimo (MST) del grafo.

2. Duplicar cada arista del MST para obtener un grafo euleriano.
3. Encontrar un recorrido de Euler.
4. Aplicar atajos (shortcut) para eliminar vértices repetidos, obteniendo un ciclo Hamiltoniano.

Factor de aproximación: 2. Es decir, $C_{approx} \leq 2 \cdot C_{opt}$.

Demostración: El MST tiene peso $\leq C_{opt}$. Duplicar las aristas da peso $2 \cdot C_{opt}$. Los atajos no aumentan el peso por la desigualdad triangular.

¿Qué es el problema SAT y por qué es más difícil que SAT? #SAT es el problema de contar el número de asignaciones que satisfacen una fórmula booleana.

Es más difícil que SAT porque:

1. Resolver SAT solo requiere saber si existe al menos una asignación.
2. Contar todas las asignaciones requiere explorar el espacio de soluciones más exhaustivamente.
3. #SAT es #P-completo, y se cree que #P es más difícil que NP.

Si se pudiera resolver #SAT en tiempo polinomial, entonces $P = NP$ (porque se podría responder SAT viendo si el conteo es > 0).

¿Qué es el problema de la programación lineal entera (ILP) y por qué es NP-hard? ILP (Integer Linear Programming) es el problema de optimizar una función lineal sujeta a restricciones lineales, con la condición de que las variables deben ser enteras.

Es NP-hard porque muchos problemas NP-completos (como KNAPSACK, VERTEX COVER) pueden formularse como ILP. Por ejemplo, KNAPSACK se formula como:

$$\text{Maximizar } \sum v_i x_i \text{ sujeta a } \sum w_i x_i \leq W, x_i \in \{0, 1\}$$

La versión sin restricción de integralidad (programación lineal) está en P. ¿Qué diferencia hay entre un algoritmo de aproximación y una heurística?

- **Algoritmo de aproximación:** Tiene una garantía demostrable sobre la calidad de la solución (ej. "nunca es peor que el doble del óptimo").
- **Heurística:** No tiene garantía teórica; funciona bien en la práctica pero puede fallar catastróficamente en algunos casos.

Ejemplo: La heurística del vecino más cercano para TSP no tiene factor de aproximación constante (puede ser arbitrariamente mala), mientras que el algoritmo basado en MST garantiza factor 2.

¿Qué es el problema de la mochila múltiple (Multiple Knapsack)? El problema de la mochila múltiple es una generalización donde hay m mochilas, cada una con capacidad W_j , y se deben asignar objetos a las mochilas maximizando el valor total.

Es NP-hard incluso para $m = 2$. Se puede reducir KNAPSACK a Multiple Knapsack con $m = 2$ (usando una mochila dummy).

Tiene aplicaciones en:

- Carga de contenedores
- Asignación de recursos
- Publicidad en línea (asignación de anuncios a espacios)

Demuestre que TSP es NP-hard. Para demostrar que TSP es NP-hard, reducimos HAMILTONIAN CYCLE a TSP:

1. Dado un grafo $G = (V, E)$ de HAMILTONIAN CYCLE.
2. Construir instancia de TSP con n ciudades.
3. Definir distancias: $d(i, j) = 1$ si $(i, j) \in E$, $d(i, j) = 2$ si no.
4. Preguntar si existe un recorrido de longitud $\leq n$.

Existe un ciclo Hamiltoniano en G si y solo si existe un recorrido TSP de longitud n .

Como HAMILTONIAN CYCLE es NP-completo, TSP es NP-hard.

¿Qué es el problema de la planificación (Job Shop Scheduling) y por qué es difícil? Job Shop Scheduling es el problema de asignar trabajos a máquinas en el tiempo. Cada trabajo consiste en una secuencia de operaciones, cada operación requiere una máquina específica por un tiempo determinado.

Objetivo: minimizar el makespan (tiempo total).

Es NP-hard incluso para 2 máquinas y casos muy restringidos. La dificultad surge de:

- Secuenciación de operaciones
- Asignación de recursos
- Conflictos por máquinas compartidas

Es un problema fundamental en ingeniería de producción y logística.

¿Qué es un esquema de aproximación (PTAS) y un esquema de aproximación totalmente polinomial (FPTAS)?

- **PTAS** (Polynomial-Time Approximation Scheme): Un algoritmo que para cualquier $\epsilon > 0$ produce una solución con factor $1 + \epsilon$ (o $1 - \epsilon$ para maximización) en tiempo polinomial en n (pero puede ser exponencial en $1/\epsilon$).
- **FPTAS** (Fully PTAS): El tiempo también es polinomial en $1/\epsilon$.

KNAPSACK tiene FPTAS, mientras que TSP no tiene PTAS a menos que $P = NP$.

¿Qué es el problema de la parada (Halting Problem) y por qué es indecidible? El Problema de la Parada pregunta: Dado un programa P y una entrada x , ¿termina P al ejecutarse con entrada x ?

Turing demostró que es indecidible (no existe algoritmo que lo resuelva para todos los casos). La demostración usa diagonalización:

1. Supongamos que existe $H(P, x)$ que decide si P termina con entrada x .
2. Construimos $D(P)$ que llama a $H(P, P)$ y entra en bucle si H dice que termina.
3. Evaluar $D(D)$ lleva a una contradicción.

Es NP-hard porque podemos reducir SAT a él (como vimos antes).

¿Qué es el problema de la correspondencia de Post (PCP)? PCP (Post Correspondence Problem): Dadas n fichas, cada una con una cadena superior u_i y una cadena inferior v_i , ¿existe una secuencia de fichas (con repetición) tal que la concatenación de las cadenas superiores es igual a la concatenación de las cadenas inferiores?

Es indecidible. Es un ejemplo clásico de problema no computable que se usa para demostrar indecidibilidad de otros problemas.

Aunque es indecidible, hay algoritmos que funcionan para instancias pequeñas.

¿Qué es un algoritmo de búsqueda local y cuándo es útil? Un algoritmo de búsqueda local comienza con una solución candidata y la mejora iterativamente mediante pequeños cambios (movimientos locales).

Ejemplo para TSP:

1. Comenzar con un recorrido aleatorio.

2. Aplicar movimientos 2-opt (intercambiar dos aristas) para mejorar.
3. Repetir hasta no encontrar mejoras.

Es útil cuando:

- El espacio de búsqueda es muy grande
- Se necesita una solución rápida (no necesariamente óptima)
- Se pueden definir movimientos locales efectivos

¿Qué es el problema de la búsqueda de cliques máximos y por qué es difícil? El problema de la clique máxima es encontrar el subconjunto de vértices completamente conectado más grande en un grafo.

Es NP-hard porque:

1. La versión de decisión ("¿existe un clique de tamaño $\geq k$?") es NP-completa.
2. La versión de optimización es al menos tan difícil.

No se conoce algoritmo de aproximación con factor constante para clique máxima (es inaproximable a menos que $P = NP$).

En la práctica, se usan algoritmos de branch-and-bound con poda.

Ejercicios Propuestos (20 Ejercicios)

1. ¿Cuál es la diferencia entre un problema NP-hard y un problema NP-completo? Dé ejemplos.
2. Demuestre que la versión de optimización de KNAPSACK es NP-hard.
3. ¿Qué es el algoritmo de aproximación para VERTEX COVER? ¿Cuál es su factor?
4. ¿Qué es un algoritmo FPT? Dé un ejemplo.
5. ¿Qué es el problema de la coloración de grafos? ¿Para qué valores de k es polinomial?
6. Demuestre que TSP (optimización) es NP-hard.
7. ¿Qué es el problema de la factorización y por qué es importante en criptografía?

8. ¿Qué es el problema de la planificación (Job Shop Scheduling)?
9. ¿Qué es un PTAS y un FPTAS? Dé ejemplos.
10. ¿Qué es el problema de la parada y por qué es indecidible?
11. ¿Qué es el problema de la correspondencia de Post?
12. ¿Qué es un algoritmo de búsqueda local? Dé un ejemplo.
13. ¿Qué es el problema de la búsqueda de cliques máximos?
14. ¿Qué es el problema de la mochila múltiple?
15. ¿Qué es el problema de la programación lineal entera?
16. ¿Qué es el problema de la alineamiento de secuencias múltiples?
17. ¿Qué es el problema QBF y cómo se relaciona con PSPACE?
18. ¿Qué es el problema SAT y por qué es más difícil que SAT?
19. ¿Qué es el algoritmo de meet-in-the-middle para SUBSET SUM?
20. Investigue el problema de la ruta más larga en grafos.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La definición formal de NP-hard y su relación con NP-completo.
- Que los problemas NP-hard no necesitan estar en NP, pueden ser incluso más difíciles (indecidibles).
- La jerarquía de complejidad más allá de NP: PSPACE, EXP, P.
- Estrategias prácticas para enfrentar problemas NP-hard: aproximación, heurísticas, FPT, algoritmos exactos exponenciales.
- Problemas NP-hard clásicos en grafos, números, planificación y cadenas.
- La importancia de estos problemas en aplicaciones reales.

Bibliografía de la Unidad

- Arora, S., & Barak, B. (2009). *Computational Complexity: A Modern Approach*. Cambridge University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 34-35.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1), 230-265.

Unidad V: Caminos Mínimos - Dijkstra y Bellman-Ford

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de caminos mínimos desde una fuente única.
- Implementar y analizar el algoritmo de Dijkstra para pesos no negativos.
- Implementar y analizar el algoritmo de Bellman-Ford para pesos negativos.
- Detectar ciclos de peso negativo utilizando Bellman-Ford.
- Aplicar estos algoritmos a problemas prácticos de optimización de rutas.
- Comparar ambos algoritmos y elegir el apropiado según el contexto.

Introducción al Problema de Caminos Mínimos

Definición Formal

Dado un grafo dirigido ponderado $G = (V, E)$ con función de peso $w : E \rightarrow R$, y un vértice fuente $s \in V$, encontrar para cada vértice $v \in V$ el camino de menor peso desde s a v .

El peso de un camino $p = \langle v_0, v_1, \dots, v_k \rangle$ es:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

La distancia mínima se define como:

$$\delta(s, v) = \begin{cases} \min \{w(p) : p \text{ es camino de } s \text{ a } v\} & \text{si existe camino} \\ \infty & \text{en caso contrario} \end{cases}$$

Propiedades Fundamentales

[Subestructura óptima] Un subcamino de un camino mínimo es también un camino mínimo.

[Desigualdad triangular] Para cualquier arista $(u, v) \in E$:

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

Relajación de Aristas

La técnica central de ambos algoritmos es la **relajación**:

[language=C, caption=Relajación de una arista] RELAX(u, v, w) if d[v] > d[u] + w(u, v) d[v] = d[u] + w(u, v) parent[v] = u

[scale=0.8] [circle, draw] (u) at (0,0) u; [circle, draw] (v) at (3,0) v; [->] (u) -- node[above] $w(u, v)$ (v);
 at (0,-1.5) $d[u] = 5$; at (3,-1.5) $d[v] = 9$;
 at (1.5,-2.5) $5 + w(u, v) < 9$;

Algoritmo de Dijkstra

Descripción General

Dijkstra encuentra caminos mínimos desde una fuente s a todos los demás vértices. El algoritmo mantiene un conjunto S de vértices cuyas distancias ya están finalizadas, y una cola de prioridad Q con los vértices restantes.

Requisitos

- Todos los pesos de las aristas deben ser **no negativos**.
- El grafo puede ser dirigido o no dirigido.

Pseudocódigo Detallado

```
[language=C, caption=Algoritmo de Dijkstra] DIJKSTRA(G, w, s) // Inicialización
for each vertex v in G.V v.d = INFINITY v.parent = NIL s.d = 0
    // Cola de prioridad mínima por d Q = G.V
    while Q is not empty u = EXTRACT-MIN(Q) // vértice con menor d
    for each vertex v in G.Adj[u] if v in Q and v.d > u.d + w(u, v) v.d = u.d + w(u, v) v.parent = u DECREASE-KEY(Q, v, v.d)
```

Ejemplo Paso a Paso

Consideremos el siguiente grafo:

```
[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1.5) a; [circle, draw] (b) at (2,-1.5) b; [circle, draw] (c) at (4,0) c; [circle, draw] (d) at (6,1) d; [circle, draw] (e) at (6,-1) e;
[->] (s) -- node[above left] 4 (a); [->] (s) -- node[below left] 2 (b); [->] (a) -- node[above] 1 (c); [->] (b) -- node[below] 5 (c); [->] (b) -- node[right] 3 (a); [->] (c) -- node[above] 2 (d); [->] (c) -- node[below] 4 (e); [->] (d) -- node[right] 1 (e);
```

Paso 1 - Inicialización:

$$d[s] = 0, d[a] = \infty, d[b] = \infty, d[c] = \infty, d[d] = \infty, d[e] = \infty$$

$$Q = \{s(0), a(\infty), b(\infty), c(\infty), d(\infty), e(\infty)\}$$

Paso 2 - Extraer s (d=0):

- Relajar (s, a): $d[a] = \min(\infty, 0 + 4) = 4$
- Relajar (s, b): $d[b] = \min(\infty, 0 + 2) = 2$

$$Q = \{b(2), a(4), c(\infty), d(\infty), e(\infty)\}$$

Paso 3 - Extraer b (d=2):

- Relajar (b, a): $d[a] = \min(4, 2 + 3 = 5) = 4$ (no cambia)
- Relajar (b, c): $d[c] = \min(\infty, 2 + 5 = 7) = 7$

$$Q = \{a(4), c(7), d(\infty), e(\infty)\}$$

Paso 4 - Extraer a (d=4):

- Relajar (a, c) : $d[c] = \min(7, 4 + 1 = 5) = 5$

$$Q = \{c(5), d(\infty), e(\infty)\}$$

Paso 5 - Extraer c ($d=5$):

- Relajar (c, d) : $d[d] = \min(\infty, 5 + 2 = 7) = 7$
- Relajar (c, e) : $d[e] = \min(\infty, 5 + 4 = 9) = 9$

$$Q = \{d(7), e(9)\}$$

Paso 6 - Extraer d ($d=7$):

- Relajar (d, e) : $d[e] = \min(9, 7 + 1 = 8) = 8$

$$Q = \{e(8)\}$$

Paso 7 - Extraer e ($d=8$): Fin.

Resultados finales:

$$d[s] = 0, d[a] = 4, d[b] = 2, d[c] = 5, d[d] = 7, d[e] = 8$$

Implementación con Montículo Binario

```
[language=C, caption=Dijkstra con montículo binario] typedef struct int
vertex; int dist; HeapNode;
void dijkstra(Graph* G, int s) int* dist = malloc(G->V * sizeof(int));
int* parent = malloc(G->V * sizeof(int)); int* inHeap = malloc(G->V *
sizeof(int));
// Inicialización for (int i = 0; i < G->V; i++) dist[i] = INF; parent[i] =
-1; inHeap[i] = 1; dist[s] = 0;
// Construir heap HeapNode* heap = malloc(G->V * sizeof(HeapNode));
int heapSize = G->V; for (int i = 0; i < G->V; i++) heap[i].vertex = i;
heap[i].dist = dist[i]; buildMinHeap(heap, heapSize);
while (heapSize > 0) HeapNode min = extractMin(heap, heapSize); int
u = min.vertex; inHeap[u] = 0;
Node* adj = G->adj[u]; while (adj != NULL) int v = adj->vertex; int
weight = adj->weight; if (inHeap[v] & dist[v] > dist[u] + weight) dist[v] =
dist[u] + weight; parent[v] = u; decreaseKey(heap, heapSize, v, dist[v]); adj
= adj->next;
// Imprimir resultados for (int i = 0; i < G->V; i++) printf("Distancia
a
```

Análisis de Complejidad

- **Con arreglo simple:** $O(V^2)$
 - EXTRACT-MIN: $O(V)$ por operación, $O(V^2)$ total
 - DECREASE-KEY: $O(1)$ con arreglo
- **Con montículo binario:** $O((V + E) \log V) = O(E \log V)$
 - EXTRACT-MIN: $O(\log V)$ por operación, $O(V \log V)$ total
 - DECREASE-KEY: $O(\log V)$ por operación, $O(E \log V)$ total
- **Con montículo de Fibonacci:** $O(E + V \log V)$
 - EXTRACT-MIN: $O(\log V)$ amortizado
 - DECREASE-KEY: $O(1)$ amortizado

Demstración de Correctitud

Dijkstra calcula correctamente $\delta(s, v)$ para todo $v \in V$ si todos los pesos de aristas son no negativos.

Por inducción en el número de vértices extraídos de Q .

Invariante: Cuando un vértice u es extraído de Q , $u.d = \delta(s, u)$.

Caso base: s es el primer vértice extraído. $s.d = 0 = \delta(s, s)$.

Paso inductivo: Supongamos que todos los vértices extraídos antes de u tienen $d = \delta$. Sea u el próximo vértice extraído. Si $u.d > \delta(s, u)$, existe un camino más corto su . Sea x el primer vértice en ese camino que no ha sido extraído, y sea y su predecesor en el camino (que sí ha sido extraído). Entonces:

$$\delta(s, u) \geq \delta(s, y) + w(y, x) \geq y.d + w(y, x) \geq x.d$$

La última desigualdad se debe a que relajamos (y, x) cuando y fue extraído. Como x no ha sido extraído y $u.d > \delta(s, u) \geq x.d$, tendríamos $u.d > x.d$, contradiciendo que u fue elegido como el mínimo.

Limitaciones y Advertencias

Dijkstra **NO** funciona con pesos negativos. En presencia de aristas negativas, un vértice puede necesitar ser actualizado después de ser extraído.

```
[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle,
draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c;
[->] (s) -- node[above] 3 (a); [->] (s) -- node[below] 4 (b); [->] (a) --
node[above] 1 (c); [->] (b) -- node[below] -2 (c);
```

Dijkstra extrae s , luego a (3), actualiza c a 4, luego extrae b (4), pero c ya fue extraído con valor 4 cuando el camino $s \rightarrow b \rightarrow c$ tiene peso $4 + (-2) = 2$, que es menor.

Algoritmo de Bellman-Ford

Descripción General

Bellman-Ford encuentra caminos mínimos desde una fuente s a todos los demás vértices, incluso con pesos negativos, y detecta ciclos de peso negativo.

Pseudocódigo Detallado

```
[language=C, caption=Algoritmo de Bellman-Ford] BELLMAN-FORD(G,
w, s) // Inicialización for each vertex v in G.V v.d = INFINITY v.parent =
NIL s.d = 0
// Relajar todas las aristas —V—1 veces for i = 1 to —V—1 for each
edge (u, v) in G.E if v.d > u.d + w(u, v) v.d = u.d + w(u, v) v.parent = u
// Verificar ciclos de peso negativo for each edge (u, v) in G.E if v.d <
u.d + w(u, v) return FALSE // ciclo de peso negativo detectado
return TRUE
```

Ejemplo Paso a Paso

Consideremos el siguiente grafo con una arista negativa:

```
[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle,
draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c;
[->] (s) -- node[above left] 4 (a); [->] (s) -- node[below left] 2 (b); [->] (a) --
node[above] 1 (c); [->] (b) -- node[below] 5 (c); [->] (b) -- node[right] -3 (a);
```

Inicialización:

$$d[s] = 0, d[a] = \infty, d[b] = \infty, d[c] = \infty$$

Iteración 1 (relajar todas las aristas):

- $(s, a): d[a] = \min(\infty, 0 + 4) = 4$
- $(s, b): d[b] = \min(\infty, 0 + 2) = 2$
- $(a, c): d[c] = \min(\infty, 4 + 1) = 5$
- $(b, c): d[c] = \min(5, 2 + 5 = 7) = 5$
- $(b, a): d[a] = \min(4, 2 + (-3) = -1) = -1$

Después de iteración 1: $d[a] = -1$, $d[b] = 2$, $d[c] = 5$

Iteración 2:

- $(s, a): d[a] = \min(-1, 0 + 4 = 4) = -1$
- $(s, b): d[b] = \min(2, 0 + 2 = 2) = 2$
- $(a, c): d[c] = \min(5, -1 + 1 = 0) = 0$
- $(b, c): d[c] = \min(0, 2 + 5 = 7) = 0$
- $(b, a): d[a] = \min(-1, 2 + (-3) = -1) = -1$

Después de iteración 2: $d[a] = -1$, $d[b] = 2$, $d[c] = 0$

Iteración 3 ($|V| - 1 = 3$ iteraciones):

- $(a, c): d[c] = \min(0, -1 + 1 = 0) = 0$

Sin cambios.

Verificación de ciclos negativos: Ninguna arista puede relajarse más.

Resultados finales: $d[s] = 0$, $d[a] = -1$, $d[b] = 2$, $d[c] = 0$

Implementación Completa

```
[language=C, caption=Bellman-Ford con detección de ciclos]
typedef struct
int u, v; int weight; Edge;
int bellmanFord(Graph* G, int s, int* dist, int* parent) {
    int V = G->V;
    int E = G->E; Edge* edges = G->edges;
    // Inicialización for (int i = 0; i < V; i++) dist[i] = INF; parent[i] = -1;
    dist[s] = 0;
    // Relajar —V—1 veces for (int i = 1; i < V; i++) {
        int updated = 0;
        for (int j = 0; j < E; j++) {
            int u = edges[j].u; int v = edges[j].v; int w =
            edges[j].weight; if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                parent[v] = u; updated = 1;
            } // Optimización: si no hubo actualizaciones,
            terminar temprano if (!updated) break;
        }
        // Verificar ciclos de peso negativo for (int j = 0; j < E; j++) {
            int u = edges[j].u; int v = edges[j].v; int w = edges[j].weight;
            if (dist[u] != INF && dist[v] > dist[u] + w) return 0; // ciclo negativo detectado
        } return 1; // éxito
    }
```

Análisis de Complejidad

- Tiempo: $O(VE)$
 - Inicialización: $O(V)$
 - $|V| - 1$ iteraciones sobre $|E|$ aristas: $O(VE)$
 - Verificación de ciclos: $O(E)$
- Espacio: $O(V)$ para distancias y padres

Demostración de Correctitud

Bellman-Ford calcula correctamente $\delta(s, v)$ para todo v si no hay ciclos de peso negativo alcanzables desde s . Si existe tal ciclo, devuelve FALSE.

Lema 1: Después de i iteraciones, $d[v]$ es el peso del camino mínimo de s a v con a lo sumo i aristas.

Demostración por inducción en i : Base $i = 0$ trivial. Paso inductivo: la i -ésima iteración relaja todas las aristas, extendiendo caminos de hasta i aristas a caminos de hasta $i + 1$ aristas.

Lema 2: Si no hay ciclos negativos, todo camino mínimo tiene a lo sumo $|V| - 1$ aristas (es simple). Por lo tanto, después de $|V| - 1$ iteraciones, $d[v] = \delta(s, v)$.

Detección de ciclos: Si después de $|V| - 1$ iteraciones aún se puede relajar una arista (u, v) , entonces existe un camino de s a v con más de $|V| - 1$ aristas y peso menor, lo que implica un ciclo negativo.

Detección de Ciclos de Peso Negativo

```
[language=C, caption=Encontrar un ciclo de peso negativo]
int findNegativeCycle(Graph* G, int s, int* cycle) {
    int V = G->V; int* dist = malloc(V * sizeof(int));
    int* parent = malloc(V * sizeof(int));
    // Ejecutar Bellman-Ford
    bellmanFord(G, s, dist, parent);
    // Buscar arista que aún puede relajarse
    for (int j = 0; j < G->E; j++) {
        int u = G->edges[j].u; int v = G->edges[j].v; int w = G->edges[j].weight;
        if (dist[u] != INF && dist[v] > dist[u] + w) {
            // Encontramos un ciclo negativo
            // Reconstruir el ciclo
            int visited[V]; memset(visited, 0, sizeof(visited));
            int node = u; int cycleLen = 0;
            while (!visited[node]) {
                visited[node] = 1;
                cycle[cycleLen++] = node;
                node = parent[node];
            }
            // cycle contiene el ciclo negativo
            return 1;
        }
    }
    return 0;
}
```

Ejemplo de Ciclo de Peso Negativo

```
[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle,
draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c;
[->] (s) -- node[above] 2 (a); [->] (s) -- node[below] 3 (b); [->] (a) --
node[above] 1 (c); [->] (b) -- node[below] 2 (c); [->] (c) -- node[right] -6 (a);
// ciclo negativo: a->c->a tiene peso -5
```

Ciclo $a \rightarrow c \rightarrow a$: $1 + (-6) = -5$. Bellman-Ford detectará que después de 3 iteraciones aún se puede relajar.

Comparación entre Dijkstra y Bellman-Ford

Característica	Dijkstra	Bellman-Ford
Pesos negativos	No	Sí
Detección de ciclos negativos	No	Sí
Complejidad temporal	$O(E \log V)$	$O(VE)$
Complejidad espacial	$O(V)$	$O(V)$
Estructura de datos	Cola de prioridad	Arreglo simple
Tipo de algoritmo	Greedy	Programación dinámica

Cuándo usar cada uno

- **Dijkstra:** Grafos con pesos no negativos, especialmente si son dispersos ($E \ll V^2$). Es más rápido en la práctica.
- **Bellman-Ford:** Grafos con pesos negativos, o cuando se necesita detectar ciclos negativos. También útil para grafos densos donde la implementación simple de Dijkstra ($O(V^2)$) puede ser competitiva.

Aplicaciones Prácticas

Navegación GPS

- Modelar calles como aristas, intersecciones como vértices.
- Dijkstra es ideal porque las distancias son positivas.
- Optimizaciones: A* (heurística) para acelerar.

Enrutamiento en Redes

- Protocolos como OSPF usan Dijkstra.
- RIP usa Bellman-Ford (detección de bucles).

Finanzas y Arbitraje

- Detección de oportunidades de arbitraje en mercados de divisas.
- Modelar tipos de cambio como logaritmos negativos.
- Ciclo de peso negativo indica oportunidad de arbitraje.

[Arbitraje de divisas] Si $1USD = 0.85EUR$, $1EUR = 1.20CHF$, $1CHF = 1.05USD$, entonces:

$$0.85 \times 1.20 \times 1.05 \approx 1.071 > 1$$

Hay arbitraje. Bellman-Ford puede detectarlo.

Ejercicios Resueltos (20 Ejercicios)

Ejecute Dijkstra desde el vértice s en el grafo con aristas: $(s, a, 5)$, $(s, b, 3)$, $(a, c, 2)$, $(b, c, 1)$, $(c, d, 4)$, $(b, d, 6)$.

1. Inicial: $d[s] = 0$, $d[a] = \infty$, $d[b] = \infty$, $d[c] = \infty$, $d[d] = \infty$
2. Extraer s : actualizar $d[a] = 5$, $d[b] = 3$
3. Extraer b : actualizar $d[c] = \min(\infty, 3+1 = 4)$, $d[d] = \min(\infty, 3+6 = 9)$
4. Extraer c : actualizar $d[d] = \min(9, 4+4 = 8)$
5. Extraer a : actualizar $d[c] = \min(4, 5+2 = 7)$ sigue 4
6. Extraer d : fin

Resultados: $d[s] = 0$, $d[a] = 5$, $d[b] = 3$, $d[c] = 4$, $d[d] = 8$.

Ejecute Bellman-Ford en el mismo grafo.

1. Inicial: $d[s] = 0$, otros ∞
2. Iteración 1: $d[a] = 5$, $d[b] = 3$, $d[c] = 4$ (vía b), $d[d] = 9$ (vía b)
3. Iteración 2: $d[d] = \min(9, 4+4 = 8)$

4. Iteración 3: sin cambios

Mismos resultados.

Demuestre que Dijkstra falla si hay una arista negativa. Contraejemplo:

```
[scale=0.8] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle,
draw] (b) at (2,-1) b; [circle, draw] (t) at (4,0) t;
[->] (s) -- node[above] 3 (a); [->] (s) -- node[below] 4 (b); [->] (a) --
node[above] 1 (t); [->] (b) -- node[below] -2 (t);
```

Dijkstra extrae s , luego a (3), actualiza t a 4, luego extrae b (4), pero t ya está extraído con valor 4 cuando el camino $s \rightarrow b \rightarrow t$ pesa $4 + (-2) = 2$, que es menor.

¿Qué hace Bellman-Ford si detecta un ciclo de peso negativo? Si Bellman-Ford detecta un ciclo de peso negativo, significa que existen caminos de peso arbitrariamente pequeño (hacia $-\infty$) para los vértices alcanzables desde ese ciclo. El algoritmo retorna FALSE, indicando que no se pueden calcular distancias finitas.

En aplicaciones como arbitraje financiero, esto indica una oportunidad de ganancia infinita.

Implemente Dijkstra para un grafo representado con matriz de adyacencia.

```
void dijkstraMatrix(int** graph, int V, int s) { int* dist = malloc(V * sizeof(int));
int* visited = malloc(V * sizeof(int));
```

```
for (int i = 0; i < V; i++) dist[i] = INF; visited[i] = 0; dist[s] = 0;
```

```
for (int count = 0; count < V - 1; count++) // Encontrar vértice no
visitado con distancia mínima int u = -1; int minDist = INF; for (int i = 0;
i < V; i++) if (!visited[i] && dist[i] < minDist) minDist = dist[i]; u = i; if (u
== -1) break; visited[u] = 1;
```

```
// Actualizar distancias de los vecinos for (int v = 0; v < V; v++) if
(graph[u][v] && !visited[v] && dist[v] > dist[u] + graph[u][v]) dist[v] = dist[u] +
graph[u][v]; Complejidad:  $O(V^2)$ .
```

Ejecute Bellman-Ford en un grafo con ciclo negativo. Grafo: $s \rightarrow a$ peso 1, $a \rightarrow b$ peso 1, $b \rightarrow a$ peso -3.

1. Inicial: $d[s] = 0$, $d[a] = \infty$, $d[b] = \infty$
2. Iteración 1: $d[a] = 1$, $d[b] = 2$
3. Iteración 2: $d[a] = \min(1, 2 + (-3)) = -1$, $d[b] = \min(2, -1 + 1 = 0) = 0$
4. Iteración 3: $d[a] = \min(-1, 0 + (-3)) = -3$, $d[b] = \min(0, -3 + 1 = -2) = -2$

Después de 3 iteraciones, aún se puede relajar. El algoritmo detecta ciclo negativo y retorna FALSE.

¿Por qué Dijkstra requiere que los pesos sean no negativos? La corrección de Dijkstra se basa en que cuando un vértice es extraído de la cola, su distancia es definitiva. Con pesos negativos, un camino más largo (en número de aristas) podría tener menor peso total, y ese camino podría llegar a un vértice después de que ya fue extraído.

Formalmente, la desigualdad $d[u] \leq d[x] + w(x, u)$ no se garantiza si hay pesos negativos, porque $w(x, u)$ podría ser negativo y hacer que $d[u] > d[x] + w(x, u)$ incluso si x aún no fue procesado.

Demuestre que Bellman-Ford puede optimizarse para terminar antes si no hay cambios en una iteración. Si en una iteración completa ninguna arista provoca una relajación, entonces $d[v]$ ya es $\delta(s, v)$ para todo v , y las iteraciones restantes no cambiarán nada. Esta optimización es especialmente útil cuando el grafo tiene caminos cortos.

En el peor caso, igual se requieren $|V| - 1$ iteraciones, pero en la práctica puede ser mucho más rápido.

Aplique Dijkstra al grafo con aristas: $(1, 2, 4)$, $(1, 3, 2)$, $(2, 3, 1)$, $(2, 4, 5)$, $(3, 4, 8)$, $(3, 5, 10)$, $(4, 5, 2)$.

1. $d[1] = 0$, otros ∞
2. Extraer 1: $d[2] = 4$, $d[3] = 2$
3. Extraer 3: $d[2] = \min(4, 2+1 = 3)$, $d[4] = 2+8 = 10$, $d[5] = 2+10 = 12$
4. Extraer 2: $d[4] = \min(10, 3+5 = 8)$, $d[5] = \min(12, 3+?)$ sin arista
5. Extraer 4: $d[5] = \min(12, 8+2 = 10)$
6. Extraer 5: fin

Resultados: $d[1] = 0$, $d[2] = 3$, $d[3] = 2$, $d[4] = 8$, $d[5] = 10$.

¿Cómo se puede usar Bellman-Ford para encontrar el camino mínimo en un grafo con pesos negativos pero sin ciclos negativos? Simplemente ejecutamos Bellman-Ford normalmente. Después de $|V| - 1$ iteraciones, $d[v]$ contiene $\delta(s, v)$. El algoritmo retorna TRUE indicando que no hay ciclos negativos. Luego se pueden reconstruir los caminos usando los punteros parent.

Es la solución estándar para grafos con pesos negativos, como los que aparecen en problemas de arbitraje financiero.

Ejercicios Propuestos (20 Ejercicios)

1. Ejecute Dijkstra desde el vértice A en un grafo de 6 vértices.
2. Demuestre que Dijkstra es correcto para pesos no negativos por inducción.
3. Ejecute Bellman-Ford en un grafo con 4 vértices y una arista negativa.
4. Implemente Dijkstra con montículo de Fibonacci.
5. ¿Qué es la optimización de cola de prioridad en Dijkstra?
6. Encuentre un ciclo de peso negativo en un grafo dado.
7. Demuestre que Bellman-Ford detecta ciclos negativos correctamente.
8. ¿Cómo se puede usar Bellman-Ford para encontrar el camino mínimo con restricciones de número de aristas?
9. Compare la complejidad de Dijkstra y Bellman-Ford en grafos densos.
10. Implemente la detección de ciclos negativos con reconstrucción del ciclo.
11. Ejecute Dijkstra en un grafo con pesos negativos y muestre el error.
12. ¿Qué es el algoritmo de D'Esopo-Pape y cuándo se usa?
13. Demuestre que si no hay ciclos negativos, Bellman-Ford termina en $O(VE)$.
14. Aplique Bellman-Ford a un grafo lineal de 5 vértices.
15. ¿Cómo se puede paralelizar Dijkstra?
16. ¿Qué es el problema de caminos mínimos con restricciones de recursos?
17. Implemente Bellman-Ford con detección temprana de convergencia.
18. Ejecute ambos algoritmos en un grafo aleatorio y compare tiempos.
19. ¿Qué es el algoritmo de Johnson para grafos dispersos con pesos negativos?
20. Investigue aplicaciones de Dijkstra en sistemas de navegación.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El problema de caminos mínimos desde una fuente única (SSSP)
- El algoritmo de Dijkstra: greedy, requiere pesos no negativos, $O(E \log V)$
- El algoritmo de Bellman-Ford: programación dinámica, permite pesos negativos, $O(VE)$
- Cómo detectar ciclos de peso negativo con Bellman-Ford
- Aplicaciones prácticas en navegación, redes y finanzas

Bibliografía de la Unidad

- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87-90.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 24.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271.
- Ford, L. R., & Fulkerson, D. R. (1962). *Flows in Networks*. Princeton University Press.

Unidad VI: Algoritmo de Floyd-Warshall - Caminos Mínimos entre Todos los Pares

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de caminos mínimos entre todos los pares de vértices (APSP).
- Implementar y analizar el algoritmo de Floyd-Warshall.
- Aplicar Floyd-Warshall a grafos con pesos positivos y negativos (sin ciclos negativos).
- Detectar ciclos de peso negativo utilizando Floyd-Warshall.
- Adaptar Floyd-Warshall para calcular el cierre transitivo de un grafo.
- Comparar Floyd-Warshall con otras soluciones al problema APSP.

Introducción al Problema APSP

Definición Formal

Dado un grafo dirigido ponderado $G = (V, E)$ con función de peso $w : E \rightarrow R$, encontrar para todo par de vértices (i, j) la distancia mínima $\delta(i, j)$.

Enfoques para APSP

- **Fuerza bruta:** Ejecutar un algoritmo de fuente única desde cada vértice.

- Con Dijkstra (pesos no negativos): $O(V \cdot E \log V) = O(V^3 \log V)$ en grafos densos
- Con Bellman-Ford: $O(V^2 E) = O(V^4)$ en grafos densos
- **Floyd-Warshall:** $O(V^3)$ tiempo, $O(V^2)$ espacio.
- **Johnson:** $O(VE \log V)$ para grafos dispersos.

Representación Matricial

El grafo se representa mediante una matriz de adyacencia W de tamaño $n \times n$:

$$W[i][j] = \begin{cases} 0 & \text{si } i = j \\ w(i, j) & \text{si } (i, j) \in E \\ \infty & \text{si } (i, j) \notin E \end{cases}$$

Algoritmo de Floyd-Warshall

Descripción General

Floyd-Warshall utiliza programación dinámica. Sea $d^{(k)}[i][j]$ la distancia mínima de i a j usando solo vértices intermedios del conjunto $\{1, 2, \dots, k\}$.

Ecuación de Recurrencia

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$$

con caso base $d^{(0)}[i][j] = W[i][j]$.

```
[scale=0.7] [circle, draw] (i) at (0,0) i; [circle, draw] (j) at (4,0) j; [circle,
draw] (k) at (2,1.5) k;
[->, thick] (i) -- node[below] camino directo (j); [->, thick, blue] (i) --
node[left] i → k (k); [->, thick, blue] (k) -- node[right] k → j (j);
```

Pseudocódigo

```
[language=C, caption=Algoritmo de Floyd-Warshall] FLOYD-WARSHALL(W,
n) // Inicialización D = W // copia de la matriz de pesos
for i = 1 to n D[i][i] = 0
// Algoritmo principal
for k = 1 to n
  for i = 1 to n
    for j = 1 to n
      if D[i][k] + D[k][j] < D[i][j]
        D[i][j] = D[i][k] + D[k][j]
  next i[j] = k // para reconstrucción
return D
```

Ejemplo Paso a Paso

Consideremos el siguiente grafo:

[scale=0.8] [circle, draw] (1) at (0,0) 1; [circle, draw] (2) at (2,1.5) 2; [circle, draw] (3) at (2,-1.5) 3; [circle, draw] (4) at (4,0) 4;
 [->] (1) -- node[above] 3 (2); [->] (1) -- node[below] 8 (3); [->] (2) -- node[above] 2 (4); [->] (3) -- node[below] 1 (4); [->] (2) -- node[right] 5 (3);

Matriz inicial $D^{(0)}$:

$$(0) \begin{matrix} & 1 & 2 & 3 & 4 \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & 0 & \infty & 8 & \infty \\ & \infty & 0 & 1 & \infty \\ & \infty & \infty & 0 & 5 \\ & \infty & \infty & \infty & 0 \end{matrix}$$

Para $k = 1$: Usamos el vértice 1 como intermedio.

- $i = 2, j = 3$: $D[2][1] + D[1][3] = \infty + 8 = \infty$, no cambia
- $i = 2, j = 4$: $\infty + \infty = \infty$, no cambia
- $i = 3, j = 2$: $\infty + \infty = \infty$, no cambia
- $i = 3, j = 4$: $\infty + \infty = \infty$, no cambia

$$D^{(1)} = D^{(0)}$$

Para $k = 2$: Usamos el vértice 2 como intermedio.

- $i = 1, j = 3$: $D[1][2] + D[2][3] = 3 + 5 = 8$, no mejora (8 vs 8)
- $i = 1, j = 4$: $D[1][2] + D[2][4] = 3 + 2 = 5$, mejora ($\infty \rightarrow 5$)
- $i = 3, j = 1$: $D[3][2] + D[2][1] = \infty + \infty = \infty$, no cambia
- $i = 3, j = 4$: $D[3][2] + D[2][4] = \infty + 2 = \infty$, no cambia

$$D^{(2)} = (0) \begin{matrix} & 1 & 2 & 3 & 4 \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & 0 & \infty & 8 & 5 \\ & \infty & 0 & 1 & \infty \\ & \infty & \infty & 0 & 5 \\ & \infty & \infty & \infty & 0 \end{matrix}$$

Para $k = 3$: Usamos el vértice 3 como intermedio.

- $i = 1, j = 4$: $D[1][3] + D[3][4] = 8 + 1 = 9$, no mejora (5 vs 9)
- $i = 2, j = 4$: $D[2][3] + D[3][4] = 5 + 1 = 6$, no mejora (2 vs 6)
- $i = 1, j = 2$: $D[1][3] + D[3][2] = 8 + \infty = \infty$, no cambia

$$D^{(3)} = D^{(2)}$$

Para $k = 4$: Usamos el vértice 4 como intermedio.

- $i = 1, j = 2$: $D[1][4] + D[4][2] = 5 + \infty = \infty$
- $i = 1, j = 3$: $D[1][4] + D[4][3] = 5 + \infty = \infty$
- $i = 2, j = 1$: $D[2][4] + D[4][1] = 2 + \infty = \infty$
- $i = 2, j = 3$: $D[2][4] + D[4][3] = 2 + \infty = \infty$
- $i = 3, j = 1$: $D[3][4] + D[4][1] = 1 + \infty = \infty$
- $i = 3, j = 2$: $D[3][4] + D[4][2] = 1 + \infty = \infty$

Matriz final:

$$D = \begin{pmatrix} 0 & 3 & 8 & 5 \\ \infty & 0 & 5 & 2 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{pmatrix}$$

Implementación Completa

```
[language=C, caption=Implementación de Floyd-Warshall] include <stdio.h>
include <limits.h>
define INF INT_MAX
define MAX 100
void floydWarshall(int graph[MAX][MAX], int n) {
    int dist[MAX][MAX];
    int next[MAX][MAX];
    // Inicializar matrices
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            dist[i][j] = graph[i][j];
    if (graph[i][j] != INF && i != j) next[i][j] = j;
    else next[i][j] = -1;
    dist[i][i] = 0;
    // Algoritmo principal
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] != INF && dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
                next[i][j] = next[i][k];
    // Detectar ciclos de peso negativo
    for (int i = 0; i < n; i++)
        if (dist[i][i] < 0)
            printf("Ciclo de peso negativo detectado");
            return;
    // Imprimir resultados
    printf("Matriz de distancias minimas:");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (dist[i][j] == INF) printf("INF");
            else printf(" %d", dist[i][j]);
    // Reconstrucción de caminos
    void printPath(int next[MAX][MAX], int i, int j) {
        if (next[i][j] == -1) printf("No hay camino");
        return;
        printf("while (i != j) { i = next[i][j]; printf(" %d", i); }");
    }
}
```

Reconstrucción de Caminos

Para reconstruir el camino de i a j , se mantiene una matriz $next[i][j]$ que indica el siguiente vértice en el camino.

```
[language=C, caption=Reconstrucción del camino] void printShortest-
Path(int next[MAX][MAX], int i, int j) if (next[i][j] == -1) printf("No
existe camino de return;
printf("Camino de int u = i; while (u != j) u = next[u][j]; printf(" -j
printf("");
```

Análisis de Complejidad

Tiempo

- Tres bucles anidados: $O(V^3)$
- Cada iteración realiza una suma y una comparación: $O(1)$
- Total: $O(V^3)$

Espacio

- Matriz de distancias: $O(V^2)$
- Matriz de siguiente vértice (opcional): $O(V^2)$
- Total: $O(V^2)$

Comparación con Alternativas

Algoritmo	Tiempo	Espacio
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Dijkstra (desde cada vértice)	$O(VE \log V)$	$O(V + E)$
Bellman-Ford (desde cada vértice)	$O(V^2E)$	$O(V)$
Johnson	$O(VE \log V)$	$O(V + E)$

Detección de Ciclos de Peso Negativo

Método

Floyd-Warshall puede detectar ciclos de peso negativo verificando la diagonal de la matriz de distancias después de ejecutar el algoritmo:

Si $D[i][i] < 0$ para algún i , existe un ciclo de peso negativo.

```
[language=C, caption=Detección de ciclos negativos] int detectNegative-
Cycle(int dist[MAX][MAX], int n) for (int i = 0; i < n; i++) if (dist[i][i] < 0)
printf("Ciclo de peso negativo encontrado en el vertice return 1; return 0;
```


Ejemplo de Ciclo Negativo

```
[scale=0.8] [circle, draw] (1) at (0,0) 1; [circle, draw] (2) at (2,1) 2; [circle,
draw] (3) at (2,-1) 3;
[->] (1) -- node[above] 1 (2); [->] (2) -- node[right] 1 (3); [->] (3) --
node[below] -3 (1);
```

Ciclo $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ tiene peso $1 + 1 + (-3) = -1$.

Después de Floyd-Warshall, $D[1][1]$ será -1 , detectando el ciclo.

Cierre Transitivo

Definición

El cierre transitivo de un grafo dirigido es una matriz T donde $T[i][j] = 1$ si existe un camino de i a j , y 0 en caso contrario.

Algoritmo

```
[language=C, caption=Cierre transitivo con Floyd-Warshall] void transitive-
Closure(int graph[MAX][MAX], int n) { int reach[MAX][MAX];
// Inicializar for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) reach[i][j]
= (i == j) ? graph[i][j] : 0;
// Algoritmo de Floyd-Warshall para cierre transitivo for (int k = 0; k < n; k++)
for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) reach[i][j] =
reach[i][j] || (reach[i][k] & reach[k][j]);
// Imprimir matriz de alcanzabilidad printf("Matriz de cierre transi-
tivo:"); for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) printf("
printf(" "); }
```

Ejemplo

```
[scale=0.8] [circle, draw] (1) at (0,0) 1; [circle, draw] (2) at (2,0) 2; [circle,
draw] (3) at (4,0) 3; [circle, draw] (4) at (6,0) 4;
[->] (1) --> (2); [->] (2) --> (3); [->] (3) --> (4); [->] (4) --> (2);
```

Cierre transitivo:

```
( 1 ) 111011101110111
```

Variantes y Optimizaciones

Floyd-Warshall para Matrices de Distancias con Pesos Grandes

```
[language=C, caption=Manejo de INF con prevención de overflow] for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) if (dist[i][k] == INF) continue;
for (int j = 0; j < n; j++) if (dist[k][j] == INF) continue; if (dist[i][k] + dist[k][j] < dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j];
```

Floyd-Warshall con Registro de Caminos

```
[language=C, caption=Versión con reconstrucción de caminos] void floyd-WarshallWithPath(int graph[MAX][MAX], int n) int dist[MAX][MAX]; int path[MAX][MAX];
// Inicialización for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) dist[i][j] = graph[i][j]; path[i][j] = (graph[i][j] != INF && i != j) ? j : -1; dist[i][i] = 0; path[i][i] = i;
// Algoritmo principal for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) if (dist[i][k] != INF && dist[k][j] != INF && dist[i][k] + dist[k][j] < dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j]; path[i][j] = path[i][k];
// Reconstrucción de caminos for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) if (i != j && dist[i][j] != INF) printf("Camino int u = i; while (u != j) u = path[u][j]; printf(" -> printf(" (distancia
```

Ejercicios Resueltos (20 Ejercicios)

Ejecute Floyd-Warshall en el grafo con 3 vértices: aristas (1, 2, 4), (1, 3, 11), (2, 3, 2), (3, 1, 3). Matriz inicial:

$$D^{(0)} = \begin{pmatrix} 0 & 4 & 11 \\ 1 & 3 & 2 \\ 3 & 1 & 3 \end{pmatrix}$$

$k = 1$:

- (2, 3): $3 + 11 = 14$ vs 2, sigue 2
- (3, 2): $\infty + 4 = \infty$, no cambia

$k = 2$:

- (1, 3): $4 + 2 = 6$ vs 11, mejora a 6

- $(3, 1)$: $\infty + 3 = \infty$, no cambia
- $(3, 3)$: $\infty + 2 = \infty$, no cambia

$$D^{(2)} = \begin{pmatrix} 0 & 4 & 6 & 3 & 0 & 2 & \infty & \infty & 0 \end{pmatrix}$$

$k = 3$:

- $(1, 2)$: $6 + \infty = \infty$, no cambia
- $(2, 1)$: $2 + \infty = \infty$, no cambia
- $(2, 2)$: $2 + \infty = \infty$, no cambia

Matriz final:

$$D = \begin{pmatrix} 0 & 4 & 6 & 3 & 0 & 2 & \infty & \infty & 0 \end{pmatrix}$$

Demuestre que Floyd-Warshall calcula correctamente las distancias mínimas. Por inducción en k . La hipótesis inductiva es que después de la iteración k , $D[i][j]$ es la distancia mínima de i a j usando solo vértices intermedios en $\{1, \dots, k\}$.

Caso base: $k = 0$, $D[i][j] = W[i][j]$, que es la distancia sin vértices intermedios (arista directa o infinito).

Paso inductivo: Supongamos cierto para $k - 1$. Para k , el camino mínimo de i a j usando vértices $\{1, \dots, k\}$ o bien no usa k (entonces es el de $k - 1$) o bien usa k (entonces se descompone en $i \rightarrow k$ y $k \rightarrow j$). La recurrencia captura exactamente esto.

Al final, $k = n$, todos los vértices son intermedios permitidos, por lo que $D[i][j] = \delta(i, j)$.

¿Cómo detecta Floyd-Warshall ciclos de peso negativo? Si existe un ciclo de peso negativo, entonces se puede mejorar la distancia de un vértice a sí mismo. Inicialmente $D[i][i] = 0$. Si hay un ciclo negativo, al usar los vértices del ciclo como intermedios, $D[i][i]$ se volverá negativo. Por lo tanto, después del algoritmo, si algún $D[i][i] < 0$, hay un ciclo negativo.

Esto también implica que no se pueden calcular distancias finitas si hay ciclos negativos alcanzables.

Aplique Floyd-Warshall para encontrar el cierre transitivo del grafo con aristas $(1, 2)$, $(2, 3)$, $(3, 1)$, $(3, 4)$. Matriz de adyacencia inicial:

$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

$$k = 1: T[3][2] = T[3][2] \vee (T[3][1] \wedge T[1][2]) = 0 \vee (1 \wedge 1) = 1$$

$k = 2$: $T[1][3] = 0 \vee (1 \wedge 1) = 1$, $T[3][1] = 1 \vee (1 \wedge 0) = 1$, $T[3][4] = 1 \vee (1 \wedge 0) = 1$

$k = 3$: $T[1][4] = 0 \vee (1 \wedge 1) = 1$, $T[2][4] = 0 \vee (1 \wedge 1) = 1$

Matriz final (todos alcanzables desde 1,2,3):

$$T = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Compare la complejidad de Floyd-Warshall con ejecutar Dijkstra V veces.

- Floyd-Warshall: $O(V^3)$ siempre.
- Dijkstra V veces:
 - Con montículo binario: $O(V \cdot E \log V)$
 - En grafos densos ($E \approx V^2$): $O(V^3 \log V)$
 - En grafos dispersos ($E \approx V$): $O(V^2 \log V)$

Floyd-Warshall es competitivo en grafos densos y es más simple de implementar. Para grafos dispersos, Dijkstra V veces o Johnson pueden ser mejores.

¿Qué sucede si el grafo tiene un ciclo de peso negativo no alcanzable desde algunos vértices? Si el ciclo negativo no es alcanzable desde i , entonces $D[i][j]$ sigue siendo finito para los j alcanzables. La detección de ciclos negativos en $D[i][i]$ solo detecta ciclos alcanzables desde i . Para una detección global, se debe verificar la diagonal de toda la matriz.

En la práctica, si hay algún ciclo negativo en cualquier parte del grafo, las distancias no están bien definidas para algunos pares.

Implemente Floyd-Warshall para un grafo representado con lista de adyacencia. `void floydWarshallFromAdjList(Graph* G) int n = G->V; int** dist = malloc(n * sizeof(int*)); for (int i = 0; i < n; i++) dist[i] = malloc(n * sizeof(int)); for (int j = 0; j < n; j++) dist[i][j] = (i == j) ? 0 : INF;`

`// Convertir lista de adyacencia a matriz for (int u = 0; u < n; u++) Node* adj = G->adj[u]; while (adj != NULL) int v = adj->vertex; int w = adj->weight; dist[u][v] = w; adj = adj->next;`

`// Floyd-Warshall for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) if (dist[i][k] != INF & dist[k][j] != INF dist[i][k] + dist[k][j] < dist[i][j]) dist[i][j] = dist[i][k] + dist[k][j];`

¿Cómo se puede modificar Floyd-Warshall para encontrar el camino más largo (en lugar del más corto)? Para encontrar caminos más largos en un grafo sin ciclos positivos (DAG), se puede cambiar min por max e inicializar con $-\infty$ en lugar de ∞ :

```

    for (int k = 0; k < n; k++) for (int i = 0; i < n; i++) for (int j = 0; j < n;
j++) if (dist[i][k] != NEG_INFdist[k][j] != NEG_INFdist[i][k] + dist[k][j] >
dist[i][j])dist[i][j] = dist[i][k] + dist[k][j];

```

Si hay ciclos positivos, el problema es NP-hard.

Aplique Floyd-Warshall al grafo completo de 3 vértices con pesos: (1, 2, 1), (2, 3, 2), (3, 1, 3), (2, 1, 4), (3, 2, 5), (1, 3, 6). Matriz inicial:

$$D^{(0)} = \begin{pmatrix} 0 & 1 & 6 \\ 4 & 0 & 2 \\ 3 & 5 & 0 \end{pmatrix}$$

$k = 1$:

- (2, 3): $4 + 6 = 10$ vs 2, sigue 2
- (3, 2): $3 + 1 = 4$ vs 5, mejora a 4

$$D^{(1)} = \begin{pmatrix} 0 & 1 & 6 \\ 4 & 0 & 2 \\ 3 & 4 & 0 \end{pmatrix}$$

$k = 2$:

- (1, 3): $1 + 2 = 3$ vs 6, mejora a 3
- (3, 1): $4 + 4 = 8$ vs 3, sigue 3

$$D^{(2)} = \begin{pmatrix} 0 & 1 & 3 \\ 4 & 0 & 2 \\ 3 & 4 & 0 \end{pmatrix}$$

$k = 3$:

- (1, 2): $3 + 4 = 7$ vs 1, sigue 1
- (2, 1): $2 + 3 = 5$ vs 4, sigue 4

Matriz final:

$$D = \begin{pmatrix} 0 & 1 & 3 \\ 4 & 0 & 2 \\ 3 & 4 & 0 \end{pmatrix}$$

Ejercicios Propuestos (20 Ejercicios)

1. Ejecute Floyd-Warshall en un grafo de 4 vértices con aristas (1, 2, 2), (1, 3, 6), (2, 3, 3), (2, 4, 1), (3, 4, 4).
2. Demuestre la corrección de Floyd-Warshall por inducción en k .
3. Implemente Floyd-Warshall con detección de ciclos negativos.

4. ¿Cómo se puede usar Floyd-Warshall para encontrar el diámetro de un grafo?
5. Aplique Floyd-Warshall a un grafo con un ciclo de peso negativo y detecte el ciclo.
6. Compare Floyd-Warshall con Johnson para grafos dispersos.
7. Implemente Floyd-Warshall para calcular el cierre transitivo.
8. ¿Qué es la desigualdad triangular y cómo la usa Floyd-Warshall?
9. Demuestre que si no hay ciclos negativos, Floyd-Warshall termina con distancias correctas.
10. Aplique Floyd-Warshall a un grafo de 5 vértices en forma de estrella.
11. ¿Cómo se puede paralelizar Floyd-Warshall?
12. Implemente Floyd-Warshall con reconstrucción de caminos.
13. ¿Qué es el problema de la ruta más confiable y cómo se adapta Floyd-Warshall?
14. Aplique Floyd-Warshall a un grafo donde todas las aristas tienen peso 1.
15. ¿Cómo se puede usar Floyd-Warshall para encontrar la cerradura transitiva de un DAG?
16. Implemente una versión optimizada que use solo una matriz.
17. ¿Qué es el algoritmo de Warshall y cómo se relaciona con Floyd-Warshall?
18. Aplique Floyd-Warshall a un grafo bipartito completo.
19. Demuestre que Floyd-Warshall funciona con pesos negativos si no hay ciclos negativos.
20. Investigue aplicaciones de Floyd-Warshall en análisis de redes sociales.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El problema de caminos mínimos entre todos los pares (APSP)
- El algoritmo de Floyd-Warshall: programación dinámica, $O(V^3)$ tiempo, $O(V^2)$ espacio
- La recurrencia $d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$
- Cómo detectar ciclos de peso negativo
- Adaptación para calcular el cierre transitivo
- Comparación con otras soluciones al problema APSP

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 25.
- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.
- Warshall, S. (1962). A theorem on Boolean matrices. *Journal of the ACM*, 9(1), 11-12.

Unidad VII: Flujo en Redes - Teorema de Flujo Máximo - Corte Mínimo

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de flujo máximo en redes.
- Definir formalmente redes de flujo, cortes y flujos.
- Comprender y demostrar el Teorema de Flujo Máximo - Corte Mínimo.
- Implementar y analizar el algoritmo de Ford-Fulkerson.
- Implementar la variante de Edmonds-Karp usando BFS.
- Aplicar flujo máximo a problemas como matching bipartito y particionamiento de redes.

Introducción al Problema de Flujo Máximo

Definición de Red de Flujo

Una **red de flujo** $G = (V, E)$ es un grafo dirigido con las siguientes propiedades:

- Una **fuentes** $s \in V$ (solo aristas salientes).
- Un **sumidero** $t \in V$ (solo aristas entrantes).
- Cada arista $(u, v) \in E$ tiene una **capacidad** $c(u, v) \geq 0$.

[scale=0.7] [circle, draw] (s) at (0,0) s; [circle, draw] (a) at (2,1) a; [circle, draw] (b) at (2,-1) b; [circle, draw] (c) at (4,0) c; [circle, draw] (t) at (6,0) t;
 [-.] (s) -- node[above] 16 (a); [-.] (s) -- node[below] 13 (b); [-.] (a) -- node[above] 12 (c); [-.] (b) -- node[below] 14 (c); [-.] (c) -- node[above] 20 (t); [-.] (a) -- node[right] 4 (b); [-.] (b) -- node[left] 9 (a);

Definición de Flujo

Un **flujo** es una función $f : V \times V \rightarrow R$ que satisface:

1. **Restricción de capacidad:** $0 \leq f(u, v) \leq c(u, v)$ para todo $u, v \in V$.
2. **Conservación del flujo:** Para todo $u \in V \setminus \{s, t\}$:

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0$$

3. **Antisimetría:** $f(u, v) = -f(v, u)$.

El **valor del flujo** es:

$$|f| = \sum_{v \in V} f(s, v) = \sum_{v \in V} f(v, t)$$

Interpretación Física

- Las aristas son tuberías con capacidad máxima.
- El flujo es la cantidad de material que viaja por la tubería.
- La fuente produce material, el sumidero lo consume.
- En los nodos intermedios, lo que entra debe salir.

Cortes en Redes de Flujo

Definición de Corte

Un **corte** (S, T) es una partición de V tal que $s \in S$ y $t \in T$.

La **capacidad del corte** es:

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

El **flujo a través del corte** es:

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u)$$

[scale=0.7] [thick, dashed] (1.5,-1.5) -- (1.5,2); at (0.5,1.5) S ; at (3,1.5) T ;
 [circle, draw] (s) at (0,0) s ; [circle, draw] (a) at (1,1) a ; [circle, draw] (b) at
 (3,1) b ; [circle, draw] (c) at (2,-1) c ; [circle, draw] (t) at (4,0) t ;
 [->] (s) -- (a); [->] (a) -- (b); [->] (b) -- (t); [->] (s) -- (c); [->] (c) -- (t);

Propiedades de los Cortes

Para cualquier flujo f y cualquier corte (S, T) :

$$|f| = f(S, T)$$

Por conservación del flujo, la suma de flujos que salen de S menos los que entran es igual al flujo neto de s a t .

Para cualquier flujo f y cualquier corte (S, T) :

$$|f| \leq c(S, T)$$

Teorema de Flujo Máximo - Corte Mínimo

[Flujo Máximo - Corte Mínimo] En cualquier red de flujo, el valor máximo de un flujo es igual a la capacidad mínima de un corte.

$$\max_f |f| = \min_{(S,T)} c(S, T)$$

[Esbozo]

1. Si f es un flujo máximo, no hay caminos de aumento en la red residual.
2. Sea S el conjunto de vértices alcanzables desde s en la red residual.
3. Entonces $t \notin S$, por lo que (S, T) es un corte.
4. Para toda arista (u, v) con $u \in S$, $v \in T$, se tiene $f(u, v) = c(u, v)$ (de lo contrario (u, v) estaría en la red residual).
5. Por lo tanto, $|f| = f(S, T) = c(S, T)$.

Red Residual y Caminos de Aumento

Definición de Red Residual

Dado un flujo f , la **red residual** G_f tiene las mismas aristas más las aristas inversas:

$$c_f(u, v) = c(u, v) - f(u, v)$$

si $(u, v) \in E$, y $c_f(v, u) = f(u, v)$.

[scale=0.7] at (0,1) Original:; [circle, draw] (u) at (0,0) u ; [circle, draw] (v) at (3,0) v ; [->] (u) -- node[above] 10 (v);
 at (6,1) Con flujo 5:; [circle, draw] (u2) at (6,0) u ; [circle, draw] (v2) at (9,0) v ; [->] (u2) -- node[above] 5 (v2); [->] (v2) -- node[below] 5 (u2);

Camino de Aumento

Un **camino de aumento** es un camino desde s hasta t en la red residual G_f .

La **capacidad residual del camino** es:

$$c_f(p) = \min_{(u,v) \in p} c_f(u, v)$$

Aumento del Flujo

Si encontramos un camino de aumento p , podemos aumentar el flujo:

$$f'(u, v) = f(u, v) + c_f(p) \text{ para } (u, v) \in p$$

$$f'(v, u) = f(v, u) - c_f(p) \text{ para } (v, u) \in p$$

Algoritmo de Ford-Fulkerson

Pseudocódigo

```
[language=C, caption=Algoritmo de Ford-Fulkerson] FORD-FULKERSON(G,
s, t) for each edge (u,v) in G.E f(u,v) = 0 f(v,u) = 0
    while exists path p from s to t in  $G_f$   $c_f(p) = \min c_f(u, v) : (u, v) \text{ in } p$  for each edge (u,v) in p  $f(u, v) + c_f(p)$   $f(v, u) = f(v, u) - c_f(p)$ 
    return f
```

Ejemplo Paso a Paso

Consideremos la siguiente red:

[scale=0.7] [circle, draw] (s) at (0,0) s ; [circle, draw] (a) at (2,1) a ; [circle, draw] (b) at (2,-1) b ; [circle, draw] (t) at (4,0) t ;
 [->] (s) -- node[above] 16 (a); [->] (s) -- node[below] 13 (b); [->] (a) -- node[above] 12 (t); [->] (b) -- node[below] 14 (t); [->] (a) -- node[right] 4 (b);
 [->] (b) -- node[left] 9 (a);

Paso 1: Inicializar flujo $f = 0$.

Paso 2: Encontrar camino de aumento. Por ejemplo, $p_1 = s \rightarrow a \rightarrow t$.

- Capacidades residuales: $c_f(s, a) = 16$, $c_f(a, t) = 12$, $c_f(p_1) = 12$
- Actualizar: $f(s, a) = 12$, $f(a, t) = 12$, $f(a, s) = -12$, $f(t, a) = -12$

Flujo actual: $|f| = 12$.

Paso 3: Otro camino, $p_2 = s \rightarrow b \rightarrow t$.

- Capacidades residuales: $c_f(s, b) = 13$, $c_f(b, t) = 14$, $c_f(p_2) = 13$
- Actualizar: $f(s, b) = 13$, $f(b, t) = 13$, $f(b, s) = -13$, $f(t, b) = -13$

Flujo actual: $|f| = 12 + 13 = 25$.

Paso 4: Otro camino, $p_3 = s \rightarrow a \rightarrow b \rightarrow t$.

- Capacidades residuales actuales: $c_f(s, a) = 16 - 12 = 4$, $c_f(a, b) = 4$ (original $a \rightarrow b$), $c_f(b, t) = 14 - 13 = 1$
- $c_f(p_3) = \min(4, 4, 1) = 1$
- Actualizar: $f(s, a) = 13$, $f(a, b) = 1$, $f(b, t) = 14$

Flujo actual: $|f| = 25 + 1 = 26$.

Paso 5: No hay más caminos de aumento desde s a t en la red residual.

Flujo máximo: $|f| = 26$.

Implementación Completa

```
[language=C, caption=Implementación de Ford-Fulkerson] include <stdio.h>
include <stdlib.h> include <limits.h> include <string.h>
define MAX_V 100 define INF INT_MAX
typedef struct { int cap[MAX_V][MAX_V]; int n; } Graph;
// DFS para encontrar camino de aumento int dfs(Graph* G, int s, int
t, int* parent, int* visited) { visited[s] = 1; if (s == t) return 1;
for (int v = 0; v < G->n; v++) if (!visited[v] & G->cap[s][v] > 0) { parent[v]
= s; if (dfs(G, v, t, parent, visited)) return 1; } return 0;
int fordFulkerson(Graph* G, int s, int t) { int parent[MAX_V]; int maxFlow =
0;
while (1) { int visited[MAX_V] = 0; if (!dfs(G, s, t, parent, visited)) break;
// Encontrar capacidad residual minima en el camino int pathFlow =
INF; for (int v = t; v != s; v = parent[v]) { int u = parent[v]; if (G->cap[u][v]
< pathFlow) pathFlow = G->cap[u][v];
}
```

```

    // Actualizar capacidades residuales for (int v = t; v != s; v = parent[v])
    int u = parent[v]; G->cap[u][v] -= pathFlow; G->cap[v][u] += pathFlow;
    maxFlow += pathFlow;
    return maxFlow;

```

Algoritmo de Edmonds-Karp

Descripción

Edmonds-Karp es una implementación de Ford-Fulkerson que utiliza BFS para encontrar caminos de aumento, garantizando que se elija siempre el camino más corto (en número de aristas).

Pseudocódigo

```

[language=C, caption=Algoritmo de Edmonds-Karp]
int bfs(Graph* G, int s, int t, int* parent) {
    int visited[MAX_V] = 0;
    int queue[MAX_V], front = 0, rear = 0;
    visited[s] = 1; queue[rear++] = s;
    while (front < rear) {
        int u = queue[front++];
        for (int v = 0; v < G->n; v++) {
            if (!visited[v] & G->cap[u][v] > 0) {
                visited[v] = 1; parent[v] = u;
                if (v == t) return 1;
                queue[rear++] = v;
            }
        }
    }
    return 0;
}

int edmondsKarp(Graph* G, int s, int t) {
    int parent[MAX_V];
    int maxFlow = 0;
    while (bfs(G, s, t, parent)) {
        int pathFlow = INF;
        for (int v = t; v != s; v = parent[v]) {
            int u = parent[v];
            if (G->cap[u][v] < pathFlow) pathFlow = G->cap[u][v];
        }
        for (int v = t; v != s; v = parent[v]) {
            int u = parent[v];
            G->cap[u][v] -= pathFlow;
            G->cap[v][u] += pathFlow;
        }
        maxFlow += pathFlow;
    }
    return maxFlow;
}

```

Análisis de Complejidad

- Cada BFS toma $O(E)$ tiempo.
- Cada arista puede volverse crítica $O(V)$ veces.
- Número total de aumentos: $O(VE)$.
- Complejidad total: $O(VE^2)$.

Demostración de Complejidad

Edmonds-Karp realiza $O(VE)$ aumentos.

[Esbozo] Cuando una arista (u, v) se vuelve crítica, la distancia de s a u aumenta en al menos 2. Como la distancia máxima es $V - 1$, cada arista puede volverse crítica $O(V)$ veces.

Teorema de Flujo Máximo - Corte Mínimo

Demostración Completa

Para cualquier red de flujo, el valor máximo de un flujo es igual a la capacidad mínima de un corte.

1. **Parte 1:** $|f| \leq c(S, T)$ para cualquier flujo f y cualquier corte (S, T) .

Por el lema anterior, $|f| = f(S, T)$. Como $f(u, v) \leq c(u, v)$ para (u, v) de S a T , y $f(v, u) \geq 0$, tenemos:

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u) \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T)$$

2. **Parte 2:** Existe un flujo f y un corte (S, T) tales que $|f| = c(S, T)$.

Sea f un flujo máximo. Construimos S como el conjunto de vértices alcanzables desde s en la red residual G_f . Como f es máximo, no hay camino de s a t en G_f , por lo que $t \notin S$. Entonces (S, T) es un corte.

Para cualquier $u \in S$, $v \in T$ con $(u, v) \in E$, no puede haber (u, v) en G_f , por lo que $f(u, v) = c(u, v)$. Para (v, u) con $v \in T$, $u \in S$, si $(v, u) \in E$, entonces $f(v, u) = 0$ (de lo contrario habría una arista en G_f).

Por lo tanto:

$$|f| = f(S, T) = \sum_{u \in S, v \in T} c(u, v) = c(S, T)$$

Aplicaciones del Flujo Máximo

Matching Máximo en Grafos Bipartitos

[scale=0.6] [circle, draw] (s) at (0,0) s; [circle, draw] (l1) at (2,1.5) l₁;
[circle, draw] (l2) at (2,0) l₂; [circle, draw] (l3) at (2,-1.5) l₃; [circle, draw]

(r1) at (5,1.5) r_1 ; [circle, draw] (r2) at (5,0) r_2 ; [circle, draw] (r3) at (5,-1.5) r_3 ; [circle, draw] (t) at (7,0) t ;
 $[-i] (s) - (l1)$; $[-i] (s) - (l2)$; $[-i] (s) - (l3)$; $[-i] (l1) - (r1)$; $[-i] (l1) - (r2)$; $[-i] (l2) - (r2)$; $[-i] (l2) - (r3)$; $[-i] (l3) - (r1)$; $[-i] (l3) - (r3)$; $[-i] (r1) - (t)$; $[-i] (r2) - (t)$; $[-i] (r3) - (t)$;

Segmentación de Imágenes

- Problema de separar objeto del fondo.
- Modelar píxeles como nodos, aristas entre píxeles adyacentes.
- Fuente conectada a píxeles de objeto probable, sumidero a píxeles de fondo.
- El corte mínimo da la segmentación óptima.

Redes de Transporte

- Maximizar flujo de mercancías de fábricas a mercados.
- Modelar carreteras con capacidades.

Problema de Asignación

- Asignar trabajadores a tareas.
- Modelar con capacidades unitarias.

Ejercicios Resueltos (20 Ejercicios)

Calcule el flujo máximo en la red con aristas: $s \rightarrow a$ (10), $s \rightarrow b$ (5), $a \rightarrow b$ (15), $a \rightarrow t$ (10), $b \rightarrow t$ (10).

1. Camino $s \rightarrow a \rightarrow t$: flujo 10. Residual: $s \rightarrow a$ (0), $a \rightarrow t$ (0), $a \rightarrow s$ (10), $t \rightarrow a$ (10)
2. Camino $s \rightarrow b \rightarrow t$: flujo 5. Residual: $s \rightarrow b$ (0), $b \rightarrow t$ (5), $b \rightarrow s$ (5), $t \rightarrow b$ (5)
3. Camino $s \rightarrow a \rightarrow b \rightarrow t$: $c_f(s, a) = 0$, no hay

Flujo total: $10 + 5 = 15$. Corte mínimo: $S = \{s\}$, $T = \{a, b, t\}$, capacidad $10 + 5 = 15$.

Demuestre que el flujo máximo es igual a la capacidad del corte mínimo. Ver demostración del Teorema de Flujo Máximo - Corte Mínimo.

Aplique Ford-Fulkerson a la red con aristas: $s \rightarrow a$ (3), $s \rightarrow b$ (2), $a \rightarrow c$ (2), $b \rightarrow c$ (3), $b \rightarrow d$ (2), $c \rightarrow t$ (4), $d \rightarrow t$ (3), $a \rightarrow b$ (1).

1. $s \rightarrow a \rightarrow c \rightarrow t$: flujo 2. Residual: $s \rightarrow a$ (1), $a \rightarrow c$ (0), $c \rightarrow t$ (2)
2. $s \rightarrow b \rightarrow c \rightarrow t$: flujo 2. Residual: $s \rightarrow b$ (0), $b \rightarrow c$ (1), $c \rightarrow t$ (0)
3. $s \rightarrow a \rightarrow b \rightarrow d \rightarrow t$: flujo 1. Residual: $s \rightarrow a$ (0), $a \rightarrow b$ (0), $b \rightarrow d$ (1), $d \rightarrow t$ (2)
4. $s \rightarrow b \rightarrow d \rightarrow t$: flujo 1

Flujo total: $2 + 2 + 1 + 1 = 6$.

¿Qué es la red residual y por qué es importante? La red residual G_f representa la capacidad adicional disponible en cada arista después de enviar un flujo f . Incluye aristas inversas que permiten "deshacer" flujo enviado previamente.

Es importante porque:

- Permite encontrar caminos de aumento.
- Los caminos en G_f corresponden a formas de aumentar el flujo.
- El teorema de flujo máximo se demuestra usando G_f .

Implemente Ford-Fulkerson con BFS (Edmonds-Karp). Ver implementación en la sección anterior.

¿Cuál es la complejidad de Edmonds-Karp y por qué? $O(VE^2)$. Cada BFS toma $O(E)$. El número de aumentos es $O(VE)$ porque cada arista puede volverse crítica $O(V)$ veces y cada aumento tiene al menos una arista crítica.

¿Cómo se puede usar flujo máximo para encontrar un matching máximo en un grafo bipartito? Se construye una red:

1. Fuente s conectada a todos los vértices de la partición izquierda (capacidad 1).
2. Sumidero t conectado desde todos los vértices de la partición derecha (capacidad 1).
3. Aristas originales del grafo bipartito (capacidad 1).

4. El flujo máximo es igual al tamaño del matching máximo.

Demuestre que si todas las capacidades son enteras, Ford-Fulkerson termina en tiempo finito. Cada aumento incrementa el flujo en al menos 1. El flujo máximo está acotado por la suma de capacidades que salen de s , que es finita. Por lo tanto, después de un número finito de aumentos se alcanza el flujo máximo.

¿Qué sucede si las capacidades son irracionales? Ford-Fulkerson puede no terminar (ejemplo de Zwick). La versión con BFS (Edmonds-Karp) termina porque siempre elige caminos de aumento más cortos.

Encuentre el corte mínimo en la red del ejercicio 1. Corte mínimo: $S = \{s, a, b\}$, $T = \{c, d, t\}$. Capacidad: $a \rightarrow c$ (0 después del flujo? original 2), $b \rightarrow c$ (1), $b \rightarrow d$ (1), $a \rightarrow t$ (0), $b \rightarrow t$ (0). Suma = 2+1+1=4? Verificar.

Ejercicios Propuestos (20 Ejercicios)

1. Calcule el flujo máximo en una red de 6 vértices.
2. Demuestre el Teorema de Flujo Máximo - Corte Mínimo.
3. Implemente Ford-Fulkerson con DFS.
4. ¿Qué es el algoritmo de push-relabel y cómo mejora Ford-Fulkerson?
5. Aplique flujo máximo para encontrar matching máximo en un grafo bipartito.
6. Demuestre que el corte mínimo es único o dé un contraejemplo.
7. ¿Cómo se puede usar flujo máximo para particionar un grafo?
8. Implemente Edmonds-Karp.
9. ¿Qué es la capacidad de un corte y cómo se calcula?
10. Encuentre el flujo máximo en una red con capacidades unitarias.
11. ¿Qué es el problema de flujo con demandas?
12. Demuestre que el flujo máximo es igual a la capacidad del corte mínimo.
13. Aplique flujo máximo a un problema de asignación de tareas.
14. ¿Qué es el teorema de descomposición de flujo?

15. Implemente una variante de Ford-Fulkerson con escalado de capacidades.
16. ¿Cómo se detecta un corte mínimo después de ejecutar Ford-Fulkerson?
17. Aplique flujo máximo a una red de 5 vértices con capacidades aleatorias.
18. ¿Qué es el problema de flujo de costo mínimo?
19. Demuestre que el algoritmo de Edmonds-Karp tiene complejidad $O(VE^2)$.
20. Investigue aplicaciones de flujo máximo en redes de telecomunicaciones.

Resumen de la Unidad

En esta unidad hemos aprendido:

- La definición formal de redes de flujo, flujos y cortes.
- El Teorema de Flujo Máximo - Corte Mínimo y su demostración.
- El algoritmo de Ford-Fulkerson y su variante Edmonds-Karp.
- El concepto de red residual y caminos de aumento.
- La complejidad $O(VE^2)$ de Edmonds-Karp.
- Aplicaciones prácticas en matching bipartito y segmentación.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 26.
- Edmonds, J., & Karp, R. M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2), 248-264.
- Ford, L. R., & Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8, 399-404.

Unidad VIII: Algoritmos de String (Cadenas)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el problema de búsqueda de patrones en cadenas.
- Implementar y analizar el algoritmo de búsqueda ingenua (fuerza bruta).
- Implementar y analizar el algoritmo KMP (Knuth-Morris-Pratt).
- Implementar y analizar el algoritmo de Boyer-Moore.
- Comprender la función de prefijo y la tabla de saltos.
- Aplicar estos algoritmos a problemas prácticos de búsqueda de texto.

Introducción al Problema de Búsqueda de Patrones

Definición del Problema

Dado un **texto** T de longitud n y un **patrón** P de longitud m ($m \leq n$), encontrar todas las ocurrencias de P en T .

$T = \text{"ABABABAC"} , P = \text{"ABA"}$ Ocurrencias en posiciones: 0, 2, 4

Aplicaciones

- Editores de texto (Ctrl+F)
- Motores de búsqueda

- Análisis de ADN
- Detección de plagio
- Compresión de datos

Notación

- $T[0..n-1]$: texto
- $P[0..m-1]$: patrón
- $T[i..j]$: subcadena de T desde i hasta j

Algoritmo Ingenuo (Fuerza Bruta)

Descripción

El algoritmo más simple: probar todas las posiciones posibles del texto donde podría comenzar el patrón.

Pseudocódigo

```
[language=C, caption=Búsqueda ingenua] NAIVE-SEARCH(T, P)
n = length(T)
m = length(P)
for i = 0 to n - m
  j = 0
  while j ≤ m and T[i+j] == P[j]
    j = j + 1
  if j == m
    print "Patrón encontrado en posición", i
```

Ejemplo

$T = "ABABABAC"$, $P = "ABA"$

```
Posición 0: A B A vs A B A
Posición 1: B A B vs A B A
Posición 2: A B A vs A B A
Posición 3: B A B vs A B A
Posición 4: A B A vs A B A
Posición 5: B A C vs A B A
```

Análisis de Complejidad

- Peor caso: $O(m \cdot (n - m + 1)) = O(mn)$
- Ejemplo de peor caso: $T = \text{"AAAAA..."}$, $P = \text{"AAA"}$
- Mejor caso: $O(n)$ (primera comparación falla siempre)

Algoritmo KMP (Knuth-Morris-Pratt)

Descripción

KMP aprovecha la información de las comparaciones previas para evitar retroceder en el texto. Utiliza una **función de prefijo** (o tabla de fallos) que indica el mayor prefijo propio del patrón que también es sufijo del patrón procesado.

Función de Prefijo

Para un patrón P , definimos $\pi[q]$ como la longitud del prefijo propio más largo de $P[0..q]$ que también es sufijo de $P[0..q]$.

$P = \text{"ABABAC"}$

- $\pi[0] = 0$ (prefijo propio de "A" es vacío)
- $\pi[1] = 0$ ("AB" no tiene prefijo propio igual a sufijo)
- $\pi[2] = 1$ ("ABA" tiene prefijo "A" igual a sufijo "A")
- $\pi[3] = 2$ ("ABAB" tiene prefijo "AB" igual a sufijo "AB")
- $\pi[4] = 3$ ("ABABA" tiene prefijo "ABA" igual a sufijo "ABA")
- $\pi[5] = 0$ ("ABABAC" no tiene)

Cálculo de la Función de Prefijo

[language=C, caption=Cálculo de la función de prefijo] COMPUTE-PREFIX-FUNCTION(P) $m = \text{length}(P)$ $\pi = \text{array of size } m$ $\pi[0] = 0$ $k = 0$ for $q = 1$ to $m-1$ while $k \neq 0$ and $P[k] \neq P[q]$ $k = \pi[k-1]$ if $P[k] == P[q]$ $k = k + 1$ $\pi[q] = k$ return π

Ejemplo de Cálculo

$P = \text{"ABABAC"}$

```

q=1, k=0: P[0]=A vs P[1]=B, diferentes, pi[1]=0
q=2, k=0: P[0]=A vs P[2]=A, iguales, k=1, pi[2]=1
q=3, k=1: P[1]=B vs P[3]=B, iguales, k=2, pi[3]=2
q=4, k=2: P[2]=A vs P[4]=A, iguales, k=3, pi[4]=3
q=5, k=3: P[3]=B vs P[5]=C, diferentes, k=pi[2]=1
          P[1]=B vs P[5]=C, diferentes, k=pi[0]=0
          P[0]=A vs P[5]=C, diferentes, pi[5]=0

```

Algoritmo KMP

[language=C, caption=Algoritmo KMP] KMP-SEARCH(T, P) $n = \text{length}(T)$
 $m = \text{length}(P)$ $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ $q = 0$ // número
 de caracteres del patrón coincidentes

```

  for i = 0 to n-1 while q  $\neq$  0 and P[q]  $\neq$  T[i] q = pi[q-1] if P[q] == T[i]
  q = q + 1 if q == m print "Patron encontrado en posicion", i - m + 1 q =
  pi[q-1] // continuar buscando

```

Ejemplo de Ejecución

$T = \text{"ABABABAC"}$, $P = \text{"ABA"}$, $\pi = [0, 0, 1]$

```

i=0, T[0]=A, q=0: P[0]=A == A, q=1
i=1, T[1]=B, q=1: P[1]=B == B, q=2
i=2, T[2]=A, q=2: P[2]=A == A, q=3, encontrado en 0, q=pi[2]=1
i=3, T[3]=B, q=1: P[1]=B == B, q=2
i=4, T[4]=A, q=2: P[2]=A == A, q=3, encontrado en 2, q=pi[2]=1
i=5, T[5]=B, q=1: P[1]=B == B, q=2
i=6, T[6]=A, q=2: P[2]=A == A, q=3, encontrado en 4, q=pi[2]=1
i=7, T[7]=C, q=1: P[1]=B vs C, q=pi[0]=0, P[0]=A vs C, q=0

```

Análisis de Complejidad

- Construcción de π : $O(m)$
- Búsqueda: $O(n)$
- Total: $O(n + m)$

Algoritmo de Boyer-Moore

Descripción

Boyer-Moore es uno de los algoritmos más eficientes en la práctica. Compara el patrón de derecha a izquierda y utiliza dos heurísticas:

1. Heurística del mal carácter (bad character rule)
2. Heurística del buen sufijo (good suffix rule)

Heurística del Mal Carácter

Cuando ocurre una falta de coincidencia, desplaza el patrón para alinear el carácter del texto con la ocurrencia más a la derecha de ese carácter en el patrón.

```
[language=C, caption=Tabla de mal carácter] COMPUTE-BAD-CHARACTER-
TABLE(P) m = length(P) // Inicializar con -1 para todos los caracteres
for each character c in alphabet lastOccurrence[c] = -1
for i = 0 to m-1 lastOccurrence[P[i]] = i
return lastOccurrence
```

Heurística del Buen Sufijo

Cuando ocurre una falta de coincidencia, desplaza el patrón para alinear el sufijo ya coincidente con otra ocurrencia del mismo sufijo en el patrón.

Tabla de Buen Sufijo

```
[language=C, caption=Tabla de buen sufijo] COMPUTE-GOOD-SUFFIX-
TABLE(P) m = length(P) suffix = array of size m
border = array of size m+1
// Calcular sufijos
suffix[m-1] = m
g = m-1
for i = m-2 downto 0
  if i != g
    and suffix[i + m - 1 - f] != i - g
    suffix[i] = suffix[i + m - 1 - f]
  else if i != g
    g = i
    f = i
    while g != 0 and P[g] == P[g + m - 1 - f]
      g = g - 1
    suffix[i] = f - g
return suffix
```

Algoritmo Completo

```
[language=C, caption=Algoritmo de Boyer-Moore] BOYER-MOORE-SEARCH(T,
P) n = length(T) m = length(P) badChar = COMPUTE-BAD-CHARACTER-
TABLE(P) goodSuffix = COMPUTE-GOOD-SUFFIX-TABLE(P)
```



```

s = 0 // desplazamiento del patrón while s ≤ n - m j = m - 1 // empezar
desde el final while j ≥ 0 and P[j] == T[s + j] j = j - 1 if j ≥ 0 print "Patron
encontrado en posicion", s s = s + goodSuffix[0] // desplazamiento else //
Calcular desplazamiento por mal carácter badShift = j - badChar[T[s + j]] if
badShift ≥ 1 badShift = 1 // Usar el máximo de las dos heurísticas goodShift
= goodSuffix[j] s = s + max(badShift, goodShift)

```

Ejemplo de Ejecución

$T = \text{"HEREISASIMPLEEXAMPLE"} , P = \text{"EXAMPLE"}$

Primera comparación: H vs E (mal carácter E en P en posición 6)
 Desplazamiento: $6 - \text{badChar}['H'] = 6 - (-1) = 7$

Segunda comparación: S vs E (mal carácter E en posición 6)
 Desplazamiento: $6 - \text{badChar}['S'] = 6 - (-1) = 7$

... (continúa)

Análisis de Complejidad

- Preprocesamiento: $O(m + |\Sigma|)$ donde Σ es el alfabeto
- Peor caso: $O(mn)$ (raro en la práctica)
- Caso promedio: $O(n/m)$ (muy eficiente)

Comparación de Algoritmos

Algoritmo	Preprocesamiento	Búsqueda	Memoria
Ingenuo	$O(1)$	$O(mn)$	$O(1)$
KMP	$O(m)$	$O(n)$	$O(m)$
Boyer-Moore	$O(m + \Sigma)$	$O(n/m)$ promedio	$O(m + \Sigma)$

Otros Algoritmos de String

Algoritmo de Rabin-Karp

Utiliza hashing para comparar subcadenas en tiempo promedio $O(n + m)$.

Algoritmo Z

Similar a KMP pero calcula la función Z.

Aho-Corasick

Para búsqueda de múltiples patrones simultáneamente.

Ejercicios Resueltos (20 Ejercicios)

Busque el patrón "ABA" en el texto "ABABABAC" usando el algoritmo ingenuo. Posiciones: 0, 2, 4. Ver ejemplo anterior.

Calcule la función de prefijo para $P = "AAABAAA"$.

```
[0]=0
i=1, P[1]=A, P[0]=A, k=1, [1]=1
i=2, P[2]=A, P[1]=A, k=2, [2]=2
i=3, P[3]=B, P[2]=A, k=[1]=1, P[1]=A vs B, k=[0]=0, P[0]=A vs B, [3]=0
i=4, P[4]=A, P[0]=A, k=1, [4]=1
i=5, P[5]=A, P[1]=A, k=2, [5]=2
i=6, P[6]=A, P[2]=A, k=3, [6]=3
```

Resultado: $\pi = [0, 1, 2, 0, 1, 2, 3]$

Ejecute KMP para $T = "AAABAAA"$, $P = "AAA"$. $\pi = [0, 1, 2]$

```
i=0, T[0]=A, q=0: P[0]=A, q=1
i=1, T[1]=A, q=1: P[1]=A, q=2
i=2, T[2]=A, q=2: P[2]=A, q=3, encontrado en 0, q=[2]=2
i=3, T[3]=B, q=2: P[2]=A vs B, q=[1]=1, P[1]=A vs B, q=[0]=0, P[0]=A vs B, q=0
i=4, T[4]=A, q=0: P[0]=A, q=1
i=5, T[5]=A, q=1: P[1]=A, q=2
i=6, T[6]=A, q=2: P[2]=A, q=3, encontrado en 4
```

Ocurrencias: 0, 4

Explique por qué KMP es más eficiente que el algoritmo ingenuo. KMP nunca retrocede en el texto. Cuando ocurre una falta de coincidencia, utiliza la función de prefijo para saber cuánto avanzar en el patrón, evitando comparaciones redundantes. En el peor caso, el algoritmo ingenuo hace $O(mn)$ comparaciones, mientras que KMP hace $O(n)$.

Calcule la tabla de mal carácter para $P = "EXAMPLE"$.

E: 0
 X: 1
 A: 2
 M: 3
 P: 4
 L: 5
 E: 6 (sobrescribe posición 0)

Para cualquier otro carácter: -1

¿Cuál es la ventaja de Boyer-Moore sobre KMP? Boyer-Moore compara de derecha a izquierda, lo que permite mayores desplazamientos. En la práctica, especialmente para alfabetos grandes y patrones largos, Boyer-Moore es significativamente más rápido (sublineal, $O(n/m)$ en promedio).

Encuentre todas las ocurrencias de "AB" en "ABABAB" usando KMP.
 $P = \text{"AB"}$, $\pi = [0, 0]$

i=0: q=1
 i=1: q=2 -> encontrado en 0, q=0
 i=2: q=1
 i=3: q=2 -> encontrado en 2
 i=4: q=1
 i=5: q=2 -> encontrado en 4

Ocurrencias: 0, 2, 4

¿Qué es la función de prefijo y cómo se calcula? La función de prefijo $\pi[q]$ es la longitud del prefijo propio más largo de $P[0..q]$ que también es sufijo de $P[0..q]$. Se calcula en $O(m)$ usando un algoritmo que aprovecha los valores ya calculados.

Demuestre que KMP tiene complejidad $O(n + m)$. Cada iteración del bucle principal incrementa i o disminuye q (a través del while). q nunca es negativo y se incrementa a lo sumo n veces. Por lo tanto, el número total de iteraciones es $O(n + m)$.

Aplique Boyer-Moore para buscar "AB" en "AABAB".

Posición 0: comparar $P[1]=B$ vs $T[1]=A$ -> mal carácter A, $\text{badShift}=1-(-1)=2$
 Posición 2: comparar $P[1]=B$ vs $T[3]=B$, $P[0]=A$ vs $T[2]=A$ -> encontrado en 2
 Posición 3: comparar $P[1]=B$ vs $T[4]=B$, $P[0]=A$ vs $T[3]=B$ -> mal carácter B, ba

Ocurrencias: 2

Ejercicios Propuestos (20 Ejercicios)

1. Busque "ABC" en "AABCABC" usando el algoritmo ingenuo.
2. Calcule la función de prefijo para "ABABAB".
3. Ejecute KMP para buscar "ABAB" en "ABABABAB".
4. Calcule la tabla de mal carácter para "MISSISSIPPI".
5. ¿Qué es la heurística del buen sufijo en Boyer-Moore?
6. Implemente el algoritmo ingenuo en C.
7. ¿En qué casos el algoritmo ingenuo es tan bueno como KMP?
8. Demuestre que la función de prefijo se puede calcular en $O(m)$.
9. Aplique Boyer-Moore a un texto largo y un patrón corto.
10. ¿Qué es el algoritmo de Rabin-Karp y cómo usa hashing?
11. Implemente KMP en Python.
12. ¿Qué es el algoritmo Z y cómo se relaciona con KMP?
13. Encuentre todas las ocurrencias de "ANA" en "BANANA".
14. ¿Cómo se manejan caracteres que no están en el patrón en Boyer-Moore?
15. Implemente la función de prefijo para un patrón dado.
16. ¿Qué es el algoritmo de Aho-Corasick y para qué sirve?
17. Compare KMP y Boyer-Moore en diferentes escenarios.
18. ¿Qué es la complejidad en el peor caso de Boyer-Moore?
19. Implemente Boyer-Moore en Java.
20. Investigue aplicaciones de estos algoritmos en bioinformática.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El problema de búsqueda de patrones en cadenas.
- El algoritmo ingenuo ($O(mn)$) como caso base.
- El algoritmo KMP ($O(n + m)$) usando función de prefijo.
- El algoritmo de Boyer-Moore (sublineal en la práctica) con heurísticas de mal carácter y buen sufijo.
- Comparación de los algoritmos y sus aplicaciones.

Bibliografía de la Unidad

- Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM*, 20(10), 762-772.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 32.
- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323-350.

Unidad IX: Algoritmos de Búsqueda de Patrones - KMP y Boyer-Moore

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender en profundidad el algoritmo KMP y su función de prefijo.
- Implementar y analizar KMP en detalle.
- Comprender las heurísticas de Boyer-Moore (mal carácter y buen su-fijo).
- Implementar y analizar Boyer-Moore.
- Comparar ambos algoritmos y elegir el apropiado según el contexto.
- Aplicar estos algoritmos a problemas de búsqueda en textos reales.

Algoritmo KMP (Knuth-Morris-Pratt)

Filosofía del Algoritmo

KMP se basa en una idea fundamental: cuando ocurre una falta de coincidencia, ya sabemos parte del texto (la parte que coincidió con el prefijo del patrón). Esta información se puede usar para evitar retroceder en el texto.

[scale=0.7] (0,0) rectangle (10,0.5); at (1,0.25) A; at (2,0.25) B; at (3,0.25) A; at (4,0.25) B; at (5,0.25) A; at (6,0.25) B; at (7,0.25) A; at (8,0.25) C; at (9,0.25) ?;
(0,-1) rectangle (3,-0.5); at (0.5,-0.75) A; at (1.5,-0.75) B; at (2.5,-0.75) A;

[red, thick] (3,-1) – (3,0.5); [red, thick] (3,-1) – (3,-0.5);
at (5,-1.5) Falta coincidencia en posición 3;

Función de Prefijo - Análisis Detallado

Definición Formal

Para un patrón P de longitud m , la función de prefijo π se define como:

$$\pi[q] = \max\{k : 0 \leq k < q \text{ y } P[0..k-1] = P[q-k..q-1]\}$$

Es decir, la longitud del prefijo propio más largo que también es sufijo de $P[0..q-1]$.

Ejemplo Detallado Para $P = \text{"ABABAC"}$:

q	Subcadena	$\pi[q]$
0	"A"	0
1	"AB"	0
2	"ABA"	1 (prefijo "A" = sufijo "A")
3	"ABAB"	2 (prefijo "AB" = sufijo "AB")
4	"ABABA"	3 (prefijo "ABA" = sufijo "ABA")
5	"ABABAC"	0

Implementación de la Función de Prefijo

```
[language=C, caption=Función de prefijo] void computePrefixFunction(char*
P, int m, int* pi) pi[0] = 0; int k = 0;
```

```
for (int q = 1; q < m; q++) // Mientras haya una falta de coincidencia,
retroceder while (k > 0 P[k] != P[q]) k = pi[k - 1]; // Si coinciden, avanzar
if (P[k] == P[q]) k++; pi[q] = k;
```

Visualización de la Función de Prefijo

[scale=0.6] (0,0) rectangle (6,0.5); at (0.5,0.25) A; at (1.5,0.25) B; at
(2.5,0.25) A; at (3.5,0.25) B; at (4.5,0.25) A; at (5.5,0.25) C;
[blue, thick] (0,-0.5) – (1,-0.5); [blue, thick] (0,-0.5) – (0,-1); [blue, thick]
(1,-0.5) – (1,-1); at (0.5,-0.8) prefijo "A";
[red, thick] (2.5,-0.5) – (3.5,-0.5); [red, thick] (2.5,-0.5) – (2.5,-1); [red,
thick] (3.5,-0.5) – (3.5,-1); at (3,-0.8) sufijo "AB";

Algoritmo KMP Completo

```
[language=C, caption=Algoritmo KMP completo] include <stdio.h> include
<string.h> include <stdlib.h>
void KMP(char* T, char* P) { int n = strlen(T); int m = strlen(P); int*
pi = (int*)malloc(m * sizeof(int));
// Calcular función de prefijo computePrefixFunction(P, m, pi);
// Búsqueda
int q = 0; // número de caracteres coincidentes
for (int i = 0; i < n; i++) // Mientras haya falta de coincidencia y q > 0,
retroceder while (q > 0 & P[q] != T[i]) q = pi[q - 1]; // Si coinciden, avanzar
if (P[q] == T[i]) q++; // Si encontramos el patrón completo if (q == m)
printf("Patrón encontrado en posición q = pi[q - 1]; // continuar buscando
ocurrencias superpuestas
free(pi);
```

Ejemplo de Ejecución Detallada

$T = "ABABABAC"$, $P = "ABA"$

i	$T[i]$	q antes	Acción	q después
0	A	0	$P[0] = A$ coincide	1
1	B	1	$P[1] = B$ coincide	2
2	A	2	$P[2] = A$ coincide	3 (encontrado en 0)
3	B	$3 \rightarrow 1$	$P[1] = B$ coincide	2
4	A	2	$P[2] = A$ coincide	3 (encontrado en 2)
5	B	$3 \rightarrow 1$	$P[1] = B$ coincide	2
6	A	2	$P[2] = A$ coincide	3 (encontrado en 4)
7	C	$3 \rightarrow 1 \rightarrow 0$	$P[0] = A \neq C$	0

Demostración de Correctitud de KMP

El algoritmo KMP encuentra todas las ocurrencias del patrón P en el texto T en tiempo $O(n + m)$.

[Esbozo]

- Corrección de la función de prefijo:** $\pi[q]$ es la longitud del prefijo propio más largo que es sufijo de $P[0..q - 1]$. Esto se demuestra por inducción.
- Corrección de la búsqueda:** El invariante es que después de procesar $T[0..i - 1]$, q es la longitud del prefijo más largo de P que es sufijo de $T[0..i - 1]$. Cuando $q = m$, se ha encontrado una ocurrencia.

3. **Complejidad:** Cada iteración incrementa i o disminuye q . q se incrementa a lo sumo n veces, y cada decremento corresponde a un incremento previo. Por lo tanto, el número total de operaciones es $O(n+m)$.

Propiedades Adicionales de KMP

- KMP es **online**: puede procesar el texto carácter por carácter.
- No requiere retroceder en el texto.
- Es óptimo en el peor caso: cualquier algoritmo de búsqueda de patrones debe hacer al menos $\Omega(n)$ comparaciones.

Algoritmo de Boyer-Moore

Filosofía del Algoritmo

Boyer-Moore compara el patrón de **derecha a izquierda**. Cuando ocurre una falta de coincidencia, puede desplazar el patrón múltiples posiciones basándose en dos heurísticas.

[scale=0.7] (0,0) rectangle (10,0.5); at (1,0.25) H; at (2,0.25) E; at (3,0.25) R; at (4,0.25) E; at (5,0.25) ; at (6,0.25) I; at (7,0.25) S; at (8,0.25) ; at (9,0.25) A;
 (3,-1) rectangle (8,-0.5); at (3.5,-0.75) E; at (4.5,-0.75) X; at (5.5,-0.75) A; at (6.5,-0.75) M; at (7.5,-0.75) P;
 [red, thick] (8,-1) – (8,0.5); [blue, thick] (4.5,-1) – (4.5,-0.5);

Heurística del Mal Carácter

Definición

Cuando se encuentra una falta de coincidencia en la posición j del patrón (comparando de derecha a izquierda), y el carácter del texto en esa posición es c , desplazamos el patrón para alinear la ocurrencia más a la derecha de c en el patrón con la posición actual del texto.

$$\text{desplazamiento} = j - \text{últimaOcurrencia}[c]$$

Si c no está en el patrón, $\text{últimaOcurrencia}[c] = -1$.

Implementación de la Tabla de Mal Carácter

```
[language=C, caption=Tabla de mal carácter] define ALPHABETSIZE256
void computeBadCharTable(char* P, int m, int* badChar) // Inicializar
con -1 for (int i = 0; i < ALPHABETSIZE; i++) badChar[i] = -1; // Guardarla última ocurrencia de cada
0; i < m; i++) badChar[(int)P[i]] = i;
```

Heurística del Buen Sufijo

Definición

Cuando ocurre una falta de coincidencia después de haber coincidido un sufijo S del patrón, desplazamos el patrón para alinear otra ocurrencia de S en el patrón (si existe), o un prefijo del patrón que sea sufijo de S .

Tipos de Buen Sufijo

1. **Caso 1:** Otra ocurrencia del sufijo S en el patrón.
2. **Caso 2:** Un prefijo del patrón que es sufijo de S .
3. **Caso 3:** Desplazamiento completo m (ninguna coincidencia parcial).

Implementación de la Tabla de Buen Sufijo

```
[language=C, caption=Tabla de buen sufijo] void computeGoodSuffixTable(char*
P, int m, int* goodSuffix) int* suffix = (int*)malloc(m * sizeof(int)); int*
border = (int*)malloc((m + 1) * sizeof(int));
// Calcular array de sufijos suffix[m - 1] = m; int g = m - 1; int f = m -
1;
for (int i = m - 2; i >= 0; i--) if (i < g && suffix[i + m - 1 - f] & i - g) suffix[i]
= suffix[i + m - 1 - f]; else if (i < g) g = i; f = i; while (g >= 0 && P[g] ==
P[g + m - 1 - f]) g--; suffix[i] = f - g;
// Inicializar goodSuffix for (int i = 0; i < m; i++) goodSuffix[i] = m;
// Calcular bordes int j = 0; for (int i = m - 1; i >= -1; i--) if (i == -1
—— suffix[i] == i + 1) for (; j < m - 1 - i; j++) if (goodSuffix[j] == m)
goodSuffix[j] = m - 1 - i;
// Casos de sufijos for (int i = 0; i < m - 1; i++) goodSuffix[m - 1 -
suffix[i]] = m - 1 - i;
free(suffix); free(border);
```

Algoritmo de Boyer-Moore Completo

```
[language=C, caption=Boyer-Moore completo] void boyerMoore(char* T,
char* P) int n = strlen(T); int m = strlen(P); int badChar[ALPHABET_SIZE]; int*
goodSuffix = (int*)malloc(m * sizeof(int));
    computeBadCharTable(P, m, badChar); computeGoodSuffixTable(P, m,
goodSuffix);
    int s = 0; // desplazamiento del patrón sobre el texto
    while (s ≤ n - m) int j = m - 1; // empezar desde el final
    // Comparar de derecha a izquierda while (j ≥ 0 P[j] == T[s + j]) j--;
    if (j ≥ 0) printf("Patrón encontrado en posición s += goodSuffix[0];
else // Calcular desplazamiento por mal carácter int badShift = j - bad-
Char[(int)T[s + j]]; if (badShift ≥ 1) badShift = 1; // Usar el máximo de las
dos heurísticas int goodShift = goodSuffix[j]; s += (badShift ≥ goodShift) ?
badShift : goodShift;
    free(goodSuffix);
```

Ejemplo Detallado de Boyer-Moore

$T = \text{"HEREISASIMPLEEXAMPLE"} , P = \text{"EXAMPLE"}$

Paso	Comparación	Desplazamiento
1	"E" vs "H" (mal carácter H)	$6 - (-1) = 7$
2	"E" vs "S" (mal carácter S)	$6 - (-1) = 7$
3	"E" vs "A" (mal carácter A)	$6 - 2 = 4$
4	"E" vs " " (espacio)	$6 - (-1) = 7$
5	"E" vs "S" (mal carácter S)	$6 - (-1) = 7$
6	"E" vs "E" (coincide), continuar	-
7	"P" vs "P" (coincide)	-
8	"L" vs "L" (coincide)	-
9	"M" vs "M" (coincide)	-
10	"A" vs "A" (coincide)	-
11	"X" vs "X" (coincide)	-
12	"E" vs " " (falta)	buen sufijo "EXAMPLE"

Análisis de Complejidad de Boyer-Moore

- **Preprocesamiento:** $O(m + |\Sigma|)$
- **Mejor caso:** $O(n/m)$ (sublineal)
- **Peor caso:** $O(mn)$ (raro, ocurre con patrones como "AAAAA")

- **Caso promedio:** $O(n)$ con una constante muy pequeña

Optimizaciones de Boyer-Moore

1. **Boyer-Moore-Horspool:** Solo usa la heurística del mal carácter (más simple).
2. **Boyer-Moore con Galil:** Optimización para patrones periódicos.
3. **Algoritmo de Apostolico-Giancarlo:** Mejora para patrones repetitivos.

Comparación entre KMP y Boyer-Moore

Característica	KMP	Boyer-Moore
Dirección de comparación	Izquierda a derecha	Derecha a izquierda
Preprocesamiento	$O(m)$	$O(m + \Sigma)$
Peor caso	$O(n + m)$	$O(mn)$ (raro)
Mejor caso	$O(n + m)$	$O(n/m)$
Memoria	$O(m)$	$O(m + \Sigma)$
Online	Sí	No (necesita el patrón completo)
Complejidad de implementación	Media	Alta

Cuándo usar cada algoritmo

- **KMP:**
 - Cuando el patrón es corto o el alfabeto es pequeño.
 - Cuando se necesita un algoritmo online.
 - Cuando se garantiza que el peor caso es importante.
- **Boyer-Moore:**
 - Cuando el patrón es largo y el alfabeto es grande.
 - En la práctica, para búsqueda en textos naturales.
 - Cuando se busca un patrón en un texto grande.

Ejercicios Resueltos (20 Ejercicios)

Calcule la función de prefijo para $P = "ABCDABD"$.

```
[0]=0
i=1, P[1]=B vs P[0]=A, [1]=0
i=2, P[2]=C vs P[0]=A, [2]=0
i=3, P[3]=D vs P[0]=A, [3]=0
i=4, P[4]=A vs P[0]=A, k=1, [4]=1
i=5, P[5]=B vs P[1]=B, k=2, [5]=2
i=6, P[6]=D vs P[2]=C, k=[1]=0, P[0]=A vs D, [6]=0
```

Resultado: [0, 0, 0, 0, 1, 2, 0]

Ejecute KMP para $T = "ABCDABD"$, $P = "ABD"$. $\pi = [0, 0, 0]$

```
i=0, T[0]=A, q=0: P[0]=A, q=1
i=1, T[1]=B, q=1: P[1]=B, q=2
i=2, T[2]=C, q=2: P[2]=D vs C, q=[1]=0, P[0]=A vs C, q=0
i=3, T[3]=D, q=0: P[0]=A vs D, q=0
i=4, T[4]=A, q=0: P[0]=A, q=1
i=5, T[5]=B, q=1: P[1]=B, q=2
i=6, T[6]=D, q=2: P[2]=D, q=3, encontrado en 4
```

Calcule la tabla de mal carácter para $P = "ABCBAB"$.

```
A: 0
B: 5 (última ocurrencia en posición 5)
C: 2
Otros caracteres: -1
```

Ejecute Boyer-Moore para $T = "ABABABAB"$, $P = "BAB"$. Tabla mal carácter: B:2, A:1

```
s=0: comparar P[2]=B vs T[2]=A, badShift=2-1=1, s=1
s=1: comparar P[2]=B vs T[3]=B, P[1]=A vs T[2]=A, encontrado en 1
s=2: comparar P[2]=B vs T[4]=B, P[1]=A vs T[3]=B, badShift=1-1=0->1, s=3
s=3: comparar P[2]=B vs T[5]=B, P[1]=A vs T[4]=A, encontrado en 3
s=4: comparar P[2]=B vs T[6]=B, P[1]=A vs T[5]=B, badShift=1-1=0->1, s=5
s=5: comparar P[2]=B vs T[7]=B, P[1]=A vs T[6]=A, encontrado en 5
```

Ocurrencias: 1, 3, 5

Ejercicios Propuestos (20 Ejercicios)

1. Calcule la función de prefijo para "ABABABAB".
2. Ejecute KMP para buscar "ABAB" en "ABABABAB".
3. Calcule la tabla de mal carácter para "MISSISSIPPI".
4. Ejecute Boyer-Moore para buscar "ISS" en "MISSISSIPPI".
5. ¿Qué ventaja tiene KMP sobre el algoritmo ingenuo?
6. ¿Qué ventaja tiene Boyer-Moore sobre KMP?
7. Implemente KMP en Python.
8. Implemente Boyer-Moore en C.
9. ¿Qué es la función de prefijo y cómo se usa?
10. ¿Qué es la heurística del buen sufijo?
11. Demuestre que KMP tiene complejidad $O(n + m)$.
12. ¿En qué casos Boyer-Moore tiene peor caso $O(mn)$?
13. ¿Qué es el algoritmo de Boyer-Moore-Horspool?
14. ¿Cómo se manejan caracteres no alfabéticos en Boyer-Moore?
15. Compare KMP y Boyer-Moore en textos con patrones repetitivos.
16. ¿Qué es el algoritmo de Galil y para qué sirve?
17. ¿Se puede usar KMP para búsqueda online? ¿Y Boyer-Moore?
18. Implemente una versión optimizada de Boyer-Moore.
19. ¿Qué es el algoritmo de Apostolico-Giancarlo?
20. Investigue aplicaciones de KMP en bioinformática.

Resumen de la Unidad

En esta unidad hemos profundizado en:

- El algoritmo KMP: función de prefijo, búsqueda en $O(n + m)$, online.
- El algoritmo de Boyer-Moore: heurísticas de mal carácter y buen sufijo, sublineal en la práctica.
- Comparación detallada de ambos algoritmos.
- Implementaciones completas y ejemplos paso a paso.
- Aplicaciones prácticas y criterios de selección.

Bibliografía de la Unidad

- Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM*, 20(10), 762-772.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 32.
- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2), 323-350.

Unidad X: Algoritmos sobre Números

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los fundamentos de la aritmética modular.
- Implementar y analizar el algoritmo de Euclides para el cálculo del MCD.
- Implementar y analizar el algoritmo de Euclides extendido.
- Comprender los conceptos básicos de la criptografía RSA.
- Aplicar estos algoritmos a problemas de teoría de números y criptografía.

Introducción a la Aritmética Modular

Definiciones Fundamentales

Decimos que $a \equiv b \pmod{n}$ si n divide a $a - b$, es decir, $a - b = kn$ para algún entero k .

Propiedades de la Aritmética Modular

- $(a + b) \bmod n = (a \bmod n + b \bmod n) \bmod n$
- $(a - b) \bmod n = (a \bmod n - b \bmod n) \bmod n$
- $(a \cdot b) \bmod n = (a \bmod n) \cdot (b \bmod n) \bmod n$
- $a^b \bmod n$ se puede calcular eficientemente mediante exponenciación modular.

Inverso Modular

El **inverso modular** de a módulo n es un entero x tal que:

$$a \cdot x \equiv 1 \pmod{n}$$

El inverso existe si y solo si $\gcd(a, n) = 1$.

Algoritmo de Euclides para el Máximo Común Divisor

Descripción

El algoritmo de Euclides calcula el máximo común divisor (MCD) de dos números enteros.

Pseudocódigo

```
[language=C, caption=Algoritmo de Euclides] int euclides(int a, int b) while (b != 0) int temp = b; b = a % b; a = temp; return a;
```

Ejemplo

Calcular $\gcd(48, 18)$:

a=48, b=18: $48 \% 18 = 12$, a=18, b=12

a=18, b=12: $18 \% 12 = 6$, a=12, b=6

a=12, b=6: $12 \% 6 = 0$, a=6, b=0

Resultado: 6

Análisis de Complejidad

- Tiempo: $O(\log \min(a, b))$ (algoritmo más rápido conocido)
- Espacio: $O(1)$

Demostración de Correctitud

El algoritmo de Euclides calcula correctamente $\gcd(a, b)$. $\gcd(a, b) = \gcd(b, a \bmod b)$. Esta propiedad se demuestra porque si d divide a a y b , entonces divide a $a \bmod b = a - \lfloor a/b \rfloor \cdot b$. Por lo tanto, los divisores comunes se preservan.

Algoritmo de Euclides Extendido

Descripción

El algoritmo de Euclides extendido encuentra enteros x e y tales que:

$$ax + by = \gcd(a, b)$$

Pseudocódigo

```
[language=C, caption=Algoritmo de Euclides Extendido] int euclidesExten-
dido(int a, int b, int* x, int* y) if (b == 0) *x = 1; *y = 0; return a;
    int x1, y1; int gcd = euclidesExtendido(b, a
    *x = y1; *y = x1 - (a / b) * y1;
    return gcd;
```

Ejemplo

Para $a = 48$, $b = 18$:

```
euclidesExtendido(48,18)
  euclidesExtendido(18,12)
    euclidesExtendido(12,6)
      euclidesExtendido(6,0) -> gcd=6, x=1, y=0
      x=0, y=1 - (12/6)*0 = 1
      gcd=6
    x=1, y=0 - (18/12)*1 = -1
    gcd=6
  x=-1, y=1 - (48/18)*(-1) = 1 + 2 = 3
Resultado: 48*(-1) + 18*3 = -48 + 54 = 6
```

Aplicaciones del Euclides Extendido

- Cálculo de inversos modulares
- Resolución de ecuaciones diofánticas
- Criptografía RSA

Cálculo del Inverso Modular

Para calcular $a^{-1} \bmod n$ (con $\gcd(a, n) = 1$): [language=C, caption=Inverso modular] int inversoModular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y); if (gcd != 1) return -1; // inverso no existe // x puede ser negativo, ajustar return (x

Exponenciación Modular

Descripción

Calcular $a^b \bmod n$ eficientemente sin calcular a^b directamente (que puede ser enorme).

Algoritmo de Exponenciación Rápida

[language=C, caption=Exponenciación modular rápida] int potenciaModular(int a, int b, int n) int resultado = 1; a = a % n; while (b != 0) if (b % 2 == 1) resultado = (resultado * a) % n; a = (a * a) % n; b = b / 2; return resultado;

Ejemplo

Calcular $2^{10} \bmod 1000$:

```
a=2, b=10, n=1000
b=10 (par): a=4, b=5
b=5 (impar): resultado=4, a=16, b=2
b=2 (par): a=256, b=1
b=1 (impar): resultado=4*256=1024, a=65536, b=0
resultado % 1000 = 24
```

$$2^{10} = 1024 \equiv 24 \pmod{1000}$$

Análisis de Complejidad

- Tiempo: $O(\log b)$ multiplicaciones modulares
- Espacio: $O(1)$

Introducción a la Criptografía RSA

Fundamentos Matemáticos

RSA se basa en:

1. La dificultad de factorizar números grandes.
2. El teorema de Euler.
3. La existencia de inversos modulares.

Generación de Claves

1. Elegir dos números primos grandes p y q .
2. Calcular $n = p \cdot q$.
3. Calcular $\phi(n) = (p - 1)(q - 1)$.
4. Elegir e tal que $1 < e < \phi(n)$ y $\gcd(e, \phi(n)) = 1$.
5. Calcular $d = e^{-1} \bmod \phi(n)$ (usando Euclides extendido).

La clave pública es (n, e) , la clave privada es (n, d) .

Cifrado y Descifrado

- **Cifrado:** $c = m^e \bmod n$
- **Descifrado:** $m = c^d \bmod n$

Ejemplo Simple

$p=61, q=53$

$n = 61 \cdot 53 = 3233$

$\phi(n) = 60 \cdot 52 = 3120$

$e = 17$ (primo con 3120)

$d = \text{inverso de } 17 \bmod 3120 = 2753$

Clave pública: $(3233, 17)$

Clave privada: $(3233, 2753)$

Mensaje $m = 65$

$c = 65^{17} \bmod 3233 = 2790$

Descifrado: $2790^{2753} \bmod 3233 = 65$

```
[language=C, caption=RSA básico] include <stdio.h> include <stdlib.h>
typedef struct { int n; int e; } PublicKey;
typedef struct { int n; int d; } PrivateKey;
// Generar claves (simplificado - solo para números pequeños) void gener-
arClaves(int p, int q, PublicKey* pub, PrivateKey* priv) { pub->n = p * q;
int phi = (p - 1) * (q - 1);
// Elegir e (usamos 65537 en la práctica) pub->e = 65537;
// Calcular d int x, y; euclidesExtendido(pub->e, phi, x, y); priv->d = (x
priv->n = pub->n;
int cifrar(int m, PublicKey* pub) { return potenciaModular(m, pub->e,
pub->n);
int descifrar(int c, PrivateKey* priv) { return potenciaModular(c, priv->d,
priv->n);
```

La seguridad de RSA se basa en:

- Dificultad de factorizar n en p y q .
- Dificultad de calcular $\phi(n)$ sin conocer p y q .
- Dificultad de calcular d a partir de e y n .

Criba de Eratóstenes

```

Para encontrar todos los números primos hasta  $n$ . [language=C, caption=Criba
de Eratóstenes] void cribaEratostenes(int n) int* esPrimo = (int*)malloc((n
+ 1) * sizeof(int)); for (int i = 2; i ≤ n; i++) esPrimo[i] = 1;
for (int i = 2; i * i ≤ n; i++) if (esPrimo[i]) for (int j = i * i; j ≤ n; j
+= i) esPrimo[j] = 0;
for (int i = 2; i ≤ n; i++) if (esPrimo[i]) printf(" free(esPrimo);

```

Algoritmo probabilístico para determinar si un número es primo.

```

[language=C, caption=Miller-Rabin] int millerRabin(int n, int k)  if (n < 2)
2) return 0; if (n == 2 — n == 3) return 1; if (n

```

```
// Escribir n-1 como d * 2^s
int d = n - 1; int s = 0; while(d % 2 == 0) { d /= 2; s++; }
for (int i = 0; i < k; i++) { int a = 2 + rand(); int x = potenciaModular(a, d, n);
if (x == 1 || x == n - 1) continue;
int compuesto = 1; for (int r = 0; r < s - 1; r++) x = (x * x) % n;
if (x == n - 1) compuesto = 0; break;
if (compuesto) return 0;
return 1; }
```

Ejercicios Resueltos (20 Ejercicios)

Calcule $\gcd(84, 36)$ usando el algoritmo de Euclides.

$84 \% 36 = 12$, $a=36$, $b=12$

$36 \% 12 = 0$, $a=12$, $b=0$

Resultado: 12

Encuentre x e y tales que $84x + 36y = \gcd(84, 36)$. Usando Euclides extendido:

$$84 = 2 \cdot 36 + 12 \rightarrow 12 = 84 - 2 \cdot 36$$

$$36 = 3 \cdot 12 + 0$$

$$x = 1, y = -2: 84 \cdot 1 + 36 \cdot (-2) = 84 - 72 = 12$$

Calcule el inverso de 7 módulo 26. $\gcd(7, 26) = 1$, existe inverso.

$$26 = 3 \cdot 7 + 5 \rightarrow 5 = 26 - 3 \cdot 7$$

$$7 = 1 \cdot 5 + 2 \rightarrow 2 = 7 - 1 \cdot 5 = 7 - (26 - 3 \cdot 7) = 4 \cdot 7 - 26$$

$$5 = 2 \cdot 2 + 1 \rightarrow 1 = 5 - 2 \cdot 2 = (26 - 3 \cdot 7) - 2 \cdot (4 \cdot 7 - 26) = 3 \cdot 26 - 11 \cdot 7$$

$-11 \equiv 15 \pmod{26}$. El inverso es 15.

Calcule $3^{20} \bmod 11$ usando exponenciación modular.

$a=3$, $b=20$, $n=11$

$b=20$ par: $a=9$, $b=10$

$b=10$ par: $a=81 \bmod 11 = 4$, $b=5$

$b=5$ impar: resultado=4, $a=16 \bmod 11 = 5$, $b=2$

$b=2$ par: $a=25 \bmod 11 = 3$, $b=1$

$b=1$ impar: resultado= $4 \cdot 3 = 12 \bmod 11 = 1$

$$3^{20} \bmod 11 = 1$$

Genere claves RSA con $p = 11$, $q = 17$.

$n = 11 * 17 = 187$
 $(n) = 10 * 16 = 160$
 $e = 7$ (primo con 160)
 $d = \text{inverso de } 7 \text{ mod } 160$
 $7 * 23 = 161 \equiv 1 \text{ mod } 160$, $d = 23$
 Clave pública: $(187, 7)$
 Clave privada: $(187, 23)$

Cifre $m = 15$ con la clave pública del ejercicio anterior. $c = 15^7 \text{ mod } 187$

$15^2 = 225 \text{ mod } 187 = 38$
 $15^4 = 38^2 = 1444 \text{ mod } 187 = 1444 - 7*187 = 1444 - 1309 = 135$
 $15^7 = 15^4 * 15^2 * 15 = 135 * 38 * 15 \text{ mod } 187$
 $135 * 38 = 5130 \text{ mod } 187 = 5130 - 27*187 = 5130 - 5049 = 81$
 $81 * 15 = 1215 \text{ mod } 187 = 1215 - 6*187 = 1215 - 1122 = 93$

$c = 93$

Ejercicios Propuestos (20 Ejercicios)

1. Calcule $\text{gcd}(144, 60)$ usando Euclides.
2. Encuentre x e y para $144x + 60y = \text{gcd}(144, 60)$.
3. Calcule el inverso de 9 módulo 14.
4. Calcule $5^{100} \text{ mod } 13$.
5. Genere claves RSA con $p = 13$, $q = 19$.
6. Cifre $m = 42$ con las claves del ejercicio anterior.
7. Descifre el resultado del ejercicio anterior.
8. Implemente el algoritmo de Euclides en Python.
9. Implemente el algoritmo de Euclides extendido en C.
10. ¿Qué es el teorema de Euler y cómo se usa en RSA?
11. ¿Por qué RSA es seguro?
12. ¿Qué es la función de Euler?

13. Calcule (100).
14. Implemente la exponenciación modular rápida.
15. ¿Qué es la criba de Eratóstenes y para qué sirve?
16. Implemente el test de Miller-Rabin.
17. ¿Qué es un número primo y por qué son importantes en criptografía?
18. ¿Cómo se eligen p y q en RSA en la práctica?
19. ¿Qué es el problema de factorización?
20. Investigue aplicaciones de RSA en la vida real.

Resumen de la Unidad

En esta unidad hemos aprendido:

- Los fundamentos de la aritmética modular.
- El algoritmo de Euclides para MCD ($O(\log n)$).
- El algoritmo de Euclides extendido para resolver $ax + by = \gcd(a, b)$.
- La exponenciación modular rápida ($O(\log b)$).
- Los conceptos básicos de RSA: generación de claves, cifrado y descifrado.
- Algoritmos adicionales: criba de Eratóstenes, test de Miller-Rabin.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
- Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120-126.
- Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman Hall/CRC.

Unidad XI: Aritmética Modular

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los fundamentos de la aritmética modular y sus propiedades.
- Realizar operaciones aritméticas básicas en módulo.
- Calcular inversos modulares utilizando el algoritmo de Euclides extendido.
- Aplicar el teorema chino del resto para resolver sistemas de congruencias.
- Comprender y aplicar el teorema de Fermat y Euler.
- Utilizar la aritmética modular en aplicaciones criptográficas.

Introducción a la Aritmética Modular

Definición de Congruencia

Sean $a, b, n \in \mathbb{Z}$ con $n > 0$. Decimos que a es **congruente** a b módulo n , denotado:

$$a \equiv b \pmod{n}$$

si n divide a $a - b$, es decir, $a - b = kn$ para algún entero k .

- $17 \equiv 2 \pmod{5}$ porque $17 - 2 = 15 = 3 \cdot 5$
- $10 \equiv 0 \pmod{2}$ (10 es par)
- $-3 \equiv 4 \pmod{7}$ porque $-3 - 4 = -7 = -1 \cdot 7$

Propiedades Básicas

La congruencia módulo n es una relación de equivalencia:

1. **Reflexiva:** $a \equiv a \pmod{n}$
2. **Simétrica:** $a \equiv b \pmod{n} \implies b \equiv a \pmod{n}$
3. **Transitiva:** $a \equiv b \pmod{n}$ y $b \equiv c \pmod{n} \implies a \equiv c \pmod{n}$

Operaciones en Aritmética Modular

Si $a \equiv b \pmod{n}$ y $c \equiv d \pmod{n}$, entonces:

1. $a + c \equiv b + d \pmod{n}$
2. $a - c \equiv b - d \pmod{n}$
3. $a \cdot c \equiv b \cdot d \pmod{n}$

Conjunto de Residuos

El conjunto de **residuos** módulo n es:

$$Z_n = \{0, 1, 2, \dots, n-1\}$$

Cada entero es congruente a exactamente un elemento de Z_n .

Operaciones Aritméticas Modulares

Suma Modular

```
[language=C, caption=Suma modular] int sumaModular(int a, int b, int n)
return (a
```

Resta Modular

```
[language=C, caption=Resta modular] int restaModular(int a, int b, int n)
return ((a
```

Multiplicación Modular

```
[language=C, caption=Multiplicación modular] int multModular(int a, int
b, int n) return ((long long)(a
```

Exponenciación Modular

Ya vista en el capítulo anterior, se utiliza para calcular $a^b \bmod n$ eficientemente.

Inverso Modular

Definición

El **inverso modular** de a módulo n es un entero x tal que:

$$a \cdot x \equiv 1 \pmod{n}$$

Se denota $x \equiv a^{-1} \pmod{n}$.

Existencia del Inverso Modular

a tiene inverso módulo n si y solo si $\gcd(a, n) = 1$.

- **Si:** $\gcd(a, n) = 1$, existen enteros x, y tales que $ax + ny = 1$. Entonces $ax \equiv 1 \pmod{n}$.
- **Solo si:** Si $ax \equiv 1 \pmod{n}$, entonces $ax - 1 = kn$, por lo que $ax - kn = 1$. Cualquier divisor común de a y n debe dividir a 1, así que $\gcd(a, n) = 1$.

Cálculo del Inverso Modular con Euclides Extendido

```
[language=C, caption=Inverso modular con Euclides extendido] int inverso-
Modular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y); if
(gcd != 1) return -1; // no existe inverso // Asegurar que el resultado sea
positivo return (x
```

Ejemplos de Inversos Modulares

a	n	$a^{-1} \bmod n$
3	7	5 (porque $3 \cdot 5 = 15 \equiv 1 \pmod{7}$)
4	9	7 (porque $4 \cdot 7 = 28 \equiv 1 \pmod{9}$)
2	10	No existe ($\gcd(2, 10) = 2 \neq 1$)

Teorema Chino del Resto (CRT)

Enunciado del Teorema

[Teorema Chino del Resto] Sean $n_1, n_2, \dots, n_k$ enteros positivos coprimos entre sí (es decir, $\gcd(n_i, n_j) = 1$ para $i \neq j$). Para cualquier conjunto de residuos $a_1, a_2, \dots, a_k$, el sistema de congruencias:

$$\{x \equiv a_1 \pmod{n_1}, x \equiv a_2 \pmod{n_2}, \dots, x \equiv a_k \pmod{n_k}\}$$

tiene una solución única módulo $N = n_1 n_2 \cdots n_k$.

Algoritmo de Solución

```
[language=C, caption=Teorema Chino del Resto] int teoremaChinoResto(int*
a, int* n, int k) { int N = 1; for (int i = 0; i < k; i++) N *= n[i];
    int resultado = 0; for (int i = 0; i < k; i++) { int Ni = N / n[i]; int inv
= inversoModular(Ni, n[i]); resultado = (resultado + a[i] * Ni * inv) % N; } return
resultado; }
```

Ejemplo

Resolver:

$$\{x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}\}$$

$$N = 3 \cdot 5 \cdot 7 = 105$$

- $N_1 = 105/3 = 35$, $35^{-1} \pmod{3} = 2$, término = $2 \cdot 35 \cdot 2 = 140$
- $N_2 = 105/5 = 21$, $21^{-1} \pmod{5} = 1$, término = $3 \cdot 21 \cdot 1 = 63$
- $N_3 = 105/7 = 15$, $15^{-1} \pmod{7} = 1$, término = $2 \cdot 15 \cdot 1 = 30$

Suma: $140 + 63 + 30 = 233 \equiv 233 - 2 \cdot 105 = 23 \pmod{105}$ Solución: $x \equiv 23 \pmod{105}$

Teorema de Fermat y Euler

Teorema de Fermat

[Pequeño Teorema de Fermat] Si p es primo y a no es divisible por p , entonces:

$$a^{p-1} \equiv 1 \pmod{p}$$

Para $p = 7$, $a = 2$: $2^6 = 64 \equiv 1 \pmod{7}$. Para $p = 5$, $a = 3$: $3^4 = 81 \equiv 1 \pmod{5}$.

Función de Euler

La función $\phi(n)$ cuenta los enteros $1 \leq k \leq n$ tales que $\gcd(k, n) = 1$.

Propiedades de

- $\phi(p) = p - 1$ si p es primo.
- $\phi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1)$.
- Si $\gcd(m, n) = 1$, entonces $\phi(mn) = \phi(m) \cdot \phi(n)$.

Teorema de Euler

[Teorema de Euler] Si $\gcd(a, n) = 1$, entonces:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

$$\phi(10) = 4, 3^4 = 81 \equiv 1 \pmod{10}.$$

Aplicaciones de la Aritmética Modular

Verificación de Números Primos

El test de primalidad de Miller-Rabin se basa en propiedades de la aritmética modular.

Criptografía RSA

La aritmética modular es la base de RSA:

- Cifrado: $c = m^e \pmod{n}$
- Descifrado: $m = c^d \pmod{n}$

Códigos de Verificación

Los dígitos de control (como el DNI o tarjetas de crédito) usan aritmética modular.

Hashing

Las funciones hash modulares son comunes en tablas hash.

Ejercicios Resueltos (20 Ejercicios)

Calcule $27 \bmod 5$. $27 \div 5 = 5$ con resto 2, entonces $27 \bmod 5 = 2$.

Demuestre que $100 \equiv 1 \pmod{9}$. $100 - 1 = 99 = 11 \cdot 9$, por lo tanto $100 \equiv 1 \pmod{9}$.

Calcule $(15 + 23) \bmod 7$. $15 \bmod 7 = 1$, $23 \bmod 7 = 2$, $1 + 2 = 3$. Por lo tanto, $(15 + 23) \bmod 7 = 3$.

Calcule $(15 \cdot 23) \bmod 7$. $15 \bmod 7 = 1$, $23 \bmod 7 = 2$, $1 \cdot 2 = 2$. Por lo tanto, $(15 \cdot 23) \bmod 7 = 2$.

Encuentre el inverso de 5 módulo 12. Probamos: $5 \cdot 5 = 25 \equiv 1 \pmod{12}$ (porque $25 - 1 = 24$ es múltiplo de 12). El inverso es 5.

¿Existe el inverso de 6 módulo 10? ¿Por qué? No existe porque $\gcd(6, 10) = 2 \neq 1$.

Calcule $3^{10} \bmod 7$ usando el teorema de Fermat. Por Fermat, $3^6 \equiv 1 \pmod{7}$. Entonces $3^{10} = 3^6 \cdot 3^4 \equiv 1 \cdot 3^4 = 81 \equiv 4 \pmod{7}$.

Calcule $\phi(12)$. Los números coprimos con 12 entre 1 y 12 son: 1, 5, 7, 11. Por lo tanto, $\phi(12) = 4$.

Resuelva $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{4}$ usando CRT. $N = 12$, $N_1 = 4$, $4^{-1} \bmod 3 = 1$, término = $2 \cdot 4 \cdot 1 = 8$. $N_2 = 3$, $3^{-1} \bmod 4 = 3$, término = $3 \cdot 3 \cdot 3 = 27$. Suma = $35 \equiv 11 \pmod{12}$.

Ejercicios Propuestos (20 Ejercicios)

1. Calcule $47 \bmod 8$.
2. Demuestre que $144 \equiv 0 \pmod{12}$.
3. Calcule $(25 + 37) \bmod 6$.
4. Calcule $(25 \cdot 37) \bmod 6$.
5. Encuentre el inverso de 3 módulo 8.
6. Encuentre el inverso de 7 módulo 15.
7. ¿Existe el inverso de 9 módulo 12? ¿Por qué?
8. Calcule $2^{15} \bmod 11$ usando Fermat.
9. Calcule $\phi(15)$.
10. Calcule $\phi(30)$.
11. Resuelva $x \equiv 1 \pmod{2}$, $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$.

12. Demuestre el teorema de Fermat.
13. Demuestre el teorema de Euler.
14. ¿Qué es un sistema completo de residuos?
15. ¿Qué es un sistema reducido de residuos?
16. Implemente la suma modular en C.
17. Implemente el cálculo del inverso modular.
18. ¿Cómo se usa el CRT en RSA?
19. ¿Qué es la función de Carmichael?
20. Investigue aplicaciones de la aritmética modular en códigos de barras.

Resumen de la Unidad

En esta unidad hemos aprendido:

- Los fundamentos de la congruencia y la aritmética modular.
- Las operaciones básicas: suma, resta, multiplicación y exponenciación modular.
- El concepto de inverso modular y su cálculo mediante Euclides extendido.
- El Teorema Chino del Resto y su aplicación.
- El teorema de Fermat y el teorema de Euler.
- Aplicaciones en criptografía y verificación.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
- Rosen, K. H. (2010). *Elementary Number Theory and Its Applications* (6th ed.). Addison-Wesley.
- Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman Hall/CRC.

Unidad XII: Algoritmo Extendido de Euclides

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el funcionamiento del algoritmo de Euclides estándar.
- Comprender y demostrar el algoritmo de Euclides extendido.
- Implementar el algoritmo de Euclides extendido en sus formas recursiva e iterativa.
- Aplicar el algoritmo para calcular inversos modulares.
- Resolver ecuaciones diofánticas lineales.
- Aplicar el algoritmo en criptografía (RSA) y otros contextos.

Algoritmo de Euclides Estándar (Repaso)

Definición del MCD

El **máximo común divisor** (MCD) de dos enteros a y b , denotado $\gcd(a, b)$, es el mayor entero d tal que d divide a a y d divide a b .

Propiedad Fundamental

Para cualquier enteros a y b con $b > 0$:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Algoritmo de Euclides

```
[language=C, caption=Algoritmo de Euclides estándar] int euclides(int a, int b) while (b != 0) int temp = b; b = a % b; a = temp; return a;
```

Ejemplo Visual

```
[scale=0.8] (0,0) rectangle (4,0.5); at (2,0.25) a = 48, b = 18;
(0,-1) rectangle (4,-0.5); at (2,-0.75) 48 % 18 = 12 → a = 18, b = 12;
(0,-2) rectangle (4,-1.5); at (2,-1.75) 18 % 12 = 6 → a = 12, b = 6;
(0,-3) rectangle (4,-2.5); at (2,-2.75) 12 % 6 = 0 → a = 6, b = 0;
[thick, blue] (0,-3.5) rectangle (4,-3); at (2,-3.25) Resultado: gcd = 6;
```

Algoritmo de Euclides Extendido

Motivación

El algoritmo de Euclides estándar solo calcula el MCD. El algoritmo extendido encuentra además coeficientes x e y tales que:

$$ax + by = \gcd(a, b)$$

Fundamento Matemático

Para cualquier par de enteros a y b (no ambos cero), existen enteros x e y tales que:

$$ax + by = \gcd(a, b)$$

Derivación del Algoritmo

Partimos de la relación recursiva del algoritmo de Euclides:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Si podemos encontrar coeficientes x_1, y_1 tales que:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = \gcd(b, a \bmod b)$$

Notando que $a \bmod b = a - \lfloor a/b \rfloor \cdot b$, tenemos:

$$b \cdot x_1 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_1 = \gcd(a, b)$$

$$a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) = \gcd(a, b)$$

Por lo tanto:

$$x = y_1, \quad y = x_1 - \lfloor a/b \rfloor \cdot y_1$$

Implementación Recursiva

```
[language=C, caption=Euclides Extendido Recursivo] include <stdio.h>
// Función que devuelve el gcd y calcula x, y tales que a*x + b*y = gcd
int euclidesExtendido(int a, int b, int* x, int* y) if (b == 0) *x = 1; *y =
0; return a;
    int x1, y1; int gcd = euclidesExtendido(b, a
    *x = y1; *y = x1 - (a / b) * y1;
    return gcd;
```

Ejemplo Detallado

Calcular coeficientes para $a = 48$, $b = 18$:

Paso	a	b	a/b	a % b
1	48	18	2	12
2	18	12	1	6
3	12	6	2	0
4	6	0	-	-

Desenrollado:

Paso 4: $b=0$, $\text{gcd}=6$, $x=1$, $y=0$

Paso 3: $a=12$, $b=6$, $q=2$

$x = y1 = 0$

$y = x1 - q*y1 = 1 - 2*0 = 1$

Paso 2: $a=18$, $b=12$, $q=1$

$x = y1 = 1$

$y = x1 - q*y1 = 0 - 1*1 = -1$

Paso 1: $a=48$, $b=18$, $q=2$

$x = y1 = -1$

$y = x1 - q*y1 = 1 - 2*(-1) = 1 + 2 = 3$

Resultado: $48 \cdot (-1) + 18 \cdot 3 = -48 + 54 = 6$

Implementación Iterativa

```
[language=C, caption=Euclides Extendido Iterativo] int euclidesExtendidoI-
terativo(int a, int b, int* x, int* y) int x0 = 1, x1 = 0; int y0 = 0, y1 = 1;
int originalA = a; int originalB = b;
    while (b != 0) int q = a / b; int r = a
    // Actualizar coeficientes int tempX = x1; int tempY = y1; x1 = x0 - q
    * x1; y1 = y0 - q * y1; x0 = tempX; y0 = tempY;
```

```

a = b; b = r;
*x = x0; *y = y0; return a;

```

Aplicaciones del Algoritmo Extendido

Cálculo del Inverso Modular

El inverso modular de a módulo n existe si y solo si $\gcd(a, n) = 1$. En ese caso, el inverso es x de la ecuación $ax + ny = 1$.

```

[language=C, caption=Inverso modular con Euclides extendido] int in-
versoModular(int a, int n) int x, y; int gcd = euclidesExtendido(a, n, x, y);
if (gcd != 1) return -1; // No existe inverso // Asegurar resultado positivo
return (x

```

Calcular $3^{-1} \bmod 7$:

```

euclidesExtendido(3, 7, &x, &y)
3* -2 + 7*1 = 1
x = -2  5 mod 7
Inverso: 5 (porque 3*5=15=1 mod 7)

```

Ecuaciones Diofánticas Lineales

Resolver $ax + by = c$ donde a, b, c son enteros.

La ecuación $ax + by = c$ tiene solución si y solo si $\gcd(a, b)$ divide a c .

```

[language=C, caption=Resolver ecuación diofántica] int resolverEcuacionDio-
fantica(int a, int b, int c, int* x, int* y) int x0, y0; int gcd = euclidesExten-
dido(a, b, x0, y0);
if (c % gcd != 0) return 0; // No tiene solución
int factor = c / gcd; *x = x0 * factor; *y = y0 * factor;
return 1; // Solución encontrada
Resolver 48x + 18y = 30:

```

$\gcd(48, 18) = 6$, 30 es divisible por 6.

Solución particular: $x_0 = -1$, $y_0 = 3$ (de $48*(-1) + 18*3 = 6$)

Multiplicando por factor = $30/6 = 5$:

$x = -5$, $y = 15$

Verificar: $48*(-5) + 18*15 = -240 + 270 = 30$

Aplicación en RSA

En RSA, se necesita calcular $d = e^{-1} \bmod \phi(n)$:

```
euclidesExtendido(e, phi, &d, &temp);
d = (d % phi + phi) % phi;
```

Fracciones Continuas

El algoritmo extendido también se puede usar para generar fracciones continuas.

Demostración de Correctitud

El algoritmo de Euclides extendido devuelve enteros x, y tales que $ax + by = \gcd(a, b)$.

Por inducción en el número de iteraciones.

Caso base: Cuando $b = 0$, $\gcd(a, 0) = a$, y la ecuación $a \cdot 1 + 0 \cdot 0 = a$ es válida.

Paso inductivo: Supongamos que para la llamada recursiva con parámetros b y $a \bmod b$ obtenemos:

$$b \cdot x_1 + (a \bmod b) \cdot y_1 = \gcd(b, a \bmod b)$$

Como $\gcd(a, b) = \gcd(b, a \bmod b)$ y $a \bmod b = a - \lfloor a/b \rfloor \cdot b$, sustituimos:

$$b \cdot x_1 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_1 = \gcd(a, b)$$

$$a \cdot y_1 + b \cdot (x_1 - \lfloor a/b \rfloor \cdot y_1) = \gcd(a, b)$$

Por lo tanto, $x = y_1$, $y = x_1 - \lfloor a/b \rfloor \cdot y_1$ satisfacen la ecuación.

Análisis de Complejidad

- **Tiempo:** $O(\log \min(a, b))$, igual que el algoritmo de Euclides estándar.
- **Espacio:** $O(\log \min(a, b))$ para la versión recursiva, $O(1)$ para la iterativa.

Comparación de Implementaciones

Característica	Recursiva	Iterativa
Claridad del código	Alta	Media
Riesgo de desbordamiento de pila	Sí (para números grandes)	No
Eficiencia	Similar	Similar
Facilidad de depuración	Media	Alta

Ejercicios Resueltos (20 Ejercicios)

Calcule $\gcd(56, 21)$ usando el algoritmo de Euclides estándar.

$$56 \% 21 = 14, \text{ a}=21, \text{ b}=14$$

$$21 \% 14 = 7, \text{ a}=14, \text{ b}=7$$

$$14 \% 7 = 0, \text{ a}=7, \text{ b}=0$$

Resultado: 7

Encuentre x e y tales que $56x + 21y = \gcd(56, 21)$.

$$56 = 2 \cdot 21 + 14 \rightarrow 14 = 56 - 2 \cdot 21$$

$$21 = 1 \cdot 14 + 7 \rightarrow 7 = 21 - 1 \cdot 14 = 21 - (56 - 2 \cdot 21) = 3 \cdot 21 - 56$$

$$x = -1, y = 3: 56 \cdot (-1) + 21 \cdot 3 = -56 + 63 = 7$$

Calcule el inverso de 17 módulo 31.

$$31 = 1 \cdot 17 + 14 \rightarrow 14 = 31 - 17$$

$$17 = 1 \cdot 14 + 3 \rightarrow 3 = 17 - 14 = 17 - (31 - 17) = 2 \cdot 17 - 31$$

$$14 = 4 \cdot 3 + 2 \rightarrow 2 = 14 - 4 \cdot 3 = (31 - 17) - 4 \cdot (2 \cdot 17 - 31) = 31 - 17 - 8 \cdot 17 + 4 \cdot 31 = 5 \cdot 31 - 11 \cdot 17$$

$$3 = 1 \cdot 2 + 1 \rightarrow 1 = 3 - 2 = (2 \cdot 17 - 31) - (5 \cdot 31 - 11 \cdot 17) = 11 \cdot 17 - 6 \cdot 31$$

$$11 \cdot 17 \equiv 1 \pmod{31}, \text{ inverso es } 11.$$

Resuelva $23x + 17y = 1$.

$$23 = 1 \cdot 17 + 6 \rightarrow 6 = 23 - 17$$

$$17 = 2 \cdot 6 + 5 \rightarrow 5 = 17 - 2 \cdot 6 = 17 - 2 \cdot (23 - 17) = 3 \cdot 17 - 2 \cdot 23$$

$$6 = 1 \cdot 5 + 1 \rightarrow 1 = 6 - 5 = (23 - 17) - (3 \cdot 17 - 2 \cdot 23) = 3 \cdot 23 - 4 \cdot 17$$

$$x = 3, y = -4: 23 \cdot 3 + 17 \cdot (-4) = 69 - 68 = 1$$

Implemente el algoritmo de Euclides extendido en Python. [language=Python] def euclides_extendido(a, b) : if b == 0 : return a, 1, 0 gcd, x1, y1 = euclides_extendido(b, a) x = y1 y = x1 - (a//b) * y1 return gcd, x, y

¿Cuál es la complejidad del algoritmo de Euclides extendido? $O(\log \min(a, b))$ operaciones aritméticas.

Ejercicios Propuestos (20 Ejercicios)

1. Calcule $\gcd(144, 60)$ usando Euclides.
2. Encuentre x, y para $144x + 60y = \gcd(144, 60)$.
3. Calcule el inverso de 19 módulo 37.
4. Resuelva $31x + 13y = 1$.
5. ¿Existe inverso de 6 módulo 14? Justifique.
6. Implemente Euclides extendido iterativo en C.
7. Demuestre la fórmula de recurrencia del algoritmo extendido.
8. ¿Cómo se usa Euclides extendido en RSA?
9. Resuelva $48x + 18y = 12$.
10. Calcule $\gcd(123456, 789012)$.
11. Encuentre el inverso de 7 módulo 26.
12. Resuelva $15x + 9y = 6$.
13. ¿Qué es una ecuación diofántica lineal?
14. Implemente la función inverso modular usando Euclides extendido.
15. Demuestre que el algoritmo de Euclides extendido es correcto.
16. ¿Cuántas iteraciones hace Euclides extendido para $a = 1, b = 10^6$?
17. Calcule $\gcd(2^{10}, 3^5)$.
18. Encuentre x e y para $100x + 37y = 1$.
19. ¿Qué relación hay entre el algoritmo de Euclides y las fracciones continuas?
20. Investigue aplicaciones del algoritmo extendido en criptografía.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El algoritmo de Euclides estándar para calcular el MCD.
- El algoritmo de Euclides extendido para encontrar $ax + by = \gcd(a, b)$.
- Implementaciones recursiva e iterativa.
- Aplicaciones: inverso modular, ecuaciones diofánticas, RSA.
- Análisis de complejidad $O(\log \min(a, b))$.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley.
- Rosen, K. H. (2010). *Elementary Number Theory and Its Applications* (6th ed.). Addison-Wesley.

Unidad XIII: Nociones de Criptografía Básica (RSA)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender los conceptos fundamentales de la criptografía.
- Distinguir entre criptografía simétrica y asimétrica.
- Comprender el funcionamiento del algoritmo RSA.
- Implementar las operaciones básicas de RSA.
- Comprender los fundamentos matemáticos de RSA.
- Analizar la seguridad de RSA y sus aplicaciones prácticas.

Introducción a la Criptografía

Definición

La **criptografía** es la práctica y estudio de técnicas para asegurar comunicaciones en presencia de adversarios. Los objetivos principales son:

- **Confidencialidad:** Solo el destinatario puede leer el mensaje.
- **Integridad:** El mensaje no puede ser alterado sin detección.
- **Autenticación:** Se verifica la identidad del remitente.
- **No repudio:** El remitente no puede negar haber enviado el mensaje.

Criptografía Simétrica

En la criptografía simétrica, el mismo secreto se usa para cifrar y descifrar.

[scale=0.8] (msg) at (0,0) Mensaje; (cifrado) at (2,0) Cifrado; (descifrado)
 at (4,0) Mensaje; (clave) at (2,1) Clave;
 [-i] (msg) – (cifrado); [-i] (cifrado) – (descifrado); [-i] (clave) – (cifrado);
 [-i] (clave) – (descifrado);

Ejemplos: AES, DES, ChaCha20.

Criptografía Asimétrica (Clave Pública)

En la criptografía asimétrica, se usan dos claves diferentes: una pública para cifrar y una privada para descifrar.

[scale=0.8] (msg) at (0,0) Mensaje; (cifrado) at (2.5,0) Cifrado; (descifrado)
 at (5,0) Mensaje; (pub) at (2.5,1) Clave Pública; (priv) at (2.5,-1) Clave
 Privada;
 [-i] (msg) – (cifrado); [-i] (cifrado) – (descifrado); [-i] (pub) – (cifrado); [-i]
 (priv) – (descifrado);

Ejemplos: RSA, ECC, ElGamal.

Fundamentos Matemáticos de RSA

Teorema de Euler

Si $\gcd(a, n) = 1$, entonces $a^{\phi(n)} \equiv 1 \pmod{n}$, donde $\phi(n)$ es la función de Euler.

Función de Euler

- $\phi(p) = p - 1$ para p primo.
- $\phi(pq) = (p - 1)(q - 1)$ para p y q primos distintos.

Pequeño Teorema de Fermat

Si p es primo y a no es múltiplo de p , entonces $a^{p-1} \equiv 1 \pmod{p}$.

El Algoritmo RSA

Generación de Claves

1. Elegir dos números primos grandes p y q .
2. Calcular $n = p \cdot q$.
3. Calcular $\phi(n) = (p - 1)(q - 1)$.
4. Elegir e tal que $1 < e < \phi(n)$ y $\gcd(e, \phi(n)) = 1$.
5. Calcular $d = e^{-1} \bmod \phi(n)$ (usando Euclides extendido).

La clave pública es (n, e) . La clave privada es (n, d) .

Cifrado

Para cifrar un mensaje m (con $0 \leq m < n$):

$$c = m^e \bmod n$$

Descifrado

Para descifrar un mensaje cifrado c :

$$m = c^d \bmod n$$

Demostración de Correctitud

Para cualquier mensaje $0 \leq m < n$, se cumple que $(m^e)^d \equiv m \pmod{n}$.

Como $ed \equiv 1 \pmod{\phi(n)}$, existe k tal que $ed = 1 + k\phi(n)$.

$$(m^e)^d = m^{ed} = m^{1+k\phi(n)} = m \cdot (m^{\phi(n)})^k$$

Por el teorema de Euler, $m^{\phi(n)} \equiv 1 \pmod{n}$, por lo que $(m^e)^d \equiv m \pmod{n}$.

Implementación de RSA

Estructuras de Datos

```
[language=C, caption=Estructuras para RSA] typedef struct long long n;
// módulo long long e; // exponente público PublicKey;
typedef struct long long n; // módulo long long d; // exponente privado
PrivateKey;
```

Generación de Claves

```
[language=C, caption=Generación de claves RSA (simplificada)] include <stdio.h>
include <stdlib.h> include <time.h>
```

```
// Función para verificar si un número es primo (simplificada) int esP-
rimo(int n) if (n < 2) return 0; for (int i = 2; i * i <= n; i++) if (n % i == 0) return 0;
return 1;
```

```
// Generar un número primo aleatorio int generarPrimo() int primo; do
primo = rand() % 1000000; while (!esPrimo(primo)); return primo;
```

```
// Euclides extendido int euclidesExtendido(int a, int b, int* x, int* y) if
(b == 0) *x = 1; *y = 0; return a; int x1, y1; int gcd = euclidesExtendido(b,
a % b, &x1, &y1); *x = y1; *y = x1 - (a / b) * y1; return gcd;
```

```
// Inverso modular int inversoModular(int a, int n) int x, y; int gcd =
euclidesExtendido(a, n, x, y); if (gcd != 1) return -1; return (x % n + n) % n;
```

```
// Exponenciación modular rápida long long potenciaModular(long long
a, long long b, long long n) long long resultado = 1; a = a % n; while (b > 0) if
(b % 2 == 1) resultado = (resultado * a) % n; a = (a * a) % n; b = b / 2; return resultado;
```

```
void generarClaves(PublicKey* pub, PrivateKey* priv) srand(time(NULL));
```

```
int p = generarPrimo(); int q = generarPrimo(); while (p == q) q =
generarPrimo();
```

```
pub->n = p * q; priv->n = pub->n;
```

```
int phi = (p - 1) * (q - 1);
```

```
// Elegir e (comúnmente 65537) pub->e = 65537;
```

```
// Calcular d priv->d = inversoModular(pub->e, phi);
```

```
long long cifrar(long long m, PublicKey* pub) return potenciaModu-
lar(m, pub->e, pub->n);
```

```
long long descifrar(long long c, PrivateKey* priv) return potenciaMod-
ular(c, priv->d, priv->n);
```

Ejemplo Completo

```
[language=C, caption=Ejemplo de uso de RSA] int main() PublicKey pub;
PrivateKey priv;
```

```
generarClaves(&pub, &priv);
```

```
printf("Clave pública: (n="); printf("Clave privada: (n=
```

```
long long mensaje = 42; printf("Mensaje original:");
```

```
long long cifrado = cifrar(mensaje, &pub); printf("Mensaje cifrado:");
```

```
long long descifrado = descifrar(cifrado, &priv); printf("Mensaje descifrado:");
```

```
return 0;
```

Seguridad de RSA

Base de la Seguridad

La seguridad de RSA se basa en la dificultad de:

1. Factorizar n en sus factores primos p y q .
2. Calcular $\phi(n)$ sin conocer p y q .
3. Calcular la raíz e -ésima módulo n .

Tamaños de Clave Recomendados

Año	Tamaño de clave (bits)
2020	2048
2030	3072
2040	4096

Ataques Conocidos

- **Ataque de factorización:** Usando el algoritmo de la criba general de cuerpos de números (GNFS).
- **Ataque de texto cifrado elegido:** Si el descifrado no está blindado.
- **Ataque de canal lateral:** Basado en tiempo, consumo de energía, etc.
- **Ataque de paridad:** Si se revela un bit de información por vez.

Aplicaciones de RSA

Firma Digital

1. El remitente calcula el hash h del mensaje.
2. Firma: $s = h^d \bmod n$.
3. El receptor verifica: $h' = s^e \bmod n$ y compara con el hash del mensaje.

Intercambio de Claves

RSA se usa para intercambiar una clave simétrica de forma segura.

Certificados Digitales (X.509)

Los certificados SSL/TLS usan RSA para autenticación.

Ejercicios Resueltos (20 Ejercicios)

Dados $p = 11$, $q = 13$, calcule n y $\phi(n)$. $n = 11 \cdot 13 = 143$ $\phi(n) = (11 - 1)(13 - 1) = 10 \cdot 12 = 120$

Con $p = 11$, $q = 13$, elija $e = 7$ y calcule d . Necesitamos d tal que $7d \equiv 1 \pmod{120}$. $7 \cdot 103 = 721 \equiv 1 \pmod{120}$, por lo que $d = 103$.

Cifre $m = 5$ usando $n = 143$, $e = 7$. $c = 5^7 \bmod 143$ $5^2 = 25$, $5^4 = 25^2 = 625 \equiv 625 - 4 \cdot 143 = 625 - 572 = 53$ $5^7 = 5^4 \cdot 5^2 \cdot 5 = 53 \cdot 25 \cdot 5 = 53 \cdot 125 = 6625 \bmod 143$: $143 \cdot 46 = 6578$, $6625 - 6578 = 47$ $c = 47$

Descifre $c = 47$ usando $n = 143$, $d = 103$. $m = 47^{103} \bmod 143$ Por construcción, debe dar 5.

¿Por qué e debe ser coprimo con $\phi(n)$? Para que exista $d = e^{-1} \bmod \phi(n)$. Si $\gcd(e, \phi(n)) \neq 1$, el inverso modular no existe.

¿Qué tamaño de clave RSA se recomienda actualmente? Al menos 2048 bits.

¿Cuál es la diferencia entre criptografía simétrica y asimétrica? Simétrica: misma clave para cifrar y descifrar. Asimétrica: clave pública para cifrar, clave privada para descifrar.

¿Qué es una firma digital RSA? Es un hash del mensaje cifrado con la clave privada. El receptor verifica descifrando con la clave pública.

¿Por qué RSA no se usa para cifrar mensajes largos? Porque es lento. En su lugar, se usa RSA para intercambiar una clave simétrica, y esa clave se usa para cifrar el mensaje largo.

¿Qué es el ataque de factorización a RSA? Consiste en encontrar p y q a partir de n , lo que permite calcular $\phi(n)$ y luego d .

Ejercicios Propuestos (20 Ejercicios)

1. Genere claves RSA con $p = 13$, $q = 17$.
2. Cifre $m = 10$ con las claves del ejercicio anterior.
3. Descifre el resultado del ejercicio anterior.
4. ¿Qué es la función ϕ de Euler?
5. ¿Qué es el teorema de Fermat?

6. ¿Por qué se elige $e = 65537$ comúnmente?
7. Implemente la generación de claves RSA en Python.
8. ¿Qué es un ataque de canal lateral?
9. ¿Qué es un certificado digital?
10. ¿Cómo se usa RSA en SSL/TLS?
11. ¿Qué es el problema de factorización?
12. ¿Qué tamaño de clave se considera seguro para 2030?
13. ¿Qué es un texto cifrado elegido?
14. Implemente una firma digital RSA.
15. ¿Qué es el hash criptográfico y para qué se usa?
16. ¿Qué es la diferencia entre RSA y ECC?
17. ¿Qué es el algoritmo de factorización GNFS?
18. ¿Cómo se generan números primos grandes para RSA?
19. ¿Qué es la criptografía post-cuántica?
20. Investigue aplicaciones de RSA en el mundo real.

Resumen de la Unidad

En esta unidad hemos aprendido:

- Los fundamentos de la criptografía simétrica y asimétrica.
- El algoritmo RSA: generación de claves, cifrado y descifrado.
- Los fundamentos matemáticos: teorema de Euler y función ϕ .
- La implementación práctica de RSA.
- La seguridad de RSA y sus aplicaciones.

Bibliografía de la Unidad

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 31.
item Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120-126.
- Stinson, D. R. (2005). *Cryptography: Theory and Practice* (3rd ed.). Chapman Hall/CRC.

Unidad XIV: Backtracking (Vuelta Atrás) - Búsqueda Exhaustiva con Poda

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender la técnica de backtracking y su aplicación en problemas combinatorios.
- Implementar algoritmos de backtracking para problemas clásicos.
- Aplicar técnicas de poda para optimizar la búsqueda.
- Analizar la complejidad de algoritmos de backtracking.
- Distinguir entre backtracking y otras técnicas como programación dinámica.
- Resolver problemas de optimización combinatoria mediante backtracking.

Introducción al Backtracking

Definición

El **backtracking** (vuelta atrás) es una técnica algorítmica para resolver problemas de manera recursiva, construyendo una solución paso a paso, y deshaciendo los pasos que no conducen a una solución válida.

Filosofía

```
[scale=0.7] [circle, draw] (root) at (0,0) Inicio; [circle, draw] (a1) at (-2,-1)
Opción 1; [circle, draw] (a2) at (0,-1) Opción 2; [circle, draw] (a3) at (2,-1)
Opción 3;
[circle, draw] (b1) at (-3,-2) 1.1; [circle, draw] (b2) at (-1,-2) 1.2; [circle,
draw] (b3) at (1,-2) 2.1; [circle, draw] (b4) at (3,-2) 3.1;
[circle, draw] (c1) at (-3.5,-3) Solución;
[->] (root) -- (a1); [->] (root) -- (a2); [->] (root) -- (a3); [->] (a1) -- (b1); [->]
(a1) -- (b2); [->] (a2) -- (b3); [->] (a3) -- (b4); [->] (b1) -- (c1);
[red, thick] (a2) -- (b3); [red, thick] (b3) -- (c1);
```

Características

- **Búsqueda exhaustiva:** Explora todas las posibilidades.
- **Poda:** Elimina ramas que no pueden llevar a una solución.
- **Recursividad:** Naturalmente recursivo.
- **Vuelta atrás:** Deshace decisiones para probar alternativas.

Cuándo usar Backtracking

- Problemas de decisión (encontrar una solución).
- Problemas de optimización (encontrar la mejor solución).
- Problemas de enumeración (contar todas las soluciones).
- Espacio de búsqueda combinatorio grande pero estructurado.

Esquema General del Backtracking

Pseudocódigo General

```
[language=C, caption=Esquema general de backtracking] int backtracking(estado
actual) if (esSolucion(estado actual)) procesarSolucion(estado actual); re-
turn 1; // o contar según problema
int soluciones = 0;
for each (movimiento in generarMovimientos(estado actual)) if (esValido(movimiento))
aplicarMovimiento(movimiento); soluciones += backtracking(nuevo estado);
deshacerMovimiento(movimiento); // vuelta atrás
return soluciones;
```

Componentes Clave

1. **Estado:** Representación de la solución parcial.
2. **Criterio de solución:** Cuándo se ha alcanzado una solución completa.
3. **Generación de movimientos:** Cómo extender la solución parcial.
4. **Validación:** Verificar si un movimiento es prometedor (poda).
5. **Aplicación y deshacer:** Modificar y restaurar el estado.

Problemas Clásicos de Backtracking

Problema de las N Reinas

Descripción

Colocar n reinas en un tablero de $n \times n$ sin que se ataquen entre sí.

Implementación

```
[language=C, caption=N Reinas] include <stdio.h> include <stdlib.h> include
<math.h>
    define N 8
    int tablero[N][N]; int soluciones = 0;
    int esSeguro(int fila, int col) // Verificar misma columna for (int i = 0;
i < fila; i++) if (tablero[i][col]) return 0;
    // Verificar diagonal superior izquierda for (int i = fila, j = col; i >= 0 j
>= 0; i--, j--) if (tablero[i][j]) return 0;
    // Verificar diagonal superior derecha for (int i = fila, j = col; i <= 0 j <
N; i--, j++) if (tablero[i][j]) return 0;
    return 1;
    void resolverNReinas(int fila) if (fila == N) // Solución encontrada
soluciones++; printf("Solución for (int i = 0; i < N; i++) for (int j = 0; j <
N; j++) printf(" "); printf(" "); return;
    for (int col = 0; col < N; col++) if (esSeguro(fila, col)) tablero[fila][col]
= 1; resolverNReinas(fila + 1); tablero[fila][col] = 0; // backtracking
```

Optimización con Arrays de Seguimiento

```
[language=C, caption=N Reinas optimizado] int columnas[N]; int diag1[2*N
- 1]; int diag2[2*N - 1]; int reinas[N]; // reinas[fila] = columna
```

```

    void resolverNReinasOptimizado(int fila) if (fila == N) soluciones++;
return;
    for (int col = 0; col < N; col++) if (!columnas[col] & !diag1[fila - col +
N - 1] & !diag2[fila + col]) columnas[col] = 1; diag1[fila - col + N - 1] = 1;
diag2[fila + col] = 1; reinas[fila] = col;
    resolverNReinasOptimizado(fila + 1);
    columnas[col] = 0; diag1[fila - col + N - 1] = 0; diag2[fila + col] = 0;

```

Problema del Laberinto

Descripción

Encontrar un camino desde el inicio hasta la salida en un laberinto.

Implementación

```

[language=C, caption=Laberinto] define TAM 5
    int laberinto[TAM][TAM] = 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1,
1, 1, 0, 0, 0, 0, 1, 1 ;
    int solucion[TAM][TAM] = 0;
    int esValido(int x, int y) return (x >= 0 & x < TAM & y >= 0 & y < TAM
laberinto[x][y] == 1 & !solucion[x][y]);
    int resolverLaberinto(int x, int y) if (x == TAM - 1 & y == TAM - 1)
solucion[x][y] = 1; return 1;
    if (esValido(x, y)) solucion[x][y] = 1;
    // Intentar en las 4 direcciones if (resolverLaberinto(x + 1, y)) return
1; if (resolverLaberinto(x - 1, y)) return 1; if (resolverLaberinto(x, y + 1))
return 1; if (resolverLaberinto(x, y - 1)) return 1;
    solucion[x][y] = 0; // backtracking return 0; return 0;

```

Problema del Sudoku

Descripción

Completar una cuadrícula de 9x9 con números del 1 al 9 sin repetir en fila, columna o subcuadrícula de 3x3.

Implementación

```

[language=C, caption=Sudoku] define TAM_SUDOKU 9
    int sudoku[TAM_SUDOKU][TAM_SUDOKU] = 5, 3, 0, 0, 7, 0, 0, 0, 0, 6, 0, 0, 1, 9, 5, 0, 0, 0, 0,

```

```

    int esValidoSudoku(int fila, int col, int num) // Verificar fila for (int i =
0; i < TAM_SUDOKU; i++) if(sudoku[fila][i] == num) return 0;
    // Verificar columna for (int i = 0; i < TAM_SUDOKU; i++) if(sudoku[i][col] == num) return 0;
    // Verificar subcuadrícula 3x3 int inicioFila = (fila / 3) * 3; int inicioCol
= (col / 3) * 3; for (int i = inicioFila; i < inicioFila + 3; i++) for (int j =
inicioCol; j < inicioCol + 3; j++) if (sudoku[i][j] == num) return 0;
    return 1;
    int resolverSudoku() for (int fila = 0; fila < TAM_SUDOKU; fila +
+) for(int col = 0; col < TAM_SUDOKU; col++) if(sudoku[fila][col] == 0) for(int num = 1; num <
10; num++) if (resolverSudoku(fila, col, num)) return 1;
    sudoku[fila][col] = 0; // backtracking return 0; return 1;

```

Problema de la Mochila (0/1) con Backtracking

Implementación con Poda

```

[language=C, caption=Mochila con backtracking] typedef struct { int peso;
int valor; } Objeto;
Objeto objetos[] = {2, 3, 3, 4, 4, 5, 5, 6}; int n = 8; int capacidad = 10; int
mejorValor = 0; int mejorSeleccion[8];
void mochilaBacktracking(int i, int pesoActual, int valorActual, int* se-
leccion) if (i == n) if (valorActual > mejorValor) mejorValor = valorActual;
for (int j = 0; j < n; j++) mejorSeleccion[j] = seleccion[j]; return;
// Opción 1: no tomar el objeto i seleccion[i] = 0; mochilaBacktracking(i
+ 1, pesoActual, valorActual, seleccion);
// Opción 2: tomar el objeto i si cabe if (pesoActual + objetos[i].peso
<= capacidad) seleccion[i] = 1; mochilaBacktracking(i + 1, pesoActual +
objetos[i].peso, valorActual + objetos[i].valor, seleccion);

```

Técnicas de Poda

Poda por Optimalidad

```

[language=C, caption=Poda por optimalidad en mochila] // Cota superior:
valor actual + valor máximo posible de los objetos restantes int cotaSupe-
rior(int i, int valorActual, int pesoRestante) int valorEstimado = valorAc-
tual; int peso = pesoRestante;
for (int j = i; j < n; j++) peso += objetos[j].peso; j++) valorEstimado +=
objetos[j].valor; peso -= objetos[j].peso;
return valorEstimado;

```

```

void mochilaBacktrackingPoda(int i, int pesoActual, int valorActual, int*
seleccion) if (i == n) if (valorActual > mejorValor) mejorValor = valorAc-
tual; for (int j = 0; j < n; j++) mejorSeleccion[j] = seleccion[j]; return;
// Poda: si la cota superior no mejora la mejor solución actual if (cota-
Superior(i, valorActual, capacidad - pesoActual) <= mejorValor) return;
// Opción 1: no tomar seleccion[i] = 0; mochilaBacktrackingPoda(i + 1,
pesoActual, valorActual, seleccion);
// Opción 2: tomar if (pesoActual + objetos[i].peso >= capacidad) se-
leccion[i] = 1; mochilaBacktrackingPoda(i + 1, pesoActual + objetos[i].peso,
valorActual + objetos[i].valor, seleccion);

```

Poda por Factibilidad

Verificar si una solución parcial puede llevar a una solución completa.

Poda por Simetría

Eliminar ramas equivalentes por simetría del problema.

Análisis de Complejidad

Complejidad en el Peor Caso

- N Reinas: $O(n!)$
- Sudoku: $O(9^{81})$ (con poda, mucho menos)
- Laberinto: $O(4^n)$ donde n es el número de celdas

Mejoras Comunes

- Ordenamiento de movimientos (los más prometedores primero)
- Poda por poda de optimalidad
- Memoización (evitar repetir estados)
- Heurísticas para ordenar la búsqueda

Ejercicios Resueltos (20 Ejercicios)

Resuelva el problema de las 4 reinas manualmente. Una solución para 4 reinas: fila0: col2, fila1: col0, fila2: col3, fila3: col1.

¿Cuántas soluciones tiene el problema de las 8 reinas? 92 soluciones distintas (12 únicas considerando simetrías).

¿Qué es la poda en backtracking? Es la técnica de descartar ramas de búsqueda que no pueden llevar a una solución mejor que la ya encontrada.

Ejercicios Propuestos (20 Ejercicios)

1. Implemente el problema de las N reinas para N=10.
2. Resuelva un laberinto de 10x10 con backtracking.
3. Implemente un generador de sudokus usando backtracking.
4. ¿Qué es el problema del caballo (knight's tour)?
5. Implemente la búsqueda de todas las soluciones del problema de la mochila.
6. ¿Cómo se puede mejorar la poda en el problema de la mochila?
7. Implemente el problema de coloreo de grafos con backtracking.
8. ¿Qué es el problema del circuito de Hamilton?
9. Implemente un algoritmo de backtracking para el problema de la suma de subconjuntos.
10. ¿Qué diferencia hay entre backtracking y ramificación y poda (branch and bound)?
11. Implemente un generador de permutaciones con backtracking.
12. Resuelva el problema de las Torres de Hanoi con backtracking.
13. ¿Qué es la poda por factibilidad?
14. Implemente la búsqueda de caminos en un grafo con backtracking.
15. ¿Cómo se puede paralelizar backtracking?

16. Implemente un algoritmo de backtracking para el problema de la asignación.
17. ¿Qué es el problema de la partición y cómo se resuelve con backtracking?
18. Implemente un algoritmo para encontrar todas las combinaciones de suma objetivo.
19. ¿Qué es la poda por simetría?
20. Investigue aplicaciones de backtracking en inteligencia artificial.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El concepto de backtracking como búsqueda exhaustiva con vuelta atrás.
- La implementación de problemas clásicos: N reinas, laberinto, sudoku, mochila.
- Las técnicas de poda para optimizar la búsqueda.
- El análisis de complejidad de algoritmos de backtracking.
- La aplicación en problemas de optimización combinatoria.

Bibliografía de la Unidad

- Brassard, G., & Bratley, P. (1995). *Fundamentals of Algorithmics*. Prentice Hall.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Knuth, D. E. (2011). *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms*. Addison-Wesley.

Unidad XV: Programación Dinámica (Dynamic Programming)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el principio de optimalidad de Bellman.
- Identificar problemas que pueden resolverse con programación dinámica.
- Implementar soluciones de programación dinámica para problemas clásicos.
- Distinguir entre enfoques top-down (memoización) y bottom-up (tabular).
- Analizar la complejidad temporal y espacial de soluciones de programación dinámica.
- Aplicar programación dinámica a problemas de optimización.

Introducción a la Programación Dinámica

Definición

La **programación dinámica** es una técnica de diseño de algoritmos que resuelve problemas complejos dividiéndolos en subproblemas más simples, almacenando los resultados de estos subproblemas para evitar recalcularlos.

Características Clave

1. **Subestructura óptima:** Una solución óptima contiene soluciones óptimas de subproblemas.
2. **Subproblemas superpuestos:** Los mismos subproblemas aparecen repetidamente.
3. **Memorización:** Almacenar resultados para reutilizarlos.

Principio de Optimalidad de Bellman

[Principio de Optimalidad] Una política óptima tiene la propiedad de que, cualquiera sea el estado inicial y la decisión inicial, las decisiones restantes deben constituir una política óptima con respecto al estado resultante de la primera decisión.

Enfoques de Programación Dinámica

[scale=0.7] (0,1) rectangle (4,2); at (2,1.5) Top-down (Memoización); at (2,0.5) Recursivo + almacenamiento; at (2,0) Procesa solo subproblemas necesarios;
 (6,1) rectangle (10,2); at (8,1.5) Bottom-up (Tabular); at (8,0.5) Iterativo + tabla; at (8,0) Procesa todos los subproblemas;

Problema de la Mochila (0/1)

Descripción del Problema

Dados n objetos, cada uno con peso w_i y valor v_i , y una mochila con capacidad W , seleccionar un subconjunto de objetos que maximice el valor total sin exceder la capacidad.

Ecuación de Recurrencia

Sea $dp[i][j]$ el valor máximo que se puede obtener con los primeros i objetos y capacidad j :

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w_i] + v_i) \quad \text{si } j \geq w_i$$

$$dp[i][j] = dp[i-1][j] \quad \text{si } j < w_i$$

Caso base: $dp[0][j] = 0$ para todo j .

Implementación Bottom-up

```
[language=C, caption=Mochila 0/1 - Bottom-up] include <stdio.h> include
<string.h>
define MAXN100define MAXW1000
int mochila01(int n, int W, int pesos[], int valores[]) { int dp[MAXN +
1][MAXW + 1];
// Inicializar casos base for (int j = 0; j ≤ W; j++) dp[0][j] = 0;
// Llenar tabla for (int i = 1; i ≤ n; i++) for (int j = 0; j ≤ W; j++) if
(j ≥ pesos[i-1]) int tomar = dp[i-1][j - pesos[i-1]] + valores[i-1]; int noTomar
= dp[i-1][j]; dp[i][j] = (tomar ≥ noTomar) ? tomar : noTomar; else dp[i][j]
= dp[i-1][j];
return dp[n][W];
```

Optimización de Espacio

```
[language=C, caption=Mochila 0/1 - Optimización de espacio] int mochila01Optimizada(int
n, int W, int pesos[], int valores[]) { int dp[MAXW + 1];
// Inicializar memset(dp, 0, sizeof(dp));
// Llenar tabla (recorriendo W hacia atrás) for (int i = 0; i ≤ n; i++)
for (int j = W; j ≥ pesos[i]; j--) int tomar = dp[j - pesos[i]] + valores[i]; if
(tomar ≥ dp[j]) dp[j] = tomar;
return dp[W];
```

Ejemplo Paso a Paso

$n = 4$, $W = 7$, objetos: (2,3), (3,4), (4,5), (5,6)

i/j	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7
3	0	0	3	4	5	7	8	9
4	0	0	3	4	5	7	8	9

Valor máximo: 9

Distancia de Edición (Levenshtein)

Descripción del Problema

Dadas dos cadenas A y B , encontrar el número mínimo de operaciones (inserción, eliminación, sustitución) para transformar A en B .

Ecuación de Recurrencia

Sea $dp[i][j]$ la distancia de edición entre los primeros i caracteres de A y los primeros j caracteres de B :

$$dp[i][j] = \min \{ dp[i-1][j] + 1 (\text{eliminación}), dp[i][j-1] + 1 (\text{inserción}), dp[i-1][j-1] + \text{costo} (\text{sustitución}) \}$$

donde $\text{costo} = 0$ si $A[i-1] = B[j-1]$, $\text{costo} = 1$ en caso contrario.

Caso base: $dp[i][0] = i$, $dp[0][j] = j$.

Implementación

```
[language=C, caption=Distancia de Levenshtein] include <stdio.h> include
<string.h> include <stdlib.h>
int min(int a, int b, int c) { int m = a; if (b < m) m = b; if (c < m) m = c;
return m;
}
int distanciaEdicion(char* A, char* B) { int m = strlen(A); int n =
strlen(B);
// Crear tabla int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int
i = 0; i <= m; i++) dp[i] = (int*)malloc((n + 1) * sizeof(int));
// Casos base for (int i = 0; i <= m; i++) dp[i][0] = i; for (int j = 0; j
<= n; j++) dp[0][j] = j;
// Llenar tabla for (int i = 1; i <= m; i++) for (int j = 1; j <= n; j++)
int costo = (A[i-1] == B[j-1]) ? 0 : 1; dp[i][j] = min( dp[i-1][j] + 1, //
eliminación dp[i][j-1] + 1, // inserción dp[i-1][j-1] + costo // sustitución );
int resultado = dp[m][n];
// Liberar memoria for (int i = 0; i <= m; i++) free(dp[i]); free(dp);
return resultado;
}
```

Ejemplo

Transformar "CASA" en "PERRO"

		C	A	S	A
	0	1	2	3	4
P	1	1	2	3	4
E	2	2	2	3	4
R	3	3	3	3	4
R	4	4	4	4	4
O	5	5	5	5	5

Distancia: 5

Subsecuencia Común Más Larga (LCS)

Descripción del Problema

Dadas dos secuencias X e Y , encontrar la subsecuencia (no necesariamente contigua) más larga que aparece en ambas.

Ecuación de Recurrencia

Sea $dp[i][j]$ la longitud de la LCS entre los primeros i caracteres de X y los primeros j caracteres de Y :

$$dp[i][j] = \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ dp[i-1][j-1] + 1 & \text{si } X[i-1] = Y[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{en caso contrario} \end{cases}$$

Implementación

```
[language=C, caption=LCS] include <stdio.h> include <string.h> include <stdlib.h>
int max(int a, int b) return (a > b) ? a : b;
int lcs(char* X, char* Y) int m = strlen(X); int n = strlen(Y);
int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int i = 0; i <= m;
i++) dp[i] = (int*)calloc(n + 1, sizeof(int));
for (int i = 1; i <= m; i++) for (int j = 1; j <= n; j++) if (X[i-1] ==
Y[j-1]) dp[i][j] = dp[i-1][j-1] + 1; else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
int resultado = dp[m][n];
// Liberar memoria for (int i = 0; i <= m; i++) free(dp[i]); free(dp);
return resultado;
```

Recuperación de la Subsecuencia

```
[language=C, caption=Recuperar LCS] void recuperarLCS(char* X, char*
Y, int** dp, int i, int j, char* resultado, int* idx) if (i == 0 || j == 0)
return;
```

```

    if (X[i-1] == Y[j-1]) recuperarLCS(X, Y, dp, i-1, j-1, resultado, idx);
    resultado[(idx)++] = X[i-1]; else if (dp[i-1][j] > dp[i][j-1]) recuperarLCS(X,
    Y, dp, i-1, j, resultado, idx); else recuperarLCS(X, Y, dp, i, j-1, resultado,
    idx);

```

Ejemplo

$X = "ABCB DAB"$, $Y = "BDCABA"$

		A	B	C	B	D	A
B		0	0	0	0	0	0
0		0	0	1	1	1	1
1	B	0	0	1	1	1	2
2	D	0	0	1	2	2	2
3	C	0	0	1	2	2	2
4	A	0	1	1	2	2	3
5	B	0	1	2	2	3	3
6	A	0	1	2	2	3	4

LCS: "BCAB" o "BDAB", longitud 4.

Optimización de Espacio para LCS

```

[language=C, caption=LCS optimizado] int lcsOptimizado(char* X, char*
Y) { int m = strlen(X); int n = strlen(Y);
    // Usar solo 2 filas
    int** dp = (int**)malloc(2 * sizeof(int*));
    dp[0] = (int*)calloc(n + 1, sizeof(int));
    dp[1] = (int*)calloc(n + 1, sizeof(int));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (X[i-1] == Y[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    int resultado = dp[m][n];
    free(dp[0]); free(dp[1]); free(dp);
    return resultado;
}

```

Comparación de Enfoques

Problema	Tiempo	Espacio	Espacio optimizado
Mochila (0/1)	$O(nW)$	$O(nW)$	$O(W)$
Distancia de edición	$O(mn)$	$O(mn)$	$O(\min(m, n))$
LCS	$O(mn)$	$O(mn)$	$O(\min(m, n))$

Ejercicios Resueltos (20 Ejercicios)

Resuelva la mochila 0/1 con $n = 3$, $W = 5$, objetos: (2,3), (3,4), (4,5).

```
dp[0][*]=0
i=1: j=2: dp=3; j=3,4,5: dp=3
i=2: j=3: dp=4; j=5: dp=3+4=7
i=3: j=4: dp=5; j=5: dp=5 vs 7 -> 7
Valor máximo: 7
```

Calcule la distancia de edición entre "gato" y "perro".

gato -> perro: 4 sustituciones (g->p, a->e, t->r, o->r?) +1 inserción = 5

Encuentre LCS de "ABCD" y "ACBD". LCS: "ABD" o "ACD", longitud 3.

Ejercicios Propuestos (20 Ejercicios)

1. Resuelva mochila 0/1 con $n=4$, $W=10$, objetos (2,5),(3,7),(4,8),(5,9).
2. Implemente distancia de edición con operaciones de costo variable.
3. Encuentre LCS de "XMJYAUZ" y "MZJAWXU".
4. ¿Qué es el principio de optimalidad de Bellman?
5. Implemente mochila 0/1 con recuperación de objetos seleccionados.
6. ¿Cuál es la diferencia entre top-down y bottom-up?
7. Resuelva el problema de la subsecuencia creciente máxima con PD.
8. Implemente distancia de edición con recuperación de operaciones.

30 UNIDAD XV: PROGRAMACIÓN DINÁMICA (DYNAMIC PROGRAMMING)

9. ¿Qué es la matriz de costos en LCS?
10. Resuelva el problema de la partición con PD.
11. Implemente LCS con recuperación de todas las soluciones.
12. ¿Cómo se puede mejorar la complejidad espacial de distancia de edición?
13. Resuelva el problema del cambio de monedas con PD.
14. Implemente mochila 0/1 con objetos fraccionarios? (no es PD)
15. ¿Qué es la subestructura óptima?
16. Resuelva el problema del viajante con PD (TSP).
17. Implemente la multiplicación de matrices con PD.
18. ¿Cuándo usar programación dinámica vs greedy?
19. Resuelva el problema de la distancia de Damerau-Levenshtein.
20. Investigue aplicaciones de PD en bioinformática.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El principio de optimalidad de Bellman.
- La diferencia entre enfoques top-down y bottom-up.
- La implementación de tres problemas clásicos: mochila 0/1, distancia de edición y LCS.
- Técnicas de optimización de espacio.
- El análisis de complejidad de soluciones de programación dinámica.

Bibliografía de la Unidad

- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 15.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.

Unidad XV: Programación Dinámica (Dynamic Programming)

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante será capaz de:

- Comprender el principio de optimalidad de Bellman.
- Identificar problemas que pueden resolverse con programación dinámica.
- Implementar soluciones de programación dinámica para problemas clásicos.
- Distinguir entre enfoques top-down (memoización) y bottom-up (tabular).
- Analizar la complejidad temporal y espacial de soluciones de programación dinámica.
- Aplicar programación dinámica a problemas de optimización.

Introducción a la Programación Dinámica

Definición

La **programación dinámica** es una técnica de diseño de algoritmos que resuelve problemas complejos dividiéndolos en subproblemas más simples, almacenando los resultados de estos subproblemas para evitar recalcularlos.

Características Clave

1. **Subestructura óptima:** Una solución óptima contiene soluciones óptimas de subproblemas.
2. **Subproblemas superpuestos:** Los mismos subproblemas aparecen repetidamente.
3. **Memorización:** Almacenar resultados para reutilizarlos.

Principio de Optimalidad de Bellman

[Principio de Optimalidad] Una política óptima tiene la propiedad de que, cualquiera sea el estado inicial y la decisión inicial, las decisiones restantes deben constituir una política óptima con respecto al estado resultante de la primera decisión.

Enfoques de Programación Dinámica

[scale=0.7] (0,1) rectangle (4,2); at (2,1.5) Top-down (Memoización); at (2,0.5) Recursivo + almacenamiento; at (2,0) Procesa solo subproblemas necesarios;
 (6,1) rectangle (10,2); at (8,1.5) Bottom-up (Tabular); at (8,0.5) Iterativo + tabla; at (8,0) Procesa todos los subproblemas;

Problema de la Mochila (0/1)

Descripción del Problema

Dados n objetos, cada uno con peso w_i y valor v_i , y una mochila con capacidad W , seleccionar un subconjunto de objetos que maximice el valor total sin exceder la capacidad.

Ecuación de Recurrencia

Sea $dp[i][j]$ el valor máximo que se puede obtener con los primeros i objetos y capacidad j :

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w_i] + v_i) \quad \text{si } j \geq w_i$$

$$dp[i][j] = dp[i-1][j] \quad \text{si } j < w_i$$

Caso base: $dp[0][j] = 0$ para todo j .

Implementación Bottom-up

```
[language=C, caption=Mochila 0/1 - Bottom-up] include <stdio.h> include
<string.h>
define MAXN100define MAXW1000
int mochila01(int n, int W, int pesos[], int valores[]) {
    int dp[MAXN + 1][MAXW + 1];
    // Inicializar casos base for (int j = 0; j ≤ W; j++) dp[0][j] = 0;
    // Llenar tabla for (int i = 1; i ≤ n; i++) for (int j = 0; j ≤ W; j++) if
    (j ≥ pesos[i-1]) int tomar = dp[i-1][j - pesos[i-1]] + valores[i-1]; int noTomar
    = dp[i-1][j]; dp[i][j] = (tomar ≥ noTomar) ? tomar : noTomar; else dp[i][j]
    = dp[i-1][j];
    return dp[n][W];
}
```

Optimización de Espacio

```
[language=C, caption=Mochila 0/1 - Optimización de espacio] int mochila01Optimizada(int
n, int W, int pesos[], int valores[]) {
    int dp[MAXW + 1];
    // Inicializar memset(dp, 0, sizeof(dp));
    // Llenar tabla (recorriendo W hacia atrás) for (int i = 0; i ≤ n; i++)
    for (int j = W; j ≥ pesos[i]; j--) int tomar = dp[j - pesos[i]] + valores[i]; if
    (tomar ≥ dp[j]) dp[j] = tomar;
    return dp[W];
}
```

Ejemplo Paso a Paso

$n = 4$, $W = 7$, objetos: (2,3), (3,4), (4,5), (5,6)

i/j	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7
3	0	0	3	4	5	7	8	9
4	0	0	3	4	5	7	8	9

Valor máximo: 9

Distancia de Edición (Levenshtein)

Descripción del Problema

Dadas dos cadenas A y B , encontrar el número mínimo de operaciones (inserción, eliminación, sustitución) para transformar A en B .

Ecuación de Recurrencia

Sea $dp[i][j]$ la distancia de edición entre los primeros i caracteres de A y los primeros j caracteres de B :

$$dp[i][j] = \min \{ dp[i-1][j] + 1 (\text{eliminación}), dp[i][j-1] + 1 (\text{inserción}), dp[i-1][j-1] + \text{costo} (\text{sustitución}) \}$$

donde $\text{costo} = 0$ si $A[i-1] = B[j-1]$, $\text{costo} = 1$ en caso contrario.

Caso base: $dp[i][0] = i$, $dp[0][j] = j$.

Implementación

```
[language=C, caption=Distancia de Levenshtein]
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int min(int a, int b, int c) {
    int m = a;
    if (b < m) m = b;
    if (c < m) m = c;
    return m;
}

int distanciaEdicion(char* A, char* B) {
    int m = strlen(A);
    int n = strlen(B);

    // Crear tabla
    int** dp = (int**)malloc((m + 1) * sizeof(int*));
    for (int i = 0; i <= m; i++) {
        dp[i] = (int*)malloc((n + 1) * sizeof(int));
    }

    // Casos base
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    // Llenar tabla
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            int costo = (A[i-1] == B[j-1]) ? 0 : 1;
            dp[i][j] = min(dp[i-1][j] + 1, // eliminación
                           dp[i][j-1] + 1, // inserción
                           dp[i-1][j-1] + costo); // sustitución
        }
    }

    int resultado = dp[m][n];

    // Liberar memoria
    for (int i = 0; i <= m; i++) free(dp[i]);
    free(dp);

    return resultado;
}
```

Ejemplo

Transformar "CASA" en "PERRO"

		C	A	S	A
	0	1	2	3	4
P	1	1	2	3	4
E	2	2	2	3	4
R	3	3	3	3	4
R	4	4	4	4	4
O	5	5	5	5	5

Distancia: 5

Subsecuencia Común Más Larga (LCS)

Descripción del Problema

Dadas dos secuencias X e Y , encontrar la subsecuencia (no necesariamente contigua) más larga que aparece en ambas.

Ecuación de Recurrencia

Sea $dp[i][j]$ la longitud de la LCS entre los primeros i caracteres de X y los primeros j caracteres de Y :

$$dp[i][j] = \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ dp[i-1][j-1] + 1 & \text{si } X[i-1] = Y[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{en caso contrario} \end{cases}$$

Implementación

```
[language=C, caption=LCS] include <stdio.h> include <string.h> include <stdlib.h>
int max(int a, int b) return (a > b) ? a : b;
int lcs(char* X, char* Y) int m = strlen(X); int n = strlen(Y);
int** dp = (int**)malloc((m + 1) * sizeof(int*)); for (int i = 0; i <= m;
i++) dp[i] = (int*)calloc(n + 1, sizeof(int));
for (int i = 1; i <= m; i++) for (int j = 1; j <= n; j++) if (X[i-1] ==
Y[j-1]) dp[i][j] = dp[i-1][j-1] + 1; else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
int resultado = dp[m][n];
// Liberar memoria for (int i = 0; i <= m; i++) free(dp[i]); free(dp);
return resultado;
```

Recuperación de la Subsecuencia

```
[language=C, caption=Recuperar LCS] void recuperarLCS(char* X, char*
Y, int** dp, int i, int j, char* resultado, int* idx) if (i == 0 || j == 0)
return;
```



```

    if (X[i-1] == Y[j-1]) recuperarLCS(X, Y, dp, i-1, j-1, resultado, idx);
    resultado[(idx)++] = X[i-1]; else if (dp[i-1][j] > dp[i][j-1]) recuperarLCS(X,
    Y, dp, i-1, j, resultado, idx); else recuperarLCS(X, Y, dp, i, j-1, resultado,
    idx);

```

Ejemplo

$X = \text{"ABCB DAB"} , Y = \text{"BDCABA"}$

		A	B	C	B	D	A
B		0	0	0	0	0	0
0		0	0	1	1	1	1
1	B	0	0	1	1	1	2
2	D	0	0	1	2	2	2
3	C	0	0	1	2	2	2
4	A	0	1	1	2	2	3
5	B	0	1	2	2	3	3
6	A	0	1	2	2	3	4

LCS: "BCAB" o "BDAB", longitud 4.

Optimización de Espacio para LCS

```

[language=C, caption=LCS optimizado] int lcsOptimizado(char* X, char*
Y) { int m = strlen(X); int n = strlen(Y);
    // Usar solo 2 filas
    int** dp = (int**)malloc(2 * sizeof(int*));
    dp[0] = (int*)calloc(n + 1, sizeof(int));
    dp[1] = (int*)calloc(n + 1, sizeof(int));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (X[i-1] == Y[j-1])
                dp[i][j] = dp[i-1][j-1] + 1;
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    int resultado = dp[m][n];
    free(dp[0]); free(dp[1]); free(dp);
    return resultado;
}

```

Comparación de Enfoques

Problema	Tiempo	Espacio	Espacio optimizado
Mochila (0/1)	$O(nW)$	$O(nW)$	$O(W)$
Distancia de edición	$O(mn)$	$O(mn)$	$O(\min(m, n))$
LCS	$O(mn)$	$O(mn)$	$O(\min(m, n))$

Ejercicios Resueltos (20 Ejercicios)

Resuelva la mochila 0/1 con $n = 3$, $W = 5$, objetos: (2,3), (3,4), (4,5).

```
dp[0][*]=0
i=1: j=2: dp=3; j=3,4,5: dp=3
i=2: j=3: dp=4; j=5: dp=3+4=7
i=3: j=4: dp=5; j=5: dp=5 vs 7 -> 7
Valor máximo: 7
```

Calcule la distancia de edición entre "gato" y "perro".

gato -> perro: 4 sustituciones (g->p, a->e, t->r, o->r?) +1 inserción = 5

Encuentre LCS de "ABCD" y "ACBD". LCS: "ABD" o "ACD", longitud 3.

Ejercicios Propuestos (20 Ejercicios)

1. Resuelva mochila 0/1 con $n=4$, $W=10$, objetos (2,5),(3,7),(4,8),(5,9).
2. Implemente distancia de edición con operaciones de costo variable.
3. Encuentre LCS de "XMJYAUZ" y "MZJAWXU".
4. ¿Qué es el principio de optimalidad de Bellman?
5. Implemente mochila 0/1 con recuperación de objetos seleccionados.
6. ¿Cuál es la diferencia entre top-down y bottom-up?
7. Resuelva el problema de la subsecuencia creciente máxima con PD.
8. Implemente distancia de edición con recuperación de operaciones.

40 UNIDAD XV: PROGRAMACIÓN DINÁMICA (DYNAMIC PROGRAMMING)

9. ¿Qué es la matriz de costos en LCS?
10. Resuelva el problema de la partición con PD.
11. Implemente LCS con recuperación de todas las soluciones.
12. ¿Cómo se puede mejorar la complejidad espacial de distancia de edición?
13. Resuelva el problema del cambio de monedas con PD.
14. Implemente mochila 0/1 con objetos fraccionarios? (no es PD)
15. ¿Qué es la subestructura óptima?
16. Resuelva el problema del viajante con PD (TSP).
17. Implemente la multiplicación de matrices con PD.
18. ¿Cuándo usar programación dinámica vs greedy?
19. Resuelva el problema de la distancia de Damerau-Levenshtein.
20. Investigue aplicaciones de PD en bioinformática.

Resumen de la Unidad

En esta unidad hemos aprendido:

- El principio de optimalidad de Bellman.
- La diferencia entre enfoques top-down y bottom-up.
- La implementación de tres problemas clásicos: mochila 0/1, distancia de edición y LCS.
- Técnicas de optimización de espacio.
- El análisis de complejidad de soluciones de programación dinámica.

Bibliografía de la Unidad

- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulo 15.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707-710.

Unidad XVII: Apéndice - Fórmulas, Recursos y Apoyo Académico

Objetivos de la Unidad

Al finalizar esta unidad, el estudiante tendrá acceso a:

- Un compendio de fórmulas y conceptos clave de la materia.
- Una bibliografía completa y actualizada.
- Recursos digitales (canales de YouTube, sitios web, blogs).
- Información de contacto para apoyo académico personalizado.

Compendio de Fórmulas y Conceptos Clave

Complejidad Computacional

- **Clase P:** Problemas resolubles en tiempo polinomial.
- **Clase NP:** Problemas verificables en tiempo polinomial.
- **NP-completo:** $L \in NP$ y todo $L' \in NP$ se reduce a L .
- **NP-hard:** Todo $L' \in NP$ se reduce a L (no necesita estar en NP).
- **Reducción polinomial:** $A \leq_p B$ si existe f polinomial tal que $x \in A \iff f(x) \in B$.

Algoritmos de Caminos Mínimos

- **Dijkstra:** $O(E \log V)$ con montículo. Requiere pesos no negativos.
- **Bellman-Ford:** $O(VE)$. Permite pesos negativos. Detecta ciclos negativos.
- **Floyd-Warshall:** $O(V^3)$. Todos los pares.

Flujo en Redes

- **Teorema de Flujo Máximo - Corte Mínimo:** $\max |f| = \min c(S, T)$
- **Ford-Fulkerson:** $O(E \cdot |f^*|)$
- **Edmonds-Karp:** $O(VE^2)$

Algoritmos de String

- **KMP:** $O(n + m)$. Función de prefijo π .
- **Boyer-Moore:** $O(n/m)$ promedio. Heurísticas de mal carácter y buen sufijo.

Aritmética Modular y RSA

- **Euclides:** $\gcd(a, b) = \gcd(b, a \bmod b)$
- **Euclides extendido:** $ax + by = \gcd(a, b)$
- **Teorema de Fermat:** $a^{p-1} \equiv 1 \pmod{p}$ si p primo y $\gcd(a, p) = 1$
- **Teorema de Euler:** $a^{\phi(n)} \equiv 1 \pmod{n}$ si $\gcd(a, n) = 1$
- **RSA:** $c = m^e \bmod n$, $m = c^d \bmod n$, $ed \equiv 1 \pmod{\phi(n)}$

Programación Dinámica

- **Mochila 0/1:** $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w_i] + v_i)$
- **Distancia de edición:** $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + cost)$
- **LCS:** $dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{si } X[i] = Y[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{cc} \end{cases}$

Backtracking

- Esquema general: recursión + validación + vuelta atrás.
- Poda por optimalidad, factibilidad y simetría.

Algoritmos Aproximados

- **Vertex Cover:** factor 2
- **TSP métrico:** factor 2 (vía MST)
- **KNAPSACK:** FPTAS con escalado de valores

Bibliografía Recomendada

Bibliografía Principal

- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
 - **Comentario:** La "biblia" de los algoritmos. Cubre todos los temas del programa con rigor matemático.
 - **Capítulos clave:** 34 (NP-Complejidad), 24-26 (Grafos), 32 (Strings), 31 (Números), 15 (PD), 35 (Aproximación).
- **Sedgewick, R., & Wayne, K. (2011).** *Algorithms* (4th ed.). Addison-Wesley.
 - **Comentario:** Enfoque práctico con implementaciones en Java. Excelente para visualización.
- **Kleinberg, J., & Tardos, É. (2006).** *Algorithm Design*. Addison-Wesley.
 - **Comentario:** Excelente para diseño de algoritmos y problemas de optimización.

Bibliografía de Complejidad

- **Arora, S., & Barak, B. (2009).** *Computational Complexity: A Modern Approach*. Cambridge University Press.

- **Garey, M. R., & Johnson, D. S. (1979).** *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- **Sipser, M. (2012).** *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

Bibliografía de Grafos

- **Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993).** *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
- **Even, S. (2011).** *Graph Algorithms* (2nd ed.). Cambridge University Press.

Bibliografía de Criptografía y Números

- **Stinson, D. R. (2005).** *Cryptography: Theory and Practice* (3rd ed.). Chapman & Hall/CRC.
- **Rosen, K. H. (2010).** *Elementary Number Theory and Its Applications* (6th ed.). Addison-Wesley.

Bibliografía de Metaheurísticas

- **Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983).** Optimization by simulated annealing. *Science*, 220(4598), 671-680.
- **Holland, J. H. (1992).** *Adaptation in Natural and Artificial Systems*. MIT Press.
- **Dorigo, M., & Stützle, T. (2004).** *Ant Colony Optimization*. MIT Press.

Canales de YouTube Recomendados

Canales en Español

- **Mitos y Leyendas de la Programación (Andrés)**
 - URL: youtube.com/c/MitosyLeyendasdeProgramacion

- **Contenido:** Algoritmos de grafos, NP-completitud, programación dinámica.
- **Deusto Ciencia**
 - **URL:** youtube.com/c/DeustoCiencia
 - **Contenido:** Complejidad computacional, teoría de la computación.
- **Algoritmos y Programación (UBA)**
 - **URL:** youtube.com/c/AlgoritmosyProgramacionUBA
 - **Contenido:** Material de las cátedras de Exactas.

Canales en Inglés

- **Abdul Bari**
 - **URL:** youtube.com/c/AbdulBari
 - **Contenido:** Algoritmos de grafos, programación dinámica, NP-completitud.
- **Reducible**
 - **URL:** youtube.com/c/Reducible
 - **Contenido:** Animaciones de alta calidad sobre algoritmos complejos.
- **Back To Back SWE**
 - **URL:** youtube.com/c/BackToBackSWE
 - **Contenido:** Preparación para entrevistas, problemas de algoritmos.
- **Computerphile**
 - **URL:** youtube.com/c/computerphile
 - **Contenido:** P vs NP, algoritmos de string, criptografía.
- **3Blue1Brown**
 - **URL:** youtube.com/c/3blue1brown
 - **Contenido:** Visualizaciones matemáticas útiles para complejidad.

Sitios Web y Blogs Recomendados

Sitios Educativos

- **VisuAlgo**
 - **URL:** visualgo.net
 - **Contenido:** Visualizaciones interactivas de algoritmos.
- **GeeksforGeeks**
 - **URL:** geeksforgeeks.org
 - **Contenido:** Artículos, implementaciones y ejercicios.
- **LeetCode**
 - **URL:** leetcode.com
 - **Contenido:** Práctica de problemas algorítmicos.
- **HackerRank**
 - **URL:** hackerrank.com
 - **Contenido:** Desafíos de algoritmos.
- **Coursera**
 - **URL:** coursera.org
 - **Contenido:** Cursos de algoritmos (Princeton, Stanford, MIT).
- **edX**
 - **URL:** edx.org
 - **Contenido:** Cursos de MIT y Harvard.

Blogs Especializados

- **CP-Algorithms**
 - **URL:** cp-algorithms.com
 - **Contenido:** Algoritmos avanzados y optimización.
- **Brilliant**
 - **URL:** brilliant.org

- **Contenido:** Matemáticas y algoritmos interactivos.
- **The Algorithms**
 - **URL:** the-algorithms.com
 - **Contenido:** Implementaciones en múltiples lenguajes.

Foros y Comunidades

- **Stack Overflow**
 - **URL:** stackoverflow.com
 - **Contenido:** Consultas específicas sobre implementaciones.
- **Reddit - r/algorithms**
 - **URL:** reddit.com/r/algorithms
 - **Contenido:** Discusión sobre algoritmos.
- **Discord: Algoritmos UBA**
 - **Contenido:** Comunidad de estudiantes de Exactas.

Profesores Particulares Recomendados

Nahuel Villafañe - @clasesconnahu

[colback=blue!5!white, colframe=blue!75!black, title=Nahuel Villafañe]
Estudiante avanzado de Ciencias Fisica

Universidad de Buenos Aires (UBA) - Facultad de Ciencias Exactas y Naturales

Experiencia:

- Autor de los tratados "*Tratado de Algoritmos y Datos II*" y "*Tratado de Algoritmos y Datos III*" y mucho otros mas

Clases Particulares:

- **Especialidad:** Fisica y Matematicas
- **Modalidad:** Presencial (CABA) y virtual (Zoom, Google Meet)

Contacto:

- Instagram: @clasesconnahu

Otros Profesores Recomendados

- **Martín Diéguez** (@martindieguez) - Especialidad: Algoritmos y Programación (UBA)
- **Facundo Palavecino** - Especialidad: Matemática discreta y complejidad
- **Clases Particulares UBA** - clasesparticularesuba.com.ar
- **Superprof Argentina** - superprof.com.ar

Recursos Adicionales

Google Drive y Material Compartido

- **Apuntes de Cátedra:** Disponibles en el Departamento de Computación (dc.uba.ar)
- **Parciales Resueltos:** Grupos de Telegram y Discord de la materia
- **GitHub:** Buscar repositorios de algoritmos con implementaciones

Grupos de Estudio

- **Telegram:** "Algo2 - Algo3 UBA" (buscar en el chat de la facultad)
- **Discord:** Servidor de Exactas UBA
- **WhatsApp:** Grupos por comisión (consultar en la cursada)

Consejos Finales para el Estudio

1. **No memorices, comprendé:** Los algoritmos son lógicos. Entendé por qué funcionan.
2. **Practicá con papel y lápiz:** Ejecutar algoritmos a mano ayuda a internalizarlos.

[width=0.5]Logo_exactas.jpg

3. **Implementá todo:** La programación refuerza la comprensión.
4. **Estudiá en grupo:** Explicar a otros es la mejor forma de aprender.
5. **Hacé simulacros de parcial:** Con tiempo limitado para practicar presión.
6. **Consultá las dudas a tiempo:** No dejes que se acumulen.

Epílogo

Querido estudiante,

Has llegado al final de este tratado. Los temas que has recorrido —desde la complejidad computacional hasta los algoritmos aproximados— son el corazón de la computación teórica y práctica.

No te desanimes si algunos conceptos parecen difíciles al principio. La belleza de los algoritmos se revela con la práctica y la paciencia. Cada problema resuelto es un paso más en tu formación.

Este tratado fue escrito con la esperanza de acompañarte en ese camino. Si algo no quedó claro, si necesitas profundizar en algún tema, o simplemente querés compartir tus avances, no dudes en contactarme a través de las redes.

Que la complejidad esté siempre de tu lado, que tus invariantes sean correctos y que tus caminos sean siempre mínimos.

Con cariño,

Nahuel Villafañe

Buenos Aires, 2026

Colofón

Este documento fue compuesto en L^AT_EX utilizando la clase `book`. Las figuras fueron elaboradas con TikZ. La tipografía principal es Computer Modern.

“La ciencia de los algoritmos es la ciencia de lo computable.”

LA TRILOGIA COMPLETA

I ALGORITMOS
Y DATOS I

II ALGORITMOS
Y DATOS II

III ALGORITMOS
Y DATOS III

Este cuaderno aborda los temas necesarios para entender algoritmos y datos I, II y III. Una guía clara, progresiva y completa para estudiantes, docentes y autodidactas.



9 789876 543210 >



HECHO EN
ARGENTINA



```
function buscar(A, x)
  i <- 0 ; n <- longitud(A)
  mientras i < n
    m <- (i + n) / 2
    si A[m] == x entonces
      devolver m
    si x < A[m] entonces
      n <- m
    sino
      i <- m + 1
  fin mientras
  devolver -1
```

```
lista <- [8, 3, 10, 1, 6, 14, 4, 7]
para i <- 1 hasta longitud(lista)-1
  clave <- lista[i]
  j <- i - 1
  mientras j >= 0 y lista[j] > clave
    lista[j + 1] <- lista[j]
  j <- j - 1
  fin mientras
  lista[j + 1] <- clave
```

